A PRACTICAL COURSE

Async PostgreSQL in Rust

tokio-postgres from the first query to a production-shaped axum service — for developers who know Rust, a little Tokio and some SQL.

20 chapters · every runnable example was compiled and run against PostgreSQL as written

python3 tools/book.py book

Contents

0 Introduction and setup . . . **1**

Contents . . . 1

Who this is for, and what it assumes . . . 2

Where tokio-postgres fits . . . 2

How each chapter is built . . . 3

stockroom in one page . . . 4

Setting up . . . 5

A ten-line taste . . . 7

Exercises . . . 8

1 Connecting . . . **9**

1.1 Two halves: `Client` and `Connection` . . . 9

1.2 Spawning the connection, and forgetting to . . . 10

1.3 Connection strings . . . 11

1.4 The `Config` builder . . . 13

1.5 What the server told you, and how a connection ends . . . 14

1.6 What failure looks like . . . 15

1.7 The playground's `connect` . . . 18

In stockroom . . . 18

Summary . . . 19

Exercises . . . 20

2 Running SQL: execute, query and parameters . . . **22**

2.1 `batch_execute`: scripts . . . 22

2.2 `execute`: changes, counted . . . 23

2.3 Parameters . . . 24

2.4 `query`, `query_one` and `query_opt`: reading . . . 26

2.5 Scalars: `query_scalar`, `query_one_scalar`, `query_opt_scalar` . . . 28

2.6 `RETURNING`: changing and reading in one statement . . . 28

2.7 The simple query protocol . . . 29

In stockroom . . . 31

Summary . . . 32

Exercises . . . 33

3 Rows and the type mapping . . . **35**

3.1 Reading a `Row` . . . 35

3.2 The type table 36
3.3 NULL 38
3.4 Borrowing from a row 39
3.5 Rows into structs 40
3.6 Dates and times 41
3.7 JSON 42
3.8 Arrays 43
3.9 Exact numbers, UUIDs 45
In stockroom 47
Summary 47
Exercises 48

4 Prepared statements and the extended protocol 50
4.1 What `query` sends 50
4.2 `prepare`: one parse, many executions 51
4.3 When the server cannot infer a type 52
4.4 `query_typed`: one round trip, no statement 53
4.5 Counting round trips 54
4.6 Caching statements, and when the cache goes stale 55
In stockroom 58
Summary 59
Exercises 59

5 Custom types: ToSql and FromSql 62
5.1 The two traits 62
5.2 A newtype by hand 63
5.3 Deriving: `transparent` newtypes 64
5.4 Enums 65
5.5 Domains 67
5.6 Composite types 69
5.7 Behind the scenes: `Type`, `Kind` and the type cache 70
In stockroom 72
Summary 73
Exercises 73

6 Errors 76
6.1 One type, many causes 76
6.2 `DbError`: the server's report 78
6.3 `SqlState`: the codes worth matching 80
6.4 `is_closed`, and the error you cannot classify 82
6.5 An error type for an application 82
6.6 An error inside a transaction 84

6.7 Notices are not errors 85
In stockroom 87
Summary 88
Exercises 88

7 One connection, many tasks: pipelining 90
7.1 Lazy futures, and the order requests are sent in 90
7.2 `join!`: N queries, about one round trip 91
7.3 One client, many tasks 92
7.4 Under the hood: a queue, in order 93
7.5 Many queries, a few at a time 94
7.6 When not to pipeline 95
In stockroom 97
Summary 98
Exercises 98

8 Transactions and concurrency control 100
8.1 `transaction`, `commit`, and rollback on drop 100
8.2 Why `transaction` takes `&mut self` 101
8.3 Isolation levels, read-only and deferrable 102
8.4 Savepoints: transactions inside transactions 103
8.5 `GenericClient`: code that runs inside or outside a transaction 104
8.6 Retrying transactions that lose a race 105
8.7 Locks 108
8.8 Optimistic concurrency: version numbers 109
8.9 Selling the last chair, four ways 110
In stockroom 114
Summary 115
Exercises 116

9 Streaming rows and portals 118
9.1 Why a `Vec<Row>` is not always enough 118
9.2 `query_raw` and `RowStream` 119
9.3 Backpressure, and the stalled connection 121
9.4 Portals: the server produces only what you ask for 123
9.5 The SQL alternative: `DECLARE` and `FETCH` 125
9.6 Keyset pagination: the HTTP answer 126
9.7 `simple_query_raw` 128
In stockroom 130
Summary 132
Exercises 132

10 Bulk data: COPY 135

10.1 `copy_in` and `CopyInSink` 135
10.2 `copy_out` and `CopyOutStream` 138
10.3 Binary COPY 139
10.4 When a load fails halfway 141
10.5 What COPY buys: the pgload table 143
10.6 COPY into a temp table, then upsert 145
In stockroom 147
Summary 149
Exercises 149

11 LISTEN/NOTIFY and notices 153
11.1 What `NOTIFY` promises 153
11.2 Driving the `Connection` yourself 154
11.3 `AsyncMessage`: notifications and notices 156
11.4 Triggers that announce changes 157
11.5 A connection of its own 158
11.6 Reliability: listen, catch up, deduplicate 159
11.7 Fan-out with `broadcast` 161
In stockroom 163
Summary 164
Exercises 165

12 Production connections: timeouts, cancellation, health and TLS 167
12.1 Dropping a future does not cancel the query 167
12.2 `CancelToken`: asking the server to stop 168
12.3 Timeouts that stop the work 169
12.4 Dead connections 172
12.5 Several hosts 174
12.6 TLS 175
12.7 Opening the connection yourself: `connect_raw` 181
In stockroom 182
Summary 184
Exercises 184

13 Connection pooling 187
13.1 Why one `Client` is not enough 187
13.2 A pool from scratch: minipool 187
13.3 deadpool-postgres 189
13.4 `Object`, `prepare_cached`, `transaction` and `Object::take` 192
13.5 How big? Measured 193
13.6 bb8, and PgBouncer 194
13.7 What not to do with a pooled connection 195

In stockroom 196
Summary 197
Exercises 197

14 Schema, migrations and the data layer 200
14.1 Embedded migrations 200
14.2 Running migrations safely 203
14.3 refinery, the ready-made alternative 205
14.4 The repository layer 206
14.5 Domain types at the boundary 208
14.6 Generating the data layer: Cornucopia 209
In stockroom 209
Summary 209
Exercises 210

15 axum I: a database-backed JSON API 212
15.1 The shape of the app 212
15.2 A `Db` extractor 214
15.3 `AppError`: from SQLSTATE to HTTP status 216
15.4 Writes in transactions 219
15.5 Keyset pagination with an opaque cursor 222
15.6 ETag and `If-Match`: optimistic concurrency over HTTP 224
15.7 Validation at the edge 226
15.8 The pipelined dashboard 228
In stockroom 230
Summary 231
Exercises 232

16 axum II: streaming, real-time and background work 234
16.1 NDJSON from a `RowStream`: who owns the connection? 234
16.2 A CSV export with `copy_out` 236
16.3 A CSV import: from the request body into `COPY` 237
16.4 Server-sent events from `LISTEN` 239
16.5 Timeouts and client disconnects 241
16.6 The outbox worker 243
16.7 Graceful shutdown 246
16.8 Health and readiness 246
In stockroom 247
Summary 247
Exercises 248

17 Testing against a real database 250
17.1 What to test where 250

17.2 A database per test, cloned from a template 251
17.3 A transaction per test, and its limits 253
17.4 HTTP tests with `oneshot` 254
17.5 Concurrency tests 255
17.6 PostgreSQL in CI: a service, or testcontainers 256
17.7 Provoking every SQLSTATE the error mapping handles 257
In stockroom 259
Summary 260
Exercises 260

18 Capstone: stockroom end to end 262
18.1 The shape 262
18.2 A request's life 263
18.3 The feature map 264
18.4 Running it 265
18.5 What production would add 267
Summary 268
Exercises 268

Appendix — cheat sheets, phrasebook, glossary and ecosystem 270
A API cheat sheet 270
B Types 273
C SQLSTATE cheat sheet 274
D Phrasebook: from sqlx and diesel 276
E Glossary 277
F The crates built on tokio-postgres 279
G Further reading 282

CHAPTER 0

Introduction and setup

This book teaches `tokio-postgres`, the asynchronous PostgreSQL driver that most of Rust's database ecosystem stands on. It takes you from the first query to a production-shaped HTTP service built with axum, and at every step it shows what the driver sends over the wire, what the server does with it, and where the sharp edges are.

Every runnable example was compiled and run against PostgreSQL 18 when the book was written, and the service the book builds, **stockroom**, is in this repository with its tests. You learn the driver and a real codebase at the same time.

Contents

Chapter	You will learn
0 · Introduction and setup (this page)	Who the book is for, where tokio-postgres fits, the setup, and stockroom in one page
1 · Connecting	`Client` and `Connection`, spawning the connection, connection strings and `Config`, what failure looks like
2 · Running SQL	`execute`, `query` and friends, parameters instead of string formatting, the simple query protocol
3 · Rows and types	Reading rows, the Rust–PostgreSQL type table, NULL, JSON, arrays, borrowing from a row
4 · Prepared statements	What goes over the wire, `Statement`, type inference, `query_typed`, round trips
5 · Custom types	`ToSql` and `FromSql` by hand and by derive: newtypes, enums, domains, composites
6 · Errors	`Error`, `DbError` and `SqlState`, and an error type for an application
7 · Pipelining	Many queries on one connection at once, and when that beats a pool
8 · Transactions	Isolation levels, savepoints, retries, locks, and selling the last chair exactly once
9 · Streaming	Rows as a stream, backpressure, portals, keyset pagination
10 · COPY	Bulk data in and out, text and binary, and how much faster it is
11 · LISTEN/NOTIFY	Driving the connection yourself, notifications, notices, reconnecting
12 · Production connections	Timeouts, cancellation, dead connections, failover and TLS

Chapter	You will learn
13 · Pooling	Why one client is not enough, a pool from scratch, deadpool-postgres
14 · The data layer	Migrations, transactional DDL, a repository layer
15 · axum I	A JSON API: extractors, errors as responses, ETags, pagination
16 · axum II	Streaming bodies, COPY over HTTP, server-sent events, an outbox worker, shutdown
17 · Testing	A database per test, HTTP tests, concurrency tests
18 · Capstone	stockroom end to end, and a map from every feature to its code
A · Appendix	Cheat sheets, a sqlx and diesel phrasebook, a glossary, and the crates built on tokio-postgres

The whole book is also available as a single PDF, tokio-postgres-book.pdf, with a linked table of contents. To print one chapter, render it yourself from the repository's root: `python3 tools/book.py pdf 3` writes `target/book/03-rows-and-types.pdf` (it needs pandoc and Typst; the README's *PDFs* section has the details).

Chapters 1–6 are the driver's foundations. Chapters 7–12 are what makes it an *async* driver, and where most production problems live. Chapters 13–18 build the service. Read them in order the first time; afterwards the appendix is the fastest way back in.

Who this is for, and what it assumes

You know three things well enough to be dangerous:

- **Rust, at an average level.** Ownership and borrowing, structs and enums with `match`, traits and generics, `Result` and `?`, closures, iterators. You have probably not had to think hard about `Pin`, higher-ranked lifetimes or why a future is not `Send`. The book explains each of those where it first matters.
- **Tokio, at a basic level.** You have written `#[tokio::main]`, `.await`ed things, called `tokio::spawn`, and seen a channel. `select!`, cancellation and streams are introduced as they are needed.
- **SQL, at a basic level.** `SELECT`, `INSERT`, `UPDATE`, `DELETE`, joins, `CREATE TABLE`, and the idea of a transaction. Isolation levels, row locks, `COPY`, `LISTEN/NOTIFY`, domains and composite types are explained when they come up.

You do not need to know anything about how a database driver works. By the end you will know what yours does on every call.

Where tokio-postgres fits

Rust has several ways to talk to PostgreSQL, and they are layered more than they compete:

Library	What it is	Built on
tokio-postgres	An asynchronous driver: connections, queries, transactions, COPY, notifications, type conversions. No query builder, no ORM, no compile-time checking.	Tokio and the PostgreSQL wire protocol
`postgres`	The same driver with a blocking API, for programs without an async runtime.	tokio-postgres
deadpool-postgres, bb8-postgres	Connection pools.	tokio-postgres
refinery	Schema migrations.	tokio-postgres (among others)
clorinde (formerly cornucopia)	Generates typed Rust functions from `.sql` files.	tokio-postgres
diesel-async (PostgreSQL backend)	Diesel's query builder, asynchronously.	tokio-postgres
sqlx	An independent async driver with compile-time-checked queries.	its own protocol implementation
SeaORM	An ORM.	sqlx

Learning tokio-postgres pays off twice. It is a good driver to use directly: small, close to the protocol, and fast, with features higher layers hide, such as running many queries on one connection at once (Chapter 7). And whatever you use on top of it, it is what actually runs when something goes wrong in production, so knowing it is knowing what your stack does.

How each chapter is built

1. **The concept**, explained from scratch, with small runnable programs.
2. **Side by side** sections. Each one states a single task and solves it three ways: at the `psql` prompt, with sqlx, and with tokio-postgres. A table of what is the **same** and what is **different** follows. `psql` ties the Rust code to SQL you already know and shows what the server is asked to do; sqlx shows which choices are specific to tokio-postgres. The sqlx snippets are illustrations and are not compiled; the tokio-postgres ones are complete and were run. In Part III, where psql has nothing to compare with, the sections drop it.
3. **In stockroom**: where the concept lives in the service this book builds, with a trimmed excerpt and a link to the file.
4. **Exercises**, each with a hint, and solutions for the ones whose answers are not obvious.

Four kinds of boxes appear where they are earned:

> **Protocol lens:** what travels over the socket. Most of tokio-postgres's API makes sense the moment you know what it sends.

> **SQL lens:** what the server does: isolation, locks, how notifications are delivered.

> **Rust refresher:** a language feature you may be hazy on, at the point where it matters.

> **Pitfall:** a mistake that compiles, passes a quick test, and fails in production.

Code blocks come in two kinds:

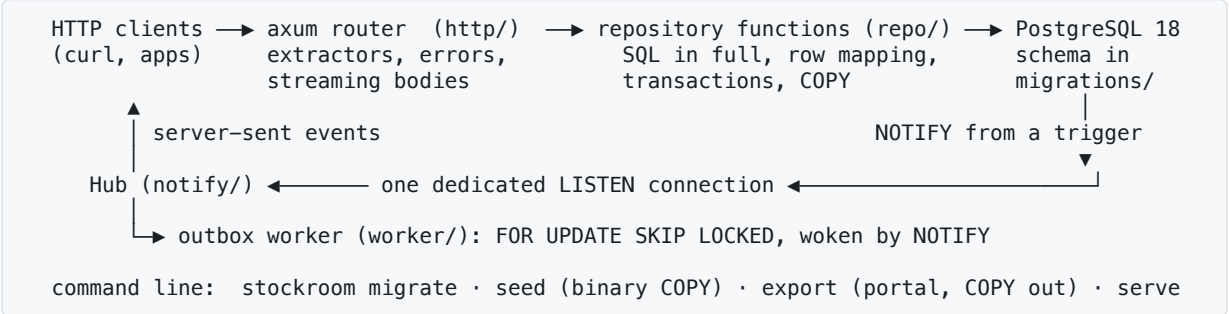
- Blocks marked as plain `rust` are **complete programs**. Each one compiles and runs as written in the playground described below, and each was run against a fresh database when the book was written. Their `assert!`s are part of the explanation: they state what the code produces.
- Blocks taken from stockroom or the example programs are **excerpts**. They compile in their own crate, not on their own, and their first line says where they come from.

(For maintainers: in the Markdown source, runnable examples are fenced with backticks and everything else with tildes, `~~~rust`, so that `tools/book.py` can extract and run every runnable block, and check every excerpt against the file it quotes. An HTML comment before a block, such as `<!-- check: needs=lagproxy -->`, tells the tool how to run it.)

---

stockroom in one page

stockroom is a click-and-collect service for a small retail chain: a catalog, stock per store, and orders that customers place online and pick up in a store. It is small enough to read in an afternoon and real enough to have the problems real services have: two customers wanting the last chair, staff screens that must update live, price lists arriving as CSV files, emails that must go out exactly when an order is ready.



Part of stockroom	What it is good for teaching	Chapters
domain/	Mapping a PostgreSQL enum, a composite type and two domains to Rust types, by hand and by derive	3, 5
error.rs	Turning SQLSTATE codes and constraint names into HTTP responses	6, 15
repo/	SQL written out in full, pipelined queries, transactions, COPY, savepoints	2–10, 14
db/	Connecting, pooling, migrations, retries, cancelling abandoned queries, TLS	1, 8, 12–14
notify/	A connection driven by hand, reconnecting, fan-out to many clients	11, 16
http/	axum on top of all of it	15, 16
tests/	A database per test, and proving that no order oversells	17

Five small programs in apps/ each do one thing a chapter talks about: pgq is a tiny psql, lagproxy puts a slow network between you and the server, pgload bulk-loads rows five ways and times them, pglisten tails notifications, and minipool is a connection pool in about a hundred lines.

Setting up

PostgreSQL

The book targets **PostgreSQL 18**. Most examples run on 14 or newer; the text says so whenever a feature needs a newer server (for example, "`MERGE` (PG 15+)"). Any of these will do:

```
# The EDB installer (macOS, Windows, Linux): https://www.postgresql.org/download/
# Homebrew (macOS):
brew install postgresql@18 && brew services start postgresql@18
# Docker:
docker run --name pg18 -e POSTGRES_PASSWORD=secret -p 5432:5432 -d postgres:18
```

You also want `psql`, PostgreSQL's command-line client. It comes with all three; with Docker, `docker exec -it pg18 psql -U postgres` works without installing anything.

Create a database for the book's examples, and tell the examples where it is with an environment variable:

```
psql "postgres://postgres:secret@localhost:5432/postgres" -c 'CREATE DATABASE tpbook'
export DATABASE_URL=postgres://postgres:secret@localhost:5432/tpbook
```

Use your own user, password and port. The examples only create temporary tables, which disappear when their connection closes, so they leave nothing behind and can run in any order.

The toolchain

The book uses Rust's 2024 edition, so you need Rust 1.85 or newer (it was written with 1.99):

```
rustup update
rustc --version
```

The playground

Make a scratch crate for running the book's examples. It must be called `playground`, because the examples use a helper from it:

```
cargo new --lib playground
cd playground
```

Replace its `Cargo.toml` with this. These are the crates the book's runnable examples use. The versions are minimums: any newer compatible release works, and `cargo update` fetches the latest one.

```
[package]
name = "playground"
version = "0.1.0"
edition = "2024"
publish = false

[dependencies]
anyhow = "1.0.104"
async-stream = "0.3.6"
axum = "0.8.9"
bytes = "1.12.1"
deadpool-postgres = "0.14.2"
futures-util = "0.3.34"
http-body-util = "0.1.5"
jiff = { version = "0.2.39", features = ["serde"] }
postgres-types = { version = "0.2.14", features = ["derive"] }
rust_decimal = { version = "1.43.0", features = ["db-tokio-postgres"] }
rustls = { version = "0.23.46", default-features = false, features = ["ring", "std",
"logging", "tls12"] }
serde = { version = "1.0.229", features = ["derive"] }
serde_json = "1.0.154"
thiserror = "2.0.21"
tokio = { version = "1.54.1", features = ["full"] }
tokio-postgres = { version = "0.7.18", features = ["with-jiff-0_2", "with-serde_json-1",
"with-uuid-1", "array-impls"] }
tokio-postgres-rustls = "0.14.0"
tokio-util = "0.7.20"
tower = { version = "0.5.3", features = ["util"] }
tower-http = { version = "0.7.1", features = ["trace", "timeout", "limit"] }
uuid = { version = "1.29.0", features = ["v7", "serde"] }
```

The `features` on `tokio-postgres` switch on conversions for other crates' types: `jiff`'s timestamps, `serde_json`'s values, `uuid`'s UUIDs, and Rust arrays. Chapter 3 explains them. `rustls` and `tokio-postgres-rustls` are for the TLS connections of Chapter 12, and `tokio-util` and `tower-http` for the services of Part III.

Then replace `src/lib.rs` with the helper every example uses to connect. Chapter 1 explains every line of it:

```
use tokio_postgres::{Client, Error, NoTls};

/// Connects to the database named by the `DATABASE_URL` environment variable and spawns the
/// connection task, returning the client.
///
/// # Panics
///
/// If `DATABASE_URL` is not set: examples are programs, not libraries, and a clear panic is
/// the most useful thing they can do about a missing setting.
pub async fn connect() -> Result<Client, Error> {
    let url = std::env::var("DATABASE_URL")
        .expect("set DATABASE_URL, e.g. postgres://postgres:secret@localhost:5432/tpbook");
    let (client, connection) = tokio_postgres::connect(&url, NoTls).await?;
```

```
    tokio::spawn(async move {
        if let Err(e) = connection.await {
            eprintln!("connection error: {e}");
        }
    });
    Ok(client)
}
```

Put each example in its own file under `examples/`, for instance `examples/ch00_taste.rs`, and run it with

```
cargo run --example ch00_taste
```

The commands you will use in this repository

```
cargo run -p pgq -- -c "SELECT version()"     # the tiny psql from apps/pgq
cargo run -p stockroom -- --help              # the service's command line
cargo test -p stockroom                       # its tests: they create and drop their own
databases
python3 tools/book.py check                   # run every example in this book
```

The repository's own `.env.example` lists the environment variables they read. Copy it to `.env` and fill in your credentials.

A ten-line taste

Connect, send one query with a parameter, and read two typed values out of the row:

```
use playground::connect;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = connect().await?;

    // `$1` is a parameter: its value travels separately from the SQL text.
    let row = client
        .query_one(
            "SELECT $1::text || ', ' || current_user AS greeting, 6 * 7 AS answer",
            &[&"hello"],
        )
        .await?;

    let greeting: &str = row.get("greeting");
    let answer: i32 = row.get("answer");
    println!("{greeting}; the answer is {answer}");
    assert!(greeting.starts_with("hello, "));
    assert_eq!(answer, 42);
    Ok(())
}
```

```
hello, postgres; the answer is 42
```

Four things in these lines are worth noticing now, and each gets a chapter later:

- `connect().await?` gives you a `Client`, but something else must keep the connection alive in the background: the helper spawns it. Forgetting that is the first mistake everyone makes (Chapter 1).
- The value `"hello"` is not pasted into the SQL. It is sent as a **parameter**, `$1`, in a separate message, which is why SQL injection cannot happen here (Chapter 2).

- `row.get("answer")` turns PostgreSQL's `integer` into Rust's `i32` because you asked for an `i32`. Ask for an `i64` and it panics: the conversion is checked, at run time (Chapter 3).
- `query_one` sent more than one message and waited for more than one answer. Knowing how many is the key to making a database-backed service fast (Chapter 4).

Exercises

1. **Set up the playground** as described above, save the ten-line taste as `examples/ch00_taste.rs`, and run it. *Hint:* if it panics with "set DATABASE_URL", the environment variable is not set in the shell you ran `cargo` from.
2. **Break it on purpose.** Change `let answer: i32` to `let answer: i64` and run it again. Read the panic message: what does it say about the types on each side? *Hint:* the SQL expression `6 * 7` has the type `integer`, which is 32 bits wide.
3. **Use the tiny psql.** From this repository's root, run `cargo run -p pgq -- -c "SELECT name, setting FROM pg_settings WHERE name LIKE 'max_conn%'"` and then the same with `--describe`. What does `--describe` tell you that the plain run does not? *Hint:* `--describe` prepares the statement instead of running it (Chapter 4 explains what a prepare returns).
4. **Run the service.** From the repository's root: `cargo run -p stockroom -- migrate`, then `cargo run -p stockroom -- seed`, then `cargo run -p stockroom -- serve`, and open stockroom.http in an editor with an HTTP client (VS Code's REST Client, or curl by hand). Place an order and watch it in another terminal with `curl -N 'http://127.0.0.1:8080/orders/events?store=store-01'`. *Hint:* `seed` creates the stores `store-01` to `store-03`.

Solutions

2. The panic reads only "error retrieving column answer: error deserializing column 1". The reason is one level down, in the error's `source()`, which a panic message does not print. With `row.try_get::<_, i64>("answer")` instead of `get`, you can print the error with `{:?}`:

```
Error { kind: FromSql(1), cause: Some(WrongType { postgres: Int4, rust: "i64" }) }
```

tokio-postgres knows the column's type from the row description the server sent, and every Rust type says which PostgreSQL types it accepts. `i64` takes only `int8`. There is no silent widening: you change the Rust type, or cast in SQL (`(6 * 7)::int8`). Chapter 6 shows how to get at the cause of any error.

3. `--describe` does not run the query. It asks the server to prepare it and reports the type of every result column (both `text` here) and, for columns that come from a table or view, which one: `oid 12104, column 1` is the first column of the view `pg_settings`, whose OID (object id) you can look up with `SELECT 12104::regclass`. The plain run uses the simple query protocol, in which every value arrives as text and types are not part of the result (Chapter 2).

CHAPTER 1

Connecting

Every other chapter starts with a connection, so this one looks at what a connection *is* in tokio-postgres, and why it comes in two halves. That split is the most important design decision in the crate. It is why one connection can be shared by many tasks, why queries can be pipelined (Chapter 7), and why notifications need special handling (Chapter 11). It is also the reason for the first bug everyone writes: a program that connects successfully and then hangs forever on its first query.

The rest of the chapter is practical: connection strings and the `Config` builder, what the server tells you when you connect, how a connection ends, and what each kind of failure looks like, so you recognize them when they happen.

1.1 Two halves: `Client` and `Connection`

`tokio_postgres::connect` returns a pair:

```
use tokio_postgres::NoTls;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let url = std::env::var("DATABASE_URL").expect("DATABASE_URL is set");

    // Two values: the client you call methods on, and the connection that does the IO.
    let (client, connection) = tokio_postgres::connect(&url, NoTls).await?;

    // The connection is a future. Until something polls it, nothing is sent or received:
    // hand it to the runtime as a task of its own.
    let task = tokio::spawn(connection);

    let version: String = client.query_one("SHOW server_version", &[]).await?.get(0);
    println!("connected to PostgreSQL {version}");
    assert!(!version.is_empty());

    // Dropping the client tells the connection there is nothing more to do: it says goodbye
    // to the server and its future completes, with `Ok(())`.
    drop(client);
    task.await.expect("the task did not panic")?;
    Ok(())
}
```

`NoTls` says "no encryption"; Chapter 12 replaces it with a real TLS connector. The rest is the shape of every program in this book.

The two halves divide the work like this:

```
  your tasks ──query──▶ Client ──requests──▶ Connection ──bytes──▶ PostgreSQL
             ◀──rows───        ◀─responses──    (a task)  ◀──bytes──
```

- The **`Connection`** owns the socket. It is a `Future` that, as long as it is polled, writes queued requests to the server, reads whatever the server sends, and routes each response to whoever is waiting for it. It finishes when the client is gone, or with an error when the connection breaks.
- The **`Client`** owns no socket at all. It holds the sending end of a channel to the connection. A method such as `query_one` turns your call into protocol messages, puts them on the channel, and waits for its own response to come back.

Two consequences matter right away. First, the `Client`'s methods take `&self`, not `&mut self`, so one client can be shared by many tasks at once (wrapped in an `Arc`); their requests queue up on the same connection. Second, nothing happens unless the connection is polled, which brings us to the first bug.

> **Rust refresher:** a `Future` is a value that describes work; it does nothing until it is polled, and `.await` is one way to poll it. `tokio::spawn(fut)` is another: it hands the future to the runtime, which polls it in the background, on any of its threads. That is why a spawned future must be `Send` (movable between threads) and `'static` (owning its data). `Connection` is both, by design.

1.2 Spawning the connection, and forgetting to

If you keep the connection but never poll it, your queries wait forever:

```
use std::time::Duration;
use tokio_postgres::NoTls;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let url = std::env::var("DATABASE_URL").expect("DATABASE_URL is set");
    let (client, connection) = tokio_postgres::connect(&url, NoTls).await?;

    // Nobody polls `connection`. The request is queued for it, and nothing happens.
    let waited = tokio::time::timeout(
        Duration::from_millis(200),
        client.query_one("SELECT 1", &[]),
    )
    .await;
    assert!(
        waited.is_err(),
        "the query timed out: it never left the process"
    );

    // Start the connection, and the next query goes through. (So does the abandoned
    // `SELECT 1`: it was still queued, and its answer is thrown away.)
    tokio::spawn(connection);
    let one: i32 = client.query_one("SELECT 1", &[]).await?.get(0);
    assert_eq!(one, 1);
    Ok(())
}
```

Note what `connect` itself did: it opened the socket, logged in and waited for the server to be ready, all *before* returning. `connect().await` succeeding means the server accepted you. The connection's background work only begins once you spawn it.

Spawning is not the only way to poll a connection; it is just the convenient one. A short program can poll the connection and its query together, in the same task, with `select!`:

```rust
use tokio_postgres::NoTls;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let url = std::env::var("DATABASE_URL").expect("DATABASE_URL is set");
    let (client, connection) = tokio_postgres::connect(&url, NoTls).await?;

    // Poll both: the query finishes first, because the connection only finishes when the
    // client is gone (or the server goes away).
    let answer: i32 = tokio::select! {
        row = client.query_one("SELECT 40 + 2", &[]) => row?.get(0),
        ended = connection => panic!("the connection ended first: {ended:?}"),
    };
    assert_eq!(answer, 42);
    Ok(())
}
```

That works, but it is fragile: the moment two queries are in flight, or one runs after the `select!`, the connection is no longer being polled. In real programs, spawn it, and do something with its result:

```rust
let (client, connection) = tokio_postgres::connect(&url, NoTls).await?;
tokio::spawn(async move {
    if let Err(e) = connection.await {
        eprintln!("connection error: {e}");
    }
});
```

The error matters. When the server restarts or the network drops, this is where you hear about it first; every query in flight fails with "connection closed" a moment later.

> **Pitfall:** `tokio::spawn(connection);` alone compiles, runs, and throws the connection's error away (the `JoinHandle` is dropped, and with it the `Result`). In a service, log it.

> **Protocol lens:** what `connect` does before it returns. The client sends a *StartupMessage* with the user, the database and any settings. The server asks for a password (for a `scram-sha-256` server, a SCRAM exchange of two more round trips), then sends a series of *ParameterStatus* messages (the server's version, encoding, time zone and so on), one *BackendKeyData* message (a process id and a secret, kept for cancelling queries, Chapter 12), and finally *ReadyForQuery*. That is three to four round trips before your first query; on a slow network you feel it, which is one reason connections are pooled (Chapter 13).

1.3 Connection strings

`tokio_postgres::connect` takes a string in either of the two forms libpq, PostgreSQL's C library, understands:

- **Key/value pairs**, separated by spaces: `host=localhost port=5432 user=shop dbname=stock`. A value with spaces, or an empty one, goes in single quotes: `password='two words'`.

- **A URL**: `postgres://user:password@host:port/database?key=value&key=value`. Every key/value setting can go in the query string. Characters with a meaning in URLs must be percent-encoded: a password `s3cr@t` is written `s3cr%40t`.

Both parse into a `Config`, and you can parse one yourself to look inside:

```
use std::time::Duration;
use tokio_postgres::Config;
use tokio_postgres::config::Host;

fn main() -> Result<(), tokio_postgres::Error> {
    let from_url: Config =
        "postgres://shop:s3cr%40t@db.example.com:5433/stock?application_name=reports&
connect_timeout=5"
            .parse()?;
    let from_pairs: Config = "host=db.example.com port=5433 user=shop password='s3cr@t' \
                              dbname=stock application_name=reports connect_timeout=5"
        .parse()?;

    for config in [&from_url, &from_pairs] {
        assert_eq!(config.get_user(), Some("shop"));
        assert_eq!(config.get_password(), Some(&b"s3cr@t"[..])); // bytes, decoded
        assert_eq!(config.get_dbname(), Some("stock"));
        assert_eq!(config.get_hosts(), [Host::Tcp("db.example.com".into())]);
        assert_eq!(config.get_ports(), [5433]);
        assert_eq!(config.get_application_name(), Some("reports"));
        assert_eq!(config.get_connect_timeout(), Some(&Duration::from_secs(5)));
    }

    // Printing a Config never shows the password.
    let printed = format!("{from_url:?}");
    assert!(printed.contains("password: Some(_)") && !printed.contains("s3cr"));
    Ok(())
}
```

The settings you will meet most often:

Key	Meaning	Default
`host`	Host name or IP address; a value starting with `/` is a Unix-socket directory	required
`port`	TCP port (or the socket file's suffix)	5432
`user`, `password`	Who you log in as	none (`user` is required)
`dbname`	The database	the user's name
`application_name`	How the session shows up in `pg_stat_activity` and the server log	none
`connect_timeout`	Seconds to wait for each connection attempt	wait as long as the OS does
`options`	Settings for the session, as server command-line options: `-c statement_timeout=5000`	none
`sslmode`	`disable`, `prefer` or `require` (Chapter 12)	`prefer`

Several hosts can be listed, separated by commas (`host=primary,replica`), and they are tried in order. Chapter 12 uses that for failover.

> **Pitfall:** tokio-postgres reads **no environment variables** and no files. libpq-based tools such as psql also look at `PGHOST`, `PGPASSWORD`, `PGSERVICE`, `~/.pgpass` and more; a connection string that works in psql because `PGPASSWORD` is set will fail with "password missing" from Rust. Everything must be in the string or the `Config`. The convention for passing that string to a program is the `DATABASE_URL` environment variable, which your program reads itself, as the playground helper does.

1.4 The `Config` builder

When a program needs settings of its own on top of what the operator configured, parse the string and add to it. Every setter takes `&mut self` and returns `&mut Config`, so calls chain:

```
use std::time::Duration;
use tokio_postgres::{Config, NoTls};

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    // Start from what the operator gave us...
    let mut config: Config = std::env::var("DATABASE_URL")
        .expect("DATABASE_URL is set")
        .parse()?;
    // ...and add what this program needs.
    config
        .application_name("chapter-1")
        .connect_timeout(Duration::from_secs(3))
        // Session settings, applied by the server before the first query.
        .options("-c statement_timeout=1500 -c TimeZone=UTC");

    let (client, connection) = config.connect(NoTls).await?;
    tokio::spawn(connection);

    let row = client
        .query_one(
            "SELECT current_setting('application_name'),
                    current_setting('statement_timeout'),
                    current_setting('TimeZone')",
            &[],
        )
        .await?;
    assert_eq!(row.get::<_, &str>(0), "chapter-1");
    assert_eq!(row.get::<_, &str>(1), "1500ms");
    assert_eq!(row.get::<_, &str>(2), "UTC");
    Ok(())
}
```

`options` deserves a moment. It is passed to the server at startup, so its settings are in force from the first statement of the session, without a round trip of their own, and a pool (Chapter 13) applies them to every connection it opens. stockroom uses it to give every session a `statement_timeout`, a `lock_timeout` and an `idle_in_transaction_session_timeout` (see *In stockroom* below).

Some settings are lists, and their setters *add* to them. That catches people who want to "point the configuration somewhere else":

```
use tokio_postgres::Config;
use tokio_postgres::config::Host;

fn main() -> Result<(), tokio_postgres::Error> {
    let mut config: Config = "postgres://app@db.internal:5432/shop".parse()?;
```

```
    // Meant: "connect to the local copy instead". Did: "try db.internal, then localhost".
    config.host("localhost").port(5433);
    assert_eq!(
        config.get_hosts(),
        [
            Host::Tcp("db.internal".into()),
            Host::Tcp("localhost".into())
        ]
    );
    assert_eq!(config.get_ports(), [5432, 5433]);

    // A Unix-domain socket is a host too: the directory that holds the socket file.
    let local: Config = "host=/tmp port=5432 user=app".parse()?;
    assert!(matches!(local.get_hosts(), [Host::Unix(dir)] if dir.as_os_str() == "/tmp"));
    Ok(())
}
```

Pitfall: `Config::host` and `Config::port` append; there is no setter that replaces the list. The first host still wins, so the mistake is silent: the program connects, just not where you think. To redirect a connection, build a fresh `Config` from the parts you need, or open the socket yourself and use `Config::connect_raw` (Chapter 12). The example programs in this repository do both, and both started with exactly this bug.

1.5 What the server told you, and how a connection ends

While logging you in, the server sent a list of its settings, and the `Connection` kept them. `Connection::parameter` reads them, and it is the only way to: once the connection has been moved into a task, you can no longer reach it, so read what you need first.

The client has two more methods about the connection itself. `is_closed` answers immediately from local state: is the connection task still there? `check_connection` (added in 0.7.15) makes one round trip to the server and fails if it does not come back.

```
use tokio_postgres::NoTls;
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let url = std::env::var("DATABASE_URL").expect("DATABASE_URL is set");
    let (client, connection) = tokio_postgres::connect(&url, NoTls).await?;

    // Reported by the server at startup: read them before the connection moves away.
    let version = connection
        .parameter("server_version")
        .unwrap_or("?")
        .to_owned();
    let encoding = connection.parameter("client_encoding").map(str::to_owned);
    let task = tokio::spawn(connection);
    println!("PostgreSQL {version}");
    assert_eq!(encoding.as_deref(), Some("UTF8"));

    client.check_connection().await?; // one round trip, no statement
    assert!(!client.is_closed());

    // End our own session from the server side, as an administrator, a restart or a crash
    // would. The query that triggers it fails with SQLSTATE 57P01, "admin shutdown".
    let error = client
```

```
        .execute("SELECT pg_terminate_backend(pg_backend_pid())", &[])
        .await
        .unwrap_err();
    assert_eq!(error.code(), Some(&SqlState::ADMIN_SHUTDOWN));

    // The connection task ends with an error, and from then on the client knows it is
closed.
    assert!(task.await.expect("the task did not panic").is_err());
    assert!(client.is_closed());
    let error = client.check_connection().await.unwrap_err();
    assert!(error.is_closed());
    Ok(())
}
```

The server reports more than a dozen such parameters. The ones worth knowing: `server_version` (for feature checks), `server_encoding` and `client_encoding` (always `UTF8` for tokio-postgres, which insists on it), `TimeZone`, `DateStyle`, `integer_datetimes`, `application_name`, `is_superuser`, `in_hot_standby` (whether you are on a read-only replica, PG 14+) and, from PG 18, `search_path`. The server also sends an update whenever one of them changes during the session, and `parameter` returns the latest value it has seen.

A connection ends in one of three ways:

How	The connection future returns	The client
You drop every `Client` (and every `Transaction`, which borrows one)	`Ok(())`, after telling the server goodbye	gone
The server ends the session (restart, `pg_terminate_backend`, `idle_session_timeout`) or the network breaks	`Err(…)`	`is_closed()` turns true; every call fails with "connection closed"
You drop the `Connection` future (abort its task)	—	`is_closed()` turns true, and the server sees the socket close

There is no reconnecting: a closed client stays closed. Making a new connection is the caller's job, which is a large part of what a pool does (Chapter 13). Chapter 12 covers how to notice a connection that died silently, without the server telling you.

1.6 What failure looks like

Connecting can fail in several distinct ways, and the error says which, if you know where to look:

```
use std::time::{Duration, Instant};
use tokio_postgres::error::SqlState;
use tokio_postgres::{Config, NoTls};

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let good: Config = std::env::var("DATABASE_URL")
        .expect("DATABASE_URL is set")
        .parse()?;

    // The server is there, but says no: there is no such database.
    let mut config = good.clone();
    config.dbname("no_such_database");
    let error = config.connect(NoTls).await.err().expect("refused");
```

```
    assert_eq!(error.code(), Some(&SqlState::INVALID_CATALOG_NAME)); // 3D000
    // The error's own text is terse; the server's message is one level down.
    assert_eq!(error.to_string(), "db error");
    let db = error.as_db_error().expect("an error from the server");
    assert_eq!(db.message(), "database \"no_such_database\" does not exist");

    // Nothing listens on the port: an IO error, and no SQLSTATE.
    let mut config = Config::new();
    config.host("localhost").port(1).user("nobody");
    let error = config.connect(NoTls).await.err().expect("refused");
    assert!(error.code().is_none() && error.as_db_error().is_none());
    let cause = std::error::Error::source(&error).expect("an underlying IO error");
    println!("{error}: {cause}"); // "error connecting to server: Connection refused …"

    // A host that never answers. Without connect_timeout, this waits as long as the
    // operating system lets a TCP connection attempt wait: often over a minute.
    let mut config = Config::new();
    config
        .host("10.255.255.1")
        .user("nobody")
        .connect_timeout(Duration::from_millis(300));
    let started = Instant::now();
    assert!(config.connect(NoTls).await.is_err());
    assert!(started.elapsed() < Duration::from_secs(2));
    Ok(())
}
```

The pattern to remember: **an error from the server has a SQLSTATE code and a `DbError` with the server's message; anything else (refused, unreachable, timed out, TLS) has neither, and its cause is the error's `source()`.** A few codes you will meet when connecting:

SQLSTATE	`SqlState` constant	Meaning
`28P01`	`INVALID_PASSWORD`	Wrong password
`28000`	`INVALID_AUTHORIZATION_SPECIFICATION`	No `pg_hba.conf` entry lets you in (wrong host, user or SSL mode)
`3D000`	`INVALID_CATALOG_NAME`	No such database
`53300`	`TOO_MANY_CONNECTIONS`	`max_connections` reached
`57P03`	`CANNOT_CONNECT_NOW`	The server is starting up or shutting down

> **Pitfall:** `error.to_string()` for any server error is just `db error`. Logging errors with `{}` hides the one thing you need. Log with `{:?}`, or reach for `as_db_error()`; Chapter 6 builds an error type that does it for you.

Side by side – connecting

The task: connect to the book's database as an application called `demo`, and ask for the server's version.

```
-- psql: the connection string is the argument; \conninfo describes the connection
-- (as a table since psql 18).
--   psql "postgres://postgres:secret@localhost:5432/tpbook?application_name=demo"
tpbook=# \conninfo
      Connection Information
      Parameter       |   Value
----------------------+-----------
 Database             | tpbook
```

```
 Client User           | postgres
 Host                  | localhost
 Host Address          | ::1
 Server Port           | 5432
 Options               |
 Protocol Version      | 3.0
 Password Used         | true
 GSSAPI Authenticated | false
 Backend PID           | 95981
 SSL Connection        | false
 Superuser             | on
 Hot Standby           | off
(13 rows)

tpbook=# SHOW server_version;
 server_version
----------------
 18.6
(1 row)
```

```
// sqlx (illustration)
use sqlx::Connection;
let mut conn = sqlx::PgConnection::connect(&url).await?; // no task to spawn
let version: String = sqlx::query_scalar("SHOW server_version").fetch_one(&mut conn).await?;
```

```
use tokio_postgres::{Config, NoTls};

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut config: Config = std::env::var("DATABASE_URL")
        .expect("DATABASE_URL is set")
        .parse()?;
    config.application_name("demo");
    let (client, connection) = config.connect(NoTls).await?;
    tokio::spawn(connection);

    let version: String = client.query_one("SHOW server_version", &[]).await?.get(0);
    assert!(version.chars().next().is_some_and(|c| c.is_ascii_digit()));
    Ok(())
}
```

	psql	**sqlx**	**tokio-postgres**
Same	A connection string in the URL or key/value form; one session on the server, with the same startup and login	The same	The same
Different	Built on libpq: reads `PG*` environment variables, `~/.pgpass` and service files	Reads the `PG*` variables too. A `PgConnection` does its IO inside each call, so it needs `&mut` and runs one query at a time; a pool provides concurrency	Reads nothing implicitly. The `Connection` is a separate task, so `Client` methods take `&self`: a client can be shared, and its queries pipelined (Chapter 7). You must spawn the connection

1.7 The playground's `connect`

From here on, the book's examples start with `playground::connect()`, which Chapter 0 asked you to put in your playground's `src/lib.rs`. Now every line of it should make sense:

```rust
use tokio_postgres::{Client, Error, NoTls};

/// Connects to the database named by the `DATABASE_URL` environment variable and spawns the
/// connection task, returning the client.
///
/// # Panics
///
/// If `DATABASE_URL` is not set: examples are programs, not libraries, and a clear panic is
/// the most useful thing they can do about a missing setting.
pub async fn connect() -> Result<Client, Error> {
    let url = std::env::var("DATABASE_URL")
        .expect("set DATABASE_URL, e.g. postgres://postgres:secret@localhost:5432/tpbook");
    let (client, connection) = tokio_postgres::connect(&url, NoTls).await?;
    tokio::spawn(async move {
        if let Err(e) = connection.await {
            eprintln!("connection error: {e}");
        }
    });
    Ok(client)
}
```

It returns `Result<Client, Error>`, so examples write `connect().await?` in a `main` that returns `Result<(), tokio_postgres::Error>`, and any failure, including at startup, ends the program with the error printed.

In stockroom

stockroom never calls `tokio_postgres::connect` with a bare string. Its settings start from the `--database-url` flag (or `STOCKROOM_DATABASE_URL`) and then get the defaults every session of the service should have, unless the operator's string already says otherwise:

```rust
// stockroom/src/config.rs (excerpt)
pub fn pg_config(&self) -> Result<Config, tokio_postgres::Error> {
    let mut config: Config = self.database_url.parse()?;
    if config.get_application_name().is_none() {
        config.application_name("stockroom");
    }
    if config.get_connect_timeout().is_none() {
        config.connect_timeout(Duration::from_secs(5));
    }
    // Notice a dead peer in about a minute rather than the default two hours.
    config.keepalives_idle(Duration::from_secs(30));
    config.keepalives_interval(Duration::from_secs(10));
    config.keepalives_retries(3);

    // Guard rails on the server: runaway statements, lock waits, and transactions left
    // open by a crashed or stuck client are all ended by PostgreSQL itself.
    let mut options = config.get_options().unwrap_or_default().to_owned();
    // … appends "-c statement_timeout=…", "-c lock_timeout=2000" and
    //   "-c idle_in_transaction_session_timeout=60000" to whatever options were given …
    config.options(options);
    Ok(config)
}
```

The application name is what makes stockroom's sessions recognizable in `pg_stat_activity` (the `/admin/sessions` endpoint shows them). The timeouts and keepalives are Chapter 12's subject. Note the care

taken with `options`: it is a single string, so new settings are appended to the operator's rather than replacing them.

Most of stockroom's connections come from a pool, which spawns each connection and discards its error (Chapter 13). The commands that need one connection of their own (`migrate`, `seed`, `export`) get it from db/connect.rs, which spawns the connection the careful way. Rather than just awaiting it, it *drives* it, so that the server's notices end up in the log:

```
// stockroom/src/db/connect.rs (excerpt)
pub async fn connect(config: &Config, tls: &MakeTls) -> Result<Client,
tokio_postgres::Error> {
    let (client, mut connection) = config.connect(tls.clone()).await?;
    tokio::spawn(async move {
        let mut messages = stream::poll_fn(move |cx| connection.poll_message(cx));
        while let Some(message) = messages.next().await {
            match message {
                // … log notices and notifications …
                Err(error) => {
                    tracing::error!(%error, "database connection failed");
                    break;
                }
            }
        }
    });
    Ok(client)
}
```

`poll_message` is Chapter 11's subject. For now, notice the outline: it is the same spawn-and-log as in §1.2, with a loop where the single `.await` was.

Summary

- `connect` returns a `Client`, which you call, and a `Connection`, a future that does all the IO. Spawn the connection, and log its error: that is where a broken connection shows up first.
- If the connection is not polled, queries wait forever. `connect().await` succeeding means only that the server accepted you.
- `Client` methods take `&self`: one client can be shared by many tasks.
- Connection strings come as key/value pairs or URLs; both parse into a `Config`. tokio-postgres reads no environment variables or password files.
- `Config` setters chain. `options` carries session settings to the server at startup. `host` and `port` *append* to lists.
- `Connection::parameter` reads what the server reported at startup; read it before spawning. `is_closed` is local and free; `check_connection` costs a round trip.
- Errors from the server have a SQLSTATE and a `DbError`; IO errors have neither and keep their cause in `source()`. `Display` of a server error is only "db error".

Exercises

1. **A connection with a name.** Connect with `application_name` set to your own name, then, from the same client, find your session in `pg_stat_activity` and print its `pid`, `application_name` and `backend_start`. *Hint:* `pg_backend_pid()` is your session's pid.
2. **Two clients, one connection?** `Client` is not `Clone`. How would you give ten tasks the same client? Write it, with each task running `SELECT pg_backend_pid()`, and check that all ten see the same pid. *Hint:* `Arc`.
3. **Settings at startup versus `SET`.** Set `statement_timeout` once through `options` and once with `client.batch_execute("SET statement_timeout = 1000")` after connecting. Both work. Name one situation where only the first one is safe. *Hint:* think of what happens to `SET` on a connection that a pool hands to someone else later, and of the connection's very first statement.
4. **What the server reports.** Print every startup parameter listed in §1.5. Then change your session's time zone with `SET TimeZone = 'Asia/Tokyo'` and check whether `connection.parameter("TimeZone")` notices. *Hint:* you need to keep polling the connection yourself to look at it after the `SET`; `select!` from §1.2 can do that in a small program.

Solutions

2.

```rust
use std::sync::Arc;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = Arc::new(playground::connect().await?);
    let mut tasks = Vec::new();
    for _ in 0..10 {
        let client = Arc::clone(&client);
        tasks.push(tokio::spawn(async move {
            let row = client.query_one("SELECT pg_backend_pid()", &[]).await?;
            Ok::<i32, tokio_postgres::Error>(row.get(0))
        }));
    }
    let mut pids = Vec::new();
    for task in tasks {
        pids.push(task.await.expect("no panic")?);
    }
    pids.dedup();
    assert_eq!(pids.len(), 1, "one connection, one server process");
    Ok(())
}
```

All ten requests went through one connection, served by one server process. Chapter 7 shows that they did not wait for each other.

3. A `SET` sticks to the session. On a pooled connection it outlives your use of it and surprises the next borrower, and it is easy to forget to undo it on an error path. Settings in `options` are part of how every connection is made, before any statement runs, so they cannot be skipped, left over or applied late. The other difference is cost: `SET` is one more round trip per connection.

4. It does: the server sends a fresh *ParameterStatus* message when a reported setting changes, and the connection updates its copy. To look at it afterwards you must still own the connection, so poll it by reference, in the same task, while the `SET` runs:

```rust
use tokio_postgres::NoTls;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let url = std::env::var("DATABASE_URL").expect("DATABASE_URL is set");
    let (client, mut connection) = tokio_postgres::connect(&url, NoTls).await?;
    let before = connection.parameter("TimeZone").map(str::to_owned);

    // `&mut connection`: polled here, but still ours afterwards.
    tokio::select! {
        result = client.batch_execute("SET TimeZone = 'Asia/Tokyo'") => result?,
        ended = &mut connection => panic!("the connection ended: {ended:?}"),
    }
    assert_eq!(connection.parameter("TimeZone"), Some("Asia/Tokyo"));
    assert_ne!(before.as_deref(), Some("Asia/Tokyo"));
    Ok(())
}
```

The server sends the new value just before it reports that the `SET` is done, so by the time `batch_execute` returns, the connection has already read it.

CHAPTER 2

Running SQL: execute, query and parameters

A `Client` has about twenty methods that run SQL. They look like a long list to learn, but they answer only three questions: do you want rows back, how many, and how should your values travel? This chapter covers the everyday ones: `batch_execute` for scripts, `execute` for changes, `query` and its *one*, *opt* and *scalar* variants for reading, and `simple_query` for when you need to see everything as text.

The most important idea in the chapter is not a method but a rule. **Values never go into the SQL text.** They travel separately, as parameters. The rule makes SQL injection impossible, and tokio-postgres's API is built around it.

2.1 `batch_execute`: scripts

`batch_execute` takes a string with any number of statements, runs them all, and returns nothing but success or failure. It is for schema scripts, migrations and setting things up, and the examples in this chapter use it to create the tables they need:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;

    // Several statements in one string: a table, and some data to start with.
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 sku         text   PRIMARY KEY,
                 name        text   NOT NULL,
                 price_cents bigint NOT NULL CHECK (price_cents > 0)
             );
             INSERT INTO products VALUES ('CHAIR-01', 'Garden chair', 4999);
             INSERT INTO products VALUES ('LAMP-01', 'Desk lamp', 2999);",
        )
        .await?;

    // The statements of one batch run in one implicit transaction: when one fails, the
    // ones before it are undone too.
    let failed = client
        .batch_execute(
            "INSERT INTO products VALUES ('VASE-01', 'Vase', 1999);
             INSERT INTO products VALUES ('RUG-01', 'Rug', 0);", // breaks the CHECK
        )
        .await;
    assert!(failed.is_err());
    let count: i64 = client
        .query_one("SELECT count(*) FROM products", &[])
```

```
        .await?
        .get(0);
    assert_eq!(count, 2, "the vase was rolled back with the rug");
    Ok(())
}
```

`CREATE TEMP TABLE` makes a table that exists only for this session and disappears when the connection closes. That is why the book's examples can run in any order without cleaning up after themselves.

> **SQL lens:** when one message carries several statements, PostgreSQL runs them as a single transaction unless they contain transaction commands of their own (`BEGIN`, `COMMIT`). That is the behaviour above, and it is often what you want from a script: all of it or none of it. One exception: statements that cannot run inside a transaction, such as `CREATE INDEX CONCURRENTLY` and `VACUUM`, cannot run in a multi-statement batch either.

`batch_execute` takes no parameters. That is the price of running several statements at once, and it makes `batch_execute` the wrong tool for anything that includes a value from outside your program. §2.3 explains why, and §2.7 why it is built that way.

2.2 `execute`: changes, counted

`execute` runs one statement and returns how many rows it affected, as a `u64`:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let created = client
        .execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, name text, price_cents
bigint)",
            &[],
        )
        .await?;
    assert_eq!(created, 0, "DDL affects no rows");

    let inserted = client
        .execute(
            "INSERT INTO products VALUES
                 ('CHAIR-01', 'Garden chair', 4999), ('LAMP-01', 'Desk lamp', 2999),
                 ('RUG-01', 'Rug', 8999)",
            &[],
        )
        .await?;
    assert_eq!(inserted, 3);

    // 10% off everything over 30.00: two rows.
    let discounted = client
        .execute(
            "UPDATE products SET price_cents = price_cents * 9 / 10 WHERE price_cents > $1",
            &[&3000_i64],
        )
        .await?;
    assert_eq!(discounted, 2);

    // Deleting something that is not there is not an error: zero rows.
    let deleted = client
        .execute("DELETE FROM products WHERE sku = $1", &[&"VASE-01"])
```

```
        .await?;
    assert_eq!(deleted, 0);
    Ok(())
}
```

The count is what the server reports when the statement completes: rows inserted, updated or deleted, rows returned by a `SELECT`, zero for everything else. The count is often the whole answer. "Was the order cancelled?" is `execute(…) == 1`, and Chapter 8 uses exactly that for optimistic locking.

`&[&3000_i64]` is the first parameter, `$1`. It is time to look at those properly.

2.3 Parameters

A statement refers to its parameters by position, `$1`, `$2` and so on, and the values follow in a slice, in the same order:

```
client.execute(
    "UPDATE products SET price_cents = $2 WHERE sku = $1",
    &[&sku, &new_price],
).await?;
```

The SQL text and the values go to the server **in different protocol messages**. The server parses the text first, with `$1` and `$2` as placeholders, and only then receives the values, as data. A value can contain quotes, semicolons or `DROP TABLE` and remain a value. Here is the difference it makes, with a value a user typed into a form:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE reviews (product text, body text, author text);
             INSERT INTO reviews VALUES ('CHAIR-01', 'Sturdy.', 'Ada'),
                                        ('CHAIR-01', 'Comfy.', 'Bob');",
        )
        .await?;

    // What someone typed into the "your name" field.
    let author = "Eve'); DELETE FROM reviews; --";

    // As a parameter, the text is a value, whatever it contains.
    client
        .execute(
            "INSERT INTO reviews (product, body, author) VALUES ($1, $2, $3)",
            &[&"CHAIR-01", &"Nice.", &author],
        )
        .await?;
    let count: i64 = client
        .query_one("SELECT count(*) FROM reviews", &[])
        .await?
        .get(0);
    assert_eq!(count, 3);

    // Pasted into the SQL text, the text *is* SQL. Never do this.
    let sql = format!(
        "INSERT INTO reviews (product, body, author) VALUES ('CHAIR-01', 'Nice.',
'{author}')"
    );
    client.batch_execute(&sql).await?;
    let count: i64 = client
        .query_one("SELECT count(*) FROM reviews", &[])
```

```
        .await?
        .get(0);
    assert_eq!(count, 0, "every review is gone");
    Ok(())
}
```

The formatted string became three statements: an `INSERT`, a `DELETE FROM reviews`, and a comment that swallowed the rest. Escaping quotes yourself is not the fix; there are more ways to break out of a string literal than quotes. Parameters are the fix.

> **Rust refresher:** the parameter slice has the type `&[&(dyn ToSql + Sync)]`: a slice of references to *trait objects*. Each element can be a different type (`&String`, `&i64`, `&Option<bool>`), as long as it implements `ToSql`, the trait that encodes a Rust value in PostgreSQL's format (Chapters 3 and 5). They are references because the values stay where they are; the slice only points at them. `Sync` is there because the query's future holds these references across `.await`, and for a future to be `Send` (spawnable), everything it borrows must be `Sync`. You rarely write the type out; `&[&a, &b]` coerces to it.

You do write it out when the parameters are not known in advance, for instance a search with optional filters. The SQL text grows one placeholder per filter, and the values go into a `Vec`:

```
use tokio_postgres::Client;
use tokio_postgres::types::ToSql;

/// SKUs of the products matching the filters that are set.
async fn search(
    client: &Client,
    name: Option<&str>,
    max_price: Option<i64>,
) -> Result<Vec<String>, tokio_postgres::Error> {
    let mut sql = String::from("SELECT sku FROM products WHERE true");
    let mut params: Vec<&(dyn ToSql + Sync)> = Vec::new();
    let pattern; // declared out here: `params` borrows it after the `if` ends
    if let Some(name) = name {
        pattern = format!("%{name}%");
        params.push(&pattern);
        sql += &format!(" AND name ILIKE ${}", params.len());
    }
    if let Some(max) = &max_price {
        params.push(max);
        sql += &format!(" AND price_cents <= ${}", params.len());
    }
    sql += " ORDER BY sku";
    let rows = client.query(&sql, &params).await?;
    Ok(rows.iter().map(|row| row.get(0)).collect())
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, name text, price_cents
bigint);
             INSERT INTO products VALUES ('CHAIR-01', 'Garden chair', 4999),
                 ('CHAIR-02', 'Office chair', 12999), ('LAMP-01', 'Desk lamp', 2999);",
        )
        .await?;
```

```
    assert_eq!(search(&client, None, None).await?.len(), 3);
    assert_eq!(
        search(&client, Some("chair"), None).await?,
        ["CHAIR-01", "CHAIR-02"]
    );
    assert_eq!(
        search(&client, Some("chair"), Some(5000)).await?,
        ["CHAIR-01"]
    );
    assert_eq!(search(&client, None, Some(3000)).await?, ["LAMP-01"]);
    Ok(())
}
```

`format!` appears there too, but notice what it inserts: placeholders (`$1`, `$2`), never values. Building SQL *structure* in Rust is fine; putting *data* into it is not.

The number of values must match the number of placeholders. tokio-postgres checks before sending anything, and fails with "expected 2 parameters but got 1".

Identifiers are not values

Parameters stand for values, not for parts of the statement's structure. A table name, a column name or a sort direction cannot be a parameter: `SELECT * FROM $1` is a syntax error (SQLSTATE 42601), because the server must know which table it is planning for before any value arrives. When structure must vary, choose it from a fixed list in Rust:

```
/// The columns a client may sort products by.
fn sort_column(requested: &str) -> Option<&'static str> {
    match requested {
        "sku" => Some("sku"),
        "price" => Some("price_cents"),
        "name" => Some("name"),
        _ => None,
    }
}
```

Only when the name genuinely comes from outside, as with the channel names a user gives pglisten, quote it as an identifier: in double quotes, with any double quote doubled. That is `quote_ident`, ten characters of code and the one place in this repository where outside text enters SQL.

> **Pitfall:** `LIMIT`, `OFFSET`, and values in `WHERE`, `SET` or `VALUES` *can* be parameters; table and column names, `ASC`/`DESC`, and SQL keywords cannot. If you find yourself writing `format!` with a variable that is not a placeholder number, stop and ask which kind it is.

2.4 `query`, `query_one` and `query_opt`: reading

`query` runs a statement and collects every row into a `Vec<Row>`. Two variants say how many rows you expect, and turn a surprise into an error:

Method	Returns	Fails when
`query`	`Vec<Row>`	never, because of the row count

Method	Returns	Fails when
`query_one`	`Row`	there is not exactly one row
`query_opt`	`Option<Row>`	there is more than one row

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, name text, price_cents
bigint);
             INSERT INTO products VALUES ('CHAIR-01', 'Garden chair', 4999),
                                         ('LAMP-01', 'Desk lamp', 2999);",
        )
        .await?;

    let all = client
        .query("SELECT sku, price_cents FROM products ORDER BY sku", &[])
        .await?;
    assert_eq!(all.len(), 2);
    assert_eq!(all[1].get::<_, &str>("sku"), "LAMP-01");

    let chair = client
        .query_one("SELECT name FROM products WHERE sku = $1", &[&"CHAIR-01"])
        .await?;
    assert_eq!(chair.get::<_, &str>("name"), "Garden chair");

    let vase = client
        .query_opt("SELECT name FROM products WHERE sku = $1", &[&"VASE-01"])
        .await?;
    assert!(vase.is_none());

    // Two rows where one was promised: an error, not a silently ignored second row.
    let error = client
        .query_one("SELECT sku FROM products", &[])
        .await
        .unwrap_err();
    assert_eq!(
        error.to_string(),
        "query returned an unexpected number of rows"
    );
    assert!(
        error.as_db_error().is_none(),
        "the server did nothing wrong"
    );
    Ok(())
}
```

Use `query_one` when a missing row is a bug (a count, a row you just inserted) and `query_opt` when it is an answer ("is there a product with this SKU?"). Mapping "not found" to a 404 starts with choosing `query_opt`. Note that the row-count error is the client's own: there is no SQLSTATE to match on, and Chapter 6 shows how to tell it apart from other errors.

> **Pitfall:** `query_one` does not add `LIMIT 1` to your statement. It reads every row the server sends and then complains if there was more than one. A query that can match many rows when you want the first needs `ORDER BY … LIMIT 1` in the SQL.

2.5 Scalars: `query_scalar`, `query_one_scalar`, `query_opt_scalar`

Many queries return a single column, and taking it out of each row is boilerplate. Since 0.7.16, `Client` has shortcuts that return the first column directly:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint);
             INSERT INTO products VALUES ('CHAIR-01', 4999), ('LAMP-01', 2999);",
        )
        .await?;

    let count: i64 = client
        .query_one_scalar("SELECT count(*) FROM products", &[])
        .await?;
    let skus: Vec<String> = client
        .query_scalar("SELECT sku FROM products ORDER BY sku", &[])
        .await?;
    let price: Option<i64> = client
        .query_opt_scalar(
            "SELECT price_cents FROM products WHERE sku = $1",
            &[&"VASE-01"],
        )
        .await?;

    assert_eq!(count, 2);
    assert_eq!(skus, ["CHAIR-01", "LAMP-01"]);
    assert_eq!(price, None);
    Ok(())
}
```

The value type must own its data (`String`, not `&str`), because the row it came from is dropped before the method returns. In 0.7.18 the scalar methods exist only on `Client`, not on `Transaction` or `GenericClient` (Chapter 8), so code that must also run inside a transaction uses `query_one(…)?.get(0)`.

2.6 `RETURNING`: changing and reading in one statement

An `INSERT`, `UPDATE` or `DELETE` can return the rows it touched, with `RETURNING`. Run such a statement with `query` (or `query_one`) instead of `execute`, and you get the generated id, the default timestamp or the new price without a second query:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 id          bigint GENERATED ALWAYS AS IDENTITY,
                 sku         text   PRIMARY KEY,
                 price_cents bigint NOT NULL
             )",
        )
        .await?;

    // The id is made by the server; RETURNING hands it back.
    let id: i64 = client
        .query_one(
            "INSERT INTO products (sku, price_cents) VALUES ($1, $2) RETURNING id",
            &[&"CHAIR-01", &4999_i64],
```

```
        )
        .await?
        .get("id");
    assert_eq!(id, 1);

    // An upsert: insert, or update the price if the SKU exists. PG 18's RETURNING old and
    // new say which one happened, and what the price was before.
    let upsert = "INSERT INTO products (sku, price_cents) VALUES ($1, $2)
                  ON CONFLICT (sku) DO UPDATE SET price_cents = EXCLUDED.price_cents
                  RETURNING old.sku IS NULL AS inserted, old.price_cents AS was";
    let row = client.query_one(upsert, &[&"LAMP-01", &2999_i64]).await?;
    assert!(row.get::<_, bool>("inserted"));
    let row = client.query_one(upsert, &[&"CHAIR-01", &3999_i64]).await?;
    assert!(!row.get::<_, bool>("inserted"));
    assert_eq!(row.get::<_, Option<i64>>("was"), Some(4999));
    Ok(())
}
```

`RETURNING old.…` and `new.…` are new in PostgreSQL 18. Before 18, the usual trick to tell an inserted row from an updated one in an upsert is `RETURNING (xmax = 0) AS inserted`, which relies on an implementation detail of how PostgreSQL marks row versions. It works, and it is worth replacing when you can.

2.7 The simple query protocol

Every method so far uses PostgreSQL's *extended* query protocol: the statement and its values travel in separate messages, and results come back in binary, typed. PostgreSQL has a second, older protocol, the *simple* one, in which a single message carries one or more statements as text, with no parameters, and every value comes back as text. `batch_execute` uses it, which is why it can run several statements and cannot take parameters.

`simple_query` uses it too, and returns everything the server sent back:

```
use tokio_postgres::SimpleQueryMessage;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint);
             INSERT INTO products VALUES ('CHAIR-01', 4999), ('LAMP-01', NULL);",
        )
        .await?;

    let messages = client
        .simple_query(
            "SELECT sku, price_cents FROM products ORDER BY sku;
             UPDATE products SET price_cents = 1 WHERE price_cents IS NULL",
        )
        .await?;

    let mut rows = Vec::new();
    let mut completed = Vec::new();
    for message in messages {
        match message {
            SimpleQueryMessage::RowDescription(columns) => {
                assert_eq!(columns[1].name(), "price_cents");
            }
            // Every value is `Option<&str>`: text, or None for NULL. No types.
            SimpleQueryMessage::Row(row) => {
```

```
                rows.push((row.get(0).map(str::to_owned), row.get(1).map(str::to_owned)));
            }
            // One per statement: rows returned or affected.
            SimpleQueryMessage::CommandComplete(count) => completed.push(count),
            _ => {} // the enum is non-exhaustive
        }
    }
    assert_eq!(rows[0], (Some("CHAIR-01".into()), Some("4999".into())));
    assert_eq!(rows[1], (Some("LAMP-01".into()), None));
    assert_eq!(completed, [2, 1]);
    Ok(())
}
```

`SimpleQueryMessage` has three variants. `RowDescription` (since 0.7.11) arrives before the rows of each statement that returns rows, with the column names. Without it, a `SELECT` that matched nothing would leave you no way to print its column headers. `Row` holds one row of text. `CommandComplete` carries a statement's count. It carries only the count: the command tag the server sends ("UPDATE 1", "CREATE TABLE") is not exposed, which is why the tiny psql in this repository prints "OK, 0 affected" where psql prints "CREATE TABLE".

When would you use it?

- **Tools that run SQL they did not write**: a migration runner, an admin console, a psql of your own. They need several statements per string and do not know the result types in advance; text suits them. pgq is built on it.
- **Commands that do not take parameters anyway**: `SET`, `LISTEN`, `VACUUM`, DDL.
- **Some connection poolers.** PgBouncer in its older modes could not carry the extended protocol's named statements across transactions, and the simple protocol always works (Chapters 4 and 13).

For everything with values from outside, use the extended protocol, that is, every method in §2.2 to §2.6.

> **Protocol lens:** the simple protocol is one round trip: a *Query* message with the text, answered by a *RowDescription*, *DataRow*s and a *CommandComplete* for each statement, then *ReadyForQuery*. The extended protocol splits the work into *Parse* (the text, with placeholders), *Bind* (the values), *Describe* and *Execute*, which is what makes parameters and binary values possible, and it needs one round trip more unless the statement was prepared before. Chapter 4 counts them.

Side by side – inserting a row and reading it back

The task: insert a product given as three values, and get back what was stored.

```
-- psql: plain SQL with literals...
tpbook=# INSERT INTO products (sku, name, price_cents) VALUES ('VASE-01', 'Vase', 1999)
tpbook-#   RETURNING sku, price_cents;
   sku    | price_cents
----------+-------------
 VASE-01 |        1999
(1 row)

INSERT 0 1
-- ...or, since psql 16, real parameters with \bind (the extended protocol):
```

```
tpbook=# INSERT INTO products (sku, name, price_cents) VALUES ($1, $2, $3)
tpbook-#   RETURNING sku, price_cents \bind 'RUG-01' 'Rug' 8999 \g
```

```
// sqlx (illustration)
let (sku, price): (String, i64) = sqlx::query_as(
    "INSERT INTO products (sku, name, price_cents) VALUES ($1, $2, $3) RETURNING sku,
price_cents",
)
.bind("VASE-01")
.bind("Vase")
.bind(1999_i64)
.fetch_one(&pool)
.await?;
```

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, name text, price_cents
bigint)",
        )
        .await?;
    let row = client
        .query_one(
            "INSERT INTO products (sku, name, price_cents) VALUES ($1, $2, $3)
             RETURNING sku, price_cents",
            &[&"VASE-01", &"Vase", &1999_i64],
        )
        .await?;
    let (sku, price): (&str, i64) = (row.get("sku"), row.get("price_cents"));
    assert_eq!((sku, price), ("VASE-01", 1999));
    Ok(())
}
```

	psql	sqlx	tokio-postgres
Same	`$n` placeholders and values sent separately (with `\bind`)	`$n` placeholders, values sent separately	`$n` placeholders, values sent separately
Different	Without `\bind`, values are literals in the text; psql variables (`:'name'`) are quoted and pasted in by psql itself	Values attached one by one with `.bind()`. The `query!` macros check the SQL and types at compile time, against a live database	Values in a slice of references, checked at run time against the types the server reports for the statement. Rows are read by name or position with typed `get` (Chapter 3)

In stockroom

Every repository function in stockroom follows the same pattern: the SQL written out in full, the values as parameters, `RETURNING` instead of a second query. Creating a store:

```
// stockroom/src/repo/stores.rs (excerpt)
pub async fn create(db: &impl GenericClient, new: &NewStore) -> Result<Store, AppError> {
    let stmt = db
        .prepare_cached(
            "INSERT INTO stores (code, name, address) VALUES ($1, $2, $3)
             RETURNING id, code, name, address, created_at",
```

```
        )
        .await?;
    let row = db
        .query_one(&stmt, &[&new.code, &new.name, &new.address])
        .await?;
    Ok(Store::try_from(&row)?)
}
```

Three things in it come later. `prepare_cached` is Chapter 4 and 13 (preparing a statement once per connection). `&new.address` is a Rust struct passed as a PostgreSQL composite value (Chapter 5). `GenericClient` lets the same function run on a pooled connection or inside a transaction (Chapter 8).

The one place stockroom uses the simple protocol is a diagnostic: `GET /admin/sessions` lists the database's sessions straight from `pg_stat_activity`, and only displays them, so text is exactly what it wants:

```
// stockroom/src/repo/reports.rs (excerpt)
let messages = client
    .simple_query(
        "SELECT pid, application_name, backend_type, state,
                wait_event_type || ':' || wait_event AS waiting_on,
                date_trunc('second', now() - query_start) AS running_for,
                left(query, 120) AS query
         FROM pg_stat_activity
         WHERE datname = current_database()
         ORDER BY pid",
    )
    .await?;
let mut sessions = Vec::new();
for message in messages {
    if let SimpleQueryMessage::Row(row) = message {
        let mut session = serde_json::Map::new();
        for (i, column) in row.columns().iter().enumerate() {
            // `Option<&str>`: NULL is `None`; everything else is text, whatever its SQL
type.
            session.insert(column.name().to_owned(), row.get(i).into());
        }
        sessions.push(session);
    }
}
```

The JSON this produces has `"pid": "60027"`, a string, where a typed query would give a number: that is the simple protocol's trade.

Summary

- `batch_execute` runs a script of several statements in one implicit transaction. No parameters, no results.
- `execute` returns the number of rows affected (or returned); 0 for DDL.
- `query` returns all rows; `query_one` exactly one; `query_opt` zero or one. A wrong row count is an error from the client, with no SQLSTATE.
- `query_scalar`, `query_one_scalar` and `query_opt_scalar` (0.7.16+, `Client` only) return the first column of each row.
- Values go in parameters, `$1…$n`, never into the SQL text. Building SQL *structure* with `format!` is fine; table names, column names and keywords come from allowlists, or `quote_ident` as a last resort.

- A dynamic parameter list is a `Vec<&(dyn ToSql + Sync)>`, with the values it points at living at least as long.
- `RETURNING` reads what a change produced. PG 18's `RETURNING old/new` shows before and after.
- `simple_query` uses the simple protocol: several statements, no parameters, all text, with `RowDescription`, `Row` and `CommandComplete` messages.

Exercises

1. **A price change.** Write `async fn reprice(client: &Client, percent: i64, min_price: i64) -> Result<u64, Error>` that raises every price at or above `min_price` by `percent` percent and returns how many products changed. *Hint:* integer arithmetic in SQL: `price_cents * (100 + $1) / 100`.
2. **One more filter.** Extend `search` from §2.3 with `min_price: Option<i64>`. Make sure the placeholder numbers stay right in every combination of filters. *Hint:* the number of each placeholder is `params.len()` right after the push.
3. **Spot the injection.** A colleague wrote this for a "sort by" dropdown. What can a client do with it, and how do you fix it?

   ```
   let direction = query_params.get("dir").unwrap_or("asc");
   let sql = format!("SELECT sku FROM products ORDER BY price_cents {direction}");
   client.query(&sql, &[]).await?
   ```

   Hint: `ORDER BY` takes a keyword, and a keyword cannot be a parameter; compare `direction` against a fixed list instead.
4. **Found, or not.** Write `async fn price_of(client: &Client, sku: &str) -> Result<Option<i64>, Error>`, which returns `Ok(None)` for an unknown SKU and an error only when something actually went wrong. *Hint:* one of the scalar methods does exactly this.
5. **A tiny psql.** Use `simple_query` to run a script typed by a user and print, for each statement, either its rows or "OK, n affected". Compare your version with `run` in apps/pgq/src/lib.rs. *Hint:* a `RowDescription` arrives even for a `SELECT` that returns no rows, and `CommandComplete` carries the count.

Solutions

3. `direction` is pasted into the SQL, so a client can send anything: `?dir=asc; DELETE FROM products` does not run here only because `query` uses the extended protocol, which allows one statement per message (the server refuses with "cannot insert multiple commands into a prepared statement"). But `?dir=, (SELECT …)` can still add subqueries to the sort, and a later refactoring to `batch_execute` would make the first attack work too. A sort direction is structure, not a value, so it cannot be a parameter. Map it from an allowlist:

```
let direction = match query_params.get("dir") {
    Some("desc") => "DESC",
    _ => "ASC",
};
let sql = format!("SELECT sku FROM products ORDER BY price_cents {direction}");
```

Now `direction` is always one of two string literals in your binary.

4.

```rust
use tokio_postgres::{Client, Error};

async fn price_of(client: &Client, sku: &str) -> Result<Option<i64>, Error> {
    client
        .query_opt_scalar("SELECT price_cents FROM products WHERE sku = $1", &[&sku])
        .await
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint);
             INSERT INTO products VALUES ('CHAIR-01', 4999);",
        )
        .await?;
    assert_eq!(price_of(&client, "CHAIR-01").await?, Some(4999));
    assert_eq!(price_of(&client, "NOPE-01").await?, None);
    Ok(())
}
```

`query_opt_scalar` returns `Ok(None)` for no row and an error for anything else, including two rows (impossible here, because `sku` is the primary key).

CHAPTER 3

Rows and the type mapping

A query hands you `Row`s, and every value in them must become a Rust value. tokio-postgres does that with one rule: **you say which Rust type you want, and the conversion checks that the column's PostgreSQL type fits.** There is no guessing and no silent widening, and a mismatch is an error at run time, so the mapping between the two type systems is something you need to know.

This chapter covers reading a `Row`, the full table of which Rust types match which PostgreSQL types (including the ones optional cargo features add), NULL, borrowing values without copying them, and the types that trip people up: timestamps, JSON, arrays and exact decimals. Chapter 5 does the same for types you define yourself.

3.1 Reading a `Row`

A `Row` knows its columns, as the server described them, and hands out values by position or by name:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let row = client
        .query_one(
            "SELECT 'CHAIR-01' AS sku, 4999::int8 AS price_cents, NULL::text AS note",
            &[],
        )
        .await?;

    // What the row knows: the names and types the server described.
    let described: Vec<(&str, &str)> = row
        .columns()
        .iter()
        .map(|c| (c.name(), c.type_().name()))
        .collect();
    assert_eq!(
        described,
        [("sku", "text"), ("price_cents", "int8"), ("note", "text")]
    );
    assert_eq!(row.len(), 3);

    // By position or by name. The Rust type you ask for decides the conversion.
    let sku: &str = row.get(0);
    let price: i64 = row.get("price_cents");
    assert_eq!((sku, price), ("CHAIR-01", 4999));

    // `get` panics on any problem; `try_get` returns it.
    let wrong_type = row.try_get::<_, i32>("price_cents").unwrap_err();
    let no_column = row.try_get::<_, i64>("prise_cents").unwrap_err();
    let was_null = row.try_get::<_, String>("note").unwrap_err();
    assert_eq!(wrong_type.to_string(), "error deserializing column 1");
    assert_eq!(no_column.to_string(), "invalid column `prise_cents`");
```

```
    assert_eq!(was_null.to_string(), "error deserializing column 2");

    // NULL becomes None, if you ask for an Option.
    let note: Option<String> = row.get("note");
    assert_eq!(note, None);
    Ok(())
}
```

Three kinds of failure, then, all at run time:

Failure	`Debug` of the error	Cause
Wrong type	`Error { kind: FromSql(1), cause: Some(WrongType { postgres: Int8, rust: "i32" }) }`	The Rust type does not accept the column's type
No such column	`Error { kind: Column("prise_cents"), cause: None }`	A name that is not in the row, or an index past the end
NULL into a non-`Option`	`Error { kind: FromSql(2), cause: Some(WasNull) }`	The value was NULL

Note how little the `Display` text says ("error deserializing column 1"): the useful part is the cause, in `source()`. Chapter 6 makes that visible.

`get` or `try_get`? `get` is convenient where a failure is a bug you want to hear about loudly: examples, tests, and columns of a query you wrote right above. In code that serves requests, prefer `try_get` with `?`: a panic in a handler takes down a request, or worse, and the error has more to say than the panic message.

By name or by position? Positions are fast and break silently when someone adds a column to the `SELECT`. Names survive that, and cost a scan of the column list (a handful of string comparisons). In application code, use names. If two columns have the same name, which happens with `SELECT *` over a join, the first one wins. Give them distinct names with `AS`.

3.2 The type table

Every Rust type that can be read from a row implements `FromSql`, and every type that can be sent as a parameter implements `ToSql` (both traits come from the `postgres-types` crate and are re-exported as `tokio_postgres::types`). Each implementation says which PostgreSQL types it accepts. Built in, with no features:

Rust	PostgreSQL
`bool`	`boolean`
`i8`	`"char"` (the one-byte internal type, not `char(n)`)
`i16`	`smallint` (`int2`)
`i32`	`integer` (`int4`)
`i64`	`bigint` (`int8`)
`u32`	`oid` (and only `oid`)
`f32`	`real` (`float4`)

Rust	PostgreSQL
`f64`	`double precision` (`float8`)
`&str`, `String`, `Box<str>`, `Cow<str>`	`text`, `varchar`, `char(n)` (`bpchar`), `name`, `unknown`, and `citext`, `ltree`, `lquery`, `ltxtquery` by name
`&[u8]`, `Vec<u8>`	`bytea`
`HashMap<String, Option<String>>`	`hstore`
`std::time::SystemTime`	`timestamp`, `timestamptz`
`std::net::IpAddr`	`inet`
`Option<T>`	whatever `T` takes, or NULL
`Vec<T>`, `&[T]`, `Box<[T]>`	arrays of whatever `T` takes (one dimension)
`[T; N]` (feature `array-impls`)	arrays of exactly `N` elements

Everything else comes with an optional feature of `tokio-postgres`, so that you only compile the integrations you use:

Feature	Rust	PostgreSQL
`with-jiff-0_2`	`jiff::Timestamp` / `civil::DateTime` / `civil::Date` / `civil::Time`	`timestamptz` / `timestamp` / `date` / `time`
`with-chrono-0_4`	`DateTime<Utc>`, `DateTime<Local>`, `DateTime<FixedOffset>` / `NaiveDateTime` / `NaiveDate` / `NaiveTime`	`timestamptz` / `timestamp` / `date` / `time`
`with-time-0_3`	`OffsetDateTime` / `PrimitiveDateTime` / `Date` / `Time`	`timestamptz` / `timestamp` / `date` / `time`
`with-uuid-1`	`uuid::Uuid`	`uuid`
`with-serde_json-1`	`serde_json::Value`, `Json<T>`	`json`, `jsonb`
`with-cidr-0_3`	`IpCidr` / `IpInet`	`cidr` / `inet`
`with-eui48-1`	`MacAddress`	`macaddr`
`with-geo-types-0_7`	`Point<f64>` / `Rect<f64>` / `LineString<f64>`	`point` / `box` / `path`
`with-bit-vec-0_8`	`BitVec`	`bit`, `varbit`

And one from outside: `rust_decimal::Decimal` implements both traits for `numeric` when you enable `rust_decimal`'s own feature `db-tokio-postgres` (§3.8).

Three gaps are worth knowing before you hit them:

- **No unsigned integers** except `u32` for `oid`. PostgreSQL has no unsigned types, and pretending would hide overflows. A `u64` count goes into a `bigint` as an `i64`, with `i64::try_from` at the boundary.
- **No `interval`.** None of the date-time features maps it. Convert in SQL (`extract(epoch FROM …)` gives seconds as `numeric`), or write the conversion yourself (Chapter 5).
- **No `money`**, and you do not want it: store cents in a `bigint`, or use `numeric`.

> **Pitfall:** SQL's own typing decides the column type, and it is not always the one you expect. `6 * 7` is `integer`, but `count(*)` is `bigint`, `sum()` of an `integer` column is `bigint`, `sum()` of a `bigint` column is **`numeric`**, and `avg()` of any integer is `numeric` too. `let total: i64 = row.get("sum")` over a `bigint` column fails. Cast in the query (`sum(total_cents)::bigint`) or read a `Decimal`. When in doubt, ask the row: `row.columns()[0].type_()`, or `pgq --describe`.

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let row = client
        .query_one(
            "SELECT count(*), sum(i), sum(i::int8), avg(i), sum(i::int8)::int8 AS total
             FROM generate_series(1, 4) AS i",
            &[],
        )
        .await?;
    let types: Vec<&str> = row.columns().iter().map(|c| c.type_().name()).collect();
    assert_eq!(types, ["int8", "int8", "numeric", "numeric", "int8"]);
    assert_eq!(row.get::<_, i64>("total"), 10);
    Ok(())
}
```

3.3 NULL

NULL is `Option` on both sides. Reading, ask for `Option<T>` wherever a column can be NULL: outer joins, nullable columns, aggregates over no rows (`max()` of nothing is NULL). Sending, pass an `Option`. A bare `None` has no type, so name it: `&None::<i64>`.

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE notes (id int, note text);
             INSERT INTO notes VALUES (1, NULL), (2, 'fragile');",
        )
        .await?;

    // A NULL parameter.
    client
        .execute("INSERT INTO notes VALUES ($1, $2)", &[&3, &None::<&str>])
        .await?;

    // `= $1` with NULL matches nothing: in SQL, NULL = NULL is not true but NULL.
    let no_note: Option<&str> = None;
    let equal = client
        .query("SELECT id FROM notes WHERE note = $1", &[&no_note])
        .await?;
    assert!(equal.is_empty());

    // IS NOT DISTINCT FROM treats two NULLs as equal: the comparison you meant.
    let same: Vec<i32> = client
        .query(
            "SELECT id FROM notes WHERE note IS NOT DISTINCT FROM $1 ORDER BY id",
            &[&no_note],
        )
        .await?
        .iter()
        .map(|row| row.get("id"))
```

```
        .collect();
    assert_eq!(same, [1, 3]);

    // max() over no rows is NULL, even though the column type is integer.
    let max: Option<i32> = client
        .query_one("SELECT max(id) FROM notes WHERE false", &[])
        .await?
        .get(0);
    assert_eq!(max, None);
    Ok(())
}
```

> **SQL lens:** an optional filter ("this note, or any note if none was given") is often written `WHERE $1::text IS NULL OR note = $1`. It works, but a prepared statement with that shape cannot use an index on `note` well, because the plan must suit both cases. When it matters, build two statements, as `search` did in Chapter 2.

3.4 Borrowing from a row

`FromSql` has a lifetime: `FromSql<'a>`, where `'a` is the lifetime of the row's data. Types like `&str` and `&[u8]` take advantage of it. They point into the row's buffer instead of copying, which makes reading many rows cheaper. The price is that they cannot outlive the row:

```
// Does not compile: the row is a temporary, gone at the end of the statement.
let name: &str = client.query_one("SELECT 'chair'", &[]).await?.get(0);
println!("{name}");
// error[E0716]: temporary value dropped while borrowed
```

Keep the row (or the `Vec<Row>`) alive for as long as you use what you borrowed, or ask for an owned type (`String`, `Vec<u8>`) instead:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let rows = client
        .query(
            "SELECT 'Product ' || i AS name FROM generate_series(1, 1000) AS i",
            &[],
        )
        .await?;

    // Borrowed: no allocation per row. `rows` outlives every `&str`.
    let total_length: usize = rows
        .iter()
        .map(|row| row.get::<_, &str>("name").len())
        .sum();
    assert_eq!(
        total_length,
        1000 * "Product ".len() + 9 + 2 * 90 + 3 * 900 + 4
    );

    // Owned: each value has its own allocation, and can go anywhere.
    let names: Vec<String> = rows.iter().map(|row| row.get("name")).collect();
    drop(rows);
    assert_eq!(names[999], "Product 1000");
    Ok(())
}
```

> **Rust refresher:** `FromSql<'a>` with `fn from_sql(ty: &Type, raw: &'a [u8]) -> Result<Self, …>` lets `Self` borrow from `raw`, the column's bytes inside the row. A type that never borrows implements it for every `'a`, which is what `FromSqlOwned` means: it is a shorthand for `for<'a> FromSql<'a>`. APIs that drop the row before handing you the value, such as the `query_scalar` family in Chapter 2, require `FromSqlOwned`. That is why `query_one_scalar::<&str, _>` does not compile.

3.5 Rows into structs

Applications want structs, not rows. tokio-postgres has no derive for this, and the usual pattern is a `TryFrom<&Row>` implementation per struct, reading columns by name:

```
use tokio_postgres::Row;

#[derive(Debug, PartialEq)]
struct Product {
    sku: String,
    name: String,
    price_cents: i64,
    discontinued: Option<jiff::civil::Date>,
}

impl TryFrom<&Row> for Product {
    type Error = tokio_postgres::Error;

    fn try_from(row: &Row) -> Result<Self, Self::Error> {
        Ok(Product {
            sku: row.try_get("sku")?,
            name: row.try_get("name")?,
            price_cents: row.try_get("price_cents")?,
            discontinued: row.try_get("discontinued")?,
        })
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 sku text PRIMARY KEY, name text NOT NULL, price_cents bigint NOT NULL,
                 discontinued date);
             INSERT INTO products VALUES ('CHAIR-01', 'Garden chair', 4999, NULL),
                                         ('STOOL-01', 'Stool', 1999, '2026-06-30');",
        )
        .await?;

    let rows = client
        .query(
            "SELECT sku, name, price_cents, discontinued FROM products ORDER BY sku",
            &[],
        )
        .await?;
    let products = rows
        .iter()
        .map(Product::try_from)
        .collect::<Result<Vec<_>, _>>()?;

    assert_eq!(products[0].discontinued, None);
    assert_eq!(
        products[1],
        Product {
```

```
            sku: "STOOL-01".into(),
            name: "Stool".into(),
            price_cents: 1999,
            discontinued: Some(jiff::civil::date(2026, 6, 30)),
        }
    );
    Ok(())
}
```

It is a few lines more per struct than a derive, and in exchange, the mapping is ordinary code you can read, step through and change: a column renamed in the query, a computed field, a nested struct read from the same row. Keep the `SELECT` list and the `TryFrom` together, in the same module. Chapter 14 shows how stockroom organizes them, and mentions clorinde, which generates both from `.sql` files.

3.6 Dates and times

PostgreSQL has two timestamp types, and they map to two different Rust types:

PostgreSQL	What it stores	jiff	chrono	time
`timestamptz`	an instant: microseconds since 2000-01-01 UTC	`Timestamp`	`DateTime<Utc>`	`OffsetDateTime`
`timestamp`	a wall-clock reading, no time zone	`civil::DateTime`	`NaiveDateTime`	`PrimitiveDateTime`
`date`	a calendar day	`civil::Date`	`NaiveDate`	`Date`
`time`	a time of day	`civil::Time`	`NaiveTime`	`Time`

This book uses jiff, whose `Timestamp` is exactly what `timestamptz` is: an instant, with no time zone attached. Despite its name, `timestamptz` does *not* store a time zone. It stores UTC and converts at the edges. And the edges tokio-postgres uses are binary, so the session's `TimeZone` setting does not affect what you read:

```
use jiff::Timestamp;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("SET TimeZone = 'America/New_York'")
        .await?;

    let row = client
        .query_one(
            "SELECT '2026-11-06 12:00:00+00'::timestamptz AS at,
                    '2026-11-06 12:00:00+00'::timestamptz::text AS as_text",
            &[],
        )
        .await?;
    let at: Timestamp = row.get("at");
    // Binary: the instant itself, whatever the session's time zone.
    assert_eq!(at.to_string(), "2026-11-06T12:00:00Z");
    // Text: rendered in the session's time zone.
    assert_eq!(row.get::<_, &str>("as_text"), "2026-11-06 07:00:00-05");

    // timestamp (without time zone) needs the civil type; Timestamp refuses it.
    let row = client
        .query_one("SELECT '2026-11-06 12:00'::timestamp", &[])
        .await?;
```

```
    assert!(row.try_get::<_, Timestamp>(0).is_err());
    let wall: jiff::civil::DateTime = row.get(0);
    assert_eq!(wall, jiff::civil::date(2026, 11, 6).at(12, 0, 0, 0));
    Ok(())
}
```

The session time zone still matters for anything the *server* computes from a `timestamptz`: `date_trunc('day', created_at)` and `created_at::date` cut the day at midnight in the session's time zone. stockroom's sales report says which one it means with `AT TIME ZONE 'UTC'`.

> **Pitfall:** PostgreSQL keeps microseconds; jiff, chrono and `time` keep nanoseconds. A value with sub-microsecond digits loses them on the way in, so it does not compare equal on the way back. `Timestamp::now()` has nanoseconds on Linux (on macOS the clock happens to stop at microseconds, which hides the problem until the tests run in CI). Truncate before you store, or compare at microsecond precision.

```
use jiff::Timestamp;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let precise = Timestamp::from_nanosecond(1_700_000_000_123_456_789).expect("in range");
    let stored: Timestamp = client
        .query_one("SELECT $1::timestamptz", &[&precise])
        .await?
        .get(0);
    assert_ne!(stored, precise);
    assert_eq!(stored.to_string(), "2023-11-14T22:13:20.123456Z");
    let truncated = Timestamp::from_microsecond(precise.as_microsecond()).expect("in
range");
    assert_eq!(stored, truncated);
    Ok(())
}
```

3.7 JSON

With `with-serde_json-1`, `serde_json::Value` reads and writes `json` and `jsonb`. For a column with a known shape, `Json<T>` does the same through serde with your own type: it wraps any `T: Serialize` going in and any `T: Deserialize` coming out.

```
use serde::{Deserialize, Serialize};
use serde_json::{Value, json};
use tokio_postgres::types::Json;

#[derive(Debug, PartialEq, Serialize, Deserialize)]
struct Customer {
    name: String,
    email: String,
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE orders (id int, customer jsonb, extra jsonb)")
```

```
        .await?;

    let ada = Customer {
        name: "Ada".into(),
        email: "ada@example.com".into(),
    };
    // A typed struct and a free-form Value, both into jsonb.
    client
        .execute(
            "INSERT INTO orders VALUES (1, $1, $2)",
            &[
                &Json(&ada),
                &json!({ "giftWrap": true, "notes": ["fragile"] }),
            ],
        )
        .await?;

    let row = client
        .query_one("SELECT customer, extra FROM orders WHERE id = 1", &[])
        .await?;
    let Json(customer): Json<Customer> = row.get("customer");
    let extra: Value = row.get("extra");
    assert_eq!(customer, ada);
    assert_eq!(extra["notes"][0], "fragile");

    // Inside the document, in SQL: ->> yields text, -> yields jsonb.
    let row = client
        .query_one(
            "SELECT customer ->> 'name' AS name, extra -> 'giftWrap' AS wrap
             FROM orders WHERE customer ->> 'email' = $1",
            &[&"ada@example.com"],
        )
        .await?;
    assert_eq!(row.get::<_, &str>("name"), "Ada");
    assert_eq!(row.get::<_, Value>("wrap"), json!(true));
    Ok(())
}
```

`json` stores the text as given; `jsonb` stores a parsed form, which is what you want for anything you query, compare or index. The Rust side works the same with both.

> **Pitfall:** `Json<T>` reads whatever is in the column and deserializes it with serde. If the document does not match `T`, for example after someone else's code wrote a slightly different shape, reading fails with a conversion error at run time. For columns written by several programs, read a `Value` and validate it, or give `T` serde defaults.

3.8 Arrays

A one-dimensional PostgreSQL array maps to `Vec<T>` (or a slice going in), for any element type `T` that maps:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE products (sku text PRIMARY KEY, tags text[])")
        .await?;
    for (sku, tags) in [
        ("CHAIR-01", vec!["garden", "sale"]),
```

```
        ("LAMP-01", vec!["lighting"]),
        ("RUG-01", vec![]),
    ] {
        client
            .execute("INSERT INTO products VALUES ($1, $2)", &[&sku, &tags])
            .await?;
    }

    let tags: Vec<String> = client
        .query_one("SELECT tags FROM products WHERE sku = 'CHAIR-01'", &[])
        .await?
        .get(0);
    assert_eq!(tags, ["garden", "sale"]);

    // "Is it one of these?" One array parameter and = ANY, not IN.
    let wanted = ["CHAIR-01", "RUG-01", "VASE-01"];
    let found: Vec<String> = client
        .query_scalar(
            "SELECT sku FROM products WHERE sku = ANY($1) ORDER BY sku",
            &[&&wanted[..]],
        )
        .await?;
    assert_eq!(found, ["CHAIR-01", "RUG-01"]);

    // "Does the array contain this?" @> (contains), indexable with GIN.
    let on_sale: Vec<String> = client
        .query_scalar(
            "SELECT sku FROM products WHERE tags @> ARRAY[$1]",
            &[&"sale"],
        )
        .await?;
    assert_eq!(on_sale, ["CHAIR-01"]);

    // Elements can be NULL: then the element type must be an Option.
    let mixed: Vec<Option<i32>> = client
        .query_one("SELECT ARRAY[1, NULL, 3]", &[])
        .await?
        .get(0);
    assert_eq!(mixed, [Some(1), None, Some(3)]);
    Ok(())
}
```

`= ANY($1)` with one array parameter replaces `IN ($1, $2, …)`. It is one statement whatever the list's length, so it can be prepared once and reused, and the list cannot grow into something you have to build with `format!`.

Two limits: an array with NULL elements needs `Vec<Option<T>>` (with `Vec<T>` it fails with `WasNull`), and arrays of more than one dimension are refused ("array contains too many dimensions"). Model a grid as a table, or as `jsonb`.

> **Pitfall:** `WHERE sku IN ($1)` does not mean "any of the values in this array". `IN` with one item is `sku = $1`, so the server types `$1` as `text`, and the client then refuses to send a `Vec` as a `text`: "cannot convert between the Rust type `Vec<&str>` and the Postgres type `text`". The `ANY` form is the one you want.

3.9 Exact numbers, UUIDs

`numeric` is PostgreSQL's arbitrary-precision decimal. `f64` cannot hold it exactly, and tokio-postgres does not pretend: there is no built-in mapping. `rust_decimal::Decimal` provides one (with its `db-tokio-postgres` feature, in the playground's `Cargo.toml`):

```
use rust_decimal::Decimal;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let row = client
        .query_one(
            "SELECT 0.1::numeric + 0.2 AS exact, 0.1::float8 + 0.2 AS binary,
                    round(avg(x), 2) AS average
             FROM (VALUES (1999), (2999), (4999)) AS prices(x)",
            &[],
        )
        .await?;
    let exact: Decimal = row.get("exact");
    let binary: f64 = row.get("binary");
    let average: Decimal = row.get("average");
    assert_eq!(exact.to_string(), "0.3");
    assert_eq!(binary.to_string(), "0.30000000000000004");
    assert_eq!(average.to_string(), "3332.33");
    Ok(())
}
```

For money that a program adds up, the simplest exact representation is an integer number of cents in a `bigint`. That is what stockroom does, with `numeric` only where SQL produces one (averages).

`uuid` maps to `uuid::Uuid` with `with-uuid-1`. PostgreSQL 18 can generate time-ordered version-7 UUIDs itself, with `uuidv7()`, which makes a good default for primary keys: unique without coordination, and roughly in insertion order, which keeps B-tree indexes compact.

```
use uuid::Uuid;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let row = client
        .query_one("SELECT uuidv7() AS first, uuidv7() AS second", &[])
        .await?;
    let (first, second): (Uuid, Uuid) = (row.get("first"), row.get("second"));
    assert_eq!(first.get_version_num(), 7);
    assert!(first < second, "time-ordered");

    // And from Rust, generated client-side, the same type goes back in.
    let mine = Uuid::now_v7();
    let echoed: Uuid = client.query_one("SELECT $1::uuid", &[&mine]).await?.get(0);
    assert_eq!(echoed, mine);
    Ok(())
}
```

Side by side – a row into a struct

The task: read a product into a struct, with a nullable column.

```
-- psql: rows are printed; \gx shows one column per line, which is how a struct reads.
tpbook=# SELECT sku, name, price_cents, discontinued FROM products WHERE sku = 'STOOL-01'
\gx
-[ RECORD 1 ]+----------
sku          | STOOL-01
```

```
name         | Stool
price_cents  | 1999
discontinued | 2026-06-30
```

```
// sqlx (illustration)
#[derive(sqlx::FromRow)]
struct Product { sku: String, name: String, price_cents: i64, discontinued:
Option<chrono::NaiveDate> }

let product: Product = sqlx::query_as("SELECT sku, name, price_cents, discontinued FROM
products WHERE sku = $1")
    .bind("STOOL-01")
    .fetch_one(&pool)
    .await?;
```

```
use tokio_postgres::Row;

#[derive(Debug, PartialEq)]
struct Product {
    sku: String,
    name: String,
    price_cents: i64,
    discontinued: Option<jiff::civil::Date>,
}

impl TryFrom<&Row> for Product {
    type Error = tokio_postgres::Error;
    fn try_from(row: &Row) -> Result<Self, Self::Error> {
        Ok(Product {
            sku: row.try_get("sku")?,
            name: row.try_get("name")?,
            price_cents: row.try_get("price_cents")?,
            discontinued: row.try_get("discontinued")?,
        })
    }
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let row = client
        .query_one(
            "SELECT 'STOOL-01' AS sku, 'Stool' AS name, 1999::int8 AS price_cents,
                    '2026-06-30'::date AS discontinued",
            &[],
        )
        .await?;
    let product = Product::try_from(&row)?;
    assert_eq!(product.discontinued, Some(jiff::civil::date(2026, 6, 30)));
    Ok(())
}
```

	psql	sqlx	tokio-postgres
Same	Columns have names and types; NULL is a value of its own	NULL is `Option`; types must match	NULL is `Option`; types must match
Different	Everything is displayed as text	A derive maps columns to fields by name; `query_as!` checks the mapping at compile time	No derive: a `TryFrom<&Row>` you write, checked at run time. Values can borrow from the row

In stockroom

Every type stockroom reads from the database has a `TryFrom<&Row>`, next to the queries that produce its columns. The catalog's `Product` mixes a domain type (`Sku`, Chapter 5), money in cents, an array, free-form JSON and two timestamps:

```
// stockroom/src/repo/products.rs (excerpt)
impl TryFrom<&Row> for Product {
    type Error = tokio_postgres::Error;

    fn try_from(row: &Row) -> Result<Self, Self::Error> {
        Ok(Product {
            id: row.try_get("id")?,
            sku: row.try_get("sku")?,
            name: row.try_get("name")?,
            price_cents: row.try_get("price_cents")?,
            tags: row.try_get("tags")?,
            attributes: row.try_get("attributes")?,
            version: row.try_get("version")?,
            created_at: row.try_get("created_at")?,
            updated_at: row.try_get("updated_at")?,
        })
    }
}
```

`id` is a `Uuid` (generated by `uuidv7()`), `tags` a `Vec<String>`, `attributes` a `serde_json::Value`, and the timestamps are jiff `Timestamps`. One generic helper turns a page of rows into structs for every such type, converting the error on the way:

```
// stockroom/src/repo/mod.rs (excerpt)
pub fn from_rows<T>(rows: &[Row]) -> Result<Vec<T>, AppError>
where
    for<'r> T: TryFrom<&'r Row, Error = tokio_postgres::Error>,
{
    rows.iter()
        .map(|row| T::try_from(row).map_err(AppError::from))
        .collect()
}
```

The `for<'r>` says "for a row borrowed for any lifetime", which is what lets `T` be built from each row in turn. An order's customer is structured JSON, read through serde:

```
// stockroom/src/repo/orders.rs (excerpt)
let Json(customer): Json<Customer> = head.try_get("customer")?;
```

Summary

- `row.get` panics and `row.try_get` returns an error, by index or by name. Prefer names, and `try_get` in code that serves requests.
- You choose the Rust type; the conversion checks it against the column's type. Mismatches, missing columns and unexpected NULLs are run-time errors whose cause is in `source()`.
- Know the table: `int4` is `i32`, `int8` is `i64`; no unsigned types except `oid`; `sum(bigint)` and `avg()` are `numeric`.
- NULL is `Option` both ways; name the type of a bare `None`. `= NULL` matches nothing; use `IS NOT DISTINCT FROM`.

- `&str` and `&[u8]` borrow from the row; owned types do not. `FromSqlOwned` means "never borrows".
- Map rows to structs with `TryFrom<&Row>`, reading columns by name.
- `timestamptz` is an instant (jiff `Timestamp`), `timestamp` a wall-clock reading (`civil::DateTime`). Binary transfer ignores the session time zone, and microseconds are the limit.
- JSON: `serde_json::Value`, or `Json<T>` with serde. Arrays: one dimension, `Vec<Option<T>>` for NULL elements, `= ANY($1)` instead of `IN`.
- `numeric` needs `rust_decimal`; `uuid` needs `with-uuid-1`; `interval` has no mapping.

Exercises

1. **A report row.** Write a struct `StoreSales { store: String, orders: i64, revenue_cents: i64 }` and fill it from `SELECT store, count(*) AS orders, sum(total_cents) AS revenue_cents FROM orders GROUP BY store` over a temporary `orders (store text, total_cents bigint)` table. Run it, read the error, and fix it two different ways. *Hint:* §3.2's pitfall.
2. **Zero copies.** Over `SELECT md5(i::text) AS hash FROM generate_series(1, 10000) AS i`, count how many hashes start with `"00"` without allocating a `String` per row. *Hint:* `&str`, and keep the rows alive.
3. **A left join.** Products and their reviews: `products (sku)` and `reviews (sku, stars int)`. Return every product with its average number of stars, or `None` when it has no reviews. Which Rust type do you read the average as? *Hint:* `avg` over an `int` column returns `numeric`, and a product without reviews gets a `NULL` from the left join.
4. **Find by tags.** Write a query that returns the products having *all* of a list of tags, and one that returns the products having *any* of them. Both take one array parameter. *Hint:* `@>` and `&&`.

Solutions

1. The error is `WrongType { postgres: Numeric, rust: "i64" }`: `sum()` of a `bigint` column is `numeric`, because a sum of many `bigint`s can overflow `bigint`. Either cast in the query, `sum(total_cents)::bigint AS revenue_cents` (which turns an overflow into an error on the server), or read a `rust_decimal::Decimal` and convert it in Rust.

```
#[derive(Debug, PartialEq)]
struct StoreSales {
    store: String,
    orders: i64,
    revenue_cents: i64,
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE orders (store text, total_cents bigint);
             INSERT INTO orders VALUES ('mitte', 4999), ('mitte', 2999), ('spandau', 999);",
        )
        .await?;
    let rows = client
        .query(
            "SELECT store, count(*) AS orders, sum(total_cents)::bigint AS revenue_cents
```

```
             FROM orders GROUP BY store ORDER BY store",
            &[],
        )
        .await?;
    let sales: Vec<StoreSales> = rows
        .iter()
        .map(|row| {
            Ok(StoreSales {
                store: row.try_get("store")?,
                orders: row.try_get("orders")?,
                revenue_cents: row.try_get("revenue_cents")?,
            })
        })
        .collect::<Result<_, tokio_postgres::Error>>()?;
    assert_eq!(
        sales[0],
        StoreSales {
            store: "mitte".into(),
            orders: 2,
            revenue_cents: 7998
        }
    );
    Ok(())
}
```

3. `avg(stars)` over an `integer` column is `numeric`, and the left join makes it NULL for a product without reviews: read an `Option<Decimal>`. If you would rather have a float, cast it, `avg(stars)::float8`, and read an `Option<f64>`.

4. All of them: `WHERE tags @> $1` ("contains"). Any of them: `WHERE tags && $1` ("overlaps"). Both take a `text[]` parameter, a `&[&str]` or `Vec<String>` on the Rust side, and both can use a GIN index on `tags`.

CHAPTER 4

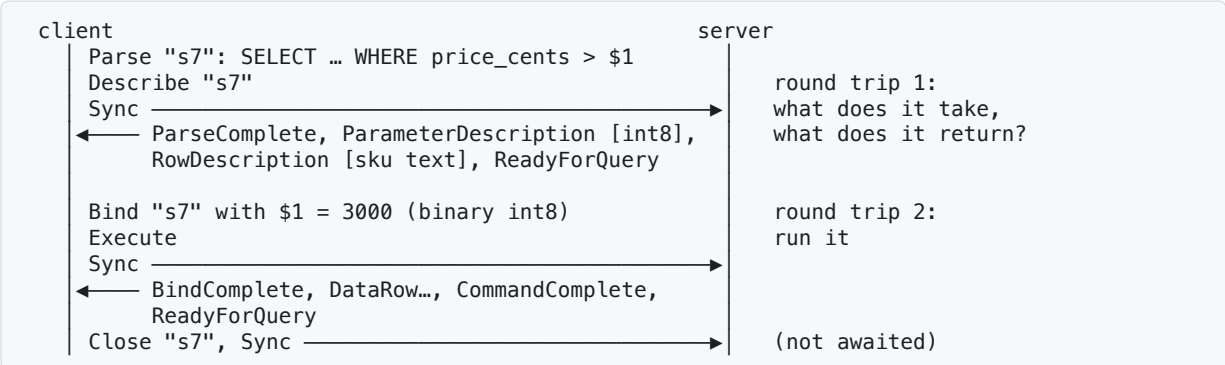
Prepared statements and the extended protocol

On localhost a query takes a few dozen microseconds, and it hardly matters how many messages it needs. Across a network, every round trip between client and server costs a fixed time (half a millisecond inside a data centre, tens of milliseconds between regions) and the count of round trips becomes the most important number in your program's performance. This chapter teaches you to count them.

It starts with what the friendly `client.query("…", &params)` actually sends, which explains why tokio-postgres's API has *statements* at all. Then come the tools for doing better: preparing a statement once and running it many times, `query_typed` for one-shot queries in a single round trip, and caching statements. Along the way you will meet the two errors that prepared statements are known for.

4.1 What `query` sends

A `Row` knows the PostgreSQL type of every column, and a parameter is checked against a PostgreSQL type before it is sent (Chapter 3). Where do those types come from? From the server, which is the only party that can tell what `$1` means in `WHERE price_cents > $1`. So when you pass SQL text to `query`, tokio-postgres does two exchanges with the server:



The first exchange *prepares* the statement: the server parses it, works out the parameter and result types, and keeps it under a name (tokio-postgres numbers them `s0`, `s1` and so on). The second *executes* it with your values, encoded in binary now that their types are known. Finally, because the statement was only needed for this one call, tokio-postgres tells the server to forget it, without waiting for an answer.

Two round trips per query is the price of passing SQL text. Most of the time you do not have to pay it.

> **Protocol lens:** these are the messages of PostgreSQL's *extended query protocol*. Chapter 2 showed the *simple* one, a single *Query* message carrying SQL text with no parameters, in one round trip. The extended protocol splits the same work into steps so that statements can be reused, values can travel separately and in binary, and results can be fetched in parts (portals, Chapter 9). Every tokio-postgres method except `simple_query` and `batch_execute` uses it.

4.2 `prepare`: one parse, many executions

`Client::prepare` does the first exchange on its own and gives you a `Statement`. Running a `Statement` costs one round trip:

```
use tokio_postgres::types::Type;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint);
             INSERT INTO products VALUES ('CHAIR-01', 4999), ('LAMP-01', 2999);",
        )
        .await?;

    // Parse and describe once...
    let cheaper_than = client
        .prepare("SELECT sku FROM products WHERE price_cents < $1 ORDER BY sku")
        .await?;
    // ...and the statement knows what the server worked out.
    assert_eq!(cheaper_than.params(), [Type::INT8]);
    assert_eq!(cheaper_than.columns()[0].name(), "sku");
    assert_eq!(cheaper_than.columns()[0].type_(), &Type::TEXT);

    // Run it many times: one round trip each.
    for (limit, expected) in [(9999_i64, 2), (3000, 1), (1000, 0)] {
        let rows = client.query(&cheaper_than, &[&limit]).await?;
        assert_eq!(rows.len(), expected);
    }

    // While the Statement lives, the server keeps it, in this session only.
    let prepared: Vec<String> = client
        .query_scalar("SELECT statement FROM pg_prepared_statements", &[])
        .await?;
    assert!(prepared.iter().any(|sql| sql.contains("price_cents < $1")));

    drop(cheaper_than); // sends Close: the server forgets it
    Ok(())
}
```

Every method that runs SQL accepts either text or a statement, through the `ToStatement` trait, implemented for `str`, `String` and `Statement`: `client.query("…", …)` and `client.query(&stmt, …)` are the same method. A `Statement` is cheap to clone (it is reference-counted), and the server-side statement is closed when the last clone is dropped.

A statement belongs to the session that prepared it. Use it on another connection and the server does not know the name:

```rust
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let one = playground::connect().await?;
    let other = playground::connect().await?;

    let statement = one.prepare("SELECT 1").await?;
    let error = other.query(&statement, &[]).await.unwrap_err();
    assert_eq!(error.code(), Some(&SqlState::INVALID_SQL_STATEMENT_NAME)); // 26000
    let message = error.as_db_error().unwrap().message();
    assert!(message.starts_with("prepared statement \"s"), "{message}");
    Ok(())
}
```

That is why a pool needs a statement cache *per connection* (§4.6, Chapter 13), and why a statement prepared through a pooled connection must not be kept after the connection goes back.

> **SQL lens:** a prepared statement saves the server work too: the parse, and for repeated executions possibly the plan. PostgreSQL plans each of the first five executions with the actual values (a *custom* plan), then may switch to a *generic* plan, made once without values, if it estimates that is not worse. For skewed data, such as a `status` column where 'pending' is rare and 'done' is everywhere, a generic plan can be a bad fit for some values. The `plan_cache_mode` setting (`force_custom_plan`) overrides the choice when you find such a case.

4.3 When the server cannot infer a type

The server infers each parameter's type from its context: from the column it is compared with or assigned to, or from an operator. Usually that works. When there is no context, preparing fails:

```rust
use tokio_postgres::error::SqlState;
use tokio_postgres::types::Type;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;

    // What the server infers from context:
    let inferred = client
        .prepare("SELECT $1 + 1, $2 || 'x', $3::int8, $4")
        .await?;
    assert_eq!(
        inferred.params(),
        [Type::INT4, Type::TEXT, Type::INT8, Type::TEXT]
    );

    // No context at all: the server gives up.
    let error = client.prepare("SELECT $1 IS NULL").await.unwrap_err();
    assert_eq!(error.code(), Some(&SqlState::INDETERMINATE_DATATYPE)); // 42P18
    assert_eq!(
        error.as_db_error().unwrap().message(),
        "could not determine data type of parameter $1"
    );

    // Fix 1: a cast in the SQL.
    let with_cast = client.prepare("SELECT $1::int8 IS NULL").await?;
    assert_eq!(with_cast.params(), [Type::INT8]);
```

```
    // Fix 2: tell prepare the types (the list may be shorter than the parameters; the
    // server infers the rest).
    let typed = client
        .prepare_typed("SELECT $1 IS NULL", &[Type::INT8])
        .await?;
    let is_null: bool = client.query_one(&typed, &[&None::<i64>]).await?.get(0);
    assert!(is_null);
    Ok(())
}
```

Note the last line of the inference examples: a bare `$4` becomes `text`. So does `SELECT $1`, which is why `client.query_one("SELECT $1", &[&42_i32])` fails on the client with "cannot convert between the Rust type `i32` and the Postgres type `text`". The server decided the type, and the Rust value has to match it, not the other way round.

A cast is usually the better fix: it documents the type where the parameter is, and it works the same in psql. `prepare_typed` is for SQL you do not control, and for the next section.

4.4 `query_typed`: one round trip, no statement

Sometimes you run a query once, or you build it dynamically and it is never the same twice. Preparing it first is a wasted round trip. `query_typed` (0.7.11+; `query_typed_one`, `query_typed_opt` and `execute_typed` since 0.7.17) skips the prepare step: you give each parameter's type along with its value, and tokio-postgres sends parse, bind and execute together, in one round trip, using an unnamed statement that nobody has to close:

```
use tokio_postgres::types::Type;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint);
             INSERT INTO products VALUES ('CHAIR-01', 4999), ('LAMP-01', 2999);",
        )
        .await?;

    // Each parameter comes with the PostgreSQL type it is sent as.
    let row = client
        .query_typed_one(
            "SELECT count(*) FROM products WHERE price_cents BETWEEN $1 AND $2",
            &[(&1000_i64, Type::INT8), (&3000_i64, Type::INT8)],
        )
        .await?;
    assert_eq!(row.get::<_, i64>(0), 1);

    // The types you declare must still fit the Rust values; this is checked before sending.
    let error = client
        .query_typed_one("SELECT $1 + 1", &[(&1_i64, Type::INT4)])
        .await
        .unwrap_err();
    assert_eq!(error.to_string(), "error serializing parameter 0");

    let changed = client
        .execute_typed(
            "UPDATE products SET price_cents = $1 WHERE sku = $2",
            &[(&3999_i64, Type::INT8), (&"CHAIR-01", Type::TEXT)],
        )
        .await?;
```

```
    assert_eq!(changed, 1);
    Ok(())
}
```

What you give up: the server parses and plans the statement on every call, and you are the one stating the types, so you can get them wrong. What you gain: one round trip instead of two, and no named statements on the server. That last point matters behind connection poolers that hand a different server connection to each transaction (PgBouncer in transaction mode, Chapter 13): a named statement prepared on one server connection does not exist on the next, and unnamed ones avoid the problem entirely.

Call	Round trips	Statement on the server
`query("…", params)`	2 (prepare, execute)	named, closed after the call
`prepare("…")` once, then `query(&stmt, params)`	1 + 1 per execution	named, kept while the `Statement` lives
`query_typed("…", params_with_types)`	1	unnamed, replaced by the next one
`simple_query("…")` / `batch_execute`	1	none (no parameters at all)

4.5 Counting round trips

On localhost the difference between those rows is invisible. To see it, put a slow network in the way: lagproxy, from this repository, forwards connections to PostgreSQL and holds every packet back for a few milliseconds in each direction. Start it, point `DATABASE_URL` at it, and the same program measures what each call costs:

```
cargo run -p lagproxy -- --upstream localhost:5432 --delay-ms 5    # listens on :6543
export DATABASE_URL=postgres://postgres:secret@127.0.0.1:6543/tpbook
export LAGPROXY_DELAY_MS=5
```

```
use std::time::{Duration, Instant};
use tokio_postgres::types::Type;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    // Through lagproxy: every round trip costs at least twice the delay.
    let client = playground::connect().await?;
    let delay: u64 = std::env::var("LAGPROXY_DELAY_MS").map_or(5, |v| v.parse().unwrap());
    let delay = Duration::from_millis(delay);
    const N: u32 = 10;

    let started = Instant::now();
    for i in 0..N as i64 {
        client.query_one("SELECT $1::int8 + 1", &[&i]).await?;
    }
    let as_text = started.elapsed() / N;

    let statement = client.prepare("SELECT $1::int8 + 1").await?;
    let started = Instant::now();
    for i in 0..N as i64 {
        client.query_one(&statement, &[&i]).await?;
    }
    let prepared = started.elapsed() / N;

    let started = Instant::now();
    for i in 0..N as i64 {
```

```
        client
            .query_typed_one("SELECT $1::int8 + 1", &[(&i, Type::INT8)])
            .await?;
    }
    let typed = started.elapsed() / N;

    println!("per query: text {as_text:?}, prepared {prepared:?}, typed {typed:?}");
    assert!(prepared >= delay * 2, "at least one round trip");
    assert!(as_text > prepared * 3 / 2, "about two round trips");
    assert!(typed < prepared * 3 / 2, "about one round trip");
    Ok(())
}
```

With a 5 ms delay each way, on the machine this book was written on:

```
per query: text 30.6ms, prepared 15.3ms, typed 15.3ms
```

A round trip through the proxy costs about 15 ms rather than 10, because a timer is never exactly on time (lagproxy's documentation explains the overshoot). The proportions are what count: passing SQL text costs two round trips, a prepared statement or `query_typed` one.

> **Pitfall:** "it's fast on my machine" is the most common way round trips hide. A request handler that runs ten small queries one after another takes 0.5 ms on localhost and 50 ms in production with a 5 ms network. Measure through lagproxy (or any latency injector) before you trust a performance number, and count the round trips of your hot paths. Chapter 7 shows how to run independent queries *without* waiting for each other.

4.6 Caching statements, and when the cache goes stale

Preparing every statement once per connection and reusing it is the way to pay one round trip per query without stating types by hand. A small cache does it: look the SQL up, prepare on a miss, keep the `Statement`.

```
use std::collections::HashMap;
use std::sync::Mutex;
use tokio_postgres::types::ToSql;
use tokio_postgres::{Client, Error, Row, Statement};

/// A client with a statement cache. One cache per connection: statements are per session.
struct CachingClient {
    client: Client,
    statements: Mutex<HashMap<String, Statement>>,
}

impl CachingClient {
    async fn prepare_cached(&self, sql: &str) -> Result<Statement, Error> {
        if let Some(statement) = self.statements.lock().unwrap().get(sql) {
            return Ok(statement.clone()); // cheap: reference-counted
        }
        // The lock is not held across the await. Two tasks may both miss and prepare the
        // same SQL; one of the statements is then dropped (and closed). Harmless.
        let statement = self.client.prepare(sql).await?;
        self.statements
            .lock()
            .unwrap()
            .insert(sql.to_owned(), statement.clone());
```

```
        Ok(statement)
    }

    async fn query(&self, sql: &str, params: &[&(dyn ToSql + Sync)]) -> Result<Vec<Row>,
Error> {
        let statement = self.prepare_cached(sql).await?;
        self.client.query(&statement, params).await
    }
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let db = CachingClient {
        client: playground::connect().await?,
        statements: Mutex::new(HashMap::new()),
    };
    for n in 0..5_i64 {
        let rows = db.query("SELECT $1::int8 * 2", &[&n]).await?;
        assert_eq!(rows[0].get::<_, i64>(0), n * 2);
    }
    assert_eq!(db.statements.lock().unwrap().len(), 1, "prepared once");
    Ok(())
}
```

You will rarely write this yourself: deadpool-postgres's pooled connections have exactly this cache built in, as `prepare_cached` (Chapter 13), and stockroom uses it everywhere. But a cache of prepared statements has a failure mode you need to know about. A statement's result columns are fixed when it is prepared. Change the table underneath, and the old statement refuses to run:

```
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE t (a int); INSERT INTO t VALUES (1)")
        .await?;
    let everything = client.prepare("SELECT * FROM t").await?;

    // A migration adds a column: `SELECT *` would now return two columns.
    client
        .batch_execute("ALTER TABLE t ADD COLUMN b int")
        .await?;
    let error = client.query(&everything, &[]).await.unwrap_err();
    assert_eq!(error.code(), Some(&SqlState::FEATURE_NOT_SUPPORTED)); // 0A000
    assert_eq!(
        error.as_db_error().unwrap().message(),
        "cached plan must not change result type"
    );

    // The cure is to prepare it again.
    let everything = client.prepare("SELECT * FROM t").await?;
    assert_eq!(client.query(&everything, &[]).await?[0].len(), 2);
    Ok(())
}
```

> **Pitfall:** "cached plan must not change result type" in production means a schema change reached a service that had statements prepared against the old schema: typically a migration run by one instance while others were serving. A per-connection cache keeps handing out the broken statement until the connection is replaced. Three habits prevent it. Never `SELECT *` in application code (an added column then changes nothing). Make migrations additive. And when you must change result types, clear the statement caches or recycle the pool's connections after migrating.

Side by side – prepared statements

The task: prepare a query with one parameter once, and run it with two different values.

```
-- psql: SQL-level PREPARE / EXECUTE, which any client can send...
tpbook=# PREPARE cheaper_than(bigint) AS
tpbook-#   SELECT sku FROM products WHERE price_cents < $1 ORDER BY sku;
PREPARE
tpbook=# EXECUTE cheaper_than(3000);
   sku
---------
 LAMP-01
(1 row)

tpbook=# DEALLOCATE cheaper_than;
DEALLOCATE
-- ...or, since psql 18, the protocol-level messages tokio-postgres uses:
tpbook=# SELECT sku FROM products WHERE price_cents < $1 ORDER BY sku \parse cheaper_than
tpbook=# \bind_named cheaper_than 9999 \g
tpbook=# \close_prepared cheaper_than
```

```
// sqlx (illustration)
// Every query is prepared on first use and cached on its connection, automatically.
let skus: Vec<String> = sqlx::query_scalar("SELECT sku FROM products WHERE price_cents < $1
ORDER BY sku")
    .bind(3000_i64)
    .fetch_all(&pool)
    .await?;
// .persistent(false) opts a query out of the cache.
```

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint);
             INSERT INTO products VALUES ('CHAIR-01', 4999), ('LAMP-01', 2999);",
        )
        .await?;
    let cheaper_than = client
        .prepare("SELECT sku FROM products WHERE price_cents < $1 ORDER BY sku")
        .await?;
    let cheap: Vec<String> = client
        .query(&cheaper_than, &[&3000_i64])
        .await?
        .iter()
        .map(|row| row.get(0))
        .collect();
    let all = client.query(&cheaper_than, &[&9999_i64]).await?;
    assert_eq!((cheap, all.len()), (vec!["LAMP-01".to_owned()], 2));
    Ok(())
}
```

	psql	sqlx	tokio-postgres
Same	A named, per-session statement, executed with values	Statements prepared per connection, executed with bound values	A named, per-session statement, executed with values
Different	`PREPARE` is SQL, parsed by the server like any statement; `\parse` (psql 18) uses the protocol	Caching is automatic and invisible: you never hold a statement	Explicit: you prepare and hold a `Statement` (or use deadpool's `prepare_cached`); text queries pay a second round trip; `query_typed` avoids both

In stockroom

Every repository function in stockroom prepares through the pool's cache, so each statement is parsed once per connection and every later call costs one round trip. The catalog's paginated listing:

```
// stockroom/src/repo/products.rs (excerpt)
pub async fn page(
    db: &impl GenericClient,
    after: Option<&Sku>,
    tag: Option<&str>,
    limit: i64,
) -> Result<Vec<Product>, AppError> {
    let stmt = db
        .prepare_cached(
            "SELECT id, sku, name, price_cents, tags, attributes, version, created_at,
updated_at
             FROM products
             WHERE sku > coalesce($1, '') AND ($2::text IS NULL OR tags @> ARRAY[$2])
             ORDER BY sku
             LIMIT $3",
        )
        .await?;
    from_rows(&db.query(&stmt, &[&after, &tag, &limit]).await?)
}
```

Look at the parameters through this chapter's eyes. `$3` gets its type, `bigint`, from `LIMIT`. `$1` is compared with a `sku` through `coalesce($1, '')`, which makes it `text`. And `$2` appears twice, so it carries an explicit cast at its first use, `$2::text`, because `$2 IS NULL` alone would be the "could not determine data type" error from §4.3. The column list is spelled out rather than `*`, so a migration that adds a column to `products` cannot break the cached statement.

stockroom also guards against the stale-statement problem at startup: `serve` runs the migrations *before* it creates the pool (app.rs), so no connection has prepared anything against the old schema.

The tiny psql in this repository uses `prepare` for something other than speed: its `--describe` mode prepares the statement and prints what the server worked out, without ever running it:

```
// apps/pgq/src/lib.rs (excerpt)
pub async fn describe(client: &Client, sql: &str) -> Result<String, Error> {
    let statement = client.prepare(sql).await?;
    // … renders statement.params() and statement.columns() as two tables …
}
```

```
$ cargo run -p pgq -- --describe -c 'SELECT relname, relpages FROM pg_class WHERE oid = $1'
 parameter | type
-----------+------
 $1        | oid

 column   | type | from table
----------+------+--------------------
 relname  | name | oid 1259, column 2
 relpages | int4 | oid 1259, column 10
```

Summary

- Passing SQL text to `query`, `execute` and friends costs two round trips: prepare (types in, types out) and execute. A temporary named statement is created and closed.
- `prepare` returns a `Statement` with `params()` and `columns()`. Executing it costs one round trip. Statements belong to one session: using one elsewhere fails with 26000.
- When a parameter's type cannot be inferred (42P18), add a cast (`$1::int8`) or use `prepare_typed`. The server decides parameter types; Rust values must fit them.
- `query_typed`, `query_typed_one`, `query_typed_opt` and `execute_typed` take the types with the values and need one round trip, with no named statement: good for one-off and dynamic SQL, and behind transaction-mode poolers.
- Cache statements per connection (deadpool's `prepare_cached` does). After a schema change, cached statements can fail with "cached plan must not change result type": avoid `SELECT *`, migrate additively, and re-prepare after migrating.
- Measure through added latency (lagproxy): round trips only show on a real network.

Exercises

1. **Count them yourself.** With lagproxy at a 10 ms delay, time a loop of 20 `execute` calls that each insert one row into a temporary table, first with SQL text and then with a prepared statement. Predict both times before you run it. *Hint:* at a 10 ms delay, a round trip through lagproxy measured about 26 ms.
2. **Infer it.** For each of these, predict the type the server gives `$1`, then check with `prepare` or `pgq --describe`: `SELECT $1 = 1`, `SELECT length($1)`, `SELECT now() - $1`, `SELECT ARRAY[$1, 2]`, `SELECT coalesce($1, 0.5)`. *Hint:* the server infers each parameter's type from where it appears; `Statement::params()` reports what it chose, and not every expression gives it enough to choose.
3. **A dynamic query in one round trip.** Rewrite Chapter 2's `search` (optional filters) with `query_typed`, so that every combination of filters costs one round trip. *Hint:* the parameter list becomes `Vec<(&(dyn ToSql + Sync), Type)>`.
4. **Stale.** Using the cache from §4.6, write a test that shows the stale-statement failure after `ALTER TABLE`, then make `CachingClient::query` recover by itself: on SQLSTATE 0A000, drop the cached statement, prepare it again and retry once. *Hint:* the error is `SqlState::FEATURE_NOT_SUPPORTED`

(0A000), "cached plan must not change result type"; Chapter 13 meets it again with deadpool's cache.

Solutions

1. About $2 \times 20 \times 26$ ms ≈ 1.04 s with text, and 20×26 ms ≈ 0.52 s (plus one round trip for `prepare`) with the statement. Every `execute` waits for its answer before the next one is sent, so the loop is a chain of round trips. Chapter 7 breaks the chain.

2. `$1 = 1` makes it `integer` (from the other operand), and `length($1)` makes it `text` (the most common `length` takes text). `now() - $1` makes it **`timestamptz`**, not `interval`: of the `-` operators with a `timestamptz` on the left, the server prefers the one whose right side has the same type, so `now() - $1` subtracts two timestamps and returns an interval. If you meant "now minus a duration", write `now() - $1::interval`. `ARRAY[$1, 2]` gives `integer`, and `coalesce($1, 0.5)` gives `numeric`, the type of the other argument. When the answer surprises you, cast.

4.

```
use std::collections::HashMap;
use std::sync::Mutex;
use tokio_postgres::error::SqlState;
use tokio_postgres::types::ToSql;
use tokio_postgres::{Client, Error, Row, Statement};

struct CachingClient {
    client: Client,
    statements: Mutex<HashMap<String, Statement>>,
}

impl CachingClient {
    async fn prepare_cached(&self, sql: &str) -> Result<Statement, Error> {
        if let Some(statement) = self.statements.lock().unwrap().get(sql) {
            return Ok(statement.clone());
        }
        let statement = self.client.prepare(sql).await?;
        self.statements
            .lock()
            .unwrap()
            .insert(sql.to_owned(), statement.clone());
        Ok(statement)
    }

    async fn query(&self, sql: &str, params: &[&(dyn ToSql + Sync)]) -> Result<Vec<Row>,
Error> {
        let statement = self.prepare_cached(sql).await?;
        match self.client.query(&statement, params).await {
            Err(e) if e.code() == Some(&SqlState::FEATURE_NOT_SUPPORTED) => {
                // Stale after a schema change: forget it, prepare again, retry once.
                self.statements.lock().unwrap().remove(sql);
                let statement = self.prepare_cached(sql).await?;
                self.client.query(&statement, params).await
            }
            other => other,
        }
    }
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let db = CachingClient {
        client: playground::connect().await?,
        statements: Mutex::new(HashMap::new()),
    };
```

```
    db.client
        .batch_execute("CREATE TEMP TABLE t (a int); INSERT INTO t VALUES (1)")
        .await?;
    assert_eq!(db.query("SELECT * FROM t", &[]).await?[0].len(), 1);
    db.client
        .batch_execute("ALTER TABLE t ADD COLUMN b int")
        .await?;
    assert_eq!(
        db.query("SELECT * FROM t", &[]).await?[0].len(),
        2,
        "recovered"
    );
    Ok(())
}
```

0A000 is "feature not supported", a broad code. Matching on it alone is acceptable here, because the retry is harmless; a stricter version also checks the message.

CHAPTER 5

Custom types: ToSql and FromSql

Chapter 3's table covers the types both languages have in common. But a well-designed schema has types of its own: an `order_status` enum, an `address` composite, a `sku` domain that only accepts valid codes. A well-designed Rust program has its own too: a `Cents` newtype that cannot be confused with a quantity, an `Email` that is known to be valid. This chapter connects the two.

There are two ways. You can implement the two conversion traits, `ToSql` and `FromSql`, by hand. That takes a dozen lines, and it is the way to add validation or to support a type nobody else supports. Or you can derive them for the common shapes: newtypes, enums, domains and composites. You will meet both in stockroom, and you will also meet the two things that make custom types tricky in practice: how domain types are described on the wire, and what happens when an enum changes underneath a running program.

5.1 The two traits

`ToSql` turns a Rust value into the bytes of a PostgreSQL value, for a given PostgreSQL type; `FromSql` does the reverse. Both live in the `postgres-types` crate, re-exported as `tokio_postgres::types`. Stripped of details:

```
pub trait ToSql: fmt::Debug {
    /// Write `self` in the binary format of `ty` to `out`.
    fn to_sql(&self, ty: &Type, out: &mut BytesMut) -> Result<IsNull, Box<dyn Error + Sync +
Send>>;
    /// Can this Rust type be sent as a value of `ty`?
    fn accepts(ty: &Type) -> bool;
    /// `to_sql`, after checking `accepts`; written for you by the `to_sql_checked!()`
macro.
    fn to_sql_checked(&self, ty: &Type, out: &mut BytesMut) -> Result<IsNull, Box<dyn Error
+ Sync + Send>>;
}

pub trait FromSql<'a>: Sized {
    /// Read a value of `ty` from its binary format in `raw`.
    fn from_sql(ty: &Type, raw: &'a [u8]) -> Result<Self, Box<dyn Error + Sync + Send>>;
    /// Can a value of `ty` be read as this Rust type?
    fn accepts(ty: &Type) -> bool;
    /// What NULL means. The default is an error (`WasNull`); `Option<T>` overrides it.
    fn from_sql_null(ty: &Type) -> Result<Self, Box<dyn Error + Sync + Send>> { … }
}
```

Three things are worth noting before you implement them.

- **`accepts` is the type check** you met in Chapter 3. tokio-postgres calls it with the type the server described, before encoding a parameter or decoding a column, and turns a `false` into the "cannot convert between the Rust type … and the Postgres type …" error.
- **Everything is binary.** tokio-postgres asks for binary values, so `raw` is the type's binary representation (big-endian integers, UTF-8 text, and so on), not text. You rarely write it by hand: delegate to the implementation of a type that already knows the format.
- **Errors are boxed.** Any error type converts into `Box<dyn Error + Sync + Send>`, including a `String`. tokio-postgres wraps yours as the cause of its own error.

5.2 A newtype by hand

The most common custom type is a validated newtype: an `Email` is a `String` that has been checked. It travels as text, so the implementation delegates encoding and decoding to `&str` and adds the check on the way out of the database as well as on the way in:

```
use bytes::BytesMut;
use postgres_types::{FromSql, IsNull, ToSql, Type, to_sql_checked};
use std::error::Error;

/// An email address, known to be well-formed.
#[derive(Debug, Clone, PartialEq)]
struct Email(String);

impl Email {
    fn parse(text: &str) -> Result<Email, String> {
        match text.split_once('@') {
            Some((user, host)) if !user.is_empty() && host.contains('.') => {
                Ok(Email(text.to_owned()))
            }
            _ => Err(format!("{text:?} is not an email address")),
        }
    }
}

impl ToSql for Email {
    fn to_sql(
        &self,
        ty: &Type,
        out: &mut BytesMut,
    ) -> Result<IsNull, Box<dyn Error + Sync + Send>> {
        self.0.as_str().to_sql(ty, out) // text's binary format: the UTF-8 bytes
    }

    fn accepts(ty: &Type) -> bool {
        <&str as ToSql>::accepts(ty) // text, varchar, …
    }

    to_sql_checked!();
}

impl<'a> FromSql<'a> for Email {
    fn from_sql(ty: &Type, raw: &'a [u8]) -> Result<Self, Box<dyn Error + Sync + Send>> {
        let text = <&str as FromSql>::from_sql(ty, raw)?;
        Ok(Email::parse(text)?) // a bad value in the database is an error, not an Email
    }

    fn accepts(ty: &Type) -> bool {
        <&str as FromSql>::accepts(ty)
    }
}
```

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE customers (id int, email text)")
        .await?;

    let ada = Email::parse("ada@example.com").expect("valid");
    client
        .execute("INSERT INTO customers VALUES (1, $1)", &[&ada])
        .await?;
    let back: Email = client
        .query_one("SELECT email FROM customers WHERE id = 1", &[])
        .await?
        .get(0);
    assert_eq!(back, ada);

    // Something not written by this program: the check runs on the way out too.
    client
        .execute("INSERT INTO customers VALUES (2, 'not an address')", &[])
        .await?;
    let row = client
        .query_one("SELECT email FROM customers WHERE id = 2", &[])
        .await?;
    let error = row.try_get::<_, Email>("email").unwrap_err();
    let cause = Error::source(&error).expect("our message").to_string();
    assert_eq!(cause, "\"not an address\" is not an email address");

    // accepts() keeps it to text-like types.
    let refused = client
        .execute("SELECT $1::int4", &[&ada])
        .await
        .unwrap_err();
    assert_eq!(refused.to_string(), "error serializing parameter 0");
    Ok(())
}
```

`to_sql_checked!()` writes the third method of `ToSql` for you: it calls `accepts`, returns the "cannot convert" error if it says no, and otherwise calls `to_sql`. Every hand-written `ToSql` ends with it.

> **Rust refresher:** `<&str as ToSql>::accepts(ty)` is the fully qualified way to call a trait function on a specific type: "`accepts`, as `&str` implements it in `ToSql`". You need it because `&str` implements both `ToSql::accepts` and `FromSql::accepts`, and the compiler cannot guess which one you mean from `<&str>::accepts`.

5.3 Deriving: `transparent` newtypes

Most newtypes need no validation at all: they exist so that the compiler can tell a price from a quantity. For those, derive the traits and say that the type is *transparent*, meaning that it travels exactly like its single field:

```
use postgres_types::{FromSql, ToSql};

/// Money, as a whole number of cents.
#[derive(Debug, Clone, Copy, PartialEq, ToSql, FromSql)]
#[postgres(transparent)]
struct Cents(i64);

#[tokio::main]
```

```
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE prices (sku text, price_cents bigint)")
        .await?;
    client
        .execute(
            "INSERT INTO prices VALUES ('CHAIR-01', $1)",
            &[&Cents(4999)],
        )
        .await?;

    let price: Cents = client
        .query_one("SELECT price_cents FROM prices", &[])
        .await?
        .get(0);
    assert_eq!(price, Cents(4999));

    // Transparent all the way down: arrays of Cents are bigint[].
    let doubled: Vec<Cents> = client
        .query_one(
            "SELECT array_agg(x * 2) FROM unnest($1::bigint[]) AS x",
            &[&vec![Cents(1), Cents(2)]],
        )
        .await?
        .get(0);
    assert_eq!(doubled, [Cents(2), Cents(4)]);
    Ok(())
}
```

`Cents` accepts exactly what `i64` accepts, `bigint`, and nothing else: sending it into an `integer` column is still a type error, which is the point of the newtype.

5.4 Enums

A PostgreSQL enum is a type with a fixed list of labels. A Rust enum of unit variants maps onto it with the derive. The type's name and the labels must match; `rename_all` saves you writing each label when they follow a convention:

```
use postgres_types::{FromSql, ToSql};

#[derive(Debug, Clone, Copy, PartialEq, ToSql, FromSql)]
#[postgres(name = "order_status", rename_all = "lowercase")]
enum OrderStatus {
    Pending,
    Paid,
    Cancelled,
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    // A temporary type, like the temporary tables: it disappears with the session.
    client
        .batch_execute(
            "CREATE TYPE pg_temp.order_status AS ENUM ('pending', 'paid', 'cancelled');
             CREATE TEMP TABLE orders (id int, status order_status);
             INSERT INTO orders VALUES (1, 'pending'), (2, 'paid'), (3, 'cancelled');",
        )
        .await?;

    client
        .execute(
            "UPDATE orders SET status = $1 WHERE id = 1",
```

```
            &[&OrderStatus::Paid],
        )
        .await?;

    // An array of enum values for = ANY, and enum values back.
    let open = vec![OrderStatus::Pending, OrderStatus::Paid];
    let rows = client
        .query(
            "SELECT id, status FROM orders WHERE status = ANY($1) ORDER BY id",
            &[&open],
        )
        .await?;
    let found: Vec<(i32, OrderStatus)> = rows
        .iter()
        .map(|row| (row.get("id"), row.get("status")))
        .collect();
    assert_eq!(found, [(1, OrderStatus::Paid), (2, OrderStatus::Paid)]);
    Ok(())
}
```

The derive's `accepts` checks the type's *name* (`order_status`, in any schema) and its *labels*, which tokio-postgres looked up from the server the first time it met the type. By default the labels must match the variants exactly: same set, nothing extra on either side. That strictness catches a typo in a `rename` on the first query. It also has a consequence that bites in production:

```
use postgres_types::{FromSql, ToSql};

#[derive(Debug, Clone, Copy, PartialEq, ToSql, FromSql)]
#[postgres(name = "order_status", rename_all = "lowercase")]
enum Strict {
    Pending,
    Paid,
}

#[derive(Debug, Clone, Copy, PartialEq, ToSql, FromSql)]
#[postgres(name = "order_status", rename_all = "lowercase", allow_mismatch)]
enum Lenient {
    Pending,
    Paid,
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TYPE pg_temp.order_status AS ENUM ('pending', 'paid')")
        .await?;
    let paid = "SELECT 'paid'::order_status";
    assert_eq!(
        client.query_one(paid, &[]).await?.get::<_, Strict>(0),
        Strict::Paid
    );

    // A migration adds a label the program does not know yet.
    client
        .batch_execute("ALTER TYPE order_status ADD VALUE 'refunded'")
        .await?;
    // A connection that loads the type afresh (any new connection, or this one after
    // clearing its type cache) now sees three labels where the strict enum has two:
    client.clear_type_cache();
    let row = client.query_one(paid, &[]).await?;
    assert!(
        row.try_get::<_, Strict>(0).is_err(),
        "even 'paid' is refused"
    );

    // allow_mismatch only fails on a label the enum lacks, when a row actually holds it.
```

```
    assert_eq!(row.get::<_, Lenient>(0), Lenient::Paid);
    let refunded = client
        .query_one("SELECT 'refunded'::order_status", &[])
        .await?;
    let error = refunded.try_get::<_, Lenient>(0).unwrap_err();
    let cause = std::error::Error::source(&error).unwrap().to_string();
    assert_eq!(cause, "invalid variant `refunded`");
    Ok(())
}
```

> **Pitfall:** with a strict enum, `ALTER TYPE … ADD VALUE` in a migration breaks every running instance of your service, gradually: each new pooled connection loads the new label list and then refuses *every* value of the type, not just the new one. The code and the database would have to change at the same instant, which never happens. For enums that can grow, add `allow_mismatch` and deploy in either order: the database first (the old code fails only on rows with the new label), or the code first (the new variant is simply never sent).

5.5 Domains

A domain is a base type with a constraint: `CREATE DOMAIN sku AS text CHECK (…)`. You would expect it to behave exactly like its base type, and on the way *out* it does: a column of type `sku` is described as `text`. On the way *in*, it does not. A parameter that goes into a `sku` column is described as the domain itself, and plain Rust types refuse it:

```
use postgres_types::{FromSql, ToSql};
use tokio_postgres::types::{Kind, Type};

/// Derived for a domain: a one-field tuple struct with the domain's name.
#[derive(Debug, Clone, PartialEq, ToSql, FromSql)]
#[postgres(name = "sku")]
struct Sku(String);

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE DOMAIN pg_temp.sku AS text CHECK (VALUE ~ '^[A-Z0-9]{3,12}(-[A-Z0-9]
{1,6})?$');
             CREATE TEMP TABLE products (sku sku PRIMARY KEY, name text);",
        )
        .await?;

    let insert = client
        .prepare("INSERT INTO products VALUES ($1, $2)")
        .await?;
    // $1 targets the sku column: it is described as the domain, not as text.
    assert_eq!(insert.params()[0].name(), "sku");
    assert!(matches!(insert.params()[0].kind(), Kind::Domain(base) if *base == Type::TEXT));

    // So a plain &str is refused, by the client, before anything is sent...
    let refused = client
        .execute(&insert, &[&"CHAIR-01", &"Chair"])
        .await
        .unwrap_err();
    assert!(refused.as_db_error().is_none());
```

```
    // ...while the derived domain type is accepted, and the server checks the constraint.
    client
        .execute(&insert, &[&Sku("CHAIR-01".into()), &"Chair"])
        .await?;
    let bad = client
        .execute(&insert, &[&Sku("chair".into()), &"Chair"])
        .await
        .unwrap_err();
    assert_eq!(bad.code().map(|c| c.code()), Some("23514")); // check_violation

    // A cast makes the parameter plain text again, if you prefer &str.
    client
        .execute(
            "INSERT INTO products VALUES ($1::text, $2)",
            &[&"LAMP-01", &"Lamp"],
        )
        .await?;

    // On the way out, the column is described as text, and both types read it.
    let row = client
        .query_one("SELECT sku FROM products WHERE name = 'Lamp'", &[])
        .await?;
    assert_eq!(row.columns()[0].type_(), &Type::TEXT);
    assert_eq!(row.get::<_, Sku>(0), Sku("LAMP-01".into()));
    assert_eq!(row.get::<_, &str>(0), "LAMP-01");
    Ok(())
}
```

So for each domain you choose one of three approaches:

Approach	Parameters	Results	When
A derived domain new-type, `#[postgres(name = "…")] struct T(Base)`	accepted as the domain	read from the base type too	the domain has a Rust type of its own
A hand-written `accepts` that takes the domain *and* the base type	accepted either way	accepted either way	you also want validation in Rust (stockroom's `Sku`)
A cast in the SQL, `$1::text`	the base type	the base type	occasional use, or SQL you do not control

Comparisons are not affected: in `WHERE sku = $1` the server types `$1` as the base type, so a `&str` works there.

> **SQL lens:** domains are a cheap way to give a constraint a name and reuse it across tables, and the server enforces it everywhere the domain is used. One limit is new in PostgreSQL 18: a *virtual* generated column (`GENERATED ALWAYS AS (…) VIRTUAL`) cannot use a user-defined type, and a domain counts, so a virtual column cannot be computed from domain-typed columns. stockroom's `stock.available` ran into exactly that.

5.6 Composite types

A composite type is a named record: `CREATE TYPE address AS (street text, city text, …)`. Tables can have composite columns, and a query can produce a composite value with `ROW(…)`. A Rust struct with named fields maps onto it, field by field, by name:

```
use postgres_types::{FromSql, ToSql};

#[derive(Debug, Clone, PartialEq, ToSql, FromSql)]
#[postgres(name = "address")]
struct Address {
    street: String,
    city: String,
    postcode: Option<String>, // a field that may be NULL
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TYPE pg_temp.address AS (street text, city text, postcode text);
             CREATE TEMP TABLE stores (code text, address address);",
        )
        .await?;

    let mitte = Address {
        street: "Main 1".into(),
        city: "Berlin".into(),
        postcode: Some("10115".into()),
    };
    client
        .execute("INSERT INTO stores VALUES ('mitte', $1)", &[&mitte])
        .await?;

    // The whole value, or one field of it, in SQL.
    let row = client
        .query_one(
            "SELECT address, (address).city AS city FROM stores WHERE code = 'mitte'",
            &[],
        )
        .await?;
    assert_eq!(row.get::<_, Address>("address"), mitte);
    assert_eq!(row.get::<_, &str>("city"), "Berlin");

    // Arrays of composites work like any array.
    let more = vec![
        Address {
            street: "Ring 2".into(),
            city: "Potsdam".into(),
            postcode: None,
        },
        Address {
            street: "Allee 3".into(),
            city: "Leipzig".into(),
            postcode: Some("04109".into()),
        },
    ];
    let codes = vec!["ring", "allee"];
    // unnest in the select list keeps each address whole...
    client
        .execute(
            "INSERT INTO stores SELECT unnest($1::text[]), unnest($2::address[])",
            &[&codes, &more],
        )
        .await?;
    // ...while in FROM, a function returning a composite is expanded into its fields.
    let error = client
```

```
        .execute(
            "INSERT INTO stores SELECT * FROM unnest($1::text[], $2::address[])",
            &[&codes, &more],
        )
        .await
        .unwrap_err();
    assert_eq!(
        error.as_db_error().unwrap().message(),
        "INSERT has more expressions than target columns"
    );

    let count: i64 = client
        .query_one("SELECT count(*) FROM stores", &[])
        .await?
        .get(0);
    assert_eq!(count, 3);
    Ok(())
}
```

The derive matches fields by name, so their order in the struct does not matter, and it checks that the type has exactly these fields. A field that can be NULL needs an `Option`, just like a column. `#[postgres(name = "…")]` on a field renames it, and `rename_all` works here too.

5.7 Behind the scenes: `Type`, `Kind` and the type cache

Built-in types are constants: `Type::INT8`, `Type::TEXT`. A type the server defines at run time (an enum, a domain, a composite, or an extension's type) has an OID that tokio-postgres cannot know in advance. The first time it meets one in a statement's description, it looks the type up in the server's catalog, with a few extra queries, and builds a `Type` whose `kind()` describes it:

`Kind`	What it is
`Simple`	A base type, such as `int8` or `text`
`Enum(labels)`	An enum, with its labels
`Domain(base)`	A domain over `base`
`Composite(fields)`	A composite type, with each field's name and type
`Array(member)`	An array of `member`
`Range(member)`, `Multirange(member)`	A range type
`Pseudo`	A pseudo-type such as `void` or `record`

The derives' `accepts` read exactly this: the enum's labels, the domain's base type, the composite's fields.

These lookups are cached *per connection*, for the connection's lifetime. That is why the strict enum in §5.4 kept working until the cache was cleared: the connection still held the old label list. Two consequences follow:

- After `ALTER TYPE` (new enum labels, changed composite fields), connections that existed before keep the old description until they are closed or call `Client::clear_type_cache()`. stockroom's migration runner calls it after applying migrations, for its own connection.
- The first query on a fresh connection that involves a custom type pays for the lookup: a few extra round trips, once. A pool hides this cost behind connection reuse (Chapter 13).

Side by side — a PostgreSQL enum

The task: define an `order_status` enum, store a value, and read it back as a typed value.

```
-- psql
tpbook=# CREATE TYPE order_status AS ENUM ('pending', 'paid', 'cancelled');
tpbook=# \dT+ order_status
                                       List of data types
 Schema |     Name     | Internal name | Size | Elements  |  Owner   | Access privileges |
Description
--------+--------------+---------------+------+-----------+----------+-------------------
+-------------
 public | order_status | order_status  | 4    | pending  +| postgres |                   |
        |              |               |      | paid     +|          |                   |
        |              |               |      | cancelled |          |                   |
(1 row)

tpbook=# SELECT 'paid'::order_status < 'cancelled';   -- labels are ordered as declared
 ?column?
----------
 t
(1 row)
```

```
// sqlx (illustration)
#[derive(sqlx::Type)]
#[sqlx(type_name = "order_status", rename_all = "lowercase")]
enum OrderStatus { Pending, Paid, Cancelled }

let status: OrderStatus = sqlx::query_scalar("SELECT
'paid'::order_status").fetch_one(&pool).await?;
```

```
use postgres_types::{FromSql, ToSql};

#[derive(Debug, Clone, Copy, PartialEq, ToSql, FromSql)]
#[postgres(name = "order_status", rename_all = "lowercase")]
enum OrderStatus {
    Pending,
    Paid,
    Cancelled,
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TYPE pg_temp.order_status AS ENUM ('pending', 'paid',
'cancelled')")
        .await?;
    let status: OrderStatus = client
        .query_one("SELECT $1::order_status", &[&OrderStatus::Paid])
        .await?
        .get(0);
    assert_eq!(status, OrderStatus::Paid);
    let pending = OrderStatus::Pending;
    assert_ne!(status, pending);
    Ok(())
}
```

	psql	sqlx	tokio-postgres
Same	An enum is a type with ordered labels	A derive maps the type name and the labels	A derive maps the type name and the labels
Different	Labels are text you type	Matches labels by name when encoding and decoding	By default, the label *set* must match exactly, checked against

psql	sqlx	tokio-postgres
		the labels looked up from the server (per connection); `allow_mismatch` relaxes that to per-value

In stockroom

stockroom's domain/ module has one of each kind. `Cents` is a transparent newtype:

```
// stockroom/src/domain/cents.rs (excerpt)
#[serde(transparent)]
#[postgres(transparent)]
pub struct Cents(pub i64);
```

`OrderStatus` is the enum, with `allow_mismatch` and the reason written next to it:

```
// stockroom/src/domain/order_status.rs (excerpt)
/// `allow_mismatch` matters in production. Without it, the derive accepts the database type
/// only if its labels are *exactly* these five: a migration that adds a sixth status would
/// make every new connection refuse every `order_status` value, `'paid'` included, until
/// this enum gets the variant too. With it, only an unknown label fails, and only when a
row
/// actually holds it, so the database can be migrated ahead of the code.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Serialize, Deserialize, ToSql, FromSql)]
#[serde(rename_all = "lowercase")]
#[postgres(name = "order_status", rename_all = "lowercase", allow_mismatch)]
pub enum OrderStatus {
    Pending,
    Paid,
    Ready,
    Collected,
    Cancelled,
}
```

A test in tests/types.rs pins that behaviour down: it adds a label to the database and checks that fresh connections can still read the others.

`Quantity` is a derived domain newtype for `positive_quantity` (which order lines use), with its own validation on the Rust side through serde. `Address` is the composite. `Sku` is written by hand, because it wants both a domain-aware `accepts` and validation when reading:

```
// stockroom/src/domain/sku.rs (excerpt)
/// `text`, `varchar` and friends, or the `sku` domain over one of them.
fn accepts_sku(ty: &Type) -> bool {
    match ty.kind() {
        Kind::Domain(base) => ty.name() == "sku" && <&str as ToSql>::accepts(base),
        _ => <&str as ToSql>::accepts(ty),
    }
}

impl ToSql for Sku {
    fn to_sql(
        &self,
        _ty: &Type,
        out: &mut BytesMut,
    ) -> Result<IsNull, Box<dyn Error + Sync + Send>> {
        // The binary format of `text` (and so of a domain over it) is just the UTF-8 bytes.
```

```
        out.extend_from_slice(self.0.as_bytes());
        Ok(IsNull::No)
    }

    fn accepts(ty: &Type) -> bool {
        accepts_sku(ty)
    }

    to_sql_checked!();
}
```

Instead of delegating, `to_sql` writes the bytes itself, which shows how simple text's binary format is. The seed command inserts stores as three arrays zipped by `unnest`, one of them an array of the composite, and its comment records §5.6's lesson:

```
// stockroom/src/cli/seed.rs (excerpt)
let store_ids: Vec<Uuid> = tx
    .query(
        "INSERT INTO stores (code, name, address)
         SELECT unnest($1::text[]), unnest($2::text[]), unnest($3::address[])
         RETURNING id",
        &[&codes, &names, &addresses],
    )
    .await?
```

Summary

- `ToSql` and `FromSql` encode and decode the binary format of a PostgreSQL type; `accepts` is the type check behind "cannot convert". End a hand-written `ToSql` with `to_sql_checked!()`.
- Hand-written implementations usually delegate to an existing type and add validation. A `FromSql` that validates turns bad data into a conversion error.
- Derives cover newtypes (`transparent`), enums (`name`, `rename_all`, `allow_mismatch`), domains (a one-field tuple struct with the domain's `name`) and composites (named fields).
- A derived enum checks the whole label set by default; for enums that grow, use `allow_mismatch`, or every new connection refuses every value after a migration.
- A domain-typed parameter is described as the domain, so plain base types are refused; result columns are described as the base type.
- In `FROM`, `unnest` of a composite array expands the fields; in the select list it does not.
- Custom types are looked up once per connection and cached; `clear_type_cache` forgets them.

Exercises

1. **A percentage.** Write `struct Percent(u8)` that maps to `smallint`, accepts 0 to 100 on both sides, and refuses anything else with a clear message. *Hint:* delegate to `i16`'s implementations, and convert with `i16::from` and `u8::try_from`.
2. **Evolve an enum.** With a strict enum and a real (not temporary) type, open two connections, add a label with one of them, and check what each connection can read. Explain the difference. *Hint:* each connection caches the types it has looked up; `clear_type_cache` empties that cache.

3. **A composite with a nested composite.** Make a `geo` composite `(lat float8, lon float8)` and an `address` composite that has a `location geo` field. Derive both in Rust, and round trip a value. *Hint:* derive `ToSql` and `FromSql` on both structs; the outer struct's `geo` field converts through the inner derive.
4. **`interval`, by hand.** tokio-postgres has no mapping for `interval`. Its binary format is 16 bytes: a big-endian `i64` of microseconds, then an `i32` of days, then an `i32` of months. Write `struct PgInterval { months: i32, days: i32, microseconds: i64 }` with a `FromSql`, and read `interval '1 year 2 months 3 days 04:05:06.5'`. *Hint:* `postgres_types::accepts!(INTERVAL)` writes `accepts` for you.

Solutions

2. The connection that ran `ALTER TYPE` still holds the type's old description in its cache, so its strict enum keeps working (until it calls `clear_type_cache`). The second connection behaves the same *if* it had already used the type. A third connection opened afterwards looks the type up fresh, sees one label too many, and refuses every value. That is how a migration breaks a service gradually, as its pool replaces connections.

4.

```rust
use postgres_types::{FromSql, Type, accepts};
use std::error::Error;

#[derive(Debug, PartialEq)]
struct PgInterval {
    months: i32,
    days: i32,
    microseconds: i64,
}

impl<'a> FromSql<'a> for PgInterval {
    fn from_sql(_: &Type, raw: &'a [u8]) -> Result<Self, Box<dyn Error + Sync + Send>> {
        let raw: &[u8; 16] = raw.try_into()?; // anything else is an error, not a panic
        Ok(PgInterval {
            microseconds: i64::from_be_bytes(raw[0..8].try_into()?),
            days: i32::from_be_bytes(raw[8..12].try_into()?),
            months: i32::from_be_bytes(raw[12..16].try_into()?),
        })
    }

    accepts!(INTERVAL);
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let interval: PgInterval = client
        .query_one("SELECT interval '1 year 2 months 3 days 04:05:06.5'", &[])
        .await?
        .get(0);
    assert_eq!(
        interval,
        PgInterval {
            months: 14,
            days: 3,
            microseconds: ((4 * 60 + 5) * 60 + 6) * 1_000_000 + 500_000,
        }
    );
    Ok(())
}
```

The three parts stay separate because they are not interchangeable. A month is not a fixed number of days, and a day is not always 24 hours across a daylight-saving change, so PostgreSQL keeps them apart and so should you. Converting to a single `Duration` is only correct when `months` and `days` are zero.

CHAPTER 6

Errors

Every chapter so far has produced errors: a refused connection, a wrong type, a missing row, a violated constraint. They all arrive as the same Rust type, `tokio_postgres::Error`, and the first thing to learn is that its `Display` text almost never tells you what happened. The information is there, one level down, in a structure the server filled in with remarkable care: a five-character SQLSTATE code, the constraint and table involved, a detail, a hint, the position of a syntax error.

This chapter takes the error apart: its causes, the server's `DbError` and its fields, and the `SqlState` codes worth matching on. Then it builds what an application needs on top: one error type for your program, classified once, with messages a user can be shown and details that go only to the log. It ends with two things that are not errors in the usual sense: a transaction that refuses to continue after one, and notices.

6.1 One type, many causes

`tokio_postgres::Error` covers everything that can go wrong, from a refused TCP connection to a deadlock detected by the server. Internally it records a *kind* and an optional *cause*. The kind is private, so what you can ask an error is limited to four questions:

Method	Answers
`as_db_error()`	Did the server report this? Then here is its `DbError`
`code()`	The SQLSTATE, if the server reported it (a shortcut through `as_db_error`)
`is_closed()`	Is it because the connection is closed?
`source()` (from `std::error::Error`)	The underlying cause: the `DbError`, an `io::Error`, a conversion error

Here is what each kind of failure looks like from the outside, as observed on PostgreSQL 18:

What happened	`Display`	`code()`	`source()`
A constraint, a syntax error, a deadlock: anything the server reports	`db error`	the SQLSTATE	the server's message, formatted as `ERROR: … DETAIL: …`
The connection was refused	`error connecting to server`	—	`Connection refused (os error 61)`
The connection is closed	`connection closed`	—	—

What happened	`Display`	`code()`	`source()`
A value did not fit the column's type (reading)	`error deserializing column 0`	—	`cannot convert between the Rust type … and the Postgres type …`
A value did not fit the parameter's type (sending)	`error serializing parameter 0`	—	the same "cannot convert"
No such column	`invalid column `x``	—	—
Wrong number of parameters	`expected 1 parameters but got 0`	—	—
`query_one`/`query_opt` saw the wrong number of rows	`query returned an unexpected number of rows`	—	—
A connection string that does not parse	`invalid connection string`	—	`invalid value for option `port``

So `Display` is a category, and the explanation is the cause. To see both, walk the chain:

```
use std::error::Error;

/// An error and every cause under it.
fn report(error: &dyn Error) -> String {
    let mut text = error.to_string();
    let mut cause = error.source();
    while let Some(c) = cause {
        text.push_str(&format!("\n  caused by: {c}"));
        cause = c.source();
    }
    text
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint);
             INSERT INTO products VALUES ('CHAIR-01', 4999);",
        )
        .await?;

    // From the server: the DbError is the cause.
    let duplicate = client
        .execute("INSERT INTO products VALUES ('CHAIR-01', 1)", &[])
        .await
        .unwrap_err();
    let text = report(&duplicate);
    assert!(text.starts_with("db error\n  caused by: ERROR: duplicate key value"));
    assert!(text.contains("DETAIL: Key (sku)=(CHAIR-01) already exists."));

    // From the client: the conversion error is the cause.
    let row = client
        .query_one("SELECT price_cents FROM products", &[])
        .await?;
    let wrong = row.try_get::<_, i32>(0).unwrap_err();
    assert_eq!(
        report(&wrong),
        "error deserializing column 0\n  caused by: cannot convert between \
         the Rust type `i32` and the Postgres type `int8`"
    );

    // Some errors have no cause at all: the category is the whole story.
    let none = client
        .query_one("SELECT 1 WHERE false", &[])
```

```
        .await
        .unwrap_err();
    assert_eq!(report(&none), "query returned an unexpected number of rows");
    assert!(none.as_db_error().is_none() && none.code().is_none());
    Ok(())
}
```

Logging `{error:?}` (`Debug`) shows the kind and the cause too, in a more technical form: `Error { kind: FromSql(0), cause: Some(WrongType { postgres: Int8, rust: "i32" }) }`. Either is fine for a log; `{error}` alone is not.

> **Pitfall:** there is no public way to ask "was this the row-count error?" or "was this a conversion error?". Only the server's errors have a code. Design so that you do not need to ask: use `query_opt` and handle `None` instead of catching `query_one`'s failure, and treat a conversion error as the bug it is.

6.2 `DbError`: the server's report

When the server reports an error, it sends far more than a message, and `DbError` keeps all of it. The fields that matter most:

Method	Content
`code()`	The SQLSTATE: five characters, the first two being the class (§6.3)
`message()`	The primary message, one line, meant for a person
`detail()`	More detail, often with the offending values: `Key (sku)=(CHAIR-01) already exists.`
`hint()`	A suggestion for fixing it, when the server has one
`severity()` / `parsed_severity()`	`ERROR` or `FATAL` (the session is over) or `PANIC`, as text or as a `Severity`
`position()`	For an error in the statement's text: where, as a 1-based character position
`schema()`, `table()`, `column()`, `datatype()`, `constraint()`	What the error is about, by name
`where_()`	The call stack, for errors raised inside functions and triggers
`routine()`, `file()`, `line()`	Where in PostgreSQL's own source the error was raised

```
use tokio_postgres::error::{ErrorPosition, Severity, SqlState};

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 sku         text   PRIMARY KEY,
```

```
                name        text   NOT NULL,
                price_cents bigint CONSTRAINT price_positive CHECK (price_cents > 0)
            )",
        )
        .await?;

    // A CHECK constraint: which one, on which table, and the row that broke it.
    let error = client
        .execute("INSERT INTO products VALUES ('LAMP-01', 'Lamp', 0)", &[])
        .await
        .unwrap_err();
    let db = error.as_db_error().unwrap();
    assert_eq!(db.code(), &SqlState::CHECK_VIOLATION);
    assert_eq!(db.constraint(), Some("price_positive"));
    assert_eq!(db.table(), Some("products"));
    assert_eq!(
        db.detail(),
        Some("Failing row contains (LAMP-01, Lamp, 0).")
    );
    assert_eq!(db.parsed_severity(), Some(Severity::Error));

    // NOT NULL names the column instead of a constraint.
    let error = client
        .execute("INSERT INTO products VALUES ('LAMP-01', NULL, 1)", &[])
        .await
        .unwrap_err();
    let db = error.as_db_error().unwrap();
    assert_eq!(db.code(), &SqlState::NOT_NULL_VIOLATION);
    assert_eq!(db.column(), Some("name"));

    // A syntax error points at the place in the statement.
    let error = client.execute("SELEC 1", &[]).await.unwrap_err();
    let db = error.as_db_error().unwrap();
    assert_eq!(db.code(), &SqlState::SYNTAX_ERROR);
    assert!(matches!(db.position(), Some(ErrorPosition::Original(1))));

    // Your own errors, raised in SQL, with your own code, detail and hint.
    let error = client
        .batch_execute(
            "DO $$ BEGIN
                 RAISE EXCEPTION 'order % is already collected', 42
                     USING ERRCODE = 'SR001', DETAIL = 'collected at 12:00',
                           HINT = 'only open orders can be cancelled';
             END $$",
        )
        .await
        .unwrap_err();
    let db = error.as_db_error().unwrap();
    assert_eq!(db.code().code(), "SR001");
    assert_eq!(db.message(), "order 42 is already collected");
    assert_eq!(db.hint(), Some("only open orders can be cancelled"));
    assert!(db.where_().unwrap().contains("at RAISE"));
    Ok(())
}
```

That last example matters more than it looks. A trigger or a function can enforce a rule the schema cannot express, and report it with a code of your own. The SQL standard reserves classes starting with `0` to `4` and `A` to `H`; the rest are for implementations, and PostgreSQL itself uses several of them (`53`, `55`, `57`, `P0`, `XX`). A class it does not use, such as `SR` above, is yours, and as easy for your Rust code to match as a built-in code.

> **Pitfall:** `detail()` often contains the data that caused the error: the duplicate email address, the failing row. Returning it to an API client can leak another customer's data, and logging it can put personal data in your logs. Decide deliberately where each field goes; §6.5 shows a policy.

6.3 `SqlState`: the codes worth matching

A SQLSTATE is five characters. The first two are a *class* that groups related errors; the whole code names one condition. tokio-postgres has a constant for each code PostgreSQL defines, and the constants work directly as patterns in a `match`. Most applications need a dozen:

Class	Code	`SqlState` constant	Usually means
`23` integrity	`23505`	`UNIQUE_VIOLATION`	duplicate: 409 Conflict
	`23503`	`FOREIGN_KEY_VIOLATION`	a reference to something that does not exist (or is still referenced)
	`23502`	`NOT_NULL_VIOLATION`	a missing required value
	`23514`	`CHECK_VIOLATION`	a rule of the data: 422
	`23P01`	`EXCLUSION_VIOLATION`	an overlapping booking (exclusion constraints)
`40` transaction rollback	`40001`	`T_R_SERIALIZATION_FAILURE`	a concurrent transaction won: **retry** (Chapter 8)
	`40P01`	`T_R_DEADLOCK_DETECTED`	a deadlock, and you were chosen to fail: **retry**
`55` object state	`55P03`	`LOCK_NOT_AVAILABLE`	`lock_timeout` or `NOWAIT`: retry later
`57` operator intervention	`57014`	`QUERY_CANCELED`	`statement_timeout` or a cancel request (Chapter 12)
	`57P01`	`ADMIN_SHUTDOWN`	the server is shutting down, or your session was terminated
`53` resources	`53300`	`TOO_MANY_CONNECTIONS`	`max_connections` reached
`25` transaction state	`25P02`	`IN_FAILED_SQL_TRANSACTION`	an earlier error aborted this transaction (§6.6)
`42` syntax and access	`42P01`, `42703`	`UNDEFINED_TABLE`, `UNDEFINED_COLUMN`	a bug, or a schema the code does not match

Matching by code is precise; matching by class handles groups. A small classifier shows both:

```
use tokio_postgres::Error;
use tokio_postgres::error::SqlState;

#[derive(Debug, PartialEq)]
enum Verdict {
    Duplicate,
    BadInput,
    RetryNow,
```

```
    RetryLater,
    TooSlow,
    Bug,
    Unavailable,
}

fn classify(error: &Error) -> Verdict {
    let Some(code) = error.code() else {
        return if error.is_closed() {
            Verdict::Unavailable
        } else {
            Verdict::Bug // client-side: conversions, row counts, parameters
        };
    };
    match *code {
        SqlState::UNIQUE_VIOLATION => Verdict::Duplicate,
        SqlState::T_R_SERIALIZATION_FAILURE | SqlState::T_R_DEADLOCK_DETECTED =>
Verdict::RetryNow,
        SqlState::LOCK_NOT_AVAILABLE => Verdict::RetryLater,
        SqlState::QUERY_CANCELED => Verdict::TooSlow,
        // The rest of class 23 (integrity) and class 22 (data exceptions, like a bad
number).
        _ if code.code().starts_with("23") || code.code().starts_with("22") =>
Verdict::BadInput,
        // Class 08 (connection), 53 (resources), 57P0x (shutting down).
        _ if code.code().starts_with("08") || code.code().starts_with("53") =>
Verdict::Unavailable,
        SqlState::ADMIN_SHUTDOWN | SqlState::CANNOT_CONNECT_NOW => Verdict::Unavailable,
        _ => Verdict::Bug,
    }
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE t (id int PRIMARY KEY, n int CHECK (n > 0));
             INSERT INTO t VALUES (1, 1);",
        )
        .await?;
    let verdict = |sql: &'static str| {
        let client = &client;
        async move { classify(&client.execute(sql, &[]).await.unwrap_err()) }
    };
    assert_eq!(
        verdict("INSERT INTO t VALUES (1, 1)").await,
        Verdict::Duplicate
    );
    assert_eq!(
        verdict("INSERT INTO t VALUES (2, 0)").await,
        Verdict::BadInput
    );
    assert_eq!(verdict("SELECT 1 / 0").await, Verdict::BadInput); // 22012
    assert_eq!(verdict("SELECT * FROM nowhere").await, Verdict::Bug); // 42P01
    assert_eq!(
        verdict("SET statement_timeout = 10; SELECT pg_sleep(1)").await,
        Verdict::Bug,
        "two statements in one prepared statement: 42601"
    );
    Ok(())
}
```

Notice the last case: it looks like a timeout test, but `execute` uses the extended protocol, which takes one statement, so the server refused the text before anything ran. Even a classifier needs tests.

> **Rust refresher:** `match *code { SqlState::UNIQUE_VIOLATION => … }` matches a value against a *constant pattern*. It works because `SqlState`'s constants are plain values of a type that derives `PartialEq` and `Eq`, so the compiler can compare structurally. The `*` dereferences the `&SqlState` that `code()` returns; you could also write `Some(&SqlState::…)` against `error.code()` directly.

6.4 `is_closed`, and the error you cannot classify

`is_closed()` is true when the connection is gone: the server restarted, a network device dropped the TCP connection, an administrator terminated the session. Nothing on this client will work again, and the remedy is a new connection, which a pool does for you (Chapter 13).

There is one situation no classification can resolve. If the connection breaks while a `COMMIT` is in flight, the client cannot know whether the server committed before the connection broke. The data may or may not be there. Retrying blindly can apply a payment twice; not retrying can lose it.

> **Pitfall:** "commit outcome unknown" is rare and real. Operations that must not happen twice need a way to find out what happened: an idempotency key stored in the same transaction (`INSERT … ON CONFLICT DO NOTHING` on the key tells you on retry whether the first attempt committed), or a status the client can query. stockroom's outbox (Chapter 16) is built on the same idea: the job row commits together with the change, or not at all.

6.5 An error type for an application

A program should classify database errors once, at the boundary, into its own error type, and decide there what is shown to whom. Three rules make that work:

1. **One `From<tokio_postgres::Error>`** that looks at the code and turns the cases the application handles into variants of its own: not found, conflict, invalid, busy. Everything else stays a database error.
2. **Constraint names are part of the schema's contract.** Name every constraint a user can trip over (`CONSTRAINT price_positive CHECK (…)`), and map the name to a message. The server's text is for developers; your message is for users.
3. **The server's text goes to the log, never to the user.** It can contain table names, values and other internals (§6.2).

```
use tokio_postgres::error::SqlState;
use tokio_postgres::{Client, Error};

#[derive(Debug, thiserror::Error)]
enum AppError {
    #[error("{0} not found")]
    NotFound(&'static str),
    #[error("{0}")]
    Conflict(&'static str),
    #[error("{0}")]
    Invalid(&'static str),
```

```
    #[error("the database is busy, try again")]
    Busy(#[source] Error),
    #[error("database error")]
    Db(#[source] Error),
}

/// What each named constraint means to a user of the API.
fn constraint_message(name: &str) -> Option<&'static str> {
    Some(match name {
        "products_pkey" => "a product with this SKU already exists",
        "price_positive" => "the price must be positive",
        _ => return None,
    })
}

impl From<Error> for AppError {
    fn from(error: Error) -> Self {
        let Some(db) = error.as_db_error() else {
            return AppError::Db(error);
        };
        let message = db.constraint().and_then(constraint_message);
        match (db.code(), message) {
            (&SqlState::UNIQUE_VIOLATION, Some(message)) => AppError::Conflict(message),
            (&SqlState::CHECK_VIOLATION, Some(message)) => AppError::Invalid(message),
            (&SqlState::T_R_SERIALIZATION_FAILURE | &SqlState::T_R_DEADLOCK_DETECTED, _) =>
{
                AppError::Busy(error)
            }
            _ => AppError::Db(error),
        }
    }
}

async fn create_product(client: &Client, sku: &str, price: i64) -> Result<(), AppError> {
    client
        .execute("INSERT INTO products VALUES ($1, $2)", &[&sku, &price])
        .await?; // classified here, by the From impl
    Ok(())
}

async fn price_of(client: &Client, sku: &str) -> Result<i64, AppError> {
    let row = client
        .query_opt("SELECT price_cents FROM products WHERE sku = $1", &[&sku])
        .await?;
    Ok(row.ok_or(AppError::NotFound("product"))?.get(0))
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (
                 sku text PRIMARY KEY,
                 price_cents bigint CONSTRAINT price_positive CHECK (price_cents > 0))",
        )
        .await?;

    create_product(&client, "CHAIR-01", 4999).await.unwrap();
    let duplicate = create_product(&client, "CHAIR-01", 1).await.unwrap_err();
    assert_eq!(
        duplicate.to_string(),
        "a product with this SKU already exists"
    );
    let free = create_product(&client, "LAMP-01", 0).await.unwrap_err();
    assert_eq!(free.to_string(), "the price must be positive");
    assert!(matches!(
        price_of(&client, "VASE-01").await,
        Err(AppError::NotFound("product"))
    ));
```

```
    // Anything unplanned keeps the original error as its source, for the log.
    let other = AppError::from(
        client
            .execute("INSERT INTO nowhere VALUES (1)", &[])
            .await
            .unwrap_err(),
    );
    assert_eq!(other.to_string(), "database error");
    let cause = std::error::Error::source(&other).unwrap();
    assert!(
        cause
            .source()
            .unwrap()
            .to_string()
            .contains("\"nowhere\" does not exist")
    );
    Ok(())
}
```

The `?` in `create_product` and `price_of` is where classification happens, through `From`. The rest of the program sees `AppErrors` with messages fit for users, and the original `tokio_postgres::Error` rides along as the `source()` of the variants that keep it, ready for the log. Chapter 15 turns each variant into an HTTP status.

6.6 An error inside a transaction

After any error, a PostgreSQL transaction refuses to do anything else until it is rolled back:

```
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY);
             INSERT INTO products VALUES ('CHAIR-01');",
        )
        .await?;

    let tx = client.transaction().await?;
    // One statement fails...
    let duplicate = tx
        .execute("INSERT INTO products VALUES ('CHAIR-01')", &[])
        .await;
    assert!(duplicate.is_err());
    // ...and every statement after it is refused, however harmless.
    let refused = tx.query_one("SELECT 1", &[]).await.unwrap_err();
    assert_eq!(refused.code(), Some(&SqlState::IN_FAILED_SQL_TRANSACTION)); // 25P02
    assert_eq!(
        refused.as_db_error().unwrap().message(),
        "current transaction is aborted, commands ignored until end of transaction block"
    );
    tx.rollback().await?;

    // A savepoint limits the damage: roll back to it, and the transaction carries on.
    let mut tx = client.transaction().await?;
    let attempt = tx.savepoint("attempt").await?;
    let duplicate = attempt
        .execute("INSERT INTO products VALUES ('CHAIR-01')", &[])
        .await;
    assert!(duplicate.is_err());
    attempt.rollback().await?; // ROLLBACK TO SAVEPOINT attempt
```

```
    tx.execute("INSERT INTO products VALUES ('LAMP-01')", &[])
        .await?;
    tx.commit().await?;

    let count: i64 = client
        .query_one("SELECT count(*) FROM products", &[])
        .await?
        .get(0);
    assert_eq!(count, 2);
    Ok(())
}
```

So "try the insert, and if it is a duplicate, do something else" does not work inside a transaction, unless the attempt runs in a savepoint. Chapter 8 covers savepoints properly, and stockroom's batch endpoint uses one per item. Often there is a better SQL answer that avoids the error altogether, such as `INSERT … ON CONFLICT DO NOTHING`.

> **Pitfall:** a 25P02 in your logs is never the real problem; it is the *second* error. Look for the first one, earlier in the same transaction. Code that swallows an error inside a transaction (a `let _ =` on a failed `execute`) produces 25P02s far away from the cause.

6.7 Notices are not errors

The server can also send *notices*: messages that do not stop anything. `DROP TABLE IF EXISTS` on a missing table sends one, as does `CREATE TABLE IF NOT EXISTS` on an existing table, or a `RAISE NOTICE` in a function. The statement succeeds:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    // Both succeed; each also makes the server send a notice.
    client.batch_execute("DROP TABLE IF EXISTS nowhere").await?;
    client
        .batch_execute("DO $$ BEGIN RAISE NOTICE 'stock is low'; END $$")
        .await?;
    Ok(())
}
```

Where did the notices go? A notice is not part of any query's response: it arrives on the connection like a notification would, and only the `Connection` sees it. When the connection is simply awaited, as the playground's `connect` does, tokio-postgres passes each notice to the `log` crate at the `info` level, so you see it only if your program installed a logger for `log`. To handle notices yourself, drive the connection with `poll_message` instead of awaiting it (Chapter 11). stockroom's migration runner does exactly that, which is how "relation … already exists, skipping" ends up in its log.

Side by side – handling a unique violation

The task: insert a product whose SKU already exists, and recognize the error as a duplicate, with the constraint's name.

```
-- psql
tpbook=# INSERT INTO products VALUES ('CHAIR-01', 1);
ERROR:  duplicate key value violates unique constraint "products_pkey"
DETAIL:  Key (sku)=(CHAIR-01) already exists.
tpbook=# \errverbose
ERROR:  23505: duplicate key value violates unique constraint "products_pkey"
DETAIL:  Key (sku)=(CHAIR-01) already exists.
SCHEMA NAME:  pg_temp_12
TABLE NAME:  products
CONSTRAINT NAME:  products_pkey
LOCATION:  _bt_check_unique, nbtinsert.c:673
```

```
// sqlx (illustration)
match sqlx::query("INSERT INTO products VALUES ($1,
$2)").bind("CHAIR-01").bind(1_i64).execute(&pool).await {
    Err(sqlx::Error::Database(e)) if e.is_unique_violation() => {
        assert_eq!(e.constraint(), Some("products_pkey"));
    }
    other => { /* … */ }
}
```

```
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint);
             INSERT INTO products VALUES ('CHAIR-01', 4999);",
        )
        .await?;
    match client
        .execute("INSERT INTO products VALUES ('CHAIR-01', 1)", &[])
        .await
    {
        Err(e) if e.code() == Some(&SqlState::UNIQUE_VIOLATION) => {
            let constraint = e.as_db_error().and_then(|db| db.constraint());
            assert_eq!(constraint, Some("products_pkey"));
        }
        other => panic!("expected a duplicate, got {other:?}"),
    }
    Ok(())
}
```

	psql	sqlx	tokio-postgres
Same	The SQLSTATE, the constraint, the detail: everything the server sent	The same fields, from the same message	The same fields, from the same message
Different	Shows the message and detail; `\errverbose` shows every field of the last error	An `Error::Database` variant, with helpers such as `is_unique_violation()`; other errors are separate variants of one public enum	One opaque `Error`: `code()` and `as_db_error()` for server errors, the cause in `source()` for the rest, and `Display` that is only a category

In stockroom

stockroom classifies database errors in exactly one place, error.rs, with the shape of §6.5 and a few more variants:

```
// stockroom/src/error.rs (excerpt)
impl From<tokio_postgres::Error> for AppError {
    fn from(error: tokio_postgres::Error) -> Self {
        let Some(db) = error.as_db_error() else {
            // Not from the server: IO, a closed connection, or a conversion on our side.
            return if error.is_closed() {
                AppError::Unavailable("the connection to the database was lost".into())
            } else {
                AppError::Db(error)
            };
        };
        match *db.code() {
            SqlState::UNIQUE_VIOLATION => {
                let (field, message) =
                    constraint_message(db).unwrap_or((None, "this already exists"));
                AppError::Conflict {
                    field,
                    message: message.into(),
                }
            }
            // … CHECK, NOT NULL and FOREIGN KEY violations become AppError::Invalid …
            SqlState::T_R_SERIALIZATION_FAILURE
            | SqlState::T_R_DEADLOCK_DETECTED
            | SqlState::LOCK_NOT_AVAILABLE => AppError::Busy(error),
            SqlState::QUERY_CANCELED => AppError::Timeout(error),
            SqlState::ADMIN_SHUTDOWN
            | SqlState::CANNOT_CONNECT_NOW
            | SqlState::TOO_MANY_CONNECTIONS =>
AppError::Unavailable(db.message().to_owned()),
            _ => AppError::Db(error),
        }
    }
}
```

`constraint_message` maps every named constraint in the schema to the request field it is about and a message, so a client gets `{"error": "conflict", "field": "sku", "message": "a product with this SKU already exists"}` instead of a server message. `Busy` is what the transaction helper of Chapter 8 retries. When an error *is* the server's fault (a 5xx), its fields go to the log as structured data and the client gets a generic message:

```
// stockroom/src/error.rs (excerpt)
fn log_server_error(error: &AppError) {
    let db = match error {
        AppError::Busy(e) | AppError::Timeout(e) | AppError::Db(e) => e.as_db_error(),
        _ => None,
    };
    match db {
        Some(db) => tracing::error!(
            code = db.code().code(),
            message = db.message(),
            detail = db.detail(),
            hint = db.hint(),
            table = db.table(),
            constraint = db.constraint(),
            "{error}"
        ),
        None => tracing::error!(error = ?error, "{error}"),
    }
}
```

There is one deliberate exception to "server text never reaches the client": errors in an uploaded CSV file (class 22, data exceptions, raised by `COPY`). Their message and position quote the client's own data back, "invalid input syntax for type bigint: "cheap" (COPY product_import, line 3, column price_cents: "cheap")", which is exactly what the uploader needs. repo/products.rs makes that exception in a function of its own, `input_error`, so that it cannot spread by accident.

Summary

- Every failure is a `tokio_postgres::Error`. Its `Display` is a category (`db error`); the explanation is in `source()`. Log `{:?}`, or walk the source chain.
- `as_db_error()` and `code()` exist for errors the server reported; client-side errors (row counts, conversions, parameters) have no code and no public kind.
- `DbError` carries the SQLSTATE, message, detail, hint, position, the schema, table, column, datatype and constraint involved, and the call stack of functions and triggers.
- Match `SqlState` constants as patterns; match classes (`23`, `40`, `08`) by prefix.
- `is_closed()` means a new connection is needed. A broken connection during `COMMIT` leaves the outcome unknown: design operations to be safely retried.
- Classify once, in a `From<tokio_postgres::Error>` for your own error type. Name your constraints, and map the names to messages. Keep the server's text out of responses.
- After an error, a transaction is aborted (25P02) until rolled back; savepoints contain the damage.
- Notices are not errors. When the connection is just awaited they go to the `log` crate; `poll_message` hands them to you.

Exercises

1. **A better log line.** Write `fn describe(error: &tokio_postgres::Error) -> String` that produces `23505 unique_violation on products (products_pkey): duplicate key value …` for server errors and falls back to the source chain for the others. *Hint:* the SQLSTATE's name is not in tokio-postgres; a short `match` on the codes you care about is enough.
2. **Retry or not?** For each, say whether a request that failed with it should be retried as is, retried after a delay, or not retried: `40001`, `40P01`, `55P03`, `57014`, `23505`, `53300`, `57P01`, a closed connection during `SELECT`, a closed connection during `COMMIT`. *Hint:* ask of each whether the same request could succeed a moment later, and whether it may already have taken effect.
3. **A rule in a trigger.** Write a trigger that refuses to delete a product that has stock, with `RAISE EXCEPTION … USING ERRCODE = 'SR002'`, and map `SR002` to a 409 Conflict variant of your `AppError`. *Hint:* a custom code has no named constant; compare `db.code().code()` with `"SR002"`, or with `SqlState::from_code("SR002")`.
4. **Find the first error.** A colleague's code logs "current transaction is aborted" every few minutes. What do you look for, and what is a likely cause in Rust code? *Hint:* the useful line is the error

before the first "aborted" one on that connection; look for code that ignores a statement's error (`let _ =`, `.ok()`) and carries on in the same transaction.

Solutions

2. `40001` and `40P01`: retry the whole transaction, immediately or after a short random pause (they are caused by concurrency, not by the request). `55P03` (lock timeout): retry after a delay, or report "busy". `57014`: do not retry blindly: it was too slow once, and will probably be too slow again; report it, and look at the query. `23505`: never; the request conflicts with data. `53300` and `57P01`: retry after a delay, preferably on another connection or server. A closed connection during `SELECT`: retry on a new connection, since reading is safe to repeat. A closed connection during `COMMIT`: the outcome is unknown; only retry if the operation is idempotent or you can check what happened first (§6.4).

4. Look for the error *before* it, in the same transaction, on the same connection: the 25P02 is the consequence. In Rust code, the classic cause is an error that was caught and ignored inside a transaction, such as `let _ = tx.execute(…).await;` or a `match` that logs and continues, after which the transaction is used again. Either use a savepoint around the statement that may fail, or let the error end the transaction.

CHAPTER 7

One connection, many tasks: pipelining

Chapter 4 counted round trips and concluded that a chain of queries, each waiting for the one before, costs one round trip per link. This chapter breaks the chain. When queries do not depend on each other, a client can send all of them and then collect the answers: the server still runs them one at a time, but nobody waits for the network in between. That is *pipelining*, and tokio-postgres does it for you, without any special API, whenever several queries on the same client are in flight at once.

That one feature explains most of tokio-postgres's design: why `Client` methods take `&self`, why the connection is a separate task, and why a single client can serve a whole program. It also has limits, and the last part of the chapter is about them: a slow query holds up everything behind it, and a transaction cannot be pipelined with anyone else's work.

7.1 Lazy futures, and the order requests are sent in

A tokio-postgres method returns a future, and like every Rust future it does nothing until it is polled. Only then does it put its request on the connection's queue. The crate's documentation states the consequence: requests are sent in the order their futures are *first polled*, not the order in which they were created.

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client.batch_execute("CREATE TEMP SEQUENCE ticket").await?;
    let next = client.prepare("SELECT nextval('ticket')").await?;

    // Created in the order a, b...
    let a = client.query_one(&next, &[]);
    let b = client.query_one(&next, &[]);
    // ...but join! polls b first, so b's request goes out first.
    let (b, a) = tokio::join!(b, a);
    let (a, b): (i64, i64) = (a?.get(0), b?.get(0));
    assert_eq!((a, b), (2, 1));
    Ok(())
}
```

There is a subtlety in that example: it uses a prepared `Statement`. A query given as SQL text is *two* requests, prepare and then execute (Chapter 4), and the second one goes out when the prepare's answer has come back and the future is polled again, which depends on scheduling. Run the example above with text instead of `&next`, and the order of the two `nextval` calls varies from run to run. So

when the order matters, use a prepared statement or `query_typed`: one request each, sent in the order you poll them.

7.2 `join!`: N queries, about one round trip

Awaiting queries one after another sends each request only after the previous answer arrived. Polling them together with `join!` (or `try_join!`, which stops at the first error) puts all the requests on the wire at once:

```
one after another                    joined
  client ──q1──▶ server                client ──q1 q2 q3──▶ server
  client ◀──r1── server                client ◀──r1 r2 r3── server
  client ──q2──▶ server
  client ◀──r2── server                1 round trip (plus the server's work)
  client ──q3──▶ server
  client ◀──r3── server
  3 round trips
```

On localhost both take microseconds. Through lagproxy, which adds 5 ms each way (Chapter 4 shows how to set it up), the difference is the whole point:

```rust
use std::time::Instant;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?; // through lagproxy
    let count = client.prepare("SELECT count(*) FROM pg_class").await?;
    let size = client
        .prepare("SELECT pg_database_size(current_database())")
        .await?;
    let version = client.prepare("SHOW server_version").await?;
    let now = client.prepare("SELECT now()").await?;

    let started = Instant::now();
    client.query_one(&count, &[]).await?;
    client.query_one(&size, &[]).await?;
    client.query_one(&version, &[]).await?;
    client.query_one(&now, &[]).await?;
    let one_by_one = started.elapsed();

    let started = Instant::now();
    let (c, s, v, n) = tokio::try_join!(
        client.query_one(&count, &[]),
        client.query_one(&size, &[]),
        client.query_one(&version, &[]),
        client.query_one(&now, &[]),
    )?;
    let joined = started.elapsed();

    println!("one by one: {one_by_one:?}, joined: {joined:?}");
    assert!(c.get::<_, i64>(0) > 0 && s.get::<_, i64>(0) > 0);
    assert!(!v.get::<_, String>(0).is_empty());
    let _: jiff::Timestamp = n.get(0);
    assert!(
        joined * 2 < one_by_one,
        "about one round trip instead of four"
    );
    Ok(())
}
```

```
one by one: 59.7ms, joined: 16.2ms
```

Four round trips became one. The server did exactly the same work in both cases, in the same order; what disappeared is the waiting on the network in between.

> **Protocol lens:** each tokio-postgres request ends with a *Sync* message, which marks the end of a unit of work for the server. Requests from different futures are just written to the socket back to back, and the server reads and answers them in order. Nothing else is needed: the protocol has always allowed this. libpq gained an explicit "pipeline mode" API only in PostgreSQL 14, and psql only in 18 (see the side by side below).

7.3 One client, many tasks

`join!` pipelines within one task. The same happens across tasks, because a `Client` can be shared: its methods take `&self`, and it is `Send` and `Sync`. Wrap it in an `Arc`, give each task a clone, and their requests meet on the one connection:

```
use std::sync::Arc;
use std::time::{Duration, Instant};

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = Arc::new(playground::connect().await?); // through lagproxy
    let delay: u64 = std::env::var("LAGPROXY_DELAY_MS").map_or(5, |v| v.parse().unwrap());
    let statement = client.prepare("SELECT $1::int8 * $1").await?;

    // Twenty "requests", each in its own task, each needing one query.
    let started = Instant::now();
    let mut tasks = Vec::new();
    for i in 0..20_i64 {
        let (client, statement) = (Arc::clone(&client), statement.clone());
        tasks.push(tokio::spawn(async move {
            let row = client.query_one(&statement, &[&i]).await?;
            Ok::<i64, tokio_postgres::Error>(row.get(0))
        }));
    }
    let mut squares = Vec::new();
    for task in tasks {
        squares.push(task.await.expect("no panic")?);
    }
    let elapsed = started.elapsed();

    assert_eq!(squares[7], 49);
    // Twenty one-round-trip queries; they all overlapped.
    let one_round_trip = Duration::from_millis(2 * delay);
    println!("20 tasks in {elapsed:?}");
    assert!(elapsed < one_round_trip * 5);
    Ok(())
}
```

It printed "20 tasks in 12.6ms": about one round trip through the proxy, for twenty queries. That is a service in miniature: many concurrent requests, each with a little database work, sharing one connection, none of them waiting for the others' round trips. Chapter 13 measures it against a connection pool over the same slow network. The shared client won by almost an order of magnitude for independent queries, because a pool of eight allows eight requests in flight, and one pipelined connection allows them all.

> **Rust refresher:** `Arc<Client>` is all the sharing takes, with no `Mutex`, because nothing needs `&mut Client`. The connection task is the only owner of the socket, and the client only talks to it through a channel. Shared state behind message passing, not behind a lock: the same *actor* pattern you would use to share any resource between tasks.

7.4 Under the hood: a queue, in order

The `Client` sends each request to the connection task through an unbounded channel, along with the sending end of a small channel for its responses. The connection writes requests to the socket in the order they arrived, and routes the server's responses, which come back in the same order, to their senders. Two properties follow.

Order is guaranteed. Requests are executed in the order they were sent, on one session. So a write followed by a read, sent without waiting, sees the write.

There is no overtaking. The server runs one statement at a time per session. A slow query holds up every request queued behind it on that connection, however fast those are. This is *head-of-line blocking*:

```
use std::sync::Arc;
use std::time::{Duration, Instant};

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let shared = Arc::new(playground::connect().await?);
    let other = playground::connect().await?;

    // Something slow starts on the shared connection...
    let slow = {
        let shared = Arc::clone(&shared);
        tokio::spawn(async move { shared.execute("SELECT pg_sleep(0.5)", &[]).await })
    };
    tokio::time::sleep(Duration::from_millis(50)).await;

    // ...and a trivial query behind it waits for it to finish.
    let started = Instant::now();
    shared.query_one("SELECT 1", &[]).await?;
    let behind_slow = started.elapsed();

    // The same query on another connection does not.
    let started = Instant::now();
    other.query_one("SELECT 1", &[]).await?;
    let elsewhere = started.elapsed();

    slow.await.expect("no panic")?;
    println!("behind the slow query: {behind_slow:?}; on another connection:
{elsewhere:?}");
    assert!(behind_slow > Duration::from_millis(300));
    assert!(elsewhere < Duration::from_millis(100));
    Ok(())
}
```

```
behind the slow query: 451.3ms; on another connection: 1.3ms
```

A program that shares one client should therefore keep slow work (reports, exports, anything that scans a lot) off it. Give that work a connection of its own, or put it behind a pool (Chapter 13), and use `statement_timeout` (Chapter 12) to bound what can hold up the queue.

> **Pitfall:** a large result consumed slowly is also slow work. The connection delivers a query's rows into a small buffer and stops reading the socket when that buffer is full, so a `RowStream` that you read slowly holds up everything behind it. Chapter 9 measures this.

7.5 Many queries, a few at a time

`join!` takes a fixed number of futures. For a list, use `futures_util`'s combinators: `try_join_all` runs them all at once, and `buffer_unordered(n)` (on a stream of futures) runs at most `n` at a time, which keeps a huge list from queueing thousands of requests:

```
use futures_util::{StreamExt, TryStreamExt, future, stream};

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE prices AS
             SELECT 'SKU' || i AS sku, i * 100 AS price_cents FROM generate_series(1, 500)
AS i",
        )
        .await?;
    let price = client
        .prepare("SELECT price_cents FROM prices WHERE sku = $1")
        .await?;
    let skus: Vec<String> = (1..=500).map(|i| format!("SKU{i}")).collect();

    // All at once: one future per SKU, all pipelined.
    let rows = future::try_join_all(skus.iter().take(10).map(|sku| {
        let price = &price;
        let client = &client;
        // The parameter slice must live as long as the future: build it inside the async
        // block. (`client.query_one(price, &[sku])` as the closure's result would borrow a
        // temporary slice that dies when the closure returns.)
        async move { client.query_one(price, &[sku]).await }
    }))
    .await?;
    assert_eq!(rows.len(), 10);

    // At most 16 in flight at a time, in any order, summed as they come.
    let total: i32 = stream::iter(&skus)
        .map(|sku| {
            let (price, client) = (&price, &client);
            async move {
                let row = client.query_one(price, &[sku]).await?;
                Ok::<i32, tokio_postgres::Error>(row.get(0))
            }
        })
        .buffer_unordered(16)
        .try_fold(0, |sum, cents| async move { Ok(sum + cents) })
        .await?;
    assert_eq!(total, (1..=500).sum::<i32>() * 100);
    Ok(())
}
```

`buffer_unordered` yields results as they complete; `buffered(n)` keeps them in input order. On one connection the server answers in order anyway, so the difference is small here, but it matters as soon as the futures do more than one query.

> **Pitfall:** the comment in that example is the error you will meet first when pipelining a list. A future returned by `query_one` borrows its parameter slice, so `|sku| client.query_one(&price, &[sku])` does not compile: the slice `&[sku]` is a temporary inside the closure. Put the call in an `async move` block (as above), or bind the slice to a variable that outlives the futures. stockroom's code has both, with a comment each time.

7.6 When not to pipeline

Pipelining is for independent work. Three situations need care.

A query that needs the previous one's answer cannot be sent before that answer arrives. That sounds obvious, but it is worth turning around: look for queries that *do not* depend on each other, because they are often written one after another out of habit. Better still, let the database do the dependent steps in one statement (a join, a subquery, `RETURNING`, a `WITH` clause).

An error in one query does not stop the others. Each request is its own unit of work, so the server runs every request it received, and with `try_join!` the futures you stop waiting for still did their work:

```
use tokio_postgres::types::Type;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE log (n int)")
        .await?;

    // Three requests, all sent before any answer: the middle one fails.
    let result = tokio::try_join!(
        client.execute_typed("INSERT INTO log VALUES ($1)", &[(&1, Type::INT4)]),
        client.execute_typed("SELECT 1 / $1", &[(&0, Type::INT4)]),
        client.execute_typed("INSERT INTO log VALUES ($1)", &[(&3, Type::INT4)]),
    );
    assert!(result.is_err(), "try_join! reports the division by zero...");

    let logged: Vec<i32> = client
        .query_scalar("SELECT n FROM log ORDER BY n", &[])
        .await?;
    assert_eq!(logged, [1, 3], "...but both inserts happened");
    Ok(())
}
```

When several changes must succeed or fail together, that is a transaction's job, not a pipeline's.

A transaction on a shared client is the dangerous case. A transaction is a state of the *session*: between `BEGIN` and `COMMIT`, every statement on the connection is part of it, whoever sent it. Start one with plain SQL on a client that other tasks share, and their statements join your transaction:

```rust
use std::sync::Arc;
use std::time::Duration;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let shared = Arc::new(playground::connect().await?);
    shared
        .batch_execute("CREATE TEMP TABLE audit (event text)")
        .await?;

    // Task A opens a transaction with plain SQL, writes, and later rolls back.
    let task_a = {
        let shared = Arc::clone(&shared);
        tokio::spawn(async move {
            shared.batch_execute("BEGIN").await?;
            shared
                .execute("INSERT INTO audit VALUES ('a: tentative')", &[])
                .await?;
            tokio::time::sleep(Duration::from_millis(100)).await;
            shared.batch_execute("ROLLBACK").await
        })
    };
    tokio::time::sleep(Duration::from_millis(30)).await;

    // Meanwhile, task B writes something that has nothing to do with A's transaction.
    shared
        .execute("INSERT INTO audit VALUES ('b: must survive')", &[])
        .await?;
    task_a.await.expect("no panic")?;

    // B's row ran inside A's transaction, and A's rollback took it away.
    let left: i64 = shared
        .query_one("SELECT count(*) FROM audit", &[])
        .await?
        .get(0);
    assert_eq!(left, 0);
    Ok(())
}
```

This is exactly why `Client::transaction` takes `&mut self`. A `Transaction` borrows the client mutably, so while it exists, nothing else can use that client, and the borrow checker turns this bug into a compile error. A client behind an `Arc` cannot give out `&mut`, so it cannot start a transaction at all. Transactions need a connection of their own for their duration: the next chapter's subject, and the main reason for connection pools.

Side by side – concurrent queries

The task: run two independent queries without waiting for the first answer before sending the second.

```
-- psql 18 has a pipeline mode of its own: \startpipeline … \endpipeline.
tpbook=# \startpipeline
tpbook=# SELECT 1 AS a \bind \sendpipeline
tpbook=# SELECT 2 AS b \bind \sendpipeline
tpbook=# \endpipeline
 a
---
 1
(1 row)

 b
---
 2
(1 row)
```

```
// sqlx (illustration)
// A PgConnection runs one query at a time (its methods take &mut self), so this does not
// compile on one connection. Concurrency comes from a pool: two queries, two connections.
let (a, b) = tokio::try_join!(
    sqlx::query_scalar::<_, i32>("SELECT 1").fetch_one(&pool),
    sqlx::query_scalar::<_, i32>("SELECT 2").fetch_one(&pool),
)?;
```

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    // One connection; both requests are written before either answer is read.
    let (a, b) = tokio::try_join!(
        client.query_one("SELECT 1", &[]),
        client.query_one("SELECT 2", &[]),
    )?;
    assert_eq!((a.get::<_, i32>(0), b.get::<_, i32>(0)), (1, 2));
    Ok(())
}
```

	psql	sqlx	tokio-postgres
Same	Requests sent back to back on one session, answered in order	Concurrent queries finish without waiting for each other	Requests sent back to back on one session, answered in order
Different	Explicit mode, statement by statement, in psql 18	No pipelining on one connection: concurrency costs one connection per query in flight	Automatic: any futures in flight at once on the same client, from one task or many

In stockroom

stockroom pipelines in three places, each for a different reason. The dashboard is the textbook case, five independent aggregates in one `try_join!`:

```
// stockroom/src/repo/reports.rs (excerpt)
pub async fn dashboard(client: &Client) -> Result<Dashboard, AppError> {
    let (products, stores, open_orders, today, low) = tokio::try_join!(
        client.query_one_scalar::<i64, _>("SELECT count(*) FROM products", &[]),
        client.query_one_scalar::<i64, _>("SELECT count(*) FROM stores", &[]),
        client.query_one_scalar::<i64, _>(
            "SELECT count(*) FROM orders WHERE status IN ('pending', 'paid', 'ready')",
            &[]
        ),
        // … today's orders and revenue, and the ten lowest stock levels …
    )?;
```

Reading an order is two queries, the order and its lines, that both need only the id. The parameter slice is bound to a name because both futures borrow it:

```
// stockroom/src/repo/orders.rs (excerpt)
// Two queries, one round trip: pipelined. (The parameter slice is bound to a name
// because both futures borrow it until they finish.)
let params: &[&(dyn ToSql + Sync)] = &[&id];
let (head, lines) = tokio::try_join!(db.query_opt(&head, params), db.query(&lines,
params))?;
```

Placing an order pipelines inside a transaction, which is safe because the transaction owns its connection. Every line's reservation is a guarded `UPDATE`, and they all go out at once:

```
// stockroom/src/repo/orders.rs (excerpt)
// Every reservation at once: pipelined, sent in this order, run one by one by the server.
let reserve = tx
    .prepare_cached(
        "UPDATE stock SET reserved = reserved + $3
         WHERE store_id = $1 AND product_id = $2 AND available >= $3",
    )
    .await?;
let (reserve, store_id) = (&reserve, &store_id);
let reserved = future::try_join_all(lines.iter().map(|l| async move {
    // The parameter slice must live as long as the future that uses it: inside the
    // `async` block, not as a temporary in the closure.
    tx.execute(reserve, &[store_id, &l.product_id, &l.quantity])
        .await
}))
.await?;
```

"Sent in this order" is a promise with consequences. The lines are sorted by product id so that every order locks stock rows in the same order and two orders cannot deadlock (Chapter 8). The promise holds because `reserve` is a prepared statement, so each reservation is a single request, and `try_join_all` polls its futures in list order. With SQL text, §7.1 showed, the order would vary.

Summary

- Futures are lazy: a request is sent when its future is first polled. Single-request calls (a `Statement`, `query_typed`) are sent in that order; a SQL-text query is two requests, and its execution order can vary.
- Futures in flight at the same time on the same client are pipelined: N independent queries cost about one round trip instead of N. `join!`, `try_join!`, `try_join_all` and `buffer_unordered` all do it.
- `Client` is `Send + Sync` with `&self` methods: share it with `Arc`, and tasks pipeline their requests on one connection.
- The server runs one statement at a time per session: a slow query, or a slowly consumed result, blocks everything queued behind it.
- A query future borrows its parameter slice: build it inside an `async move` block or bind it to a name.
- Every request is executed, even if you stop waiting for it; errors do not cancel neighbours.
- Never start a transaction with SQL on a shared client: other tasks' statements join it. `transaction()` takes `&mut self` to make that impossible.

Exercises

1. **Measure the dashboard.** Through lagproxy, time stockroom's five dashboard queries one after another, then with `try_join!` as in the excerpt. Predict both from Chapter 4's numbers first. *Hint:* the queries are SQL text, so count two round trips each.
2. **Order or not.** Repeat §7.1's example with SQL text instead of the prepared statement, in a loop of 20. How often is the order reversed? Explain the result in terms of requests. *Hint:* count the requests that each `query_one` with SQL text sends, and when its second one leaves.

3. **A bulk lookup.** Given 10,000 SKUs, fetch their prices with at most 32 queries in flight, and compare the time with one `= ANY($1)` query for all of them. Which is faster, and when would you still choose the first? *Hint:* `stream::iter(skus).map(…).buffer_unordered(32)` (§7.5) for the first; `sku = ANY($1)` with a `&[String]` for the second.
4. **Fix the shared transaction.** Rewrite §7.6's last example so that task A's rollback cannot touch task B's row, while B still uses the shared client. *Hint:* A needs a connection of its own.

Solutions

1. Ten round trips one after another (five prepares, five executes), so about 150 ms at a 5 ms delay. Joined, the five prepares go out together and the five executes go out as their prepares complete: about two round trips, 30 ms. Preparing the five statements once (`prepare_cached`, as stockroom does elsewhere) would make it one round trip.

3. One `= ANY($1)` query is far faster: one statement, one plan, one result, instead of 10,000 executions. Pipelining hides network latency, not the cost of 10,000 statements on the server. The first approach is still the right one when each item needs a different query, or the work per item is more than a lookup.

4.

```rust
use std::sync::Arc;
use std::time::Duration;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let shared = Arc::new(playground::connect().await?);
    // Temporary tables belong to one session, so the table must be a real one here, visible
    // to both connections. A schema of its own keeps it apart, and goes away at the end.
    shared
        .batch_execute(
            "DROP SCHEMA IF EXISTS ch07 CASCADE; CREATE SCHEMA ch07;
             CREATE TABLE ch07.audit (event text);",
        )
        .await?;

    // Task A: a connection of its own, and a real transaction on it.
    let task_a = tokio::spawn(async move {
        let mut own = playground::connect().await?;
        let tx = own.transaction().await?;
        tx.execute("INSERT INTO ch07.audit VALUES ('a: tentative')", &[])
            .await?;
        tokio::time::sleep(Duration::from_millis(100)).await;
        tx.rollback().await
    });
    tokio::time::sleep(Duration::from_millis(30)).await;

    shared
        .execute("INSERT INTO ch07.audit VALUES ('b: must survive')", &[])
        .await?;
    task_a.await.expect("no panic")?;

    let left: Vec<String> = shared
        .query_scalar("SELECT event FROM ch07.audit", &[])
        .await?;
    assert_eq!(left, ["b: must survive"]);
    shared.batch_execute("DROP SCHEMA ch07 CASCADE").await?;
    Ok(())
}
```

CHAPTER 8

Transactions and concurrency control

A store has five garden chairs and twenty customers who want one, all clicking "buy" in the same second. A program that reads "five available", decides "fine, one more", and writes "one less" will sell more than five chairs, every time, because twenty copies of it read "five" before any of them wrote. That is the problem this chapter solves, and the tools are PostgreSQL's: transactions, isolation levels, row locks and constraints.

tokio-postgres's part is small but strict. `Transaction` borrows the client mutably, so the compiler enforces what Chapter 7 showed the hard way: nothing else can run on a connection while a transaction owns it. The chapter covers transactions and that borrow, isolation levels, savepoints, writing functions that work inside or outside a transaction, retrying transactions that lose a race, locks, and optimistic concurrency. It ends by solving the chairs problem four ways, and proving each one with twenty concurrent customers.

8.1 `transaction`, `commit`, and rollback on drop

`Client::transaction` sends `BEGIN` and returns a `Transaction`. It has the same query methods as a `Client`. `commit` ends it successfully. Anything else ends it with a rollback: an explicit `rollback`, or simply dropping it, which includes returning early with `?`:

```
use tokio_postgres::{Client, Error};

/// Moves money between two accounts: both updates, or neither.
async fn transfer(client: &mut Client, from: &str, to: &str, cents: i64) -> Result<(),
Error> {
    let tx = client.transaction().await?;
    tx.execute(
        "UPDATE accounts SET cents = cents - $2 WHERE name = $1",
        &[&from, &cents],
    )
    .await?; // an error here returns, drops `tx`, and so rolls back
    tx.execute(
        "UPDATE accounts SET cents = cents + $2 WHERE name = $1",
        &[&to, &cents],
    )
    .await?;
    tx.commit().await
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let mut client = playground::connect().await?;
    client
        .batch_execute(
```

```
            "CREATE TEMP TABLE accounts (name text PRIMARY KEY, cents bigint CHECK (cents >=
0));
             INSERT INTO accounts VALUES ('ada', 100), ('bob', 0);",
        )
        .await?;

    transfer(&mut client, "ada", "bob", 30).await?;
    // Too much: the first UPDATE breaks the CHECK, and nothing changes.
    assert!(transfer(&mut client, "ada", "bob", 500).await.is_err());

    // A transaction that is dropped without commit is rolled back.
    {
        let tx = client.transaction().await?;
        tx.execute("DELETE FROM accounts", &[]).await?;
    }

    let balances: Vec<(String, i64)> = client
        .query("SELECT name, cents FROM accounts ORDER BY name", &[])
        .await?
        .iter()
        .map(|row| (row.get(0), row.get(1)))
        .collect();
    assert_eq!(balances, [("ada".into(), 70), ("bob".into(), 30)]);
    Ok(())
}
```

Rollback on drop is a safety net, not a way to report errors. A `Drop` implementation cannot wait, so the dropped transaction only *queues* a `ROLLBACK` on the connection, and nobody hears whether it worked. Call `rollback().await` when you want to know, as stockroom's retry helper does (§8.6).

8.2 Why `transaction` takes `&mut self`

A transaction is a state of the *session*. Between `BEGIN` and `COMMIT`, every statement on the connection belongs to it, whoever sent it. Chapter 7 showed what happens when another task's statement slips into a transaction started with plain SQL on a shared client: it is rolled back with it. tokio-postgres prevents that at compile time. `transaction()` borrows the client mutably, and the `Transaction` keeps that borrow until it is committed, rolled back or dropped:

```
let tx = client.transaction().await?;
client.query("SELECT 1", &[]).await?;   // the client is busy with `tx`
tx.commit().await?;
// error[E0502]: cannot borrow `client` as immutable because it is also borrowed as mutable
// note: mutable borrow might be used here, when `tx` is dropped and runs the `Drop` code
//       for type `Transaction`
```

And a client behind an `Arc`, shared by many tasks, cannot start a transaction at all, because nobody can get `&mut` through an `Arc`:

```
let shared = Arc::new(client);
let tx = shared.transaction().await?;
// error[E0596]: cannot borrow data in an `Arc` as mutable
```

That is the design working, not getting in the way: a transaction needs a connection of its own for as long as it runs. A program with concurrent transactions therefore needs several connections, and handing them out is a pool's job (Chapter 13). The examples below that need concurrency simply open one connection per task.

8.3 Isolation levels, read-only and deferrable

What a transaction sees of other transactions' changes is its *isolation level*. `build_transaction` chooses it, along with two more options, before sending `BEGIN`:

```
let tx = client
    .build_transaction()
    .isolation_level(IsolationLevel::RepeatableRead) // ReadCommitted is the default
    .read_only(true)                                 // writes fail with 25006
    .deferrable(true)                                // only matters for SERIALIZABLE READ
ONLY
    .start()
    .await?;
```

> **SQL lens:** PostgreSQL never lets a reader see uncommitted changes, and readers never block writers or the other way round (that is MVCC: every statement reads a *snapshot*). The levels differ in when the snapshot is taken and what is checked at commit. **READ COMMITTED** (the default) takes a new snapshot for every *statement*, so a transaction sees changes that others committed while it was running. **REPEATABLE READ** takes one snapshot for the whole *transaction*: everything it reads is as of its first statement, and if it tries to update a row that someone else changed since, it fails with a serialization error. **SERIALIZABLE** adds a check that the combined result of all committed transactions is one that running them one at a time could have produced, and fails one of them (40001) when that is not true. `READ UNCOMMITTED` exists in name only: PostgreSQL treats it as READ COMMITTED.

The difference is easy to see with two connections, one reading in a transaction while the other changes the row in between. The table must be a real one, visible to both sessions, so the example makes a schema of its own and drops it at the end:

```
use tokio_postgres::IsolationLevel;
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut reader = playground::connect().await?;
    let writer = playground::connect().await?;
    writer
        .batch_execute(
            "DROP SCHEMA IF EXISTS ch08_isolation CASCADE;
             CREATE SCHEMA ch08_isolation;
             CREATE TABLE ch08_isolation.stock (sku text PRIMARY KEY, on_hand int);
             INSERT INTO ch08_isolation.stock VALUES ('CHAIR-01', 10);",
        )
        .await?;
    let read = "SELECT on_hand FROM ch08_isolation.stock WHERE sku = 'CHAIR-01'";
    let set = "UPDATE ch08_isolation.stock SET on_hand = $1 WHERE sku = 'CHAIR-01'";

    for (level, second_read) in [
        (IsolationLevel::ReadCommitted, 7), // sees the other connection's commit
        (IsolationLevel::RepeatableRead, 10), // keeps its snapshot
    ] {
        writer.execute(set, &[&10]).await?;
        let tx = reader
            .build_transaction()
            .isolation_level(level)
```

```
            .start()
            .await?;
        let first: i32 = tx.query_one(read, &[]).await?.get(0);
        writer.execute(set, &[&7]).await?; // commits at once: no transaction
        let second: i32 = tx.query_one(read, &[]).await?.get(0);
        tx.commit().await?;
        assert_eq!((first, second), (10, second_read), "{level:?}");
    }

    // A read-only transaction refuses to write.
    let tx = reader.build_transaction().read_only(true).start().await?;
    let error = tx.execute(set, &[&1]).await.unwrap_err();
    assert_eq!(error.code(), Some(&SqlState::READ_ONLY_SQL_TRANSACTION)); // 25006
    drop(tx);

    writer
        .batch_execute("DROP SCHEMA ch08_isolation CASCADE")
        .await?;
    Ok(())
}
```

`deferrable(true)` has an effect only together with `SERIALIZABLE` and `read_only(true)`: the transaction may wait at its start until it can take a snapshot that is guaranteed to be consistent, and from then on it can never fail with a serialization error, nor make anyone else's fail. That is the right setting for long reports and exports, and stockroom's `export` command uses it.

8.4 Savepoints: transactions inside transactions

`Transaction` has a `transaction()` method of its own, and `savepoint(name)`, which does the same with a name of your choice. Both start a *nested* transaction with `SAVEPOINT`. Committing it (`RELEASE SAVEPOINT`) keeps its changes as part of the outer transaction, and rolling it back (`ROLLBACK TO SAVEPOINT`) undoes only what happened since the savepoint. Either way, the outer transaction carries on, healthy:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut client = playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE products (sku text PRIMARY KEY)")
        .await?;

    let mut tx = client.transaction().await?;
    tx.execute("INSERT INTO products VALUES ('CHAIR-01')", &[])
        .await?;
    for sku in ["LAMP-01", "CHAIR-01", "VASE-01"] {
        // One savepoint per item: a failed insert spoils only its own savepoint.
        let item = tx.savepoint("item").await?;
        match item
            .execute("INSERT INTO products VALUES ($1)", &[&sku])
            .await
        {
            Ok(_) => item.commit().await?,
            Err(_) => item.rollback().await?, // the duplicate CHAIR-01
        }
    }
    // Nested transactions nest further, and their commit is still provisional:
    {
        let mut inner = tx.transaction().await?;
        let innermost = inner.transaction().await?;
        innermost
            .execute("INSERT INTO products VALUES ('RUG-01')", &[])
            .await?;
        innermost.commit().await?;
```

```
        // `inner` is dropped without commit: RUG-01 goes with it.
    }
    tx.commit().await?;

    let skus: Vec<String> = client
        .query_scalar("SELECT sku FROM products ORDER BY sku", &[])
        .await?;
    assert_eq!(skus, ["CHAIR-01", "LAMP-01", "VASE-01"]);
    Ok(())
}
```

Savepoints are cheap, but not free: each one is a subtransaction on the server, and a very large number of them in one transaction (tens of thousands) has a measurable cost. One per item in a batch of a few hundred is fine, and it is exactly how stockroom's batch endpoint keeps the good items when some fail (Chapter 6).

8.5 `GenericClient`: code that runs inside or outside a transaction

Most data-access functions do not care whether they run inside a transaction. Writing them against `&Client` locks them out of transactions, and writing them against `&Transaction` locks them in. `GenericClient` is the trait both implement, with the query methods they share:

```
use tokio_postgres::{Error, GenericClient};

/// Works on a plain client and inside a transaction alike.
async fn stock_of(db: &impl GenericClient, sku: &str) -> Result<i32, Error> {
    let row = db
        .query_one("SELECT on_hand FROM stock WHERE sku = $1", &[&sku])
        .await?;
    Ok(row.get(0))
}

async fn receive(db: &impl GenericClient, sku: &str, units: i32) -> Result<(), Error> {
    db.execute(
        "UPDATE stock SET on_hand = on_hand + $2 WHERE sku = $1",
        &[&sku, &units],
    )
    .await?;
    Ok(())
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let mut client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE stock (sku text PRIMARY KEY, on_hand int);
             INSERT INTO stock VALUES ('CHAIR-01', 5);",
        )
        .await?;

    assert_eq!(stock_of(&client, "CHAIR-01").await?, 5); // a Client

    let tx = client.transaction().await?;
    receive(&tx, "CHAIR-01", 3).await?; // a Transaction
    assert_eq!(stock_of(&tx, "CHAIR-01").await?, 8);
    tx.rollback().await?;

    assert_eq!(stock_of(&client, "CHAIR-01").await?, 5);
    Ok(())
}
```

Three things to know about it. It is *sealed*: only tokio-postgres can implement it, so you cannot add your own client type. Its async methods are written with the `async-trait` style and return boxed `Send` futures, which is why generic functions over it spawn without trouble. And it is not object-safe, so use it as a generic bound (`&impl GenericClient`), not as `&dyn GenericClient`. Pools have their own version: deadpool-postgres's `GenericClient` adds `prepare_cached`, and stockroom's repository functions use that one (Chapter 14).

8.6 Retrying transactions that lose a race

At REPEATABLE READ and SERIALIZABLE, PostgreSQL resolves conflicts between concurrent transactions by failing one of them with SQLSTATE 40001, *serialization failure*. Deadlocks are resolved the same way at any level, with 40P01. Neither error says anything is wrong with the transaction itself: run it again, from `BEGIN`, and it will most likely succeed.

Here is a conflict that only SERIALIZABLE catches. Two doctors are on call, and the rule is that at least one must stay on call. Two transactions each check the rule and then take one doctor off; each change is fine on its own, but together they break the rule (a *write skew*):

```
use tokio_postgres::IsolationLevel;
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut first = playground::connect().await?;
    let mut second = playground::connect().await?;
    first
        .batch_execute(
            "DROP SCHEMA IF EXISTS ch08_oncall CASCADE; CREATE SCHEMA ch08_oncall;
             CREATE TABLE ch08_oncall.doctors (name text PRIMARY KEY, on_call bool);
             INSERT INTO ch08_oncall.doctors VALUES ('alice', true), ('bob', true);",
        )
        .await?;
    let count = "SELECT count(*) FROM ch08_oncall.doctors WHERE on_call";
    let leave = "UPDATE ch08_oncall.doctors SET on_call = false WHERE name = $1";

    let a = first
        .build_transaction()
        .isolation_level(IsolationLevel::Serializable)
        .start()
        .await?;
    let b = second
        .build_transaction()
        .isolation_level(IsolationLevel::Serializable)
        .start()
        .await?;
    // Both check the rule: two on call, so one may leave.
    assert_eq!(a.query_one(count, &[]).await?.get::<_, i64>(0), 2);
    assert_eq!(b.query_one(count, &[]).await?.get::<_, i64>(0), 2);
    a.execute(leave, &[&"alice"]).await?;
    b.execute(leave, &[&"bob"]).await?;
    a.commit().await?;
    // Committing b too would leave nobody on call. SERIALIZABLE notices.
    let error = b.commit().await.unwrap_err();
    assert_eq!(error.code(), Some(&SqlState::T_R_SERIALIZATION_FAILURE));
    println!("{}", error.as_db_error().unwrap().message());

    first
        .batch_execute("DROP SCHEMA ch08_oncall CASCADE")
        .await?;
```

```
    Ok(())
}
```

```
could not serialize access due to read/write dependencies among transactions
```

At READ COMMITTED, both commits succeed and nobody is on call. At SERIALIZABLE, the second transaction fails, and the right reaction is to run it again: its new check sees one doctor on call, and it declines. Code that uses SERIALIZABLE must therefore be ready to retry, which calls for a helper: run the transaction, and on 40001 or 40P01, wait a little and start over.

Modern Rust writes the "transaction body" as an *async closure* (`async |tx| …`, stable since Rust 1.85), which the helper calls once per attempt with that attempt's transaction:

```
use std::time::Duration;
use tokio_postgres::error::SqlState;
use tokio_postgres::{Client, Error, IsolationLevel, Transaction};

/// Runs `work` in a SERIALIZABLE transaction, again from the start after a serialization
/// failure or a deadlock, up to 10 times.
async fn serializable<T>(
    client: &mut Client,
    mut work: impl AsyncFnMut(&Transaction<'_>) -> Result<T, Error>,
) -> Result<T, Error> {
    let mut attempt = 1;
    loop {
        let tx = client
            .build_transaction()
            .isolation_level(IsolationLevel::Serializable)
            .start()
            .await?;
        let outcome = match work(&tx).await {
            Ok(value) => tx.commit().await.map(|()| value),
            Err(error) => Err(error), // `tx` is dropped here: rolled back
        };
        match outcome {
            Err(e)
                if attempt < 10
                    && matches!(
                        e.code(),
                        Some(
                            &SqlState::T_R_SERIALIZATION_FAILURE |
&SqlState::T_R_DEADLOCK_DETECTED
                        )
                    ) =>
            {
                // A random pause, so that the losers do not collide again in lockstep.
                tokio::time::sleep(Duration::from_millis(fastrand_ms(attempt))).await;
                attempt += 1;
            }
            other => return other,
        }
    }
}

/// Up to 2, 4, 8 … ms, capped at 100, at random (no extra crate needed for an example).
fn fastrand_ms(attempt: u32) -> u64 {
    let ceiling = (2u64 << attempt).min(100);
    let nanos = std::time::SystemTime::now()
        .duration_since(std::time::UNIX_EPOCH)
        .unwrap()
        .subsec_nanos() as u64;
    1 + nanos % ceiling
}

#[tokio::main]
```

```
async fn main() -> Result<(), Error> {
    let setup = playground::connect().await?;
    setup
        .batch_execute(
            "DROP SCHEMA IF EXISTS ch08_retry CASCADE; CREATE SCHEMA ch08_retry;
             CREATE TABLE ch08_retry.doctors (name text PRIMARY KEY, on_call bool);
             INSERT INTO ch08_retry.doctors VALUES ('alice', true), ('bob', true);",
        )
        .await?;

    // Both doctors try to leave at the same time, each in a task of its own.
    let mut tasks = Vec::new();
    for name in ["alice", "bob"] {
        let name = name.to_owned(); // owned: see below
        tasks.push(tokio::spawn(async move {
            let mut client = playground::connect().await?;
            // `async move`: the closure owns what it uses; every attempt borrows from it.
            serializable(&mut client, async move |tx| {
                let on_call: i64 = tx
                    .query_one("SELECT count(*) FROM ch08_retry.doctors WHERE on_call", &[])
                    .await?
                    .get(0);
                if on_call < 2 {
                    return Ok(false); // someone must stay
                }
                tokio::time::sleep(Duration::from_millis(20)).await; // a slow decision
                tx.execute(
                    "UPDATE ch08_retry.doctors SET on_call = false WHERE name = $1",
                    &[&name],
                )
                .await?;
                Ok(true)
            })
            .await
        }));
    }
    let mut left = 0;
    for task in tasks {
        if task.await.expect("no panic")? {
            left += 1;
        }
    }
    assert_eq!(left, 1, "exactly one doctor left");
    let still: i64 = setup
        .query_one("SELECT count(*) FROM ch08_retry.doctors WHERE on_call", &[])
        .await?
        .get(0);
    assert_eq!(still, 1);
    setup
        .batch_execute("DROP SCHEMA ch08_retry CASCADE")
        .await?;
    Ok(())
}
```

Two rules make such a helper safe.

The body must be safe to run more than once. Everything it does to the database is rolled back with the failed attempt, but anything it does *outside* (sending an email, calling another service, incrementing a counter in memory) happens once per attempt. Keep side effects out of transaction bodies, or record them in the database and act on them after the commit: the outbox pattern (Chapter 16).

The closure must own what it uses, with no references inside. Note the `async move` in `main`, and the `to_owned()` before it: the closure takes ownership of `name` as a `String` (an owned request struct, in a web handler), and each attempt's future borrows from the closure. Had the closure borrowed a local

variable of the task instead, the program would not compile. Surprisingly, neither does capturing the original `name`, a `&'static str`: any reference in the captures trips the same check:

```
let request = load_request();
tokio::spawn(async move {
    serializable(&mut client, async |tx| apply(tx, &request).await).await
    //                        ^^^^^ borrows `request` from the enclosing scope
});
// error: implementation of `Send` is not general enough
```

> **Rust refresher:** `AsyncFnMut(&Transaction<'_>) -> T` is a bound on an async closure that can be called repeatedly, each call borrowing a transaction for some lifetime of its own, and returning a future that may borrow both the transaction and the closure's captured state. The compiler must prove that this future is `Send` *for every possible lifetime* before `tokio::spawn` (or axum) accepts it. With owned captures it can; when the captures contain a reference (to the caller's locals, or even a `&'static str`), today's compiler cannot, and reports "not general enough", with a note about "`Send` … for any lifetime `'0`". The fix is to move owned data into the closure, as above. In older code you will also see the pre-async-closure form, `for<'t> FnMut(&'t Transaction<'t>) -> BoxFuture<'t, Result<T>>` with `Box::pin(async move { … })` at each call site, which states `Send` explicitly.

8.7 Locks

Isolation decides what a transaction *sees*. Locks decide who *waits*. An `UPDATE` or `DELETE` locks the rows it changes until the transaction ends, and a `SELECT` can take the same lock before it decides anything:

Clause	Meaning
`FOR UPDATE`	Lock the selected rows as if about to update them; others wait to update or lock them
`FOR NO KEY UPDATE`	The same, but foreign-key checks from other tables do not wait (use it when you will not change the key)
`… NOWAIT`	Do not wait for a locked row: fail at once with 55P03
`… SKIP LOCKED`	Do not wait for a locked row: leave it out of the result

`SET LOCAL lock_timeout = '100ms'` bounds the wait for the rest of the transaction (55P03 again when it runs out). *Advisory locks* lock a number you choose instead of a row, for coordination that has no row to lock: `pg_advisory_xact_lock(key)` waits, and `pg_try_advisory_xact_lock(key)` returns `false` instead of waiting. Both are released at the end of the transaction.

```
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut a = playground::connect().await?;
    let mut b = playground::connect().await?;
    a.batch_execute(
        "DROP SCHEMA IF EXISTS ch08_locks CASCADE; CREATE SCHEMA ch08_locks;
         CREATE TABLE ch08_locks.jobs (id int PRIMARY KEY, done bool DEFAULT false);
         INSERT INTO ch08_locks.jobs (id) SELECT generate_series(1, 6);",
```

```
    )
    .await?;
    let claim = "SELECT id FROM ch08_locks.jobs WHERE NOT done ORDER BY id LIMIT 2
                 FOR UPDATE SKIP LOCKED";

    let worker_a = a.transaction().await?;
    let worker_b = b.transaction().await?;
    // Two workers claim jobs at the same time: SKIP LOCKED gives them different ones.
    let ids =
        |rows: Vec<tokio_postgres::Row>| -> Vec<i32> { rows.iter().map(|r|
r.get(0)).collect() };
    let mine = ids(worker_a.query(claim, &[]).await?);
    let theirs = ids(worker_b.query(claim, &[]).await?);
    assert_eq!((mine, theirs), (vec![1, 2], vec![3, 4]));

    // NOWAIT on a row someone holds: an error now, instead of a wait.
    let error = worker_b
        .query(
            "SELECT * FROM ch08_locks.jobs WHERE id = 1 FOR UPDATE NOWAIT",
            &[],
        )
        .await
        .unwrap_err();
    assert_eq!(error.code(), Some(&SqlState::LOCK_NOT_AVAILABLE)); // 55P03
    worker_b.rollback().await?;

    // An advisory lock: a number, not a row. The second taker is told no.
    let locked: bool = worker_a
        .query_one("SELECT pg_try_advisory_xact_lock(42)", &[])
        .await?
        .get(0);
    let also: bool = b
        .query_one("SELECT pg_try_advisory_xact_lock(42)", &[])
        .await?
        .get(0);
    assert!(locked && !also);
    worker_a.commit().await?; // releases the row locks and the advisory lock

    a.batch_execute("DROP SCHEMA ch08_locks CASCADE").await?;
    Ok(())
}
```

`FOR UPDATE SKIP LOCKED` is how a table becomes a job queue that several workers can share without stepping on each other: each worker claims a few rows, and the rows stay claimed exactly as long as its transaction is open. stockroom's outbox worker is built on it (Chapter 16).

> **Pitfall:** locks are held until the transaction ends, so a transaction that waits on something slow outside the database (an HTTP call, a user) holds its locks, and its connection, the whole time. Keep transactions short, and set `idle_in_transaction_session_timeout` on the server as a backstop for the ones that are not.

8.8 Optimistic concurrency: version numbers

Locks are *pessimistic*: they assume a conflict and make others wait. For a change that a user prepares slowly (an edit form open for minutes), holding a lock is out of the question. *Optimistic* concurrency assumes there will be no conflict, and checks at the end: every row carries a version number, an update only applies if the version is still the one the client read, and the update increments it.

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE products (sku text PRIMARY KEY, price_cents bigint,
                                         version int NOT NULL DEFAULT 1);
             INSERT INTO products VALUES ('CHAIR-01', 4999);",
        )
        .await?;
    let update = "UPDATE products SET price_cents = $2, version = version + 1
                  WHERE sku = $1 AND version = $3
                  RETURNING version";

    // Two editors read the product at version 1.
    let seen = 1_i32;
    // The first one saves: version 1 is current, so it applies and becomes version 2.
    let saved = client
        .query_opt(update, &[&"CHAIR-01", &3999_i64, &seen])
        .await?;
    assert_eq!(saved.map(|row| row.get::<_, i32>(0)), Some(2));
    // The second one saves what it saw at version 1: nothing matches, nothing changes.
    let lost = client
        .query_opt(update, &[&"CHAIR-01", &2999_i64, &seen])
        .await?;
    assert!(
        lost.is_none(),
        "a conflict, reported instead of overwriting"
    );

    let price: i64 = client
        .query_one("SELECT price_cents FROM products", &[])
        .await?
        .get(0);
    assert_eq!(price, 3999);
    Ok(())
}
```

Zero rows affected is the conflict signal, and the caller decides what to do: reload and show the user what changed. Over HTTP the version becomes an ETag and the check an `If-Match` header (Chapter 15).

8.9 Selling the last chair, four ways

Back to the five chairs and twenty customers. Each strategy below runs twenty concurrent tasks, each with its own connection, all started at the same moment, and counts what was sold.

```
use std::sync::Arc;
use std::time::Duration;
use tokio::sync::Barrier;
use tokio_postgres::error::SqlState;
use tokio_postgres::{Client, Error, IsolationLevel};

const CUSTOMERS: usize = 20;
const CHAIRS: i32 = 5;

#[derive(Clone, Copy, Debug)]
enum Strategy {
    ReadThenWrite, // the bug
    GuardedUpdate,
    ForUpdate,
    SerializableRetry,
}
```

```rust
/// One customer buys one chair; returns whether they got it.
async fn buy(client: &mut Client, strategy: Strategy) -> Result<bool, Error> {
    let available = "SELECT on_hand - reserved FROM ch08_chairs.stock";
    let reserve = "UPDATE ch08_chairs.stock SET reserved = reserved + 1";
    match strategy {
        Strategy::ReadThenWrite => {
            let left: i32 = client.query_one(available, &[]).await?.get(0);
            tokio::time::sleep(Duration::from_millis(20)).await; // thinking…
            if left < 1 {
                return Ok(false);
            }
            client.execute(reserve, &[]).await?;
            Ok(true)
        }
        Strategy::GuardedUpdate => {
            // The check and the change in one statement. A concurrent update makes this one
            // wait, then re-check the condition against the row as the other one left it.
            let changed = client
                .execute(
                    "UPDATE ch08_chairs.stock SET reserved = reserved + 1
                     WHERE on_hand - reserved >= 1",
                    &[],
                )
                .await?;
            Ok(changed == 1)
        }
        Strategy::ForUpdate => {
            let tx = client.transaction().await?;
            // Lock first: everyone else queues here until this transaction ends.
            let left: i32 = tx
                .query_one(&format!("{available} FOR UPDATE"), &[])
                .await?
                .get(0);
            tokio::time::sleep(Duration::from_millis(20)).await;
            if left < 1 {
                return Ok(false); // dropping `tx` rolls back and releases the lock
            }
            tx.execute(reserve, &[]).await?;
            tx.commit().await?;
            Ok(true)
        }
        Strategy::SerializableRetry => loop {
            let tx = client
                .build_transaction()
                .isolation_level(IsolationLevel::Serializable)
                .start()
                .await?;
            let attempt: Result<bool, Error> = async {
                let left: i32 = tx.query_one(available, &[]).await?.get(0);
                if left < 1 {
                    return Ok(false);
                }
                tx.execute(reserve, &[]).await?;
                Ok(true)
            }
            .await;
            match attempt {
                Ok(bought) => match tx.commit().await {
                    Ok(()) => return Ok(bought),
                    Err(e) if e.code() == Some(&SqlState::T_R_SERIALIZATION_FAILURE) => {}
                    Err(e) => return Err(e),
                },
                Err(e) if e.code() == Some(&SqlState::T_R_SERIALIZATION_FAILURE) => {}
                Err(e) => return Err(e),
            }
            // Lost the race: try again from BEGIN, after a short pause.
            tokio::time::sleep(Duration::from_millis(2)).await;
        },
    }
}
```

```
async fn run(strategy: Strategy) -> Result<(i64, i32), Error> {
    let setup = playground::connect().await?;
    setup
        .batch_execute(&format!(
            "DROP SCHEMA IF EXISTS ch08_chairs CASCADE; CREATE SCHEMA ch08_chairs;
             CREATE TABLE ch08_chairs.stock (on_hand int, reserved int);
             INSERT INTO ch08_chairs.stock VALUES ({CHAIRS}, 0);"
        ))
        .await?;
    // Connect everyone first, then start them all at the same moment.
    let start = Arc::new(Barrier::new(CUSTOMERS));
    let mut customers = Vec::new();
    for _ in 0..CUSTOMERS {
        let mut client = playground::connect().await?;
        let start = Arc::clone(&start);
        customers.push(tokio::spawn(async move {
            start.wait().await;
            buy(&mut client, strategy).await
        }));
    }
    let mut sold = 0;
    for customer in customers {
        if customer.await.expect("no panic")? {
            sold += 1;
        }
    }
    let reserved: i32 = setup
        .query_one("SELECT reserved FROM ch08_chairs.stock", &[])
        .await?
        .get(0);
    setup
        .batch_execute("DROP SCHEMA ch08_chairs CASCADE")
        .await?;
    Ok((sold, reserved))
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let (sold, reserved) = run(Strategy::ReadThenWrite).await?;
    println!("read, then write: sold {sold} of {CHAIRS}");
    assert!(sold > CHAIRS as i64 && reserved > CHAIRS, "oversold");

    for strategy in [
        Strategy::GuardedUpdate,
        Strategy::ForUpdate,
        Strategy::SerializableRetry,
    ] {
        let (sold, reserved) = run(strategy).await?;
        println!("{strategy:?}: sold {sold} of {CHAIRS}");
        assert_eq!((sold, reserved), (CHAIRS as i64, CHAIRS), "{strategy:?}");
    }
    Ok(())
}
```

```
read, then write: sold 20 of 5
GuardedUpdate: sold 5 of 5
ForUpdate: sold 5 of 5
SerializableRetry: sold 5 of 5
```

Twenty chairs sold out of five: every customer read "five left" before anyone wrote. The three correct strategies have different costs:

Strategy	How it is correct	Cost
Guarded update	The condition and the change are one statement; after waiting for a row lock, PostgreSQL re-checks the condition against the newest row version	One statement, no retries. The best choice whenever the decision fits in a `WHERE` clause
`SELECT … FOR UPDATE`	Everyone queues on the row lock; the check runs with the lock held	Customers wait for each other, one at a time, for as long as each holds the lock
SERIALIZABLE + retry	Any interleaving that is not equivalent to a serial order fails, and is run again	Retries; the right tool when the rule spans several rows or tables and fits no `WHERE` clause

A `CHECK (reserved <= on_hand)` constraint on the table is the safety net under all of them: if the code is ever wrong, the database refuses the bad row instead of storing it. stockroom has that constraint, and still uses the guarded update.

Side by side – a transaction

The task: move 30 cents between two accounts, both changes or neither.

```
-- psql
tpbook=# BEGIN;
BEGIN
tpbook=*# UPDATE accounts SET cents = cents - 30 WHERE name = 'ada';
UPDATE 1
tpbook=*# UPDATE accounts SET cents = cents + 30 WHERE name = 'bob';
UPDATE 1
tpbook=*# COMMIT;
COMMIT
```

```
// sqlx (illustration)
let mut tx = pool.begin().await?; // a connection from the pool, with BEGIN sent
sqlx::query("UPDATE accounts SET cents = cents - $1 WHERE name =
$2").bind(30_i64).bind("ada").execute(&mut *tx).await?;
sqlx::query("UPDATE accounts SET cents = cents + $1 WHERE name =
$2").bind(30_i64).bind("bob").execute(&mut *tx).await?;
tx.commit().await?; // dropped without commit: rolled back
```

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE accounts (name text PRIMARY KEY, cents bigint);
             INSERT INTO accounts VALUES ('ada', 100), ('bob', 0);",
        )
        .await?;
    let tx = client.transaction().await?;
    let move_money = "UPDATE accounts SET cents = cents + $2 WHERE name = $1";
    tx.execute(move_money, &[&"ada", &-30_i64]).await?;
    tx.execute(move_money, &[&"bob", &30_i64]).await?;
    tx.commit().await?;
    let bob: i64 = client
        .query_one("SELECT cents FROM accounts WHERE name = 'bob'", &[])
        .await?
        .get(0);
    assert_eq!(bob, 30);
    Ok(())
}
```

	psql	sqlx	tokio-postgres
Same	`BEGIN` … `COMMIT` on one session; the prompt shows `*` inside a transaction	A transaction object; rollback when it is dropped uncommitted	A transaction object; rollback when it is dropped uncommitted
Different	Nothing stops you from forgetting `COMMIT`	`pool.begin()` takes a connection out of the pool for the transaction's lifetime	`transaction()` borrows a client mutably: the compiler stops other use of that connection, and a shared (`Arc`) client cannot start one

In stockroom

stockroom's retry helper is §8.6's, with stockroom's error type, deadpool's connections, a proper random backoff, and an explicit rollback so that a failing rollback is at least logged:

```
// stockroom/src/db/retry.rs (excerpt)
pub async fn in_transaction<T>(
    client: &mut Object,
    isolation: IsolationLevel,
    mut work: impl AsyncFnMut(&Transaction<'_>) -> Result<T, AppError>,
) -> Result<T, AppError> {
    let mut attempt = 1;
    loop {
        let tx = client
            .build_transaction()
            .isolation_level(isolation)
            .start()
            .await?;
        let outcome = match work(&tx).await {
            Ok(value) => tx.commit().await.map(|()| value).map_err(AppError::from),
            Err(error) => {
                // Explicitly, rather than on drop, so the next attempt starts from a clean
                // session and a failed rollback is at least logged.
                if let Err(rollback) = tx.rollback().await {
                    tracing::debug!(%rollback, "rollback failed");
                }
                Err(error)
            }
        };
        match outcome {
            Err(error) if error.is_retryable() && attempt < MAX_ATTEMPTS => {
                tracing::debug!(attempt, %error, "transaction failed under concurrency;
retrying");
                tokio::time::sleep(backoff(attempt)).await;
                attempt += 1;
            }
            other => return other,
        }
    }
}
```

`is_retryable` is true for 40001, 40P01 and 55P03, which Chapter 6's `From` impl turned into `AppError::Busy`. A handler hands it an `async move` closure that owns the request it extracted:

```
// stockroom/src/http/orders.rs (excerpt)
pub async fn place(
    Db(mut db): Db,
    AppJson(order): AppJson<NewOrder>,
```

```
) -> Result<impl IntoResponse, AppError> {
    // `async move`: the closure owns the request, and each attempt borrows it from there.
    let order = db::in_transaction(&mut db, IsolationLevel::ReadCommitted, async move |tx| {
        orders::place(tx, &order).await
    })
    .await?;
    Ok((StatusCode::CREATED, etag(order.version), Json(order)))
}
```

Placing an order uses the guarded update from §8.9, one per order line, at READ COMMITTED, so it rarely needs the retry. It only does when two orders deadlock, which the code prevents by locking stock rows in product-id order (Chapter 7 showed why that order is reliable). Transfers between stores are different: they read a stock level, then change two rows, and the read must still be true at the end, so they run at SERIALIZABLE. Their first statement locks both rows, in a fixed order, because the first version of this code deadlocked:

```
// stockroom/src/repo/stock.rs (excerpt)
// Lock both rows first, always in the same order. Otherwise a transfer A→B and one B→A
// each lock one row and wait for the other's: a deadlock, which PostgreSQL only notices
// after `deadlock_timeout` (a second). In this order the second transaction just waits,
// and when the first commits, SERIALIZABLE fails it at once ("could not serialize access
// due to concurrent update") for `in_transaction` to retry.
tx.execute(
    "SELECT FROM stock WHERE product_id = $1 AND store_id IN ($2, $3) ORDER BY store_id FOR
UPDATE",
    &[&product_id, &from_id, &to_id],
)
.await?;
```

The test suite proves all of it with concurrency: tests/concurrency.rs places fifty orders for twenty chairs at once (exactly twenty succeed), runs forty orders for two products listed in either order (no deadlock surfaces), and forty transfers in both directions (no unit is lost).

Summary

- `transaction()` sends `BEGIN`; `commit()` commits; everything else rolls back, including a drop (which cannot report errors) and an early `?`.
- `transaction()` borrows the client mutably: nothing else can use the connection meanwhile, and a shared `Arc<Client>` cannot start a transaction. Concurrent transactions need connections of their own.
- `build_transaction()` sets the isolation level, `read_only` and `deferrable`. READ COMMITTED sees others' commits between statements; REPEATABLE READ keeps one snapshot; SERIALIZABLE fails anything not equivalent to a serial order.
- Savepoints (`savepoint`, nested `transaction`) contain failures within a transaction.
- `GenericClient` lets one function run on a `Client` or a `Transaction`: sealed, boxed `Send` futures, a generic bound rather than `dyn`.
- 40001 and 40P01 mean "run the whole transaction again". A retry helper takes an async closure; the closure must own what it uses, and its body must be safe to repeat.
- `FOR UPDATE`, `NOWAIT`, `SKIP LOCKED`, `lock_timeout` and advisory locks control waiting.
- Optimistic concurrency: a version column, and zero rows updated means someone else won.

- To sell the last chair once: a guarded `UPDATE … WHERE` first, `FOR UPDATE` or SERIALIZABLE with retries when the rule does not fit one statement, and a `CHECK` constraint underneath.

Exercises

1. **Write skew at REPEATABLE READ.** Run §8.6's doctors example at REPEATABLE READ instead of SERIALIZABLE. What happens, and why does snapshot isolation not catch it? *Hint:* each transaction checks a snapshot taken at its start, and neither writes a row that the other one wrote.
2. **A transfer without SERIALIZABLE.** Rewrite stockroom's transfer logic (read the source stock, check it, update two rows) so that it is correct at READ COMMITTED. *Hint:* guarded updates, and `RETURNING`.
3. **A deadlock on purpose.** With two connections, make two transactions update rows 1 and 2 in opposite orders, and observe 40P01. How long did it take, and which setting decides that? *Hint:* `SHOW deadlock_timeout`.
4. **Claim a job.** Using `FOR UPDATE SKIP LOCKED`, write `claim_next_job(tx)` that returns the oldest unclaimed job or `None`, and run three workers concurrently on a table of ten jobs. Check that every job is processed exactly once. *Hint:* `SELECT … ORDER BY id LIMIT 1 FOR UPDATE SKIP LOCKED`, and mark the job done in the same transaction.
5. **Where does the email go?** A handler places an order and sends a confirmation email, inside the retried transaction body. What can go wrong, and how does stockroom avoid it? *Hint:* the body of a retried transaction may run more than once, and a rollback cannot unsend an email.

Solutions

1. Both commits succeed and nobody is on call. REPEATABLE READ gives each transaction a consistent snapshot and fails a transaction that *updates a row someone else changed*, but here the two transactions update *different* rows; each read a row the other changed, and only SERIALIZABLE tracks those read/write dependencies.

2. Do the decrement as a guarded update and the increment unconditionally, in one transaction:

```
UPDATE stock SET on_hand = on_hand - $3
WHERE store_id = $1 AND product_id = $2 AND available >= $3
RETURNING on_hand;   -- no row: not enough stock; roll back
INSERT INTO stock (store_id, product_id, on_hand) VALUES ($4, $2, $3)
ON CONFLICT (store_id, product_id) DO UPDATE SET on_hand = stock.on_hand + EXCLUDED.on_hand;
```

The guard makes the check and the change atomic, and lock order still matters: two opposite transfers can deadlock (40P01), so either lock both rows first in a fixed order, or retry.

3. It takes about one second: the server only looks for deadlocks after a transaction has waited for `deadlock_timeout` (default `1s`), and then fails one of them with 40P01. That second is why stockroom's first version of the transfer was slow under load, before it locked rows in a fixed order.

5. The body can run several times: a serialization failure or deadlock means one email per attempt, and a failed commit after a sent email means an email for an order that does not exist. stockroom never sends anything from a transaction. The order's trigger writes a row to the outbox *in the same*

transaction, so the job exists if and only if the order committed, and a worker sends it afterwards (Chapter 16).

CHAPTER 9

Streaming rows and portals

Every query so far has returned a `Vec<Row>`: the whole result, in memory, before your code sees the first row. That is the right default for the results most programs handle, a page of products or one order and its lines. It is the wrong one for an export of the whole catalog, a report over a year of stock movements, or an HTTP response that should start before the last row is read.

This chapter is about the other ways to read a result. A `RowStream` hands you rows as they arrive. A *portal* goes further: the server produces only the rows you ask for, a batch at a time. SQL has its own version of portals, cursors, and HTTP has its own answer, keyset pagination. Each comes with a cost, and the most important one is easy to miss: a stream that you read slowly holds up its whole connection.

9.1 Why a `Vec<Row>` is not always enough

`query` is a convenience. In 0.7.18 it is literally `query_raw(…)` followed by `try_collect()` (`client.rs`): it reads the stream this chapter is about into a vector and hands you the vector. Two costs follow, and both grow with the result:

- **Memory.** Every row is held at once. Chapter 3's `Row::raw_size_bytes` measures it.
- **Latency.** Nothing reaches your code until the last row has arrived.

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let rows = client
        .query(
            "SELECT i, repeat('x', 100) AS filler FROM generate_series(1, 200000) i",
            &[],
        )
        .await?;
    let bytes: usize = rows.iter().map(|row| row.raw_size_bytes()).sum();
    println!(
        "{} rows, {} MB of row data, all in memory at once",
        rows.len(),
        bytes / 1_000_000
    );
    assert_eq!(rows.len(), 200_000);
    assert!(bytes > 20_000_000);
    Ok(())
}
```

```
200000 rows, 22 MB of row data, all in memory at once
```

Twenty-two megabytes is affordable once. It is not affordable per request when a hundred clients ask for the export at the same time, and it grows with the table. The fix is to handle each row and let it go before the next one arrives.

9.2 `query_raw` and `RowStream`

`query_raw` takes the same statement as `query` and returns a `RowStream`, a `Stream` of `Result<Row, Error>`. Its parameters are different:

```
pub async fn query_raw<T, P, I>(&self, statement: &T, params: I) -> Result<RowStream, Error>
where
    T: ?Sized + ToStatement,
    P: BorrowToSql,
    I: IntoIterator<Item = P>,
    I::IntoIter: ExactSizeIterator,
```

Instead of a slice of `&(dyn ToSql + Sync)`, it takes any iterator of values that can lend a `&dyn ToSql` (`BorrowToSql`), as long as the iterator knows its length: the parameter count is checked against the statement before anything is sent. The `_raw` family shares that signature: `query_raw`, `execute_raw`, and, on a `Transaction`, `bind_raw` (§9.4). `query_typed_raw` takes `(value, Type)` pairs instead.

To pass the usual mixed list of parameters, you need a small adapter from a slice to that iterator. tokio-postgres has one internally but does not export it, so add it to the playground's `src/lib.rs`:

```
use tokio_postgres::types::ToSql;

/// Adapts a parameter slice, the kind `query` takes, to the iterator that the `*_raw`
/// methods take: `client.query_raw(&statement, slice_iter(&[&a, &b]))`.
pub fn slice_iter<'a>(
    params: &'a [&'a (dyn ToSql + Sync)],
) -> impl ExactSizeIterator<Item = &'a dyn ToSql> + Send + Sync + 'a {
    params.iter().map(|p| *p as _)
}
```

Reading a stream then looks like this. `try_next` comes from `futures_util::TryStreamExt`:

```
use futures_util::TryStreamExt;
use std::pin::pin;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let statement = client
        .prepare("SELECT n, $2::text || n FROM generate_series(1, $1::int) n")
        .await?;
    let rows = client
        .query_raw(&statement, playground::slice_iter(&[&5i32, &"item "]))
        .await?;
    let mut rows = pin!(rows);

    // The number of rows is in the message that ends the result, so it is not known yet.
    assert_eq!(rows.rows_affected(), None);

    let mut labels = Vec::new();
    while let Some(row) = rows.try_next().await? {
        labels.push(row.get::<_, String>(1)); // handle the row; then it is dropped
    }
    assert_eq!(labels, ["item 1", "item 2", "item 3", "item 4", "item 5"]);
    assert_eq!(rows.rows_affected(), Some(5));
```

```
    Ok(())
}
```

`rows_affected()` reads the count from the *CommandComplete* message that ends every result, so it is `None` until the stream has been read to the end. For an `INSERT … RETURNING` whose output you stream, it gives the count without your counting the rows.

When the parameters all have one type, or are decided at run time, the adapter is not needed:

```
use futures_util::TryStreamExt;
use tokio_postgres::types::{ToSql, Type};

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;

    // All of one type: any collection of values, by value or by reference.
    let words: Vec<String> = vec!["stock".into(), "room".into()];
    let rows: Vec<_> = client
        .query_raw("SELECT $1 || $2", words)
        .await?
        .try_collect()
        .await?;
    assert_eq!(rows[0].get::<_, String>(0), "stockroom");

    client
        .batch_execute("CREATE TEMP TABLE word (w text)")
        .await?;
    let inserted = client
        .execute_raw("INSERT INTO word VALUES ($1), ($2)", ["stock", "room"])
        .await?;
    assert_eq!(inserted, 2);

    // Decided at run time: boxed trait objects. Only the placeholder *numbers* are
formatted
    // into the SQL; the values travel as parameters.
    let (min_price, tag): (Option<i64>, Option<&str>) = (Some(500), Some("a"));
    let mut sql = String::from(
        "SELECT count(*) FROM (VALUES (100::bigint, 'a'), (900, 'b'), (1200, 'a')) AS
p(price, tag)
         WHERE true",
    );
    let mut params: Vec<Box<dyn ToSql + Sync + Send>> = Vec::new();
    if let Some(min) = min_price {
        params.push(Box::new(min));
        sql += &format!(" AND price >= ${}", params.len());
    }
    if let Some(tag) = tag {
        params.push(Box::new(tag.to_owned()));
        sql += &format!(" AND tag = ${}", params.len());
    }
    let rows: Vec<_> = client.query_raw(&sql, params).await?.try_collect().await?;
    assert_eq!(rows[0].get::<_, i64>(0), 1);

    // query_typed_raw: each value with its type, no prepare round trip (Chapter 4).
    let rows: Vec<_> = client
        .query_typed_raw(
            "SELECT $1 + 1, $2",
            [
                (&41i32 as &dyn ToSql, Type::INT4),
                (&"answer" as &dyn ToSql, Type::TEXT),
            ],
        )
        .await?
        .try_collect()
        .await?;
    assert_eq!(rows[0].get::<_, i32>(0), 42);
```

```
    Ok(())
}
```

Rust refresher: `try_next` and `next` need a stream that is `Unpin`, and `RowStream` is declared `!Unpin`, so it must be *pinned* first: promised never to move again. There are two ways. `std::pin::pin!(rows)` pins it where it is, on the stack, and gives you a `Pin<&mut RowStream>` that you can poll but not move or return. `Box::pin(rows)` moves it to the heap and gives you a `Pin<Box<RowStream>>`, which you can return from a function, store in a struct or send to a task, at the cost of one allocation. Combinators that take the stream by value, like `try_collect` above or `try_fold`, pin it themselves.

Pitfall: `drop(rows)` on a stream pinned with `pin!` drops the `Pin<&mut _>`, a reference, and the stream itself lives on until the end of the enclosing block. It compiles without a warning, and §9.3 shows what an unread stream does to its connection: the next query on the client never finishes. To end a stream early, let it go out of scope (put it in a block), or pin it with `Box::pin`, which owns it, so that `drop` really drops it.

9.3 Backpressure, and the stalled connection

A stream does not make the server wait for you. The server runs the query and sends rows as fast as the socket accepts them. What makes it wait is that tokio-postgres stops *reading* the socket when you stop reading the stream.

Protocol lens: each request's responses go to the caller through a channel created with a capacity of one (`mpsc::channel(1)` in `client.rs`). When that channel is full, the connection task parks the message it is holding and stops reading the socket. The operating system's receive buffer fills, TCP tells the server to stop sending, and the server's backend blocks in a write, which `pg_stat_activity` shows as the wait event `ClientWrite`. Memory stays flat all the way down. The price is that one socket serves the whole connection: nothing else queued on it can be answered until the stream moves again.

Here is that state, produced on purpose: read one row of a large result, then try another query on the same client:

```
use futures_util::TryStreamExt;
use std::pin::pin;
use std::time::Duration;
use tokio::time::timeout;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
```

```
    let other = playground::connect().await?;
    let pid: i32 = client
        .query_one("SELECT pg_backend_pid()", &[])
        .await?
        .get(0);
    let wait = Duration::from_millis(500);
    {
        let rows = client
            .query_raw(
                "SELECT i, repeat('x', 100) FROM generate_series(1, 200000) i",
                playground::slice_iter(&[]),
            )
            .await?;
        let mut rows = pin!(rows);
        let first = rows.try_next().await?.expect("a first row");
        assert_eq!(first.get::<_, i32>(0), 1);

        // The stream is not being read. A query on the same client waits behind it...
        let same = timeout(wait, client.query_one("SELECT 1", &[])).await;
        // ...while the same query on another connection does not.
        let elsewhere = timeout(wait, other.query_one("SELECT 1", &[])).await;
        // And the server is stuck writing to the client.
        let event: Option<String> = other
            .query_one(
                "SELECT wait_event FROM pg_stat_activity WHERE pid = $1",
                &[&pid],
            )
            .await?
            .get(0);
        println!(
            "same connection timed out: {}; other connection timed out: {}; server waits on
{:?}",
            same.is_err(),
            elsewhere.is_err(),
            event
        );
        assert!(same.is_err());
        assert!(elsewhere.is_ok());
        assert_eq!(event.as_deref(), Some("ClientWrite"));
    } // the stream is dropped here

    // With the stream gone, the connection discards the rest of its rows and the queue
moves.
    client.query_one("SELECT 1", &[]).await?;
    Ok(())
}
```

```
same connection timed out: true; other connection timed out: false; server waits on
Some("ClientWrite")
```

Three consequences shape the rest of this chapter and Part III:

- **A `RowStream` does not borrow the `Client`.** The compiler lets you hold it for as long as you like, pass it to another task, or return it from a function, and the connection stalls for all that time. A slow consumer is a slow query, as Chapter 7 put it, as far as everyone else on the connection is concerned.
- **Stream on a connection of its own.** Never stream on a client that other tasks share. In a service, the stream should own the pooled connection it reads from, and keep it until the last row (stockroom's `/products/stream`, at the end of this chapter).
- **Dropping a stream does not stop the query.** The connection reads and throws away every remaining row, and the server computes all of them. §9.4 measures what that costs and shows the alternative. Chapter 12 shows how to cancel a query outright.

9.4 Portals: the server produces only what you ask for

When you run a query through the extended protocol, the server keeps its execution state in a *portal*. tokio-postgres normally uses the unnamed portal and asks for every row at once. Inside a transaction, you can make a named portal yourself with `Transaction::bind` and then ask for rows with `query_portal(&portal, max_rows)`. Each call is one round trip and returns at most `max_rows` rows; between calls, the query is suspended on the server, with its place kept:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut client = playground::connect().await?;
    let tx = client.transaction().await?;
    let statement = tx.prepare("SELECT generate_series(1, 10)").await?;
    let portal = tx.bind(&statement, &[]).await?;

    let mut batches = Vec::new();
    loop {
        let rows = tx.query_portal(&portal, 4).await?;
        batches.push(rows.len());
        if rows.len() < 4 {
            break; // a short batch is the last one
        }
    }
    assert_eq!(batches, [4, 4, 2]);
    // An exhausted portal returns nothing.
    assert!(tx.query_portal(&portal, 4).await?.is_empty());
    tx.commit().await?;
    Ok(())
}
```

If the total is a multiple of the batch size, the last full batch is followed by an empty one, which the same `<` test handles. `bind_raw` and `query_portal_raw` are the iterator and stream versions. Two details of the stream version, and the portal's lifetime, are worth seeing once:

```
use futures_util::TryStreamExt;
use std::pin::pin;
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut client = playground::connect().await?;
    let tx = client.transaction().await?;
    let statement = tx.prepare("SELECT generate_series(1, $1::int)").await?;
    let portal = tx.bind_raw(&statement, [7i32]).await?;

    // A batch of 3 as a stream. The portal is suspended, not finished: no count.
    let batch = tx.query_portal_raw(&portal, 3).await?;
    let mut batch = pin!(batch);
    let mut n = 0;
    while batch.try_next().await?.is_some() {
        n += 1;
    }
    assert_eq!((n, batch.rows_affected()), (3, None));

    // max_rows = 0 means "all that is left". The portal completes, and the count it reports
    // covers this last batch only.
    let rest = tx.query_portal_raw(&portal, 0).await?;
    let mut rest = pin!(rest);
    let mut m = 0;
    while rest.try_next().await?.is_some() {
        m += 1;
    }
    assert_eq!((m, rest.rows_affected()), (4, Some(4)));
    tx.commit().await?;
```

```
    // A portal lives and dies with its transaction.
    let tx = client.transaction().await?;
    let err = tx.query_portal(&portal, 1).await.unwrap_err();
    assert_eq!(err.code(), Some(&SqlState::INVALID_CURSOR_NAME)); // 34000
    println!("{}", err.as_db_error().expect("a server error").message());
    Ok(())
}
```

```
portal "p0" does not exist
```

The compiler does not tie a `Portal` to its transaction, so using one after the commit is a runtime error from the server. Dropping a `Portal` sends a *Close* for it, and closing a portal that no longer exists is not an error, so the drop itself is harmless.

Stopping early

The difference between a portal and a stream shows when you stop before the end. A stream asked the server for everything, so stopping early still costs the whole result (§9.3). A portal asked for ten rows, so ten rows is all the server made:

```
use futures_util::TryStreamExt;
use std::pin::pin;
use std::time::Instant;

const MANY: &str = "SELECT generate_series(1, 3000000)";

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut client = playground::connect().await?;

    // A stream: read one row, drop it, and wait until the connection is free again.
    let started = Instant::now();
    {
        let rows = client.query_raw(MANY, playground::slice_iter(&[])).await?;
        let mut rows = pin!(rows);
        rows.try_next().await?;
    }
    client.query_one("SELECT 1", &[]).await?;
    let stream_cost = started.elapsed();

    // A portal: ask for ten rows, then end the transaction.
    let started = Instant::now();
    let tx = client.transaction().await?;
    let portal = tx.bind(MANY, &[]).await?;
    assert_eq!(tx.query_portal(&portal, 10).await?.len(), 10);
    tx.commit().await?;
    client.query_one("SELECT 1", &[]).await?;
    let portal_cost = started.elapsed();

    println!("stopping after the first rows: stream {stream_cost:?}, portal
{portal_cost:?}");
    assert!(portal_cost < stream_cost / 10);
    Ok(())
}
```

```
stopping after the first rows: stream 181.19775ms, portal 554.125µs
```

(Measured on localhost against PG 18.6, in a debug build. The stream's cost is the server producing three million rows that nobody reads.)

> **SQL lens:** a portal stops early only if the plan can. `generate_series` in the select list produces one row at a time, as the example needs. In `FROM`, as `SELECT i FROM generate_series(1, 3000000) i`, it is a function scan that computes the whole series before returning the first row: the same ten-row portal took 111 ms. An `ORDER BY` that no index provides, a `GROUP BY` or a `DISTINCT` likewise read all their input first. Check the plan with `EXPLAIN` before relying on a portal to save work.

A portal keeps its transaction open, and with it a snapshot, for as long as you page through it. On a busy database that has a cost of its own: the rows that other transactions delete or update cannot be vacuumed away while your snapshot might still need them. Keep portal transactions as short as the job allows, and read-only. stockroom's export, at the end of this chapter, uses the isolation level made for this job.

9.5 The SQL alternative: `DECLARE` and `FETCH`

A portal made with `bind` is the protocol's version of a SQL *cursor*. The server treats them as the same kind of object, and the `pg_cursors` view lists both:

```
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut client = playground::connect().await?;

    // Outside a transaction, a plain cursor would end with its own statement, so the server
    // refuses to create one.
    let err = client
        .batch_execute("DECLARE numbers CURSOR FOR SELECT 1")
        .await
        .unwrap_err();
    assert_eq!(err.code(), Some(&SqlState::NO_ACTIVE_SQL_TRANSACTION)); // 25P01

    let tx = client.transaction().await?;
    tx.batch_execute("DECLARE numbers CURSOR FOR SELECT generate_series(1, 10)")
        .await?;
    let first = tx.query("FETCH 4 FROM numbers", &[]).await?;
    let rest = tx.query("FETCH ALL FROM numbers", &[]).await?;
    assert_eq!((first.len(), rest.len()), (4, 6));

    // A portal made with bind() sits beside the cursor.
    let statement = tx.prepare("SELECT 42").await?;
    let _portal = tx.bind(&statement, &[]).await?;
    let names: Vec<String> = tx
        .query(
            "SELECT name FROM pg_cursors WHERE name <> '' ORDER BY name",
            &[],
        )
        .await?
        .iter()
        .map(|row| row.get(0))
        .collect();
    println!("{names:?}");
    assert_eq!(names.len(), 2);
    assert!(names.contains(&"numbers".to_owned()));
    tx.commit().await?;

    // WITH HOLD: the cursor outlives the transaction that created it.
    client
        .batch_execute("DECLARE kept CURSOR WITH HOLD FOR SELECT generate_series(1, 10)")
```

```
        .await?;
    assert_eq!(client.query("FETCH 4 FROM kept", &[]).await?.len(), 4);
    client.batch_execute("CLOSE kept").await?;
    Ok(())
}
```

```
["numbers", "p0"]
```

(The unnamed portal, `''`, is the one running the `pg_cursors` query itself. tokio-postgres names its portals `p0`, `p1` and so on, from a counter shared by the whole program.)

So when would you write `DECLARE` instead of calling `bind`? When you need what only the SQL form offers:

- **`WITH HOLD`**, to keep reading after the transaction commits. The documentation is frank about the price: "the rows represented by a held cursor are copied into a temporary file or memory area so that they remain available for subsequent transactions." The whole remaining result is computed at the commit.
- **`SCROLL`**, to move backwards or jump (`FETCH PRIOR`, `FETCH ABSOLUTE 100`, `MOVE`).
- **A cursor driven from SQL**, by a PL/pgSQL function or by another tool that only speaks SQL.

Otherwise prefer `bind`: the parameters are typed and sent separately, no names to invent, and the portal is closed for you when it is dropped. Each `FETCH` is also a statement in its own right, so it pays the prepare round trip of Chapter 4 unless you prepare it once.

9.6 Keyset pagination: the HTTP answer

Neither a stream nor a portal survives the end of an HTTP request: the next page arrives as a new request, often on another connection from the pool, in another transaction. The page has to be described in the query itself. `LIMIT … OFFSET …` is the obvious way, and it gets slower the deeper you go, because, in the words of the documentation, "the rows skipped by an `OFFSET` clause still have to be computed inside the server".

Keyset pagination (also called the seek method) remembers the last key of the page instead, and asks for the rows after it. With an index on the key, every page costs the same. `EXPLAIN ANALYZE` makes the difference countable:

```
use tokio_postgres::{Client, Error};

/// How many rows the scan under the `LIMIT` produced, from `EXPLAIN (ANALYZE, FORMAT
JSON)`.
async fn rows_scanned(client: &Client, explain: &str) -> Result<f64, Error> {
    let plan: serde_json::Value = client.query_one(explain, &[]).await?.get(0);
    Ok(plan[0]["Plan"]["Plans"][0]["Actual Rows"]
        .as_f64()
        .expect("a row count"))
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
```

```
            "CREATE TEMP TABLE item (id int PRIMARY KEY, name text);
             INSERT INTO item SELECT i, 'item ' || i FROM generate_series(1, 100000) i;
             ANALYZE item;",
        )
        .await?;

    // Page 1,981, fifty rows a page, both ways.
    let offset = rows_scanned(
        &client,
        "EXPLAIN (ANALYZE, FORMAT JSON, COSTS OFF, TIMING OFF)
         SELECT * FROM item ORDER BY id LIMIT 50 OFFSET 99000",
    )
    .await?;
    let keyset = rows_scanned(
        &client,
        "EXPLAIN (ANALYZE, FORMAT JSON, COSTS OFF, TIMING OFF)
         SELECT * FROM item WHERE id > 99000 ORDER BY id LIMIT 50",
    )
    .await?;
    println!("OFFSET read {offset} rows from the index; keyset read {keyset}");
    assert_eq!((offset, keyset), (99_050.0, 50.0));

    // Walking the whole table: each page starts after the last id of the one before.
    let page = client
        .prepare("SELECT id FROM item WHERE id > $1 ORDER BY id LIMIT $2")
        .await?;
    let (mut after, mut pages) = (0i32, 0);
    loop {
        let rows = client.query(&page, &[&after, &1000i64]).await?;
        let Some(last) = rows.last() else { break };
        after = last.get(0);
        pages += 1;
    }
    assert_eq!((pages, after), (100, 100_000));
    Ok(())
}
```

```
OFFSET read 99050 rows from the index; keyset read 50
```

(PostgreSQL 18 reports actual row counts with decimals, `99050.0` in the JSON, which is why the helper reads an `f64`.)

Three rules make keyset pagination correct, not just fast:

- **The key must be unique.** If the sort column has duplicates (a price, a date), add a tie-breaker and compare row values: `WHERE (price, id) > ($1, $2) ORDER BY price, id`, with an index on `(price, id)`. Exercise 3 builds it.
- **The client gets an opaque cursor, not the key.** Encode the last key (base64 of JSON, for instance) so that clients cannot start building URLs from your column values, and so that you can change the key later.
- **Ask for one row more than the page.** If it comes back, there is a next page; drop it and return a cursor. If not, this was the last page, and no empty request is needed to find out.

Unlike a portal, keyset pages are separate queries with separate snapshots: a row inserted between two requests appears if it sorts after the cursor. For an HTTP API, that is usually what you want.

9.7 `simple_query_raw`

The simple protocol (Chapter 2) has a streaming form too. `simple_query_raw` returns a `SimpleQueryStream` of the same `SimpleQueryMessage`s that `simple_query` collects into a vector, for any number of statements:

```
use futures_util::TryStreamExt;
use std::pin::pin;
use tokio_postgres::SimpleQueryMessage;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let messages = client
        .simple_query_raw("SELECT generate_series(1, 3) AS n; SELECT 'done' AS status")
        .await?;
    let mut messages = pin!(messages);
    let mut seen = Vec::new();
    while let Some(message) = messages.try_next().await? {
        seen.push(match message {
            SimpleQueryMessage::RowDescription(columns) => format!("columns {}",
columns[0].name()),
            SimpleQueryMessage::Row(row) => format!("row {}", row.get(0).unwrap_or("NULL")),
            SimpleQueryMessage::CommandComplete(n) => format!("complete {n}"),
            _ => "something new".to_owned(), // the enum is #[non_exhaustive]
        });
    }
    assert_eq!(
        seen,
        [
            "columns n",
            "row 1",
            "row 2",
            "row 3",
            "complete 3",
            "columns status",
            "row done",
            "complete 1",
        ]
    );
    Ok(())
}
```

It is the tool for programs that run SQL they did not write, the way psql does. pgq's `--stream` mode is built on it, and prints each row as soon as it arrives:

```
// apps/pgq/src/lib.rs (excerpt)
pub async fn stream(client: &Client, sql: &str, mut print: impl FnMut(&str)) -> Result<u64,
Error> {
    let mut messages = pin!(client.simple_query_raw(sql).await?);
    let mut rows = 0;
    while let Some(message) = messages.try_next().await? {
        match message {
            SimpleQueryMessage::RowDescription(columns) => {
                let names: Vec<&str> = columns.iter().map(|c| c.name()).collect();
                print(&names.join("\t"));
            }
            SimpleQueryMessage::Row(row) => {
                let cells: Vec<&str> = (0..row.len()).map(|i|
row.get(i).unwrap_or("")).collect();
                print(&cells.join("\t"));
                rows += 1;
            }
            _ => {}
        }
    }
    Ok(rows)
}
```

apps/pgq/src/lib.rs

Everything in §9.3 applies unchanged: the same channel, the same stall.

Side by side — *iterating over a million rows*

The task: add up a million values without holding them all in memory.

```
-- psql prints the rows in groups of FETCH_COUNT as they arrive, instead of collecting the
-- whole result first. (psql 18 declares no cursor for this: pg_cursors stays empty.)
tpbook=# \set FETCH_COUNT 1000
tpbook=# \o /dev/null
tpbook=# SELECT generate_series(1, 1000000) AS n;
tpbook=# \o
-- psql cannot add them up itself; the server can, and sends one row:
tpbook=# SELECT sum(n) FROM generate_series(1, 1000000) n;
     sum
--------------
 500000500000
(1 row)
```

```
// sqlx (illustration)
use futures_util::TryStreamExt;
let mut rows = sqlx::query_scalar::<_, i32>("SELECT generate_series(1,
1000000)").fetch(&pool);
let mut sum = 0i64;
while let Some(n) = rows.try_next().await? {
    sum += i64::from(n);
}
```

```
use futures_util::TryStreamExt;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let rows = client
        .query_raw(
            "SELECT generate_series(1, 1000000)",
            playground::slice_iter(&[]),
        )
        .await?;
    // try_fold takes the stream by value and pins it itself.
    let sum = rows
        .try_fold(0i64, |sum, row| async move {
            Ok(sum + i64::from(row.get::<_, i32>(0)))
        })
        .await?;
    assert_eq!(sum, 500_000_500_000);
    Ok(())
}
```

	psql	sqlx	tokio-postgres
Same	Rows are handled in groups as they arrive; memory stays flat	`fetch` returns a stream of rows	`query_raw` returns a stream of rows
Different	A display setting, `FETCH_COUNT`, rather than an API	The stream borrows its executor (`&pool` here), and a pool lends it a connection of its own	The stream borrows nothing, and stalls its connection until it is read or dropped

The psql line deserves its comment: the best way to add up a million values is to let the server do it and send one row. Stream when the client really needs every row.

In stockroom

`GET /products/stream` sends the whole catalog as newline-delimited JSON, one line per row, with memory use that does not grow with the catalog. The handler takes a connection from the pool, starts the query, and hands the stream to the response body:

```rust
// stockroom/src/http/products.rs (excerpt)
pub async fn stream(Db(db): Db) -> Result<Response, AppError> {
    let stmt = db.prepare_cached(products::ALL_IN_SKU_ORDER).await?;
    let rows = db.query_raw(&stmt, db::slice_iter(&[])).await?;
    let body = Body::from_stream(ndjson(db, rows));
    Ok(([(header::CONTENT_TYPE, "application/x-ndjson")], body).into_response())
}

/// The body stream owns the pooled connection, so the connection stays checked out until
the
/// last row is sent. `RowStream` does not borrow the client: had `db` gone back to the pool
/// when the handler returned, the next borrower's queries would queue behind these rows.
fn ndjson(db: Object, rows: RowStream) -> impl Stream<Item = Result<Bytes, AppError>> + Send
{
    async_stream::try_stream! {
        let _checked_out = db;
        let mut rows = pin!(rows);
        while let Some(row) = rows.try_next().await? {
            let mut line = serde_json::to_vec(&Product::try_from(&row)?)?;
            line.push(b'\n');
            yield Bytes::from(line);
        }
    }
}
```

stockroom/src/http/products.rs

The line `let _checked_out = db;` is the whole of §9.3's advice in one statement. The handler returns as soon as the query has started, long before the last row is sent. If `db`, the pooled connection, were dropped with the handler, it would go back to the pool with a stream still running on it, and the next request to borrow it would wait behind the rest of the catalog. Moving it into the body's stream keeps it checked out until the last row is written, or until the client hangs up and axum drops the body. The underscore name keeps the value alive; a bare `_` would drop it on the spot. And the backpressure works end to end: a slow HTTP client slows the body, which slows the stream, which leaves the server waiting in `ClientWrite`, all without buffering.

`stockroom export products` reads the same catalog through a portal instead, in batches of `--batch` rows (500 by default), inside the transaction mode made for long reads:

```rust
// stockroom/src/cli/export.rs (excerpt)
/// A portal lives in a transaction. `SERIALIZABLE, READ ONLY, DEFERRABLE` is the one made
/// for exports: it waits, if needed, for a moment when a consistent snapshot is guaranteed,
/// and then can never fail with a serialization error, nor slow anyone else down.
async fn products(
    client: &mut Client,
    batch: i32,
    out: &mut (impl AsyncWrite + Unpin),
```

```
) -> anyhow::Result<u64> {
    let tx = client
        .build_transaction()
        .isolation_level(IsolationLevel::Serializable)
        .read_only(true)
        .deferrable(true)
        .start()
        .await?;
    let statement = tx.prepare(ALL_IN_SKU_ORDER).await?;
    let portal = tx.bind(&statement, &[]).await?;
    let mut total = 0;
    loop {
        let rows = tx.query_portal(&portal, batch).await?;
        for row in &rows {
            let mut line = serde_json::to_vec(&Product::try_from(row)?)?;
            line.push(b'\n');
            out.write_all(&line).await?;
        }
        total += rows.len() as u64;
        if rows.len() < batch as usize {
            break; // a short batch is the last one
        }
    }
    tx.commit().await?;
    Ok(total)
}
```

stockroom/src/cli/export.rs

Why a portal here and a stream in the HTTP handler? The export writes to a file, so one round trip per 500 rows is noise, and the batches give it natural points to flush. The HTTP handler wants the first byte out as soon as possible and lets TCP set the pace. Both hold one connection for the whole job; the portal also holds one snapshot, which is what makes the export consistent.

The paged listing, `GET /products?cursor=…&limit=…`, uses §9.6's keyset pagination, with the one-extra-row trick and an opaque cursor:

```
// stockroom/src/repo/products.rs (excerpt)
let stmt = db
    .prepare_cached(
        "SELECT id, sku, name, price_cents, tags, attributes, version, created_at,
updated_at
         FROM products
         WHERE sku > coalesce($1, '') AND ($2::text IS NULL OR tags @> ARRAY[$2])
         ORDER BY sku
         LIMIT $3",
    )
    .await?;
```

```
// stockroom/src/http/products.rs (excerpt)
// One more than asked for: if it comes back, there is a next page.
let mut items = products::page(&db, after.as_ref(), query.tag.as_deref(), limit + 1).await?;
let next_cursor = if items.len() as i64 > limit {
    items.truncate(limit as usize);
    items.last().map(|last| encode_cursor(&last.sku))
} else {
    None
};
```

stockroom/src/repo/products.rs · stockroom/src/http/products.rs

`sku` is unique, so it needs no tie-breaker.

Summary

- `query` collects a `RowStream` into a `Vec`. For large results, read the stream itself with `query_raw`: rows arrive one by one, and memory stays flat.
- The `_raw` methods take an `ExactSizeIterator` of `BorrowToSql` values. Use `slice_iter` for mixed parameters, a plain collection for parameters of one type, and `Vec<Box<dyn ToSql + Sync + Send>>` for parameters decided at run time.
- `RowStream` is `!Unpin`: pin it with `pin!` (on the stack, cannot be dropped early) or `Box::pin` (owned, movable). `rows_affected()` is `None` until the stream ends.
- An unread stream stalls its whole connection: the response channel is bounded at one, the connection stops reading, and the server waits in `ClientWrite`. Stream only on a connection that nothing else uses, and keep that connection with the stream.
- Dropping a stream does not stop the query; the server still produces every row. A portal (`bind`, `query_portal`) produces only the rows you ask for, a batch per round trip, inside a transaction, if the plan can produce rows lazily.
- `DECLARE … CURSOR` is the SQL form of a portal. Use it for `WITH HOLD`, `SCROLL`, or SQL-only callers; otherwise `bind` is simpler.
- Across HTTP requests, use keyset pagination: a unique key, an opaque cursor, one extra row to detect the last page. Its cost does not grow with the page number; `OFFSET`'s does.
- `simple_query_raw` streams the simple protocol's messages, statement after statement.

Exercises

1. **Return a stream.** Write `async fn numbers(client: &Client, n: i32) -> Result<impl Stream<Item = Result<Row, Error>>, Error>` that streams `1..=n`, and use it from `main` with `try_next`. *Hint:* `RowStream` borrows nothing, so it can leave the function; `Box::pin` it so that the caller can call `try_next` without pinning.
2. **Batches from a cursor.** Rewrite §9.4's batch loop with `DECLARE` and `FETCH` instead of `bind` and `query_portal`. What does each batch cost in round trips, and how would you make it cost one? *Hint:* `FETCH 4 FROM numbers` is a statement; Chapter 4 counted what a statement given as text costs.
3. **A tie-breaker.** Create a temp table of 1,000 products with only ten distinct prices, then page through it in price order, 64 rows a page, with keyset pagination. Check that every product is seen exactly once. *Hint:* `WHERE (price, id) > ($1, $2) ORDER BY price, id`.
4. **The cost of a consistent export.** stockroom's export keeps one transaction open for the whole catalog. What does a long-lived snapshot cost the rest of the database? How would exporting with keyset pages change that, and what would the export lose? *Hint:* think of `VACUUM`, and of a product renamed halfway through the export.

Solutions

1.

```
use futures_util::{Stream, TryStreamExt};
use tokio_postgres::{Client, Error, Row};

async fn numbers(
    client: &Client,
    n: i32,
) -> Result<impl Stream<Item = Result<Row, Error>> + Send + 'static, Error> {
    let rows = client
        .query_raw("SELECT generate_series(1, $1::int)", [n])
        .await?;
    Ok(Box::pin(rows))
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let client = playground::connect().await?;
    let mut rows = numbers(&client, 3).await?;
    let mut seen = Vec::new();
    while let Some(row) = rows.try_next().await? {
        seen.push(row.get::<_, i32>(0));
    }
    assert_eq!(seen, [1, 2, 3]);
    Ok(())
}
```

The `'static` in the return type is the point: the stream outlives the borrow of `client`. It also means nothing stops the caller from holding it while using `client` for something else, with §9.3's result.

2. As text, each `FETCH` is a prepare and an execute, two round trips per batch. Prepare it once and each batch is one round trip, the same as `query_portal`:

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut client = playground::connect().await?;
    let tx = client.transaction().await?;
    tx.batch_execute("DECLARE numbers CURSOR FOR SELECT generate_series(1, 10)")
        .await?;
    let fetch = tx.prepare("FETCH 4 FROM numbers").await?;
    let mut batches = Vec::new();
    loop {
        let rows = tx.query(&fetch, &[]).await?;
        batches.push(rows.len());
        if rows.len() < 4 {
            break;
        }
    }
    assert_eq!(batches, [4, 4, 2]);
    tx.commit().await?;
    Ok(())
}
```

On the wire, the two differ only in who names the portal and how the batch size travels: `query_portal` sends an *Execute* with a row limit to a portal made by *Bind*; `FETCH` runs a statement whose result is the next rows of a portal made by `DECLARE`.

3.

```
use std::collections::HashSet;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE product (id int PRIMARY KEY, price int NOT NULL);
```

```
                INSERT INTO product SELECT i, (i % 10) * 100 FROM generate_series(1, 1000) i;
                CREATE INDEX ON product (price, id);",
            )
            .await?;
        let first = client
            .prepare("SELECT price, id FROM product ORDER BY price, id LIMIT $1")
            .await?;
        let next = client
            .prepare(
                "SELECT price, id FROM product WHERE (price, id) > ($1, $2)
                 ORDER BY price, id LIMIT $3",
            )
            .await?;

        let mut seen = HashSet::new();
        let mut last: Option<(i32, i32)> = None;
        loop {
            let rows = match last {
                None => client.query(&first, &[&64i64]).await?,
                Some((price, id)) => client.query(&next, &[&price, &id, &64i64]).await?,
            };
            let Some(end) = rows.last() else { break };
            last = Some((end.get(0), end.get(1)));
            for row in &rows {
                assert!(seen.insert(row.get::<_, i32>(1)), "a product seen twice");
            }
        }
        assert_eq!(seen.len(), 1000);
        Ok(())
}
```

With `WHERE price > $1` alone, every page would skip the rest of the products sharing the last price on the page before.

4. While the export's snapshot is open, `VACUUM` cannot remove any row version that the snapshot might still see, in any table of the database: dead rows from every update and delete accumulate until the export commits, and tables and indexes grow. With keyset pages, each page is a short transaction and nothing is held between pages. What the export loses is consistency: each page shows the catalog at a different moment, so a product whose SKU changes halfway through can appear twice or not at all, and the file no longer describes one moment in time. For a backup-like export, keep the snapshot; for a feed that is refreshed anyway, page.

CHAPTER 10

Bulk data: COPY

Loading twenty thousand rows with one `INSERT` each means twenty thousand statements: twenty thousand times the server parses parameters, plans (or reuses a plan), executes and reports, and, unless you pipeline them, twenty thousand round trips. `COPY` is PostgreSQL's answer for moving many rows: one statement, after which the rows simply stream, in either direction, as text, CSV or a binary format.

tokio-postgres exposes `COPY` as a `Sink` going in and a `Stream` coming out, plus a pair of helpers for the binary format. This chapter covers all of them, what happens when a load fails halfway, and how to turn a bulk load into an upsert. It ends with measurements: on the same machine, binary `COPY` loaded rows about a hundred times faster than one `INSERT` at a time, and more than two hundred times faster across a 1 ms network delay.

10.1 `copy_in` and `CopyInSink`

`Client::copy_in` takes a `COPY … FROM STDIN` statement and returns a `CopyInSink<T>`, a `futures` `Sink` that accepts any `T: Buf`, usually `Bytes`. You send it chunks of data with `SinkExt::send`, then call `finish()`, which ends the COPY and returns the number of rows loaded. Like `RowStream`, the sink is `!Unpin`, so it is pinned first.

The data is one byte stream in the format the statement names. The chunks you send do not have to end at row boundaries; the protocol says so (COPY operations): "The message boundaries are not required to have anything to do with row boundaries".

```
use bytes::Bytes;
use futures_util::SinkExt;
use std::pin::pin;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE item (sku text PRIMARY KEY, name text, price_cents bigint)",
        )
        .await?;

    let sink = client
        .copy_in("COPY item (sku, name, price_cents) FROM STDIN (FORMAT csv, HEADER)")
        .await?;
    let mut sink = pin!(sink);
    // One byte stream: the first chunk ends in the middle of a row.
```

```
    for chunk in [
        "sku,name,price_cents\nA-1,\"Chair, oak\",4999\nA-2,\"Tab",
        "le \"\"Nordic\"\"\",12900\nA-3,Lamp,2500\n",
    ] {
        sink.send(Bytes::from(chunk)).await?;
    }
    let rows = sink.as_mut().finish().await?;
    assert_eq!(rows, 3);

    let name: String = client
        .query_one("SELECT name FROM item WHERE sku = 'A-2'", &[])
        .await?
        .get(0);
    assert_eq!(name, "Table \"Nordic\"");
    Ok(())
}
```

Chunk size is yours to choose. In 0.7.18 the sink gathers chunks of 4 KiB or less until they pass 4 KiB, but `SinkExt::send` flushes after every item, and a flush sends whatever has been gathered (`copy_in.rs`). So `send` makes one protocol message per chunk, however small. If you produce many small pieces, one per row for instance, either `feed` them and call `finish` (or `flush`) at the end, or build larger chunks yourself, as pgload does with 64 KiB of CSV at a time. The binary writer of §10.3 batches its rows on its own.

The text and CSV formats come with the usual escaping rules, which the next section shows on the way out. Use a CSV library (pgload and stockroom use the `csv` crate) rather than `format!` and hope.

`finish`, or nothing

Until `finish()`, nothing has been committed to the table. That is a feature: if your program fails halfway through producing the data, the half that was sent is thrown away. Dropping the sink without calling `finish()` sends a *CopyFail* message, and the server aborts the `COPY`:

```
use bytes::Bytes;
use futures_util::SinkExt;
use std::pin::pin;
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut client = playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE item (id int, name text)")
        .await?;
    {
        let sink = client.copy_in("COPY item FROM STDIN (FORMAT csv)").await?;
        let mut sink = pin!(sink);
        sink.send(Bytes::from_static(b"1,chair\n2,table\n")).await?;
        // No finish(): the sink is dropped at the end of this block.
    }
    let count: i64 = client
        .query_one("SELECT count(*) FROM item", &[])
        .await?
        .get(0);
    assert_eq!(count, 0); // aborted; the connection is fine

    // In a transaction, the aborted COPY is an error like any other: the transaction is
over.
    let tx = client.transaction().await?;
    {
        let sink = tx.copy_in("COPY item FROM STDIN (FORMAT csv)").await?;
        let mut sink = pin!(sink);
```

```
        sink.send(Bytes::from_static(b"1,chair\n")).await?;
    }
    let err = tx
        .query_one("SELECT count(*) FROM item", &[])
        .await
        .unwrap_err();
    assert_eq!(err.code(), Some(&SqlState::IN_FAILED_SQL_TRANSACTION)); // 25P02
    Ok(())
}
```

> **Pitfall:** a `?` between the sends and `finish()` is a silent abort. The function returns the error, the sink is dropped, and the rows you thought you had loaded are gone. That is usually what you want, but make sure the error is logged rather than swallowed, since the COPY itself reports nothing.

A COPY that fails to start loses the connection

One failure mode of `copy_in` is not like the others, and it is worth knowing before it finds you. If the server rejects the `COPY` *as it starts*, because the table or a column does not exist, or an option is not allowed, then in 0.7.18 the connection does not survive:

```
use bytes::Bytes;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let Err(err) = client
        .copy_in::<_, Bytes>("COPY no_such_table FROM STDIN")
        .await
    else {
        panic!("the table does not exist");
    };
    println!(
        "copy_in: {}",
        err.as_db_error().expect("a server error").message()
    );

    // The next request on this connection fails, and so does every one after it.
    let next = client.query_one("SELECT 1", &[]).await;
    println!(
        "the next query: {}",
        next.as_ref()
            .map(|_| "ok".into())
            .unwrap_or_else(|e| e.to_string())
    );
    assert!(next.is_err());
    Ok(())
}
```

```
copy_in: relation "no_such_table" does not exist
the next query: unexpected message from server
```

The next query receives a response that belongs to no request, and the connection task ends with the same error; every request after that fails with "connection closed".

> **Protocol lens:** `copy_in` sends *Bind*, *Execute* and *Sync* together, then waits for the server to enter copy-in mode. When the server answers with an error instead, it consumes the *Sync* and sends *ReadyForQuery*. Meanwhile `copy_in` returns the error and drops its side of the copy, and the connection, as for any abandoned copy, sends *CopyFail* and another *Sync*. The server, no longer in copy mode, has nothing to abort, and answers the second *Sync* with a second *ReadyForQuery* that nobody asked for, and the connection task, which matches responses to requests strictly in order, gives up with "unexpected message from server". The design problem is tracked as rust-postgres issue #789.

Errors *during* the copy (a malformed line, a value of the wrong type, a header that does not match) and errors from `finish()` leave the connection healthy; §10.4 relies on that. So the defence is about the start:

- **Copy into a table you just created**, in the same transaction, with the columns you name. That is the staging-table pattern of §10.4 and §10.6, and it leaves nothing to reject at the start.
- **Treat an error from `copy_in` itself as the loss of the connection.** On a connection of your own, reconnect. On a pooled one, do not give it back for the next request to trip over: detach it and drop it (deadpool's `Object::take`, Chapter 13).

10.2 `copy_out` and `CopyOutStream`

`copy_out` takes a `COPY … TO STDOUT` statement and returns a `CopyOutStream`, a `Stream` of `Result<Bytes, Error>`. The server sends "CopyData messages (always one per row)", as the protocol documentation puts it, so each chunk is one row. Any query can be the source, with `COPY (SELECT …) TO STDOUT`.

The example also shows how CSV and the default text format write the awkward values, which matters as soon as someone reads the file back:

```
use bytes::Bytes;
use futures_util::TryStreamExt;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE note (id int, body text);
             INSERT INTO note VALUES (1, NULL), (2, ''), (3, 'a,b'), (4, 'say \"hi\"');",
        )
        .await?;

    let chunks: Vec<Bytes> = client
        .copy_out("COPY (SELECT * FROM note ORDER BY id) TO STDOUT (FORMAT csv)")
        .await?
        .try_collect()
        .await?;
    assert_eq!(chunks.len(), 4); // one chunk per row
    let csv = String::from_utf8(chunks.concat()).expect("UTF-8");
    print!("CSV:\n{csv}");
    assert_eq!(csv, "1,\n2,\"\"\n3,\"a,b\"\n4,\"say \"\"hi\"\"\"\n");

    let chunks: Vec<Bytes> = client
```

```
        .copy_out("COPY (SELECT * FROM note ORDER BY id) TO STDOUT")
        .await?
        .try_collect()
        .await?;
    print!(
        "text:\n{}",
        String::from_utf8(chunks.concat()).expect("UTF-8")
    );
    Ok(())
}
```

```
CSV:
1,
2,""
3,"a,b"
4,"say ""hi"""
text:
1   \N
2
3   a,b
4   say "hi"
```

In CSV, `NULL` is an empty field and the empty string is `""`; in the text format, `NULL` is `\N` and the empty string is nothing at all. Both read back exactly as they were written, but only by `COPY` with the same format: a spreadsheet, or another program's CSV reader, may well turn `""` into an empty field and lose the difference. Decide what "no value" means in your file and say it in the query (stockroom exports "no tags" as `NULL`, with `nullif`).

Everything Chapter 9 said about `RowStream` holds for `CopyOutStream`: it borrows nothing, and reading it slowly holds up the connection. Treat a large `COPY … TO STDOUT` as a stream that owns its connection.

10.3 Binary COPY

In the binary format, every value travels in the same binary encoding that query parameters and results use: no number formatting, no quoting, no parsing. tokio-postgres wraps both directions in `binary_copy`:

- `BinaryCopyInWriter::new(sink, &[Type])` takes the sink and the type of each column, in the statement's column order. Pinned, it accepts one row at a time with `write(&[&dyn ToSql + Sync])` (or `write_raw` with an iterator), and `finish()` returns the row count.
- `BinaryCopyOutStream::new(stream, &[Type])` turns a `copy_out` stream into a stream of `BinaryCopyOutRow`s, read with `get` and `try_get` by column index.

```
use futures_util::TryStreamExt;
use jiff::Timestamp;
use std::pin::pin;
use tokio_postgres::binary_copy::{BinaryCopyInWriter, BinaryCopyOutStream};
use tokio_postgres::types::Type;

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE reading (sensor text, at timestamptz, value float8, tags
text[])",
```

```
        )
        .await?;
    let types = [
        Type::TEXT,
        Type::TIMESTAMPTZ,
        Type::FLOAT8,
        Type::TEXT_ARRAY,
    ];

    // In: the column types once, then the rows.
    let sink = client
        .copy_in("COPY reading FROM STDIN (FORMAT binary)")
        .await?;
    let writer = BinaryCopyInWriter::new(sink, &types);
    let mut writer = pin!(writer);
    let tags = vec!["lab".to_owned()];
    for i in 0..1000i32 {
        let at = Timestamp::from_second(1_700_000_000 + i64::from(i))?;
        let value = f64::from(i) / 3.0;
        writer.as_mut().write(&[&"t-1", &at, &value, &tags]).await?;
    }
    assert_eq!(writer.as_mut().finish().await?, 1000);

    // Out: the same types, each row decoded as it arrives.
    let stream = client
        .copy_out("COPY (SELECT * FROM reading ORDER BY at) TO STDOUT (FORMAT binary)")
        .await?;
    let rows: Vec<_> = BinaryCopyOutStream::new(stream, &types)
        .try_collect()
        .await?;
    assert_eq!(rows.len(), 1000);
    let second = &rows[1];
    assert_eq!(second.get::<&str>(0), "t-1");
    assert_eq!(
        second.get::<Timestamp>(1),
        Timestamp::from_second(1_700_000_001)?
    );
    assert_eq!(second.get::<f64>(2), 1.0 / 3.0);
    assert_eq!(second.get::<Vec<String>>(3), ["lab"]);
    Ok(())
}
```

Binary `COPY` is the fastest of the formats (§10.5), and the types are exact on both sides. The price is that nobody checks your list of types against the table. Three mistakes, three outcomes:

```
use std::pin::pin;
use tokio_postgres::binary_copy::BinaryCopyInWriter;
use tokio_postgres::error::SqlState;
use tokio_postgres::types::Type;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE counter (n bigint); CREATE TEMP TABLE gauge (x
real)")
        .await?;

    // INT4 declared for a bigint column: the client cannot know. The server reads four
bytes
    // where it expects eight.
    {
        let sink = client
            .copy_in("COPY counter FROM STDIN (FORMAT binary)")
            .await?;
        let writer = BinaryCopyInWriter::new(sink, &[Type::INT4]);
        let mut writer = pin!(writer);
        writer.as_mut().write(&[&5i32]).await?;
```

```
        let err = writer.as_mut().finish().await.unwrap_err();
        let db = err.as_db_error().expect("a server error");
        println!(
            "server: {} ({})",
            db.message(),
            db.where_().unwrap_or_default()
        );
        assert_eq!(db.code(), &SqlState::PROTOCOL_VIOLATION); // 08P01
    }

    // INT8 declared, but an i32 given: the client refuses to encode it. (The block matters:
    // the writer must be dropped, aborting its COPY, before the connection can move on.)
    {
        let sink = client
            .copy_in("COPY counter FROM STDIN (FORMAT binary)")
            .await?;
        let writer = BinaryCopyInWriter::new(sink, &[Type::INT8]);
        let mut writer = pin!(writer);
        let err = writer.as_mut().write(&[&5i32]).await.unwrap_err();
        println!("client: {err}");
        assert!(err.as_db_error().is_none());
    }
    let count: i64 = client
        .query_one("SELECT count(*) FROM counter", &[])
        .await?
        .get(0);
    assert_eq!(count, 0);

    // INT4 declared for a `real` column: four bytes where four are expected. Nobody
notices,
    // and the bits of the integer 5 are stored as a floating-point number.
    {
        let sink = client
            .copy_in("COPY gauge FROM STDIN (FORMAT binary)")
            .await?;
        let writer = BinaryCopyInWriter::new(sink, &[Type::INT4]);
        let mut writer = pin!(writer);
        writer.as_mut().write(&[&5i32]).await?;
        assert_eq!(writer.as_mut().finish().await?, 1);
    }
    let stored: f32 = client.query_one("SELECT x FROM gauge", &[]).await?.get(0);
    println!("stored: {stored:e}");
    assert!(stored > 0.0 && stored < 1e-40);
    Ok(())
}
```

```
server: insufficient data left in message (COPY counter, line 1, column n)
client: error serializing parameter 0
stored: 7e-45
```

The client checks each Rust value against the type you declared, with the same `ToSql::accepts` that guards query parameters (Chapter 3). Nothing checks the declared type against the column: the server decodes each field with the column's own receive function, and finds out only when the bytes do not fit. When they do fit, as the four bytes of an `int4` do in a `real`, the load succeeds with wrong data. Get the types from the table definition, keep the list next to the `COPY` statement in column order, and test the load by reading a row back. A domain column, such as stockroom's `sku`, takes its base type (`TEXT`).

10.4 When a load fails halfway

By default `COPY` is all or nothing: one bad line, and nothing is loaded. The error says where, in `DbError::where_()` (Chapter 6), which is what makes it useful to whoever sent the file. PostgreSQL 17

added `ON_ERROR ignore`, which skips the lines whose values do not convert to the column types, and PostgreSQL 18 added `REJECT_LIMIT`, which puts a ceiling on how many may be skipped:

```
use bytes::Bytes;
use futures_util::SinkExt;
use std::pin::pin;
use tokio_postgres::error::SqlState;
use tokio_postgres::{Client, Error};

const CSV: &str = "1,chair\n2,table\nthree,lamp\n4,desk\nfive,shelf\n";

/// Runs one `COPY … FROM STDIN` with `data`; returns the rows loaded.
async fn load(client: &Client, copy: &str, data: &'static str) -> Result<u64, Error> {
    let sink = client.copy_in(copy).await?;
    let mut sink = pin!(sink);
    sink.send(Bytes::from_static(data.as_bytes())).await?;
    sink.as_mut().finish().await
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE item (id int, name text)")
        .await?;

    // The default, ON_ERROR stop: nothing is loaded, and the error names the line.
    let err = load(&client, "COPY item FROM STDIN (FORMAT csv)", CSV)
        .await
        .unwrap_err();
    let db = err.as_db_error().expect("a server error");
    println!(
        "{}\n  where: {}",
        db.message(),
        db.where_().unwrap_or_default()
    );
    assert_eq!(db.code(), &SqlState::INVALID_TEXT_REPRESENTATION); // 22P02

    // ON_ERROR ignore (PG 17+): the two bad lines are skipped.
    let copy = "COPY item FROM STDIN (FORMAT csv, ON_ERROR ignore)";
    assert_eq!(load(&client, copy, CSV).await?, 3);

    // REJECT_LIMIT (PG 18+): skipping is fine, up to a point.
    let copy = "COPY item FROM STDIN (FORMAT csv, ON_ERROR ignore, REJECT_LIMIT 1)";
    let err = load(&client, copy, CSV).await.unwrap_err();
    println!("{}", err.as_db_error().expect("a server error").message());

    let count: i64 = client
        .query_one("SELECT count(*) FROM item", &[])
        .await?
        .get(0);
    assert_eq!(count, 3); // only the ON_ERROR ignore load went in
    Ok(())
}
```

```
invalid input syntax for type integer: "three"
  where: COPY item, line 3, column id: "three"
skipped more than REJECT_LIMIT (1) rows due to data type incompatibility
```

`ON_ERROR ignore` has limits worth knowing before you rely on it:

- It skips **data type errors only**. A line with an extra column is still fatal ("extra data after last expected column", 22P04), and so is a constraint violation: a duplicate key or a failed `CHECK` aborts the whole `COPY`.

- It is **not allowed with the binary format**, and asking for it fails as the `COPY` starts ("only ON_ERROR STOP is allowed in BINARY mode", 42601), which is exactly §10.1's connection-losing case.
- It reports the skipped lines as **notices**: a summary ("2 rows were skipped due to data type incompatibility"), and one notice per line with `LOG_VERBOSITY verbose`. With `playground::connect`, notices go to the `log` crate and nobody sees them (Chapter 6); Chapter 11 shows how to receive them.

The staging table

When the rows come from outside (an upload, a partner's export), the robust pattern loads them first into a temporary table that cannot reject anything, and then moves them with SQL:

1. `CREATE TEMP TABLE … ON COMMIT DROP`, inside the transaction, with loose types (often `text`) and no constraints. The `COPY` cannot fail to start, and fails only on lines that are not CSV at all.
2. `COPY` into it.
3. Validate with queries: duplicates, unknown references, values that do not convert (`pg_input_is_valid`, PG 16+, answers "would this text convert to that type?" without an error). Report everything wrong at once, which a single `COPY` error cannot do.
4. Move the rows into the real table with `INSERT … SELECT`, `INSERT … ON CONFLICT` or `MERGE` (§10.6), converting types on the way.

The real table is touched only by step 4, in one statement, in the same transaction: the upload goes in whole, or not at all.

10.5 What COPY buys: the pgload table

`pgload` (in `apps/pgload`) loads the same 20,000 generated rows, with commas and quotes in them, in five ways, and checks that all five store identical data:

Strategy	Statements	What travels
`INSERT`, one at a time	20,000, each awaited	20,000 round trips
`INSERT`, pipelined	20,000, in flight together (Chapter 7)	about one round trip per window
`INSERT … SELECT FROM unnest(…)`	20, of 1,000 rows each	one array per column
`COPY`, CSV	1	the rows, as CSV
`COPY`, binary	1	the rows, in binary

The `unnest` strategy deserves a look, because it is the fastest way to load rows without `COPY`, and so the one to use where `COPY` does not fit (inside a larger statement, or with `RETURNING`). Each column travels as one array parameter, and `unnest` zips the arrays back into rows:

```
// apps/pgload/src/lib.rs (excerpt)
async fn unnest(tx: &Transaction<'_>, rows: &[Row]) -> Result<(), Error> {
    // …
            SELECT * FROM unnest($1::bigint[], $2::text[], $3::text[], $4::bigint[],
$5::timestamptz[])",
    // …
```

```
        tx.execute(&statement, &[&ids, &skus, &names, &prices, &times])
            .await?;
    // …
}
```

apps/pgload/src/lib.rs

Measured with a release build, PG 18.6 on localhost, on Apple Silicon. The second run goes through an in-process lagproxy with a 1 ms delay each way, with ten times fewer rows (the first strategy would take minutes otherwise):

```
cargo run -p pgload --release -- --rows 20000
cargo run -p pgload --release -- --rows 2000 --delay-ms 1
```

```
20000 rows, no added one-way delay

| Strategy | Time | Rows per second | Relative |
|---|---:|---:|---:|
| INSERT, one at a time | 0.761 s | 26283 | 103.0× |
| INSERT, pipelined | 0.118 s | 169524 | 16.0× |
| INSERT … unnest, batches | 0.012 s | 1607792 | 1.7× |
| COPY, CSV | 0.011 s | 1773115 | 1.5× |
| COPY, binary | 0.007 s | 2707215 | 1.0× |

2000 rows, 1 ms one-way delay

| Strategy | Time | Rows per second | Relative |
|---|---:|---:|---:|
| INSERT, one at a time | 9.663 s | 207 | 220.1× |
| INSERT, pipelined | 0.200 s | 9978 | 4.6× |
| INSERT … unnest, batches | 0.047 s | 42650 | 1.1× |
| COPY, CSV | 0.052 s | 38655 | 1.2× |
| COPY, binary | 0.044 s | 45547 | 1.0× |
```

What the numbers say:

- **Round trips dominate as soon as there is a network.** One `INSERT` at a time went from 100× slower than binary `COPY` on localhost to 220× slower with 2 ms per round trip. Production networks are rarely faster than that.
- **Pipelining recovers most of it, not all.** It removes the waiting, but the server still runs 20,000 statements.
- **Statements are the next cost.** `unnest` batches send the same data in 20 statements and come within a factor of two of `COPY`; across the network the three are close, because each is now a handful of round trips plus the bytes themselves.
- **Binary beats CSV** by not formatting and parsing text. The margin is clear on localhost and shrinks once the network dominates.

So: for thousands of rows or more, use `COPY`. For tens to hundreds, or when you need `RETURNING` or `ON CONFLICT` in the same statement, `unnest` is nearly as good and much simpler. Never loop over single `INSERT`s, and if you must, pipeline them.

10.6 COPY into a temp table, then upsert

`COPY` only inserts. To load a file that both adds new rows and changes existing ones, combine the staging table with an upsert. `MERGE` (PG 15+) says what to do in each case, and since PostgreSQL 17 it can return `merge_action()` for each row it touched, which turns into a report for free:

```
use bytes::Bytes;
use futures_util::SinkExt;
use std::pin::pin;
use tokio_postgres::error::SqlState;
use tokio_postgres::{Client, Error};

const MERGE: &str = "MERGE INTO product AS p
     USING price_import AS i ON p.sku = i.sku
     WHEN MATCHED AND p.price_cents <> i.price_cents THEN
         UPDATE SET price_cents = i.price_cents
     WHEN NOT MATCHED THEN
         INSERT VALUES (i.sku, i.price_cents)
     RETURNING merge_action()";

/// Loads a price list into `product`; returns what happened to each changed row.
async fn import(client: &mut Client, csv: &'static [u8]) -> Result<Vec<String>, Error> {
    let tx = client.transaction().await?;
    tx.batch_execute(
        "CREATE TEMP TABLE price_import (sku text, price_cents bigint) ON COMMIT DROP",
    )
    .await?;
    let sink = tx
        .copy_in("COPY price_import FROM STDIN (FORMAT csv)")
        .await?;
    let mut sink = pin!(sink);
    sink.send(Bytes::from_static(csv)).await?;
    sink.as_mut().finish().await?;

    let mut actions: Vec<String> = tx
        .query(MERGE, &[])
        .await?
        .iter()
        .map(|row| row.get(0))
        .collect();
    tx.commit().await?;
    actions.sort();
    Ok(actions)
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let mut client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE product (sku text PRIMARY KEY, price_cents bigint NOT NULL);
             INSERT INTO product VALUES ('A-1', 4999), ('A-2', 12900);",
        )
        .await?;

    // A-1 unchanged, A-2 repriced, A-3 new.
    let actions = import(&mut client, b"A-1,4999\nA-2,11900\nA-3,2500\n").await?;
    assert_eq!(actions, ["INSERT", "UPDATE"]);

    // The same new SKU twice: the second INSERT collides with the first.
    let err = import(&mut client, b"A-4,100\nA-4,200\n")
        .await
        .unwrap_err();
    assert_eq!(err.code(), Some(&SqlState::UNIQUE_VIOLATION)); // 23505

    // The same existing SKU twice: one row cannot be changed twice by one statement.
    let err = import(&mut client, b"A-1,100\nA-1,200\n")
        .await
```

```
        .unwrap_err();
    assert_eq!(err.code(), Some(&SqlState::CARDINALITY_VIOLATION)); // 21000
    println!("{}", err.as_db_error().expect("a server error").message());

    // Neither failed import changed anything.
    let total: i64 = client
        .query_one("SELECT sum(price_cents)::bigint FROM product", &[])
        .await?
        .get(0);
    assert_eq!(total, 4999 + 11900 + 2500);
    Ok(())
}
```

```
MERGE command cannot affect row a second time
```

The duplicate cases are the ones that surprise. A key listed twice in the file is a *unique violation* when the key is new, because `MERGE` decides "not matched" for both rows before inserting either, and a *cardinality violation* when the key exists. `INSERT … ON CONFLICT DO UPDATE` fails with the cardinality error in both cases ("ON CONFLICT DO UPDATE command cannot affect row a second time"). Neither error names the key. So check the staging table for duplicates first and report them by name, as stockroom does below, or decide which line wins (Exercise 4).

Side by side — *a bulk load from a CSV file*

The task: load a CSV file with a header line into a table.

```
-- \copy reads the file on the client and streams it to the server. (Plain COPY … FROM
-- 'items.csv' would ask the *server* to open a file on its own disk.)
tpbook=# CREATE TEMP TABLE item (sku text, name text, price_cents bigint);
CREATE TABLE
tpbook=# \copy item FROM 'items.csv' (FORMAT csv, HEADER)
COPY 3
```

```
// sqlx (illustration)
let mut conn = pool.acquire().await?;
let mut copy = conn
    .copy_in_raw("COPY item FROM STDIN (FORMAT csv, HEADER)")
    .await?;
copy.read_from(tokio::fs::File::open("items.csv").await?).await?;
let rows = copy.finish().await?;
```

```
use bytes::BytesMut;
use futures_util::SinkExt;
use std::pin::pin;
use tokio::io::AsyncReadExt;

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let path = std::env::temp_dir().join(format!("items-{}.csv", std::process::id()));
    tokio::fs::write(
        &path,
        "sku,name,price_cents\nA-1,\"Chair, oak\",4999\nA-2,\"Table
\"\"Nordic\"\"\",12900\nA-3,Lamp,2500\n",
    )
    .await?;

    let client = playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE item (sku text, name text, price_cents bigint)")
        .await?;
    let sink = client
```

```
        .copy_in("COPY item FROM STDIN (FORMAT csv, HEADER)")
        .await?;
    let mut sink = pin!(sink);
    let mut file = tokio::fs::File::open(&path).await?;
    let mut buf = BytesMut::new();
    loop {
        buf.reserve(64 * 1024);
        if file.read_buf(&mut buf).await? == 0 {
            break;
        }
        sink.send(buf.split().freeze()).await?;
    }
    let rows = sink.as_mut().finish().await?;
    tokio::fs::remove_file(&path).await?;
    assert_eq!(rows, 3);
    Ok(())
}
```

	psql	sqlx	tokio-postgres
Same	One `COPY … FROM STDIN`; the file streams through the client	One `COPY … FROM STDIN`; data sent in chunks, row count at the end	One `COPY … FROM STDIN`; data sent in chunks, row count at the end
Different	A meta-command that names a local file	Raw bytes only, from slices or any `AsyncRead`; `finish` or `abort` must be called, or the connection reports an error the next time it is used	A `Sink` of any `Buf`, plus a typed writer for binary rows; dropping the sink aborts the `COPY`

In stockroom

`POST /products/import` takes a CSV price list as the request body and streams it, chunk by chunk as it arrives, into a staging table. Then it checks for duplicates and merges:

```
// stockroom/src/repo/products.rs (excerpt)
pub async fn import_csv<S, E>(tx: &Transaction<'_>, chunks: S) -> Result<ImportReport,
AppError>
where
    S: Stream<Item = Result<Bytes, E>>,
    E: Display,
{
    tx.batch_execute(
        "CREATE TEMP TABLE product_import (sku text, name text, price_cents bigint, tags
text)
         ON COMMIT DROP",
    )
    .await?;

    // HEADER MATCH (PG 15+): the first line must name exactly these columns.
    let sink = tx
        .copy_in("COPY product_import (sku, name, price_cents, tags) FROM STDIN (FORMAT csv,
HEADER MATCH)")
        .await?;
    let mut sink = pin!(sink);
    let mut chunks = pin!(chunks);
    while let Some(chunk) = chunks.next().await {
        let chunk = chunk.map_err(|e| AppError::Invalid {
            field: None,
            message: format!("could not read the upload: {e}"),
```

```
        })?;
        sink.send(chunk).await.map_err(input_error)?;
    }
    // Without `finish`, dropping the sink would abort the COPY.
    let rows = sink.as_mut().finish().await.map_err(input_error)?;

    // A SKU listed twice would make MERGE insert it twice (a unique violation) or update it
    // twice (a cardinality violation). Naming the SKU is kinder than either.
    let twice = tx
        .query_opt(
            "SELECT sku FROM product_import GROUP BY sku HAVING count(*) > 1 LIMIT 1",
            &[],
        )
        .await?;
    // … then MERGE … RETURNING merge_action(), counted into an ImportReport
```

stockroom/src/repo/products.rs

Each piece of this chapter is in it. The HTTP body is already a stream of `Bytes`, so the upload goes from socket to `COPY` without being held in memory, and chunk boundaries do not matter. The staging table exists before the `COPY` starts, so the `COPY` cannot fail at the start. `HEADER MATCH` (PG 15+) rejects a file whose header names other columns, with "column name mismatch in header line field 2: got "title", expected "name"" and its line. And the errors that a client can fix, class 22, are passed back with their line (`input_error`, Chapter 6), while everything else stays in the log.

`stockroom seed` fills a development database with binary `COPY`, naming the types in column order. Note the domain:

```
// stockroom/src/cli/seed.rs (excerpt)
// Products: binary COPY. Every value is sent in its binary wire format, so the column
// types are named up front, one per column, in COPY's column order. The `sku` column is
// a domain, and in COPY a domain travels as its base type: `TEXT`.
let sink = tx
    .copy_in(
        "COPY products (sku, name, price_cents, tags, attributes) FROM STDIN (FORMAT
binary)",
    )
    .await?;
let writer = BinaryCopyInWriter::new(
    sink,
    &[
        Type::TEXT,
        Type::TEXT,
        Type::INT8,
        Type::TEXT_ARRAY,
        Type::JSONB,
    ],
);
let mut writer = pin!(writer);
for i in 1..=args.products {
    // …
    writer
        .as_mut()
        .write(&[&sku, &name, &price, &tags, &attributes])
        .await?;
}
let products = writer.as_mut().finish().await?;
```

stockroom/src/cli/seed.rs

The other direction is `GET /products/export.csv`, which passes `COPY … TO STDOUT (FORMAT csv, HEADER)` straight to the response body, and `stockroom export movements`, which reads the stock ledger with

`BinaryCopyOutStream` and writes CSV. Both hold their connection for the whole transfer, for the reasons of Chapter 9.

Summary

- `copy_in` returns a `CopyInSink`: `send` chunks of bytes (they need not align with rows), then `finish()`, which returns the row count. Dropping the sink before `finish()` aborts the `COPY`, and aborts the surrounding transaction.
- In 0.7.18, a `COPY … FROM STDIN` that the server rejects as it starts (missing table or column, invalid option) leaves the connection unusable (issue #789). Copy into a table you just created, and treat an error from `copy_in` itself as a lost connection.
- `copy_out` returns a `CopyOutStream` of `Bytes`, one chunk per row; any query can be the source. CSV writes `NULL` as an empty field and `''` as `""`.
- Binary `COPY` (`BinaryCopyInWriter`, `BinaryCopyOutStream`) is the fastest format. Name the column types exactly, in order: the client checks values against your types, nothing checks your types against the table, and a same-sized wrong type loads wrong data without an error.
- `COPY` is all or nothing, and its error names the line. `ON_ERROR ignore` (PG 17) skips values that do not convert, `REJECT_LIMIT` (PG 18) caps it; neither works for structural errors, constraints or binary.
- For data from outside, load a staging table, validate with SQL, then `INSERT … SELECT` or `MERGE … RETURNING merge_action()`. Deduplicate first: duplicate keys fail the merge.
- Measured: binary `COPY` was about 100× faster than single `INSERT`s on localhost and 220× across a 1 ms delay; `unnest` batches came within 2× of it.

Exercises

1. **A faithful round trip.** Export a table with `NULL`s, empty strings, commas, quotes and a newline in its text values to CSV with `copy_out`, load the result into a second table with `copy_in`, and prove with SQL that the two tables are identical. *Hint:* `EXCEPT` in both directions, or compare `count(*)` of a `FULL JOIN … IS NOT DISTINCT FROM`.
2. **Every bad line at once.** Load a CSV of `sku,price_cents` into a staging table so that the `COPY` cannot fail on a bad price, then report the line number of every price that is not a valid `bigint`, in one query. *Hint:* a staging table of `text` columns, plus `line bigint GENERATED ALWAYS AS IDENTITY` left out of the `COPY` column list; then `pg_input_is_valid` (PG 16+).
3. **Who checks what.** In §10.3's second example, why does a wrong *Rust* value fail on the client while a wrong *declared* type fails on the server, or not at all? Would declaring `Type::TEXT` for a `varchar(10)` column work, and what happens with an 11-character value? *Hint:* the binary formats of `text` and `varchar` are the same; the length limit is a property of the column.
4. **The last line wins.** Change §10.6's import so that when a SKU appears more than once, the line nearest the end of the file wins, instead of failing. *Hint:* the identity column of Exercise 2, and `DISTINCT ON (sku) … ORDER BY sku, line DESC`.

Solutions

1.

```
use bytes::Bytes;
use futures_util::{SinkExt, TryStreamExt};
use std::pin::pin;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE source (id int, body text);
             INSERT INTO source VALUES
                 (1, NULL), (2, ''), (3, 'a,b'), (4, 'say \"hi\"'), (5, E'two\\nlines');
             CREATE TEMP TABLE target (LIKE source);",
        )
        .await?;

    let chunks: Vec<Bytes> = client
        .copy_out("COPY source TO STDOUT (FORMAT csv)")
        .await?
        .try_collect()
        .await?;
    let sink = client
        .copy_in("COPY target FROM STDIN (FORMAT csv)")
        .await?;
    let mut sink = pin!(sink);
    for chunk in chunks {
        sink.send(chunk).await?;
    }
    assert_eq!(sink.as_mut().finish().await?, 5);

    let differences: i64 = client
        .query_one(
            "SELECT count(*) FROM (
                 (SELECT * FROM source EXCEPT SELECT * FROM target)
                 UNION ALL
                 (SELECT * FROM target EXCEPT SELECT * FROM source)
             ) d",
            &[],
        )
        .await?
        .get(0);
    assert_eq!(differences, 0);
    Ok(())
}
```

`EXCEPT` compares rows with `IS NOT DISTINCT FROM` semantics, so a `NULL` that came back as `''` would show up as a difference. It does not: `COPY` reads its own CSV back exactly, quoted newline included.

2.

```
use bytes::Bytes;
use futures_util::SinkExt;
use std::pin::pin;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE price_import (
                 line bigint GENERATED ALWAYS AS IDENTITY,
                 sku text,
                 price_cents text
             )",
        )
```

```
        .await?;
    let sink = client
        .copy_in("COPY price_import (sku, price_cents) FROM STDIN (FORMAT csv, HEADER)")
        .await?;
    let mut sink = pin!(sink);
    sink.send(Bytes::from_static(
        b"sku,price_cents\nA-1,4999\nA-2,12.900\nA-3,2500\nA-4,\nA-5,lots\n",
    ))
    .await?;
    sink.as_mut().finish().await?;

    // The header is line 1, so data row n is line n + 1.
    let bad: Vec<i64> = client
        .query(
            "SELECT line + 1 FROM price_import
             WHERE price_cents IS NULL OR NOT pg_input_is_valid(price_cents, 'bigint')
             ORDER BY line",
            &[],
        )
        .await?
        .iter()
        .map(|row| row.get(0))
        .collect();
    assert_eq!(bad, [3, 5, 6]);
    Ok(())
}
```

The identity column numbers the rows in the order `COPY` inserts them, which is the order of the file. An empty CSV field arrives as `NULL`, hence the `IS NULL` test.

3. The client has a Rust value and a declared `Type`, and `ToSql::accepts` decides whether that value may be encoded as that type: `i32` may not be `INT8`. It never sees the table. The server has the table and a stream of bytes, and decodes each field with the receive function of the *column's* type; it notices a wrong declared type only when the bytes do not decode, as with 4 bytes for a `bigint`, and a `real` takes the 4 bytes of an `int4` without complaint. `TEXT` for a `varchar(10)` column works, because the two types have the same binary format, and an 11-character value then fails on the server like any other insert: "value too long for type character varying(10)" (22001), a constraint of the column, not of the format.

4.

```
use bytes::Bytes;
use futures_util::SinkExt;
use std::pin::pin;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut client = playground::connect().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE product (sku text PRIMARY KEY, price_cents bigint NOT NULL);
             INSERT INTO product VALUES ('A-1', 4999);",
        )
        .await?;

    let tx = client.transaction().await?;
    tx.batch_execute(
        "CREATE TEMP TABLE price_import (
             line bigint GENERATED ALWAYS AS IDENTITY, sku text, price_cents bigint
         ) ON COMMIT DROP",
    )
    .await?;
    let sink = tx
        .copy_in("COPY price_import (sku, price_cents) FROM STDIN (FORMAT csv)")
```

```
        .await?;
    let mut sink = pin!(sink);
    sink.send(Bytes::from_static(b"A-1,100\nA-9,1\nA-1,200\nA-9,2\n"))
        .await?;
    sink.as_mut().finish().await?;
    tx.execute(
        "MERGE INTO product AS p
         USING (
             SELECT DISTINCT ON (sku) sku, price_cents
             FROM price_import
             ORDER BY sku, line DESC
         ) AS i ON p.sku = i.sku
         WHEN MATCHED THEN UPDATE SET price_cents = i.price_cents
         WHEN NOT MATCHED THEN INSERT VALUES (i.sku, i.price_cents)",
        &[],
    )
    .await?;
    tx.commit().await?;

    let prices: Vec<(String, i64)> = client
        .query("SELECT sku, price_cents FROM product ORDER BY sku", &[])
        .await?
        .iter()
        .map(|row| (row.get(0), row.get(1)))
        .collect();
    assert_eq!(prices, [("A-1".to_owned(), 200), ("A-9".to_owned(), 2)]);
    Ok(())
}
```

CHAPTER 11

LISTEN/NOTIFY and notices

Everything so far has been request and response: the client asks, the server answers. But the server can also speak first. A session that has run `LISTEN orders` receives a message whenever any session runs `NOTIFY orders`, with no query in flight and no polling. That is enough to build live dashboards, cache invalidation and job queues that wake up the moment there is work, with the database as the only moving part.

The server also sends *notices*, the messages that `RAISE NOTICE` and many commands produce along the way. Both arrive unprompted, and both are thrown away unless you drive the `Connection` yourself. This chapter shows how, then builds the parts of a reliable listener: a dedicated connection, a trigger that announces changes, a catch-up query for whatever was missed, and fan-out to many subscribers.

11.1 What `NOTIFY` promises

> **SQL lens:** `NOTIFY` is transactional. A notification is queued by the statement and delivered when the transaction commits, and only then; after a rollback it is never delivered. Within one transaction, identical notifications (same channel, same payload) are delivered once. The payload is text, and "in the default configuration it must be shorter than 8000 bytes". Nothing is stored: sessions that are not listening at the moment of the commit never hear of it.

Here is each of those rules, observed from a listening session. `connect_with_messages`, a new playground helper that §11.2 explains, gives a client plus a receiver for what the server sends unprompted:

```
use std::time::Duration;
use tokio::sync::mpsc::UnboundedReceiver;
use tokio_postgres::AsyncMessage;

/// The payloads of the notifications that arrive within 200 ms.
async fn payloads(messages: &mut UnboundedReceiver<AsyncMessage>) -> Vec<String> {
    let mut seen = Vec::new();
    while let Ok(Some(message)) =
        tokio::time::timeout(Duration::from_millis(200), messages.recv()).await
    {
        if let AsyncMessage::Notification(n) = message {
            seen.push(n.payload().to_owned());
        }
    }
    seen
}
```

```
#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let (listener, mut messages) = playground::connect_with_messages().await?;
    let mut notifier = playground::connect().await?;
    listener.batch_execute("LISTEN stock").await?;

    // Outside a transaction: delivered when the statement's implicit transaction commits.
    notifier.batch_execute("NOTIFY stock, 'restocked'").await?;
    assert_eq!(payloads(&mut messages).await, ["restocked"]);

    // Inside one: nothing until the commit...
    let tx = notifier.transaction().await?;
    tx.execute("SELECT pg_notify('stock', 'counted')", &[])
        .await?;
    assert!(payloads(&mut messages).await.is_empty());
    tx.commit().await?;
    assert_eq!(payloads(&mut messages).await, ["counted"]);

    // ...and nothing at all after a rollback.
    let tx = notifier.transaction().await?;
    tx.execute("SELECT pg_notify('stock', 'never')", &[])
        .await?;
    tx.rollback().await?;
    assert!(payloads(&mut messages).await.is_empty());

    // Identical notifications in one transaction are delivered once, in first-sent order.
    notifier
        .batch_execute("BEGIN; NOTIFY stock, 'a'; NOTIFY stock, 'b'; NOTIFY stock, 'a';
COMMIT")
        .await?;
    assert_eq!(payloads(&mut messages).await, ["a", "b"]);

    // 8,000 bytes is one too many.
    let err = notifier
        .execute("SELECT pg_notify('stock', $1)", &[&"x".repeat(8000)])
        .await
        .unwrap_err();
    println!("{}", err.as_db_error().expect("a server error").message());
    Ok(())
}
```

```
payload string too long
```

Two ways to send a notification appear in that example. The `NOTIFY channel, 'payload'` statement takes literals only: `NOTIFY stock, $1` is a syntax error, so a payload computed by your program goes through the function `pg_notify(channel, payload)`, which takes ordinary parameters. Use `pg_notify` from code.

The "at most once, only if committed" rule is what makes `NOTIFY` safe to put inside the transaction that changes the data: listeners never hear about a change that did not happen. The "nothing is stored" rule is what §11.6 is about.

11.2 Driving the `Connection` yourself

Chapter 1 spawned the `Connection` and awaited it. That runs all of its IO, and in 0.7.18 it does this with anything the server sends unprompted (`connection.rs`):

```
// tokio-postgres 0.7.18, the Future implementation of Connection
fn poll(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Result<(), Error>> {
    while let Some(message) = ready!(self.poll_message(cx)?) {
        if let AsyncMessage::Notice(notice) = message {
```

```
            info!("{}: {}", notice.severity(), notice.message());
        }
    }
    Poll::Ready(Ok(()))
}
```

Notices go to the `log` crate at `info` level, and notifications are dropped on the floor. The method it calls, `Connection::poll_message`, is public: it does all the same work (sending requests, routing responses) and returns each unprompted message as `Some(Ok(AsyncMessage))`. `None` means the connection closed cleanly and `Some(Err(_))` that it failed; after either, it must not be polled again.

So the recipe is: spawn a task that calls `poll_message` in a loop and forwards each message to whoever wants it. Add this to the playground's `src/lib.rs`, below `connect`, whose imports it shares:

```
use tokio::sync::mpsc;
use tokio_postgres::AsyncMessage;

/// Like `connect`, but drives the connection with `poll_message`, and forwards what the
/// server sends unprompted (notifications and notices) to the returned receiver instead of
/// dropping it. The channel is unbounded, which suits examples: nothing is lost, and memory
is
/// the limit if nobody reads.
pub async fn connect_with_messages()
-> Result<(Client, mpsc::UnboundedReceiver<AsyncMessage>), Error> {
    let url = std::env::var("DATABASE_URL")
        .expect("set DATABASE_URL, e.g. postgres://postgres:secret@localhost:5432/tpbook");
    let (client, mut connection) = tokio_postgres::connect(&url, NoTls).await?;
    let (forward, messages) = mpsc::unbounded_channel();
    tokio::spawn(async move {
        loop {
            match std::future::poll_fn(|cx| connection.poll_message(cx)).await {
                // A closed receiver only means nobody is interested any more; the
connection
                // must still be driven for the client's queries.
                Some(Ok(message)) => {
                    let _ = forward.send(message);
                }
                Some(Err(e)) => {
                    eprintln!("connection error: {e}");
                    break;
                }
                None => break,
            }
        }
    });
    Ok((client, messages))
}
```

`std::future::poll_fn` turns one call of a `poll_*` method into a future, which is all a loop needs. The crate's own documentation uses the stream version, `futures_util::stream::poll_fn`, with `StreamExt::forward`; stockroom and pglisten use that one with `next()`. The three are equivalent.

The loop must keep going for as long as the client is used, because it *is* the connection: the client's requests are written and their responses read by these `poll_message` calls. That is why a full channel must not block it.

> **Pitfall:** forwarding through a *bounded* channel with `send().await` gives backpressure, and stockroom does it (capacity 1,024), but it couples two things. If the consumer falls behind and the channel fills, the driver waits, and while it waits nobody polls the connection: the client's own queries stall too, and the server's notification queue grows. Keep the consumer of a listener connection fast, or use an unbounded channel and watch its length, and keep the listening connection for listening (§11.5).

11.3 `AsyncMessage`: notifications and notices

`AsyncMessage` has two variants in 0.7.18, and is `#[non_exhaustive]`, so a `match` needs a `_` arm:

- `Notification(Notification)`, with `channel()`, `payload()`, and `process_id()`: the backend PID of the session that sent it.
- `Notice(DbError)`: the same type as a server error (Chapter 6), with `severity()`, `code()` and `message()`, but not an error. `RAISE NOTICE`, `RAISE WARNING` and many commands (`DROP TABLE IF EXISTS` on a missing table, for one) send them.

```
use std::time::Duration;
use tokio_postgres::AsyncMessage;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let (client, mut messages) = playground::connect_with_messages().await?;
    let pid: i32 = client
        .query_one("SELECT pg_backend_pid()", &[])
        .await?
        .get(0);
    client
        .batch_execute(r#"LISTEN "Orders"; LISTEN orders"#)
        .await?;
    client
        .batch_execute(
            r#"NOTIFY Orders, 'unquoted';
               SELECT pg_notify('Orders', 'pg_notify keeps the case');
               DO $$ BEGIN
                   RAISE NOTICE 'checked % rows', 42;
                   RAISE WARNING 'stock is low';
                   RAISE DEBUG 'not sent: below client_min_messages';
               END $$;"#,
        )
        .await?;

    let mut seen = Vec::new();
    while let Ok(Some(message)) =
        tokio::time::timeout(Duration::from_millis(200), messages.recv()).await
    {
        match message {
            AsyncMessage::Notification(n) => {
                assert_eq!(n.process_id(), pid); // a session hears its own notifications
                seen.push(format!(
                    "notification on {:?}: {}",
                    n.channel(),
                    n.payload()
                ));
            }
            AsyncMessage::Notice(notice) => {
                seen.push(format!(
                    "{} ({}): {}",
                    notice.severity(),
```

```
                    notice.code().code(),
                    notice.message()
                ));
            }
            _ => {} // the enum is #[non_exhaustive]
        }
    }
    for line in &seen {
        println!("{line}");
    }
    assert_eq!(seen.len(), 4);
    Ok(())
}
```

```
NOTICE (00000): checked 42 rows
WARNING (01000): stock is low
notification on "orders": unquoted
notification on "Orders": pg_notify keeps the case
```

Three details are in that output:

- **Notices come during execution, notifications at the commit.** The batch is one implicit transaction, so both notifications arrived after both notices, although they were sent first.
- **Channel names are identifiers.** Unquoted, `NOTIFY Orders` and `LISTEN Orders` fold to `orders`; `pg_notify('Orders', …)` takes a string and keeps the case, so only a quoted `LISTEN "Orders"` hears it. When channel names come from configuration, quote them (pglisten's `quote_ident`, at the end of this chapter), and they then match `pg_notify` exactly.
- **`client_min_messages`** (default `notice`) decides which notices the server sends at all: the `DEBUG` line never left the server.

11.4 Triggers that announce changes

Calling `pg_notify` from application code works, but every code path that changes the data must remember to do it. A trigger cannot forget. It runs in the transaction of the change, so the notification shares its fate: delivered if the change commits, never if it rolls back.

```
use serde::Deserialize;
use std::time::Duration;
use tokio_postgres::AsyncMessage;

#[derive(Debug, Deserialize, PartialEq)]
#[serde(rename_all = "camelCase")]
struct StockEvent {
    product: String,
    on_hand: i32,
    previous: Option<i32>,
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let (client, mut messages) = playground::connect_with_messages().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE stock (product text PRIMARY KEY, on_hand int NOT NULL);

             CREATE FUNCTION pg_temp.announce_stock() RETURNS trigger
             LANGUAGE plpgsql AS $$
             BEGIN
```

```
                PERFORM pg_notify('stock_events', json_build_object(
                    'product',  NEW.product,
                    'onHand',   NEW.on_hand,
                    'previous', CASE WHEN TG_OP = 'UPDATE' THEN OLD.on_hand END
                )::text);
                RETURN NULL;
            END $$;

            CREATE TRIGGER stock_announce
                AFTER INSERT OR UPDATE OF on_hand ON stock
                FOR EACH ROW EXECUTE FUNCTION pg_temp.announce_stock();

            LISTEN stock_events;",
        )
        .await?;

    client
        .batch_execute(
            "INSERT INTO stock VALUES ('chair', 10);
             UPDATE stock SET on_hand = on_hand - 3 WHERE product = 'chair';",
        )
        .await?;

    let mut events = Vec::new();
    while let Ok(Some(message)) =
        tokio::time::timeout(Duration::from_millis(200), messages.recv()).await
    {
        if let AsyncMessage::Notification(n) = message {
            events.push(serde_json::from_str::<StockEvent>(n.payload())?);
        }
    }
    assert_eq!(
        events,
        [
            StockEvent {
                product: "chair".into(),
                on_hand: 10,
                previous: None
            },
            StockEvent {
                product: "chair".into(),
                on_hand: 7,
                previous: Some(10)
            },
        ]
    );
    Ok(())
}
```

(The function lives in `pg_temp`, like the table, so that the example leaves nothing behind; in a real schema it is an ordinary function created by a migration.)

Keep payloads small. The 8,000-byte limit is one reason; another is that a notification is a *hint*, not a record. Send the identifiers and the little that a listener needs to decide whether to care, and let it read the current state if it does. A payload that carries the whole row tempts listeners to trust it, and §11.6 shows why they should not trust it alone.

11.5 A connection of its own

The listening session must be a connection that does nothing else, and in particular not one from the pool:

- **A pool awaits its connections.** deadpool-postgres spawns each `Connection` as a future (`connection.await`, in its `lib.rs`), so notifications that arrive on a pooled connection are dropped, as §11.2 showed.
- **`LISTEN` belongs to the session.** A pooled connection is someone else's session a moment later. deadpool's `RecyclingMethod::Clean` even runs `UNLISTEN *` when the connection comes back (Chapter 13).
- **A listener in a transaction hears nothing until the transaction ends.** The server holds notifications back from a session that is inside a transaction, so a connection that also does work delivers them late:

```
use std::time::Duration;
use tokio_postgres::AsyncMessage;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let (mut listener, mut messages) = playground::connect_with_messages().await?;
    let notifier = playground::connect().await?;
    listener.batch_execute("LISTEN jobs").await?;

    let tx = listener.transaction().await?;
    tx.execute("SELECT 1", &[]).await?;
    notifier
        .execute("SELECT pg_notify('jobs', 'work')", &[])
        .await?;
    let during = tokio::time::timeout(Duration::from_millis(300), messages.recv()).await;
    assert!(
        during.is_err(),
        "nothing arrives while the listener is in a transaction"
    );

    tx.commit().await?;
    let after = tokio::time::timeout(Duration::from_millis(300), messages.recv()).await;
    assert!(matches!(after, Ok(Some(AsyncMessage::Notification(_)))));
    Ok(())
}
```

A listener connection is cheap: one per process, however many channels and subscribers. Open it from the same `Config` as the pool, drive it with `poll_message`, `LISTEN` on every channel the process needs, and distribute what arrives inside the process (§11.7).

11.6 Reliability: listen, catch up, deduplicate

Notifications are not stored. A listener that is reconnecting, restarting, or not yet started misses whatever was committed in the meantime, and nothing tells it so. A reliable consumer therefore treats notifications as a fast path over a slow path that is always correct: a query that finds everything it has not handled yet.

The order of the steps matters:

1. `LISTEN` first.
2. Then run the catch-up query, which reads everything committed so far.
3. Then handle notifications as they arrive, and ignore the ones the catch-up already covered.

A change committed between steps 1 and 2 is seen twice, once by each path, which deduplication handles. Run the steps the other way round, and a change committed between the query and the `LISTEN` is seen by neither. This example plays both kinds of change on one session, which makes the order deterministic:

```
use std::collections::BTreeSet;
use std::time::Duration;
use tokio_postgres::AsyncMessage;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let (client, mut messages) = playground::connect_with_messages().await?;
    client
        .batch_execute(
            "CREATE TEMP TABLE event (id bigint GENERATED ALWAYS AS IDENTITY, body text);
             CREATE FUNCTION pg_temp.announce_event() RETURNS trigger
             LANGUAGE plpgsql AS $$
             BEGIN
                 PERFORM pg_notify('events', NEW.id::text);
                 RETURN NULL;
             END $$;
             CREATE TRIGGER event_announce AFTER INSERT ON event
                 FOR EACH ROW EXECUTE FUNCTION pg_temp.announce_event();",
        )
        .await?;
    let insert = client
        .prepare("INSERT INTO event (body) VALUES ($1)")
        .await?;

    client
        .execute(&insert, &[&"before anyone listened"])
        .await?; // id 1
    // 1. LISTEN first.
    client.batch_execute("LISTEN events").await?;
    client
        .execute(&insert, &[&"between LISTEN and catch-up"])
        .await?; // id 2
    // 2. Catch up on everything committed so far.
    let mut handled = BTreeSet::new();
    for row in client
        .query("SELECT id FROM event ORDER BY id", &[])
        .await?
    {
        handled.insert(row.get::<_, i64>(0));
    }
    client.execute(&insert, &[&"after catch-up"]).await?; // id 3

    // 3. Notifications, deduplicated against what was handled.
    let mut duplicates = 0;
    while let Ok(Some(message)) =
        tokio::time::timeout(Duration::from_millis(200), messages.recv()).await
    {
        if let AsyncMessage::Notification(n) = message {
            let id: i64 = n.payload().parse().expect("an id");
            if !handled.insert(id) {
                duplicates += 1;
            }
        }
    }
    assert_eq!(handled.into_iter().collect::<Vec<_>>(), [1, 2, 3]);
    assert_eq!(duplicates, 1); // event 2, seen by both paths
    Ok(())
}
```

In a service, "everything committed so far" is rarely the whole table: it is the rows newer than a checkpoint the consumer stored (an id or a timestamp), or the rows of an *outbox* table not yet marked

done. And the steps repeat after every reconnect, because a new connection is a new session that has missed everything since the old one died. stockroom publishes a `Resync` event to its subscribers each time its listener reconnects, which tells them to re-read instead of trusting the stream.

One server-side limit completes the picture. Notifications wait in a queue ("8GB in a standard installation") until every listening session has read them, and "no cleanup can take place if a session executes `LISTEN` and then enters a transaction for a very long time" (`NOTIFY`). If the queue fills, transactions that `NOTIFY` fail at commit: one forgotten listener in a long transaction can break every writer. `SELECT pg_notification_queue_usage()` reports how full it is, as a fraction; watch it in production.

11.7 Fan-out with `broadcast`

One listener connection per process, but many consumers inside it: one per open Server-Sent Events stream, say (Chapter 16). `tokio::sync::broadcast` is made for this. Every subscriber gets every message, and the channel holds the last `capacity` messages. A subscriber that falls further behind than that does not block anyone: its next `recv` returns `RecvError::Lagged(n)`, saying how many it missed, and continues with the oldest message still held. That is exactly the "you missed something, re-read" signal of §11.6:

```
use tokio::sync::broadcast;
use tokio_postgres::AsyncMessage;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let (client, mut messages) = playground::connect_with_messages().await?;
    client.batch_execute("LISTEN prices").await?;
    client
        .batch_execute(
            "BEGIN;
             SELECT pg_notify('prices', i::text) FROM generate_series(1, 10) i;
             COMMIT;",
        )
        .await?;

    // The listener task publishes; capacity 4 keeps the last four events.
    let (publish, mut slow_subscriber) = broadcast::channel::<String>(4);
    let mut published = 0;
    while published < 10 {
        if let Some(AsyncMessage::Notification(n)) = messages.recv().await {
            let _ = publish.send(n.payload().to_owned()); // Err only means no subscribers
            published += 1;
        }
    }

    // This subscriber did not read while the ten were published.
    let mut received = Vec::new();
    let mut lagged = 0;
    loop {
        match slow_subscriber.try_recv() {
            Ok(payload) => received.push(payload),
            Err(broadcast::error::TryRecvError::Lagged(n)) => lagged += n, // re-read here
            Err(_) => break,                                               // empty
        }
    }
    assert_eq!(lagged, 6);
    assert_eq!(received, ["7", "8", "9", "10"]);
    Ok(())
}
```

`recv().await` reports the same thing as `Err(RecvError::Lagged(n))`. Pick the capacity for the slowest subscriber you are willing to keep up to date without a resync, and make the resync cheap: a subscriber that lagged re-reads the current state, which is the same query as the catch-up of §11.6.

Side by side — *subscribing to events*

The task: receive the notifications sent on `order_events`.

```
-- psql prints notifications after the next command it runs, or while it waits for input.
tpbook=# LISTEN order_events;
LISTEN
tpbook=# NOTIFY order_events, '{"orderId": 42, "status": "paid"}';
NOTIFY
Asynchronous notification "order_events" with payload "{"orderId": 42, "status": "paid"}"
received from server process with PID 28381.
```

```
// sqlx (illustration)
let mut listener = sqlx::postgres::PgListener::connect_with(&pool).await?;
listener.listen("order_events").await?;
loop {
    // Reconnects by itself after a lost connection; what was sent meanwhile is lost.
    let notification = listener.recv().await?;
    println!("{}: {}", notification.channel(), notification.payload());
}
```

```
use tokio_postgres::AsyncMessage;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let (client, mut messages) = playground::connect_with_messages().await?;
    client.batch_execute("LISTEN order_events").await?;
    client
        .execute(
            "SELECT pg_notify('order_events', $1)",
            &[&r#"{"orderId": 42, "status": "paid"}"#],
        )
        .await?;
    loop {
        match messages.recv().await {
            Some(AsyncMessage::Notification(n)) => {
                println!("{}: {}", n.channel(), n.payload());
                assert_eq!(n.channel(), "order_events");
                break;
            }
            Some(_) => {}
            None => panic!("the connection closed"),
        }
    }
    Ok(())
}
```

	psql	sqlx	tokio-postgres
Same	`LISTEN` on a session; notifications arrive after commit, with channel, payload and sender PID	`LISTEN` on a session; notifications arrive after commit, with channel, payload and sender PID	`LISTEN` on a session; notifications arrive after commit, with channel, payload and sender PID
Different	Printed between commands	A dedicated `PgListener` connection that reconnects and	You drive the `Connection` with `poll_message`; reconnecting, re-

psql	sqlx	tokio-postgres
	re-listens by itself, losing what was sent meanwhile	listening and catching up are yours to write

In stockroom

stockroom announces orders with a trigger, in the migration that creates the outbox:

```
-- stockroom/migrations/0002_order_events.sql (excerpt)
CREATE FUNCTION announce_order_event() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
    event jsonb := jsonb_build_object(
        'orderId',  NEW.id,
        'storeId',  NEW.store_id,
        'status',   NEW.status,
        'previous', CASE WHEN TG_OP = 'UPDATE' THEN OLD.status END,
        'version',  NEW.version
    );
BEGIN
    PERFORM pg_notify('order_events', event::text);
    INSERT INTO outbox (topic, payload) VALUES ('order.' || NEW.status, event);
    PERFORM pg_notify('outbox', '');
    RETURN NULL;
END
$$;
```

stockroom/migrations/0002_order_events.sql

Each change is announced twice. `order_events` carries a small JSON event for live screens, which may miss some and resync. The outbox row is the durable record for the background worker (Chapter 16), and the payload-less `pg_notify('outbox', '')` only wakes it: since identical notifications in one transaction are delivered once, a transaction that places a hundred orders wakes the worker once.

`notify::listener` holds the dedicated connection. It drives it with `poll_message` through a bounded channel, listens, publishes `Resync`, and then dispatches; `run` wraps it in a reconnect loop with exponential backoff:

```
// stockroom/src/notify/listener.rs (excerpt)
let (client, mut connection) = config.connect(tls.clone()).await?;
hub.set_server_info(server_info(&connection));

let (forward, mut messages) = mpsc::channel(1024);
let driver = tokio::spawn(async move {
    let mut incoming = stream::poll_fn(move |cx| connection.poll_message(cx));
    while let Some(message) = incoming.next().await {
        if forward.send(message?).await.is_err() {
            break; // the listener is gone
        }
    }
    Ok::<_, tokio_postgres::Error>(())
});

client
    .batch_execute("LISTEN order_events; LISTEN outbox")
    .await?;
tracing::info!("listening for order events");
// Anything sent while nobody was listening is lost for good. Tell subscribers to
// re-read, and the worker to look for jobs it was not woken for.
```

```
hub.publish(HubEvent::Resync);
hub.wake_outbox();
// … then a select! loop over shutdown and `messages`, dispatching each notification
```

stockroom/src/notify/listener.rs · stockroom/src/notify/mod.rs

The `Hub` in `notify/mod.rs` is §11.7's pattern: a `broadcast` channel of `HubEvents`, where `HubEvent::Resync` is the "re-read" signal, plus a `tokio::sync::Notify` for the worker. A `Notify` keeps one pending wake-up even if the worker is busy when it arrives, so a burst of outbox notifications costs the worker one extra look, never a missed job.

The listener also records what the server said about itself when it connected, through `Connection::parameter` (Chapter 1): `GET /admin/sessions` shows the server version and time zone that the listener saw.

pglisten (`apps/pglisten`) is the same design as a command-line tool, with the catch-up query of §11.6 as an option and a `check_connection` ping so that a dead connection is noticed even when nothing is notified (Chapter 12):

```
cargo run -p pglisten -- order_events outbox
cargo run -p pglisten -- --catch-up "SELECT id, topic FROM outbox WHERE done_at IS NULL"
outbox
```

Channel names come from the command line, and identifiers cannot be parameters (`LISTEN $1` is a syntax error), so this is one of the few places where text from outside goes into SQL. It goes through `quote_ident`:

```
// apps/pglisten/src/lib.rs (excerpt)
pub fn quote_ident(name: &str) -> String {
    format!("\"{}\"", name.replace('"', "\"\""))
}
```

apps/pglisten/src/lib.rs

Summary

- `NOTIFY` is delivered at commit, never after a rollback, once per identical channel-and-payload within a transaction, with a payload shorter than 8,000 bytes. Nothing is stored. Use `pg_notify(channel, payload)` from code: it takes parameters.
- Awaiting a `Connection` drops notifications and logs notices. Drive it with `poll_message` in a loop and forward what it returns; never stop polling while the client is in use.
- `AsyncMessage` is `Notification` (channel, payload, sender PID) or `Notice` (a `DbError` that is not an error), and is non-exhaustive. Notices arrive during execution, notifications at commit. Unquoted channel names fold to lower case.
- Announce changes from a trigger, in the transaction of the change, with a small payload of identifiers.
- Listen on a dedicated connection, never a pooled one, and keep it out of transactions.

- Reliability: `LISTEN`, then catch up with a query, then deduplicate notifications against what the query found. Repeat after every reconnect.
- Fan out inside the process with `broadcast`; treat `Lagged` as "resync".

Exercises

1. **How many wake-ups?** A transaction inserts 1,000 orders into stockroom. How many notifications does a listener on `order_events` receive, and how many on `outbox`? Check your answer against stockroom's trigger. *Hint:* §11.1's rule about identical notifications.
2. **The gap.** Use pglisten with a catch-up query, and kill its connection from psql with `SELECT pg_terminate_backend(pid) FROM pg_stat_activity WHERE application_name = …`. While it reconnects, insert an outbox row. Which path reports the row, and what would have happened without the catch-up query? *Hint:* pglisten backs off for 100 ms before its first reconnect; `\watch` can insert rows in a loop.
3. **A stalled driver.** Change a copy of `connect_with_messages` to forward through `mpsc::channel(1)` with `send().await`, and never read the receiver. Run a `DO` block that raises two notices, with a timeout. What happens, and why? *Hint:* who reads the statement's own answer from the socket?
4. **A safe payload.** Rewrite §11.4's trigger so that it sends the JSON when it fits in 7,999 bytes, and only the product key otherwise. *Hint:* `octet_length(text)`.

Solutions

1. 1,000 on `order_events`, because each payload carries a different order id, and one on `outbox`, whose payload is always empty: a thousand identical notifications in one transaction are delivered once.

2. The catch-up query reports it, after the reconnect. The notification was sent at commit to the sessions listening at that moment, and pglisten's new session did not exist yet, so without the catch-up query the row would wait until the next notification on `outbox` happened to wake someone.

3. The statement times out, although the server finished it at once. The driver task receives the first notice and sends it into the channel, which fills it; the second `send().await` waits for room that never comes. While it waits, nothing calls `poll_message`, and that is the call that reads responses from the socket and writes new requests to it. The statement's *CommandComplete* stays unread, and every later request stays unsent:

```
use std::time::Duration;
use tokio::sync::mpsc;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let url = std::env::var("DATABASE_URL").expect("DATABASE_URL");
    let (client, mut connection) = tokio_postgres::connect(&url,
tokio_postgres::NoTls).await?;
    let (forward, _never_read) = mpsc::channel(1);
    tokio::spawn(async move {
        while let Some(Ok(message)) = std::future::poll_fn(|cx|
connection.poll_message(cx)).await {
            if forward.send(message).await.is_err() {
                break;
            }
```

```
        }
    });

    // The first notice fills the channel; the second leaves the driver waiting for room.
    let batch = client.batch_execute("DO $$ BEGIN RAISE NOTICE 'one'; RAISE NOTICE 'two';
END $$");
    let result = tokio::time::timeout(Duration::from_millis(500), batch).await;
    assert!(
        result.is_err(),
        "the driver is stuck, so the answer is never read"
    );
    Ok(())
}
```

4.

```
CREATE FUNCTION announce_stock() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
    payload text := json_build_object(
        'product', NEW.product,
        'onHand',  NEW.on_hand,
        'previous', CASE WHEN TG_OP = 'UPDATE' THEN OLD.on_hand END
    )::text;
BEGIN
    IF octet_length(payload) >= 8000 THEN
        payload := json_build_object('product', NEW.product)::text;
    END IF;
    PERFORM pg_notify('stock_events', payload);
    RETURN NULL;
END $$;
```

Listeners must then accept both shapes, which they should anyway: a notification is a hint to re-read, not the data itself. (A product key long enough to break even this is a schema problem: limit it with a `CHECK`.)

CHAPTER 12

Production connections: timeouts, cancellation, health and TLS

On a laptop, a connection is a socket to `localhost` that never fails, and every query finishes. In production, queries run long because a table grew, locks are held by someone else, the network between the service and the database drops packets or disappears, the database fails over to a replica, and the traffic crosses networks that you do not own, which is why it must be encrypted.

This chapter is about the settings and techniques that keep a service healthy through all of that: what really happens when you stop waiting for a query, how to cancel one, how to make the server enforce limits, how to notice that a connection has died, how to connect to several hosts, and how to connect over TLS, which tokio-postgres treats differently from libpq. The last section opens the connection yourself, with `connect_raw`.

12.1 Dropping a future does not cancel the query

`tokio::time::timeout` around a query looks like a query timeout. It is not one. When the timeout fires, it drops the query's future, so *your code* stops waiting. The request was already sent, though, and the server knows nothing about your future: it keeps executing the statement. And since the connection answers requests in order (Chapter 7), the next query on that connection waits until the abandoned one finishes:

```
use std::time::{Duration, Instant};

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let observer = playground::connect().await?;
    let pid: i32 = client
        .query_one("SELECT pg_backend_pid()", &[])
        .await?
        .get(0);

    let started = Instant::now();
    let gave_up = tokio::time::timeout(
        Duration::from_millis(100),
        client.execute("SELECT pg_sleep(1)", &[]),
    )
    .await;
    assert!(gave_up.is_err()); // we stopped waiting...

    // ...but the server did not stop working,
```

```
    let row = observer
        .query_one(
            "SELECT state, query FROM pg_stat_activity WHERE pid = $1",
            &[&pid],
        )
        .await?;
    let (state, query): (Option<String>, Option<String>) = (row.get(0), row.get(1));
    println!("server: {state:?}, {query:?}");
    assert_eq!(state.as_deref(), Some("active"));

    // and the next query on the connection waits for it.
    client.query_one("SELECT 1", &[]).await?;
    let elapsed = started.elapsed();
    println!("the next query answered after {elapsed:?}");
    assert!(elapsed > Duration::from_millis(800));
    Ok(())
}
```

```
server: Some("active"), Some("SELECT pg_sleep(1)")
the next query answered after 1.003539792s
```

> **Pitfall:** the same happens when axum drops a handler's future because the client hung up, or when a `select!` picks another branch. The query you abandoned keeps the server busy, holds its locks, and, on a pooled connection, delays whoever borrows the connection next. Abandoning a query is free for your code and not free for anyone else.

12.2 `CancelToken`: asking the server to stop

To stop a running statement you must ask the server, and you cannot ask on the busy connection: it is waiting for the answer. PostgreSQL's protocol has a separate mechanism (Canceling Requests in Progress): the client "opens a new connection to the server and sends a CancelRequest message", which names the backend by its process ID and a secret key the server handed out at startup.

`Client::cancel_token` returns a `CancelToken` holding exactly that: the key, plus how to reach the server. It is `Clone` and `Send`, and it does not borrow the client, so it can go to whichever task decides that a query has run long enough. `cancel_query(tls)` connects and sends the request; the statement then fails with SQLSTATE 57014, `query_canceled`:

```
use std::sync::Arc;
use std::time::Duration;
use tokio_postgres::NoTls;
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = Arc::new(playground::connect().await?);
    let token = client.cancel_token();

    let slow = {
        let client = Arc::clone(&client);
        tokio::spawn(async move { client.execute("SELECT pg_sleep(5)", &[]).await })
    };
    tokio::time::sleep(Duration::from_millis(100)).await;
    token.cancel_query(NoTls).await?;
```

```
    let err = slow.await.expect("no panic").unwrap_err();
    println!("{}", err.as_db_error().expect("a server error").message());
    assert_eq!(err.code(), Some(&SqlState::QUERY_CANCELED)); // 57014

    // Only the statement was cancelled; the connection is fine.
    client.query_one("SELECT 1", &[]).await?;
    Ok(())
}
```

```
canceling statement due to user request
```

The `tls` argument is a connector of the same kind as the connection's (`NoTls` here), because the cancel request travels on a new connection with the same TLS settings. `Client::cancel_query` also exists, deprecated since 0.6.0 in favour of the token.

> **Protocol lens:** the server never answers a cancel request: "for reasons of both security and efficiency, the frontend has no direct way to tell whether a cancel request has succeeded". `cancel_query` returns once the request is sent. The effect is a signal to the backend process, and the documentation is candid about the timing: "if it arrives after the backend has finished processing the query, then it will have no effect." The proof that it worked is the 57014 error on the original query.

12.3 Timeouts that stop the work

A timeout that cancels

Put the two together: wait with a timeout, and when it fires, cancel *and keep waiting* for the query to end. Waiting matters. The query then fails with 57014 (or succeeds, if it finished just as the cancel was sent), and when the helper returns, the connection is free for the next request:

```
use std::future::Future;
use std::pin::pin;
use std::time::Duration;
use tokio_postgres::error::SqlState;
use tokio_postgres::{Client, Error, NoTls};

/// Runs `query`, and asks the server to cancel it if it takes longer than `limit`. After a
/// cancel it still waits for the query to end, so the connection is idle when this returns.
async fn cancel_after<T>(
    client: &Client,
    limit: Duration,
    query: impl Future<Output = Result<T, Error>>,
) -> Result<T, Error> {
    let mut query = pin!(query);
    match tokio::time::timeout(limit, query.as_mut()).await {
        Ok(result) => result,
        Err(_elapsed) => {
            client.cancel_token().cancel_query(NoTls).await?;
            query.await // usually 57014 now
        }
    }
}

#[tokio::main]
async fn main() -> Result<(), Error> {
```

```
    let client = playground::connect().await?;
    let limit = Duration::from_millis(200);

    let fast = cancel_after(&client, limit, client.query_one("SELECT 1", &[])).await?;
    assert_eq!(fast.get::<_, i32>(0), 1);

    let err = cancel_after(&client, limit, client.execute("SELECT pg_sleep(5)", &[]))
        .await
        .unwrap_err();
    assert_eq!(err.code(), Some(&SqlState::QUERY_CANCELED));
    Ok(())
}
```

The helper has one gap, which follows from the protocol lens above. If the query finishes at the very moment the cancel request is sent, the request may reach the backend while it runs the connection's *next* statement, and cancel that one instead. Code that owns its connection can live with a rare spurious 57014. A pool cannot hand that risk to the next borrower, which is why stockroom discards a connection after cancelling on it (§ In stockroom).

Limits enforced by the server

A client-side timeout needs the client to be alive and well. Server-side limits work even when it is not, and they are the first line of defence:

Setting	Ends	Error
`statement_timeout`	a statement that runs longer than this	57014, the statement
`lock_timeout`	a wait for a lock (row, table, advisory) longer than this	55P03, the statement
`idle_in_transaction_session_timeout`	a session idle inside a transaction for longer than this	25P03, the whole session
`transaction_timeout` (PG 17+)	a transaction older than this, idle or not	25P04, the whole session

There are three places to set them. `SET LOCAL` changes a setting until the end of the current transaction, which is the right scope for "this report may take 30 seconds". A startup option in `Config::options` (`-c name=value`, as `PGOPTIONS` does for psql) sets it for every statement of the session from the first. And `ALTER ROLE … SET` makes it the default for every session of a role.

```
use tokio_postgres::error::SqlState;
use tokio_postgres::{Config, NoTls};

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    // For one transaction: SET LOCAL ends with the transaction.
    let mut client = playground::connect().await?;
    let tx = client.transaction().await?;
    tx.batch_execute("SET LOCAL statement_timeout = '100ms'")
        .await?;
    let err = tx.execute("SELECT pg_sleep(1)", &[]).await.unwrap_err();
    println!("{}", err.as_db_error().expect("a server error").message());
    assert_eq!(err.code(), Some(&SqlState::QUERY_CANCELED));
    drop(tx); // rolled back, and the setting with it
    let after: String = client
        .query_one("SHOW statement_timeout", &[])
        .await?
        .get(0);
```

```
    assert_eq!(after, "0");

    // For a whole session: startup options, in force before the first query.
    let url = std::env::var("DATABASE_URL").expect("DATABASE_URL");
    let mut config: Config = url.parse()?;
    config.options("-c statement_timeout=100ms -c lock_timeout=50ms");
    let (guarded, connection) = config.connect(NoTls).await?;
    tokio::spawn(async move {
        if let Err(e) = connection.await {
            eprintln!("connection error: {e}");
        }
    });
    let settings: Vec<String> = guarded
        .query(
            "SELECT setting FROM pg_settings
             WHERE name IN ('lock_timeout', 'statement_timeout') ORDER BY name",
            &[],
        )
        .await?
        .iter()
        .map(|row| row.get(0))
        .collect();
    assert_eq!(settings, ["50", "100"]); // in milliseconds

    // lock_timeout: wait at most 50 ms for an advisory lock that someone else holds.
    client.execute("SELECT pg_advisory_lock(912)", &[]).await?;
    let err = guarded
        .execute("SELECT pg_advisory_lock(912)", &[])
        .await
        .unwrap_err();
    println!("{}", err.as_db_error().expect("a server error").message());
    assert_eq!(err.code(), Some(&SqlState::LOCK_NOT_AVAILABLE)); // 55P03
    client
        .execute("SELECT pg_advisory_unlock(912)", &[])
        .await?;
    Ok(())
}
```

```
canceling statement due to statement timeout
canceling statement due to lock timeout
```

The last two settings in the table end the *session*, because a transaction cannot be safely continued once the server has given up on it. The client sees its connection close:

```
use std::time::Duration;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut client = playground::connect().await?;
    client
        .batch_execute("SET idle_in_transaction_session_timeout = '200ms'")
        .await?;
    let tx = client.transaction().await?;
    tx.execute("SELECT 1", &[]).await?;
    // A slow HTTP call, made while the transaction is open...
    tokio::time::sleep(Duration::from_millis(500)).await;
    // ...and the server has ended the session meanwhile.
    let err = tx.execute("SELECT 1", &[]).await.unwrap_err();
    println!("{err}");
    assert!(err.is_closed());
    Ok(())
}
```

```
connection error: db error
connection closed
```

The first line comes from `playground::connect`'s connection task, which received the server's FATAL message (25P03) as the connection's final error; the query sees only "connection closed". `transaction_timeout` reports its FATAL 25P04 to the running statement. Note that 0.7.18's `SqlState` has no named constant for 25P04, which is newer than the crate's table: compare `code().code()` with `"25P04"` if you need to match it.

stockroom sets `statement_timeout`, `lock_timeout` and `idle_in_transaction_session_timeout` for every session (§ In stockroom). The last one is the safety net for a bug that otherwise causes slow, mysterious trouble: a code path that starts a transaction and then awaits something slow, holding locks and a snapshot all the while.

12.4 Dead connections

A connection can die in two ways, and they are noticed very differently.

Loudly. The server ends the session (an administrator's `pg_terminate_backend`, a restart, a session timeout) and sends a FATAL message, or the operating system resets the TCP connection. The connection task reads that at once, finishes with an error, and `Client::is_closed()` turns `true`:

```
use std::time::{Duration, Instant};

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    let admin = playground::connect().await?;
    let pid: i32 = client
        .query_one("SELECT pg_backend_pid()", &[])
        .await?
        .get(0);
    client.check_connection().await?; // one round trip, no statement

    admin
        .execute("SELECT pg_terminate_backend($1)", &[&pid])
        .await?;
    let started = Instant::now();
    while !client.is_closed() && started.elapsed() < Duration::from_secs(5) {
        tokio::time::sleep(Duration::from_millis(1)).await;
    }
    println!("is_closed() after {:?}", started.elapsed());
    assert!(client.is_closed());
    let err = client.check_connection().await.unwrap_err();
    assert!(err.is_closed());
    Ok(())
}
```

```
connection error: db error
is_closed() after 2.552166ms
```

`is_closed()` costs nothing and asks no one: it reports whether the connection task has ended. `check_connection()` (0.7.15+) asks the server, with one round trip and no statement, so it also catches a connection that died a moment ago and whose task has not noticed yet. Chapter 13 weighs the two for a pool.

Silently. A network path that disappears (a NAT that forgets the connection, a cable, a cloud load balancer's idle timeout, a virtual machine that freezes) sends nothing. An idle connection then looks

alive forever, and a query sent on it waits until TCP gives up, which takes many minutes by default. Three settings shorten that:

`Config` setting	Default in 0.7.18	What it detects
`keepalives(bool)`	`true`	turns TCP keepalive probes on
`keepalives_idle(Duration)`	2 hours	how long an idle connection waits before the first probe
`keepalives_interval(Duration)`, `keepalives_retries(u32)`	the operating system's	the time between probes, and how many may go unanswered
`tcp_user_timeout(Duration)`	none	how long sent data may stay unacknowledged; **Linux only** (`connect_socket.rs` sets it only there)
`connect_timeout(Duration)`	none	how long one connection attempt may take, per host

With keepalives probing after 30 seconds, every 10 seconds, 3 times, a dead idle connection is found within about a minute, as in stockroom's settings. A query timeout (§12.3) covers the case keepalives cannot: a connection that is not idle but waiting for an answer.

In code, with each setting spelled out, plus `host_path`, which connects through a Unix-domain socket instead of TCP where the server offers one (its directory is `unix_socket_directories` in the server's configuration; this book's server uses `/tmp`):

```
use std::path::Path;
use std::time::Duration;
use tokio_postgres::config::LoadBalanceHosts;
use tokio_postgres::{Config, NoTls};

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut config: Config = std::env::var("DATABASE_URL")?.parse()?;
    config
        .keepalives(true)
        .keepalives_idle(Duration::from_secs(30))
        .keepalives_interval(Duration::from_secs(10))
        .keepalives_retries(3)
        .tcp_user_timeout(Duration::from_secs(60)) // applied on Linux only
        .connect_timeout(Duration::from_secs(5))
        .load_balance_hosts(LoadBalanceHosts::Disable);
    let (client, connection) = config.connect(NoTls).await?;
    tokio::spawn(connection);
    client.check_connection().await?;
    assert_eq!(config.get_keepalives_idle(), Duration::from_secs(30));

    // The same server through its Unix-domain socket, if it has one in /tmp: `host_path`
names
    // the directory, and the port picks the socket file in it.
    let port = config.get_ports()[0];
    let dir = Path::new("/tmp");
    if dir.join(format!(".s.PGSQL.{port}")).exists() {
        let mut local = Config::new();
        local
            .user(config.get_user().expect("a user"))
            .password(config.get_password().expect("a password"))
            .dbname(config.get_dbname().expect("a database"))
            .host_path(dir)
            .port(port);
```

```
        let (client, connection) = local.connect(NoTls).await?;
        tokio::spawn(connection);
        // No client IP address: the connection did not use TCP.
        let address: Option<String> = client
            .query_one("SELECT inet_client_addr()::text", &[])
            .await?
            .get(0);
        assert_eq!(address, None);
        println!(
            "connected through {}",
            dir.join(format!(".s.PGSQL.{port}")).display()
        );
    }
    Ok(())
}
```

```
connected through /tmp/.s.PGSQL.5433
```

What to do about a dead connection depends on who owns it. A pool discards it and opens another (Chapter 13). A long-lived connection of your own, like Chapter 11's listener, needs a reconnect loop, with a backoff so that a database that is down is not hammered with attempts. pglisten's is typical:

```
// apps/pglisten/src/lib.rs (excerpt)
let mut delay = Duration::from_millis(100);
loop {
    let started = Instant::now();
    let error = match session(&config, &options, &mut on_event, &shutdown).await {
        Ok(()) => return, // shut down
        Err(error) => error,
    };
    if started.elapsed() > Duration::from_secs(60) {
        delay = Duration::from_millis(100); // a new failure, not the same one again
    }
    // … report the loss, then sleep for `delay` unless shut down …
    delay = (delay * 2).min(Duration::from_secs(30));
}
```

apps/pglisten/src/lib.rs

12.5 Several hosts

A `Config` holds a list of hosts, and `connect` tries them in order until one accepts, with `connect_timeout` applying to each. In a URL they are comma-separated (`postgres://user@db1:5432,db2:5432/app`), and in key/value form, `host=db1,db2 port=5432,5432`. Two settings refine the choice:

- `target_session_attrs` (`Any`, `ReadWrite`, `ReadOnly`) checks each server after connecting and moves on if it is not the kind asked for. `ReadWrite` finds the primary of a replicated cluster after a failover. In 0.7.18 the check is a `SHOW transaction_read_only` query on each new connection (`connect.rs`), so it costs one round trip per host tried.
- `load_balance_hosts` (`Disable`, `Random`) shuffles the list first, spreading clients over read replicas.

```
use tokio_postgres::config::{Host, TargetSessionAttrs};
use tokio_postgres::{Config, NoTls};

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let url = std::env::var("DATABASE_URL")?;
    let base: Config = url.parse()?;
```

```
    let Host::Tcp(host) = &base.get_hosts()[0] else {
        panic!("a TCP host")
    };

    // `host` *appends*: this is a two-host configuration now, not a redirected one.
    let mut appended = base.clone();
    appended.host("replica.example.internal");
    assert_eq!(appended.get_hosts().len(), 2);

    // A first host where nothing listens, then the real server: the second one is used.
    let mut config = Config::new();
    config
        .user(base.get_user().expect("a user"))
        .password(base.get_password().expect("a password"))
        .dbname(base.get_dbname().expect("a database"))
        .host("localhost")
        .port(1)
        .host(host)
        .port(base.get_ports()[0]);
    let (client, connection) = config.connect(NoTls).await?;
    tokio::spawn(async move {
        if let Err(e) = connection.await {
            eprintln!("connection error: {e}");
        }
    });
    let port: i32 = client
        .query_one("SELECT inet_server_port()", &[])
        .await?
        .get(0);
    assert_eq!(port, i32::from(base.get_ports()[0]));

    // Ask for a read-only server: this one is a primary, so no host qualifies.
    config.target_session_attrs(TargetSessionAttrs::ReadOnly);
    let Err(err) = config.connect(NoTls).await else {
        panic!("this connection attempt should fail")
    };
    println!(
        "{err}: {}",
        std::error::Error::source(&err).expect("a cause")
    );
    Ok(())
}
```

```
error connecting to server: database is not read only
```

The `host` call in the middle of that example is a trap worth remembering: `Config::host` and `Config::port` add to the lists, so "parse the URL, then point it somewhere else" keeps the original host first and silently connects there. Build a new `Config`, as the example does, when you need different hosts.

12.6 TLS

What `sslmode` means in tokio-postgres

In libpq, `sslmode` has six values, and `verify-ca` and `verify-full` make libpq check the server's certificate. tokio-postgres splits the job in two. The connection string decides *whether* TLS is used, with three values only (`SslMode`):

`sslmode`	Meaning
`disable`	never TLS
`prefer` (the default)	TLS if the connector and the server both support it; otherwise plaintext, silently

`sslmode`	**Meaning**
`require`	TLS, or fail

And the **connector** you pass to `connect` decides *how* TLS is done, including whether the certificate is trusted. `NoTls` cannot do TLS at all. The rustls, native-tls and OpenSSL connectors verify the certificate chain and the host name by default, which is libpq's `verify-full`. So `sslmode=require` with a verifying connector is the secure setting, and the two common mistakes look like this, against this book's server (which has TLS turned off):

```
use std::sync::Arc;
use tokio_postgres::Config;
use tokio_postgres::config::{ChannelBinding, SslMode};

fn a_rustls_connector() -> tokio_postgres_rustls::MakeRustlsConnect {
    let provider = Arc::new(rustls::crypto::ring::default_provider());
    let config = rustls::ClientConfig::builder_with_provider(provider)
        .with_safe_default_protocol_versions()
        .expect("the ring provider supports the default versions")
        .with_root_certificates(rustls::RootCertStore::empty())
        .with_no_client_auth();
    tokio_postgres_rustls::MakeRustlsConnect::new(config)
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let config: Config = std::env::var("DATABASE_URL")?.parse()?;

    // 1. A TLS connector, and the default sslmode=prefer: plaintext, without a word.
    let (client, connection) = config.connect(a_rustls_connector()).await?;
    tokio::spawn(connection);
    let ssl: bool = client
        .query_one(
            "SELECT ssl FROM pg_stat_ssl WHERE pid = pg_backend_pid()",
            &[],
        )
        .await?
        .get(0);
    assert!(!ssl, "connected, but not encrypted");

    // 2. sslmode=require, as it should be: now the missing TLS is an error.
    let mut required = config.clone();
    required.ssl_mode(SslMode::Require);
    let Err(err) = required.connect(a_rustls_connector()).await else {
        panic!("this connection attempt should fail")
    };
    println!(
        "{err}: {}",
        std::error::Error::source(&err).expect("a cause")
    );

    // 3. NoTls with sslmode=require: the server says "no TLS" first, so the same error. (A
    //    server that offers TLS gets "no TLS implementation configured" instead.)
    let Err(err) = required.connect(tokio_postgres::NoTls).await else {
        panic!("this connection attempt should fail")
    };
    println!(
        "{err}: {}",
        std::error::Error::source(&err).expect("a cause")
    );

    // Channel binding (SCRAM-SHA-256-PLUS) needs TLS too: requiring it over plaintext
fails.
    let mut binding = config.clone();
    binding.channel_binding(ChannelBinding::Require);
    let Err(err) = binding.connect(tokio_postgres::NoTls).await else {
```

```
        panic!("this connection attempt should fail")
    };
    println!(
        "{err}: {}",
        std::error::Error::source(&err).expect("a cause")
    );
    Ok(())
}
```

```
error performing TLS handshake: server does not support TLS
error performing TLS handshake: server does not support TLS
authentication error: server did not use channel binding
```

> **Pitfall:** with `sslmode=prefer`, any server that does not offer TLS, or anything in the network pretending to be that server, gets a plaintext connection, password exchange included. Put `sslmode=require` in every production connection string, and pass a connector that verifies.

The connectors

The connector is anything that implements `MakeTlsConnect<Socket>`: given the host name, `make_tls_connect` returns a `TlsConnect`, whose `connect` turns the socket into a TLS stream. The stream type also reports the certificate data that channel binding needs. You will rarely implement these traits; three crates do, for the three common TLS libraries:

Crate (version checked 2026-11-06)	TLS library	Trusts by default
`tokio-postgres-rustls` 0.14.0	rustls, pure Rust	what you put in its `ClientConfig`
`postgres-native-tls` 0.5.3	the platform's: Security.framework, SChannel or OpenSSL	the operating system's store
`postgres-openssl` 0.5.3	OpenSSL	OpenSSL's configured store

This book uses rustls, which needs no system library and builds the same everywhere. The example below connects to the TLS server of `docker compose` (on port 5434, accepting TLS connections only), trusting the certificate authority that `docker/gen-certs.sh` created:

```
use rustls::pki_types::CertificateDer;
use rustls::pki_types::pem::PemObject;
use std::sync::Arc;
use tokio_postgres_rustls::MakeRustlsConnect;

/// A rustls connector that trusts the certificate authorities in a PEM file.
fn rustls_connector(ca_file: &str) -> Result<MakeRustlsConnect, Box<dyn std::error::Error>>
{
    let mut roots = rustls::RootCertStore::empty();
    for cert in CertificateDer::pem_file_iter(ca_file)? {
        roots.add(cert?)?;
    }
    let provider = Arc::new(rustls::crypto::ring::default_provider());
    let config = rustls::ClientConfig::builder_with_provider(provider)
        .with_safe_default_protocol_versions()?
        .with_root_certificates(roots)
        .with_no_client_auth();
    Ok(MakeRustlsConnect::new(config))
```

```
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    // For this example DATABASE_URL is the TLS server's: …@localhost:5434/tpbook?
sslmode=require
    let url = std::env::var("DATABASE_URL")?;
    let ca_file = std::env::var("TLS_CA_FILE")?;
    let (client, connection) = tokio_postgres::connect(&url,
rustls_connector(&ca_file)?).await?;
    tokio::spawn(async move {
        if let Err(e) = connection.await {
            eprintln!("connection error: {e}");
        }
    });
    let row = client
        .query_one(
            "SELECT ssl, version, cipher FROM pg_stat_ssl WHERE pid = pg_backend_pid()",
            &[],
        )
        .await?;
    let (ssl, version, cipher): (bool, String, String) = (row.get(0), row.get(1),
row.get(2));
    println!("ssl: {ssl}, {version}, {cipher}");
    assert!(ssl);
    Ok(())
}
```

```
ssl: true, TLSv1.3, TLS_AES_256_GCM_SHA384
```

To run it yourself: `docker/gen-certs.sh && docker compose up -d --wait`, then set `TLS_DATABASE_URL` and `TLS_CA_FILE` as in `.env.example`, and point `DATABASE_URL` at the former. In production, trust the roots your platform provides (stockroom loads them with `rustls-native-certs`) or the CA of your database provider.

Verification fails in the two ways it should. An unknown certificate authority, and a certificate for another name, are both refused during the handshake; `hostaddr` makes the second one easy to show, by connecting to 127.0.0.1 while presenting another host name:

```
use std::sync::Arc;
use tokio_postgres::Config;
use tokio_postgres::config::SslMode;

fn connector(roots: rustls::RootCertStore) -> tokio_postgres_rustls::MakeRustlsConnect {
    let provider = Arc::new(rustls::crypto::ring::default_provider());
    let config = rustls::ClientConfig::builder_with_provider(provider)
        .with_safe_default_protocol_versions()
        .expect("the ring provider supports the default versions")
        .with_root_certificates(roots)
        .with_no_client_auth();
    tokio_postgres_rustls::MakeRustlsConnect::new(config)
}

fn trusted_roots() -> Result<rustls::RootCertStore, Box<dyn std::error::Error>> {
    use rustls::pki_types::{CertificateDer, pem::PemObject};
    let mut roots = rustls::RootCertStore::empty();
    for cert in CertificateDer::pem_file_iter(std::env::var("TLS_CA_FILE")?)? {
        roots.add(cert?)?;
    }
    Ok(roots)
}

fn cause(err: &tokio_postgres::Error) -> String {
    std::error::Error::source(err)
```

```
        .map(|e| e.to_string())
        .unwrap_or_default()
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let config: Config = std::env::var("DATABASE_URL")?.parse()?;

    // A connector that trusts nobody: the server's certificate has an unknown issuer.
    let Err(err) = config
        .connect(connector(rustls::RootCertStore::empty()))
        .await
    else {
        panic!("this connection attempt should fail")
    };
    println!("{}", cause(&err));

    // The right CA, but the name we connect by is not on the certificate.
    let mut renamed = Config::new();
    renamed
        .user(config.get_user().expect("a user"))
        .password(config.get_password().expect("a password"))
        .dbname(config.get_dbname().expect("a database"))
        .host("db.example.internal") // the name TLS checks
        .hostaddr("127.0.0.1".parse()?) // the address TCP uses
        .port(config.get_ports()[0])
        .ssl_mode(SslMode::Require);
    let Err(err) = renamed.connect(connector(trusted_roots()?)).await else {
        panic!("this connection attempt should fail")
    };
    println!("{}", cause(&err));

    // NoTls against a server that accepts TLS connections only.
    let mut plain = config.clone();
    plain.ssl_mode(SslMode::Prefer);
    let Err(err) = plain.connect(tokio_postgres::NoTls).await else {
        panic!("this connection attempt should fail")
    };
    println!("{}", err.as_db_error().expect("a server error").message());
    Ok(())
}
```

```
invalid peer certificate: UnknownIssuer
invalid peer certificate: certificate not valid for name "db.example.internal"; certificate
is only valid for DnsName("localhost") or IpAddress(127.0.0.1)
no pg_hba.conf entry for host "192.168.65.1", user "postgres", database "tpbook", no
encryption
```

(The last message is the server's: its `pg_hba.conf` accepts only `hostssl` connections from outside the container, so a plaintext connection is refused before authentication.)

Direct TLS and channel binding

Two newer options make TLS connections faster and harder to intercept:

- **`sslnegotiation=direct`** (PG 17+ servers). Normally the client first asks "do you speak TLS?" (an *SSLRequest*) and waits for the answer before starting the handshake. With direct negotiation it starts the handshake at once, saving a round trip. The server then insists on the TLS ALPN extension with the protocol name `postgresql` (protocol docs), and tokio-postgres-rustls 0.14 does not set it: add it to the `ClientConfig` yourself, or the server closes the connection. (postgres-native-tls and postgres-openssl have a `set_postgresql_alpn()` method for this.)

- **`channel_binding=require`**. With SCRAM authentication over TLS, the client can bind the password exchange to the server's certificate (SCRAM-SHA-256-PLUS), so that a man-in-the-middle holding a different certificate cannot relay the authentication. The default, `prefer`, uses it when it can, which also means it hides a connector that never can: tokio-postgres-rustls 0.13 had exactly that bug, fixed in 0.14 (Appendix F).

```
use rustls::pki_types::{CertificateDer, pem::PemObject};
use std::sync::Arc;
use tokio_postgres::Config;
use tokio_postgres::config::{ChannelBinding, SslNegotiation};

fn connector(
    alpn: bool,
) -> Result<tokio_postgres_rustls::MakeRustlsConnect, Box<dyn std::error::Error>> {
    let mut roots = rustls::RootCertStore::empty();
    for cert in CertificateDer::pem_file_iter(std::env::var("TLS_CA_FILE")?)? {
        roots.add(cert?)?;
    }
    let provider = Arc::new(rustls::crypto::ring::default_provider());
    let mut config = rustls::ClientConfig::builder_with_provider(provider)
        .with_safe_default_protocol_versions()?
        .with_root_certificates(roots)
        .with_no_client_auth();
    if alpn {
        config.alpn_protocols = vec![b"postgresql".to_vec()];
    }
    Ok(tokio_postgres_rustls::MakeRustlsConnect::new(config))
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut config: Config = std::env::var("DATABASE_URL")?.parse()?; // sslmode=require
    config
        .ssl_negotiation(SslNegotiation::Direct)
        .channel_binding(ChannelBinding::Require);

    // Without ALPN, the server hangs up on a direct TLS connection.
    let Err(err) = config.connect(connector(false)?).await else {
        panic!("this connection attempt should fail")
    };
    println!("without ALPN: {err}");
    assert!(err.is_closed());

    // With it: one round trip less, and the authentication bound to the certificate.
    let (client, connection) = config.connect(connector(true)?).await?;
    tokio::spawn(connection);
    let ssl: bool = client
        .query_one(
            "SELECT ssl FROM pg_stat_ssl WHERE pid = pg_backend_pid()",
            &[],
        )
        .await?
        .get(0);
    println!("with ALPN: ssl = {ssl}");
    assert!(ssl);
    Ok(())
}
```

```
without ALPN: connection closed
with ALPN: ssl = true
```

12.7 Opening the connection yourself: `connect_raw`

`tokio_postgres::connect` and `Config::connect` do the networking: resolve the host, open a TCP or Unix socket, set its options, try the next host on failure. That is the `runtime` feature, on by default, which brings `connect`, `Config::connect`, the `Socket` type, and `CancelToken::cancel_query`. Underneath, `Config::connect_raw` runs the PostgreSQL startup over any stream you give it: anything `AsyncRead + AsyncWrite + Unpin`, a TCP stream you opened yourself, a stream through a proxy or tunnel, an in-memory pipe in a test. lagproxy's users (pgload, minipool) connect that way, and without the `runtime` feature it is the only way to connect at all.

```
use std::sync::Arc;
use std::time::Duration;
use tokio::net::TcpStream;
use tokio_postgres::config::Host;
use tokio_postgres::error::SqlState;
use tokio_postgres::{Config, NoTls};

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let config: Config = std::env::var("DATABASE_URL")?.parse()?;
    let Host::Tcp(host) = &config.get_hosts()[0] else {
        panic!("a TCP host")
    };
    let address = (host.as_str(), config.get_ports()[0]);

    let stream = TcpStream::connect(address).await?;
    assert!(!stream.nodelay()?); // the operating system's default: Nagle's algorithm on
    stream.set_nodelay(true)?; // as `connect` would have done
    let (client, connection) = config.connect_raw(stream, NoTls).await?;
    tokio::spawn(async move {
        if let Err(e) = connection.await {
            eprintln!("connection error: {e}");
        }
    });
    let client = Arc::new(client);
    let one: i32 = client.query_one("SELECT 1", &[]).await?.get(0);
    assert_eq!(one, 1);

    // Cancelling works the same way: bring your own stream to cancel_query_raw.
    let slow = {
        let client = Arc::clone(&client);
        tokio::spawn(async move { client.execute("SELECT pg_sleep(5)", &[]).await })
    };
    tokio::time::sleep(Duration::from_millis(100)).await;
    client
        .cancel_token()
        .cancel_query_raw(TcpStream::connect(address).await?, NoTls)
        .await?;
    let err = slow.await?.unwrap_err();
    assert_eq!(err.code(), Some(&SqlState::QUERY_CANCELED));
    Ok(())
}
```

`connect_raw` uses the stream exactly as given, and that is the catch. `connect` sets `TCP_NODELAY` on every socket (`connect_socket.rs`), because PostgreSQL's small request and response messages suffer badly from Nagle's algorithm; with `connect_raw`, setting it is your job, and so are keepalives and the connect timeout. Of the connection settings in the `Config`, `connect_raw` uses only those of the startup itself: user, password, database name, options and application name, plus the TLS mode and channel binding.

Side by side — *a query timeout*

The task: make a query that runs longer than 100 ms fail, and stop it on the server.

```
tpbook=# SET statement_timeout = '100ms';
SET
tpbook=# SELECT pg_sleep(1);
ERROR:  canceling statement due to statement timeout
```

```
// sqlx (illustration)
let mut conn = pool.acquire().await?;
sqlx::query("SET statement_timeout = '100ms'").execute(&mut *conn).await?;
let err = sqlx::query("SELECT pg_sleep(1)").execute(&mut *conn).await.unwrap_err();
// err.as_database_error().and_then(|e| e.code()) == Some("57014".into())
```

```
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;
    client
        .batch_execute("SET statement_timeout = '100ms'")
        .await?;
    let err = client.execute("SELECT pg_sleep(1)", &[]).await.unwrap_err();
    assert_eq!(err.code(), Some(&SqlState::QUERY_CANCELED));
    Ok(())
}
```

	psql	sqlx	tokio-postgres
Same	`statement_timeout` on the server: the statement fails with 57014 and the session goes on	`statement_timeout` on the server: the statement fails with 57014 and the session goes on	`statement_timeout` on the server: the statement fails with 57014 and the session goes on
Different	Ctrl-C sends a cancel request for the running statement	Set per connection, on a connection borrowed with `&mut`	Set per connection or per transaction; a `CancelToken` can also cancel from any task

In stockroom

Every stockroom session gets the same guard rails, applied to the `Config` before the pool or the listener sees it. The connection string can override the application name and connect timeout; the keepalives and server-side limits are always set:

```
// stockroom/src/config.rs (excerpt)
pub fn pg_config(&self) -> Result<Config, tokio_postgres::Error> {
    let mut config: Config = self.database_url.parse()?;
    if config.get_application_name().is_none() {
        config.application_name("stockroom");
    }
    if config.get_connect_timeout().is_none() {
        config.connect_timeout(Duration::from_secs(5));
    }
    // Notice a dead peer in about a minute rather than the default two hours.
    config.keepalives_idle(Duration::from_secs(30));
    config.keepalives_interval(Duration::from_secs(10));
    config.keepalives_retries(3);
```

```
    // Guard rails on the server: runaway statements, lock waits, and transactions left
    // open by a crashed or stuck client are all ended by PostgreSQL itself.
    let mut options = config.get_options().unwrap_or_default().to_owned();
    for setting in [
        format!("statement_timeout={}", self.statement_timeout_ms),
        "lock_timeout=2000".to_owned(),
        "idle_in_transaction_session_timeout=60000".to_owned(),
    ] {
        // … appended to `options` as "-c name=value" …
    }
    config.options(options);
    Ok(config)
}
```

stockroom/src/config.rs

`GET /reports/sales` can be slow, and its request may be abandoned: the router's timeout fires, or the client disconnects, and axum drops the handler's future. `CancelOnDrop` owns the pooled connection while the report runs. If it is dropped before `finish`, it cancels the query and then discards the connection instead of returning it, for the race of §12.3:

```
// stockroom/src/db/cancel.rs (excerpt)
impl Drop for CancelOnDrop {
    fn drop(&mut self) {
        let Some(conn) = self.conn.take() else { return };
        let token = conn.cancel_token();
        let tls = self.tls.clone();
        // `drop` cannot await, so the cancel request goes out from a task of its own.
        let Ok(runtime) = tokio::runtime::Handle::try_current() else {
            return;
        };
        runtime.spawn(async move {
            match token.cancel_query(tls).await {
                Ok(()) => tracing::info!("cancelled a query nobody was waiting for"),
                Err(error) => tracing::warn!(%error, "could not cancel an abandoned query"),
            }
            // Detach from the pool and close.
            drop(Object::take(conn));
        });
    }
}
```

stockroom/src/db/cancel.rs · stockroom/src/http/reports.rs

When the router's timeout fires before a `?delayMs=5000` report is done, the client gets 503, and `pg_stat_activity` shows no `pg_sleep` left running: §16.5 demonstrates it, and stockroom's test `an_abandoned_report_is_cancelled_on_the_server` checks it with a 300 ms timeout.

TLS is a cargo feature. Without it, `MakeTls` is `NoTls`; with `--features tls`, it is rustls, trusting `--db-ca-file` or the system's roots:

```
// stockroom/src/db/tls.rs (excerpt)
#[cfg(not(feature = "tls"))]
pub type MakeTls = tokio_postgres::NoTls;

#[cfg(feature = "tls")]
pub type MakeTls = tokio_postgres_rustls::MakeRustlsConnect;
```

stockroom/src/db/tls.rs

The rest of the code is written against `MakeTls` and compiles either way. One wrinkle: `NoTls` is `Copy`, while rustls's connector is only `Clone`, so code that must work with both calls `.clone()`, and clippy's

`clone_on_copy` lint, which fires in the `NoTls` build, is allowed once for the crate. The same binary runs against the TLS server:

```
cargo run -p stockroom --features tls -- --db-ca-file docker/certs/ca.crt migrate
```

Summary

- Dropping a query future (a timeout, a `select!`, a disconnected HTTP client) does not stop the query: the server keeps working, and the connection's next request waits behind it.
- `client.cancel_token()` gives a `Clone + Send` token; `cancel_query(tls)` sends a cancel request on a new connection, and the query fails with 57014. The server never confirms, and a late request can hit the next statement.
- A client-side timeout should cancel and then wait for the query to end. Server-side, `statement_timeout` and `lock_timeout` end statements; `idle_in_transaction_session_timeout` and `transaction_timeout` (PG 17+) end sessions. Set them with `SET LOCAL`, startup options (`Config::options`) or `ALTER ROLE`.
- `is_closed()` reports a connection whose task has ended; `check_connection()` asks the server. Keepalives (default: first probe after 2 hours) and `tcp_user_timeout` (Linux) detect silent network failures; long-lived connections need a reconnect loop with backoff.
- Several hosts are tried in order; `target_session_attrs` picks a primary or a replica, and `load_balance_hosts` shuffles. `Config::host` appends.
- In tokio-postgres, `sslmode` (disable, prefer, require) decides whether TLS is used, and the connector decides whether the certificate is trusted. Use `sslmode=require` with a verifying connector; `prefer` falls back to plaintext silently.
- `sslnegotiation=direct` (PG 17+) needs ALPN `postgresql`, which you set on the rustls `ClientConfig`; `channel_binding=require` binds SCRAM to the TLS certificate.
- `connect_raw` runs the startup over a stream you opened; then `TCP_NODELAY`, keepalives and timeouts are yours to set.

Exercises

1. **A timeout for a transaction.** Put a whole transaction (begin, two statements, commit), written as an `async` block, under a timeout that cancels. Why can't it be passed to `cancel_after` as written? What happens to the transaction when the cancel lands in the middle, and in what state is the connection afterwards? *Hint:* a cancelled statement is an error like any other; Chapter 8 said what an error does to a transaction, and what dropping a `Transaction` does.
2. **Who is waiting for whom?** Reproduce a lock wait between two sessions on a real table, and from a third session list the blocked and the blocking backend with `pg_blocking_pids()`. Then end it twice: once with a cancel request from the waiting client's `CancelToken`, once with `lock_timeout`. *Hint:* `SELECT pid, pg_blocking_pids(pid), wait_event_type FROM pg_stat_activity WHERE cardinality(pg_blocking_pids(pid)) > 0`.

3. **Reconnect.** Write a `connect_with_backoff(config: &Config) -> Client` that retries with delays of 100 ms, 200 ms, 400 ms … up to 5 s, and test it by starting it against a port where nothing listens. *Hint:* `tokio::time::timeout` around the whole retry loop keeps the test bounded.
4. **Prefer is not secure.** Explain why `sslmode=prefer` with a connector that verifies certificates still does not protect the password against an attacker on the network path. *Hint:* what does the attacker answer to the client's SSLRequest?

Solutions

1. `cancel_after` takes the client by `&Client`, and the transaction needs it by `&mut` (`transaction()` takes `&mut self`), so the two borrows cannot coexist. Take the `CancelToken` first, since it borrows nothing, and inline the helper. The cancel makes the running statement fail with 57014; the `?` in the async block returns that error, the `Transaction` is dropped and rolls back, and only then does the timeout code get its answer, so the connection is idle and outside any transaction:

```
use std::pin::pin;
use std::time::Duration;
use tokio_postgres::error::SqlState;
use tokio_postgres::{Error, NoTls};

#[tokio::main]
async fn main() -> Result<(), Error> {
    let mut client = playground::connect().await?;
    client
        .batch_execute("CREATE TEMP TABLE audit (note text)")
        .await?;

    let token = client.cancel_token(); // before the transaction borrows the client
    let err = {
        let work = async {
            let tx = client.transaction().await?;
            tx.execute("INSERT INTO audit VALUES ('started')", &[])
                .await?;
            tx.execute("SELECT pg_sleep(5)", &[]).await?; // cancelled here
            tx.commit().await
        };
        let mut work = pin!(work);
        match tokio::time::timeout(Duration::from_millis(200), work.as_mut()).await {
            Ok(result) => result.unwrap_err(),
            Err(_elapsed) => {
                token.cancel_query(NoTls).await?;
                work.await.unwrap_err()
            }
        }
    }; // the pinned block ends here, and with it the borrow of `client`
    assert_eq!(err.code(), Some(&SqlState::QUERY_CANCELED));

    // Rolled back, and the connection is usable, outside any transaction.
    let rows: i64 = client
        .query_one("SELECT count(*) FROM audit", &[])
        .await?
        .get(0);
    assert_eq!(rows, 0);
    Ok(())
}
```

The block around the pinned future is not decoration: `pin!` keeps the future, and its borrow of `client`, alive until the end of the enclosing block (Chapter 9's pitfall), so without it the last query would not compile.

2. The waiting session shows `wait_event_type = 'Lock'` and the blocker's PID in `pg_blocking_pids`. A cancel request ends the wait with 57014 ("canceling statement due to user request"), `lock_timeout` with 55P03 ("canceling statement due to lock timeout"); in both cases only the waiting statement fails, and the blocker never notices.

3.

```
use std::time::Duration;
use tokio_postgres::{Client, Config, Error, NoTls};

async fn connect_with_backoff(config: &Config) -> Client {
    let mut delay = Duration::from_millis(100);
    loop {
        match config.connect(NoTls).await {
            Ok((client, connection)) => {
                tokio::spawn(async move {
                    if let Err(e) = connection.await {
                        eprintln!("connection error: {e}");
                    }
                });
                return client;
            }
            Err(error) => {
                eprintln!("connect failed: {error}; retrying in {delay:?}");
                tokio::time::sleep(delay).await;
                delay = (delay * 2).min(Duration::from_secs(5));
            }
        }
    }
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    // Nothing listens on port 1: the loop keeps trying, and the timeout stops it.
    let mut unreachable = Config::new();
    unreachable.host("localhost").port(1).user("nobody");
    let attempt = tokio::time::timeout(
        Duration::from_millis(500),
        connect_with_backoff(&unreachable),
    )
    .await;
    assert!(attempt.is_err());

    // The real server answers on the first try.
    let url = std::env::var("DATABASE_URL").expect("DATABASE_URL");
    let client = connect_with_backoff(&url.parse()?).await;
    client.check_connection().await?;
    Ok(())
}
```

A production version adds jitter (Chapter 8) so that a hundred instances restarting together do not retry in lockstep, and resets the delay after a connection has been healthy for a while, as pglisten does.

4. With `prefer`, the client sends an SSLRequest and accepts either answer. An attacker in the middle answers "no" (`N`), and the client continues in plaintext, sending its authentication through the attacker; the connector never runs, so its careful verification never happens. Only `require` turns "no TLS" into an error. (SCRAM never sends the password itself, but a plaintext exchange can still be relayed to the real server, which is what channel binding prevents, and every query and result after it is readable.)

CHAPTER 13

Connection pooling

Part II showed how much one connection can do: it pipelines, streams, copies and listens. A web service still needs more than one, because a request that runs a transaction needs a connection to itself, a slow report must not hold up everyone else, and a connection that dies should take one request down with it, not all of them. Opening a connection per request is the wrong answer: it costs a TCP handshake, perhaps TLS, authentication and a new server process, several round trips and milliseconds before the first query.

A *pool* keeps a set of open connections and lends them out. This chapter builds one from scratch to show every decision a pool makes, then uses the one stockroom uses, deadpool-postgres, and measures when a pool helps and when one shared client does better.

13.1 Why one `Client` is not enough

Chapter 7 shared one `Client` between tasks, and for independent queries that is hard to beat. Three things break it:

- **Transactions.** `Client::transaction` takes `&mut self` (Chapter 8), so a client shared through an `Arc` cannot start one. That is the borrow checker enforcing a real rule: a transaction is a state of the *session*, and anything else sent on the session joins it.
- **Head-of-line blocking.** The server runs one statement at a time per session. A slow query, or a slowly read stream, holds up every request behind it (Chapters 7 and 9).
- **Failure isolation.** One connection is one point of failure. If it dies, every task using it fails at once; if one task's query is cancelled, the others must not be affected.

A pool answers all three: each borrower gets a session of its own, for as long as it needs it. It does not answer them for free, and §13.5 shows the price.

13.2 A pool from scratch: minipool

minipool (`apps/minipool`) is a working pool in about a hundred lines, written to make each decision visible. Its state is a semaphore and a stack:

```
// apps/minipool/src/lib.rs (excerpt)
struct Inner {
    config: Config,
    options: Options,
```

```
    permits: Arc<Semaphore>,
    // A std mutex, not tokio's: it is never held across an `.await`.
    idle: Mutex<Vec<Client>>,
}
```

How many connections at most? A `tokio::sync::Semaphore` with `max_size` permits. A borrower holds a permit for as long as it holds a connection, so there are never more than `max_size` in use. **What if all are busy?** Wait for a permit, but not forever: an overloaded service should fail fast instead of queueing requests without bound. **Which idle connection next?** The most recently used, which keeps a few connections warm. **Is it still alive?** Check before lending:

```
// apps/minipool/src/lib.rs (excerpt)
pub async fn get(&self) -> Result<Conn, PoolError> {
    let permit = tokio::time::timeout(
        self.inner.options.wait_timeout,
        Arc::clone(&self.inner.permits).acquire_owned(),
    )
    .await
    .map_err(|_| PoolError::Timeout)?
    .expect("the semaphore is never closed");

    loop {
        let candidate = self.inner.idle.lock().expect("not poisoned").pop();
        let Some(client) = candidate else { break };
        if client.is_closed() {
            continue; // its connection task has ended: drop it, try the next
        }
        if self.inner.options.verify && client.check_connection().await.is_err() {
            continue;
        }
        return Ok(self.lend(client, permit));
    }

    let (client, connection) = self.inner.config.connect(NoTls).await?;
    tokio::spawn(async move {
        let _ = connection.await; // ends when the client is dropped or the server goes away
    });
    Ok(self.lend(client, permit))
}
```

The borrowed connection is a guard, `Conn`, which derefs to `Client` (mutably too, so that `transaction()` works) and puts the client back when it is dropped:

```
// apps/minipool/src/lib.rs (excerpt)
pub struct Conn {
    client: Option<Client>,
    pool: Arc<Inner>,
    // Declared last, so it is dropped last: the client is back on the idle stack before the
    // permit lets the next caller in. The other way round, that caller could find the stack
    // empty and open a connection too many.
    _permit: OwnedSemaphorePermit,
}

impl Drop for Conn {
    fn drop(&mut self) {
        if let Some(client) = self.client.take()
            && !client.is_closed()
        {
            self.pool.idle.lock().expect("not poisoned").push(client);
        }
    }
}
```

apps/minipool/src/lib.rs

> **Rust refresher:** struct fields are dropped in declaration order, after the struct's own `Drop::drop` runs. `Conn::drop` pushes the client back first; then `pool` is dropped, then `_permit`, which releases the semaphore. The comment on `_permit` is load-bearing: in the opposite order, a waiter let in by the permit can find the stack still empty and open a connection beyond `max_size`.

Two more lessons hide in this code. What goes back to the pool is *whatever the borrower left behind*: a dropped `Transaction` has already queued its `ROLLBACK`, but a `SET`, a temporary table or a `LISTEN` stays with the session. And `is_closed()` only knows what the connection task has seen, which §13.3 puts to the test.

13.3 deadpool-postgres

deadpool-postgres (0.14.2, on deadpool 0.13) is the same design, finished: a `Manager` that knows how to create and recycle tokio-postgres connections, and a generic `Pool` with timeouts, hooks and metrics. It is built from a `tokio_postgres::Config`, a TLS connector, and a `ManagerConfig` that holds the one decision deadpool leaves to you, the `RecyclingMethod`:

Method	Runs when a connection is reused	Catches
`Fast` (the default)	`is_closed()` only	connections whose task has ended
`Verified`	`is_closed()` and an empty query: one round trip	connections that died without the task noticing yet
`Clean`	`is_closed()` and `CLOSE ALL; SET SESSION AUTHORIZATION DEFAULT; RESET ALL; UNLISTEN *; SELECT pg_advisory_unlock_all(); DISCARD TEMP; DISCARD SEQUENCES;`	the above, plus session state left behind
`Custom(sql)`	`is_closed()` and your SQL	what your SQL checks

`Clean` is `DISCARD ALL` without `DEALLOCATE ALL` and `DISCARD PLANS`, so that the statement cache of §13.4 stays valid. Here is what each one does with state a borrower leaves behind, in a pool of one connection so that the second borrower is sure to get the first one's session:

```
use deadpool_postgres::{Manager, ManagerConfig, Pool, RecyclingMethod, Runtime};
use tokio_postgres::{Config, NoTls};

fn pool_of_one(recycling_method: RecyclingMethod) -> Pool {
    let config: Config = std::env::var("DATABASE_URL")
        .expect("DATABASE_URL")
        .parse()
        .expect("a valid connection string");
    let manager = Manager::from_config(config, NoTls, ManagerConfig { recycling_method });
    Pool::builder(manager)
        .max_size(1)
        .runtime(Runtime::Tokio1)
        .build()
        .expect("a valid pool configuration")
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    for method in [
```

```
        RecyclingMethod::Fast,
        RecyclingMethod::Verified,
        RecyclingMethod::Clean,
    ] {
        let pool = pool_of_one(method.clone());
        {
            let first = pool.get().await?;
            first
                .batch_execute(
                    "SET application_name = 'left behind';
                     CREATE TEMP TABLE scratch (x int);
                     LISTEN left_behind;",
                )
                .await?;
        } // back to the pool
        let second = pool.get().await?;
        let row = second
            .query_one(
                "SELECT current_setting('application_name'),
                        to_regclass('pg_temp.scratch') IS NOT NULL,
                        (SELECT count(*) FROM pg_listening_channels())",
                &[],
            )
            .await?;
        let (name, temp, listening): (String, bool, i64) = (row.get(0), row.get(1),
row.get(2));
        println!(
            "{method:?}: application_name {name:?}, temp table {temp}, listening on
{listening}"
        );
        assert_eq!(temp, method != RecyclingMethod::Clean);
    }
    Ok(())
}
```

```
Fast: application_name "left behind", temp table true, listening on 1
Verified: application_name "left behind", temp table true, listening on 1
Clean: application_name "", temp table false, listening on 0
```

So `Fast` is right only for code that leaves nothing behind, which is a property of your code, not of the pool. stockroom promises it (§ In stockroom); a service that cannot, wants `Clean`.

The difference between `Fast` and `Verified` shows when a connection dies while it is idle in the pool: `Fast` can hand it out before its connection task has read the server's goodbye:

```
use deadpool_postgres::{Manager, ManagerConfig, Pool, RecyclingMethod, Runtime};
use tokio_postgres::{Config, NoTls};

fn pool_of_one(recycling_method: RecyclingMethod) -> Pool {
    let config: Config = std::env::var("DATABASE_URL")
        .expect("DATABASE_URL")
        .parse()
        .expect("a valid connection string");
    let manager = Manager::from_config(config, NoTls, ManagerConfig { recycling_method });
    Pool::builder(manager)
        .max_size(1)
        .runtime(Runtime::Tokio1)
        .build()
        .expect("a valid pool configuration")
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let admin = playground::connect().await?;
    for method in [RecyclingMethod::Fast, RecyclingMethod::Verified] {
        let pool = pool_of_one(method.clone());
```

```
        let pid: i32 = {
            let conn = pool.get().await?;
            conn.query_one("SELECT pg_backend_pid()", &[]).await?.get(0)
        };
        // The idle pooled connection is killed; its task may not have noticed yet.
        admin
            .execute("SELECT pg_terminate_backend($1)", &[&pid])
            .await?;
        let conn = pool.get().await?;
        let first_query = conn.query_one("SELECT pg_backend_pid()", &[]).await;
        println!(
            "{method:?}: first query {}",
            match &first_query {
                Ok(_) => "succeeded".to_owned(),
                Err(e) => format!("failed: {e}"),
            }
        );
        if method == RecyclingMethod::Verified {
            let new_pid: i32 = first_query?.get(0);
            assert_ne!(new_pid, pid); // a fresh connection
        }
    }
    Ok(())
}
```

```
Fast: first query failed: db error
Verified: first query succeeded
```

(The `Fast` line depends on a race, and it lost on every run here: `pg_terminate_backend` returns before the backend's goodbye has reached the client.) `Verified` pays one round trip on every checkout for that guarantee. On localhost that is noise; across a 2 ms network it is a 2 ms tax on every request. Many services choose `Fast` and handle the rare failure with a retry, or the HTTP 503 it becomes; a service whose database fails over often chooses `Verified`.

The `Pool` itself is configured with a builder:

```
// stockroom/src/db/pool.rs (excerpt)
pub fn create(config: Config, tls: MakeTls, max_size: usize) -> Result<Pool, BuildError> {
    let manager = Manager::from_config(
        config,
        tls,
        ManagerConfig {
            recycling_method: RecyclingMethod::Fast,
        },
    );
    Pool::builder(manager)
        .max_size(max_size)
        .runtime(Runtime::Tokio1)
        // Waiting longer than this for a free connection means the service is overloaded:
        // better to answer 503 than to queue requests without bound.
        .wait_timeout(Some(Duration::from_secs(5)))
        .create_timeout(Some(Duration::from_secs(5)))
        .recycle_timeout(Some(Duration::from_secs(5)))
        .build()
}
```

stockroom/src/db/pool.rs

The three timeouts bound the three ways `get()` can wait: for a free slot, for a new connection to open, and for the recycling check. They need a runtime to run their timers, hence `.runtime(Runtime::Tokio1)`; without one, a configured timeout makes `get()` fail with `NoRuntimeSpecified`. With none configured,

`get()` waits as long as it takes, which under overload means forever. deadpool also has `post_create`, `pre_recycle` and `post_recycle` hooks, for a `SET` that every new session should run, say.

13.4 `Object`, `prepare_cached`, `transaction` and `Object::take`

`pool.get()` returns an `Object` (deadpool-postgres also calls it `Client`), which goes back to the pool when dropped. It derefs to deadpool's `ClientWrapper`, which derefs to `tokio_postgres::Client`, so every method of the earlier chapters works on it, and `&object` coerces to `&tokio_postgres::Client`. The wrapper adds three things:

- **`prepare_cached(sql)`**: a per-connection statement cache. The first call prepares (one round trip); later calls with the same SQL on the same connection return the cached `Statement` without asking the server.
- **`transaction()` and `build_transaction()`**, returning deadpool's `Transaction`, which also has `prepare_cached`.
- **A `GenericClient` trait** of its own, implemented by `Object` and deadpool's `Transaction`, with `prepare_cached` in it. Repository functions written against it work in and out of transactions (Chapter 14).

And `Object::take(object)` detaches the connection from the pool for good: the pool forgets it and may open another in its place.

```
use deadpool_postgres::{Manager, Object, Pool, Runtime};
use std::time::{Duration, Instant};
use tokio_postgres::{Config, NoTls};

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let config: Config = std::env::var("DATABASE_URL")?.parse()?;
    let pool = Pool::builder(Manager::new(config, NoTls))
        .max_size(2)
        .runtime(Runtime::Tokio1)
        .wait_timeout(Some(Duration::from_millis(100)))
        .build()?;

    let a = pool.get().await?;
    let mut b = pool.get().await?;
    println!("{:?}", pool.status());
    // Both are out: the third waits 100 ms, then gives up.
    let err = pool.get().await.unwrap_err();
    println!("{err}");

    // The statement cache: the first call asks the server, the second does not.
    let started = Instant::now();
    let statement = b.prepare_cached("SELECT $1::int + 1").await?;
    let first = started.elapsed();
    let started = Instant::now();
    b.prepare_cached("SELECT $1::int + 1").await?;
    let second = started.elapsed();
    println!("prepare_cached: {first:?}, then {second:?}");
    assert!(second < first);
    assert_eq!(b.statement_cache.size(), 1);

    // A deadpool transaction, which has prepare_cached too.
    let tx = b.transaction().await?;
    let cached = tx.prepare_cached("SELECT $1::int + 1").await?;
    assert_eq!(cached.columns().len(), statement.columns().len());
    let two: i32 = tx.query_one(&cached, &[&1i32]).await?.get(0);
```

```
    tx.commit().await?;
    assert_eq!(two, 2);

    // Detach `a` for good: the pool shrinks, and `a` lives on as a plain client.
    let detached = Object::take(a);
    drop(b);
    println!("{:?}", pool.status());
    detached.query_one("SELECT 1", &[]).await?;
    Ok(())
}
```

```
Status { max_size: 2, size: 2, available: 0, waiting: 0 }
Timeout occurred while waiting for a slot to become available
prepare_cached: 1.251833ms, then 3.458µs
Status { max_size: 2, size: 1, available: 1, waiting: 0 }
```

`Object::take` is for connections that must not go back: one whose session state you cannot undo, one with a cancel request possibly in flight (Chapter 12's `CancelOnDrop`), or one you want to keep for a long-lived job.

> **Pitfall:** `prepare_cached` caches per connection, by SQL text. A statement prepared before a migration changes a table's columns can fail afterwards with "cached plan must not change result type" (0A000) on every connection that cached it. After schema changes at run time, clear the caches (`pool.manager().statement_caches.clear()`) or restart the service, which is what most deployments do anyway.

13.5 How big? Measured

PostgreSQL limits connections with `max_connections` (100 by default, of which `superuser_reserved_connections`, 3 by default, are kept for superusers), and each connection is a server process with its own memory. Every instance of every service that connects takes from that budget: four instances with pools of 25 already use it all. So the question is never "how many could help?", but "how few are enough?".

`minipool`'s binary measures the trade-off. 64 tasks each run 50 units of work through one shared client, through minipool, and through deadpool, with pools of 8. A unit is either one `query_typed` (one round trip) or a transaction around it (`BEGIN`, query, `COMMIT`: three round trips that cannot be pipelined with anyone else's). Release build, PG 18.6 on localhost, Apple Silicon:

```
cargo run -p minipool --release -- --tasks 64 --pool-size 8
cargo run -p minipool --release -- --tasks 64 --pool-size 8 --delay-ms 2
```

```
64 tasks × 50 iterations = 3200 operations, pools of 8, no added one-way delay

| Setup | Work | Time | Operations per second |
|---|---|---:|---:|
| one shared client | query | 0.050 s | 63403 |
| minipool, 8 connections | query | 0.041 s | 78587 |
| deadpool, 8 connections | query | 0.041 s | 77514 |
| minipool, 8 connections | transaction | 0.107 s | 29786 |
```

```
| deadpool, 8 connections | transaction | 0.107 s | 30006 |

64 tasks × 50 iterations = 3200 operations, pools of 8, 2 ms one-way delay

| Setup | Work | Time | Operations per second |
|---|---|---:|---:|
| one shared client | query | 0.383 s | 8350 |
| minipool, 8 connections | query | 3.084 s | 1038 |
| deadpool, 8 connections | query | 3.328 s | 962 |
| minipool, 8 connections | transaction | 9.577 s | 334 |
| deadpool, 8 connections | transaction | 9.093 s | 352 |
```

What the numbers say:

- **On localhost, a pool is a little faster for independent queries**: eight server processes work in parallel, while one connection's statements run one at a time.
- **Across a network, one pipelined client is eight times faster for independent queries.** A pool of 8 has at most 8 requests in flight, each waiting a full round trip; the shared client has all 64 in flight on one connection. This is Chapter 7's lesson with a number on it.
- **Transactions are where a pool is required**, and their cost is round trips: three per transaction, with the connection held for all of them. 352 transactions a second on eight connections is 44 per connection, about 23 ms each: three round trips of about 8 ms through lagproxy at a 2 ms delay. Only more connections, or fewer round trips, raise that ceiling.
- **minipool and deadpool are equal.** The cost is in the round trips, not the pool.

So size a pool for the transactions in flight at once, and no larger: roughly the number of concurrent requests that need a session to themselves, times the round trips each holds it for. Throughput beyond that comes from fewer round trips per transaction (pipelining inside it, Chapter 7; doing the work in one statement), not from more connections.

13.6 bb8, and PgBouncer

bb8 is the other common pool. Its PostgreSQL adapter, `bb8-postgres` (0.9.0, on bb8 0.9.1, checked 2026-11-06), wraps the same `tokio_postgres::Client`. The design differs in emphasis: bb8 runs a background "reaper" that closes idle connections after `idle_timeout` and old ones after `max_lifetime`, keeps `min_idle` connections open, and checks connections on checkout with `test_on_check_out` (on by default), much like `Verified`. bb8-postgres has no statement cache; with it you prepare statements yourself or use `query_typed`. For a service like stockroom the two are interchangeable, and this book uses deadpool for its statement cache and simpler API.

```rust
// bb8 (illustration)
let manager = bb8_postgres::PostgresConnectionManager::new(config, NoTls);
let pool = bb8::Pool::builder()
    .max_size(8)
    .connection_timeout(Duration::from_secs(5))
    .idle_timeout(Some(Duration::from_secs(600)))
    .build(manager)
    .await?;
let conn = pool.get().await?; // a PooledConnection that derefs to tokio_postgres::Client
```

PgBouncer is a pool outside the process: a proxy that many service instances connect to, multiplexing them onto a few server connections. In *transaction* pooling mode, the most useful one, a client's consecutive transactions may run on different server connections, so session state does not survive between transactions, which rules out everything §13.7 lists, and traditionally named prepared statements too. PgBouncer 1.21 (2023) added support for protocol-level named prepared statements, and since 1.24 it is on by default (`max_prepared_statements = 200`), so tokio-postgres's `prepare` and deadpool's `prepare_cached` work through current versions. Behind an older PgBouncer, or one configured without it, use `query_typed` (Chapter 4): it sends the unnamed statement with its parameter types in one round trip, and never depends on the server remembering anything.

13.7 What not to do with a pooled connection

A pooled connection is borrowed, and the next borrower inherits its session. So:

- **No `LISTEN`.** deadpool awaits its connections' tasks, so notifications are dropped, and `Clean` unlistens anyway. Listen on a dedicated connection (Chapter 11).
- **No streams that outlive their `Object`.** A `RowStream` or `CopyOutStream` does not borrow the connection; if the `Object` returns to the pool while one is unread, the next borrower's queries queue behind it. Move the `Object` into the stream (Chapter 9's `let _checked_out = db;`).
- **No session `SET`.** Use `SET LOCAL` inside a transaction, which ends with it, or startup options on the pool's `Config` (Chapter 12), which every connection gets.
- **No session-level advisory locks or temporary tables that outlive a transaction.** Use `pg_advisory_xact_lock` and `ON COMMIT DROP`.
- **No long transactions.** A borrowed connection held for seconds is one connection fewer for everyone else (§13.5), and its snapshot holds back `VACUUM` (Chapter 9).

Side by side — *a pool*

The task: a pool of at most 8 connections that gives up after 5 seconds of waiting, and a transaction on a connection from it.

```
// sqlx (illustration)
let pool = sqlx::postgres::PgPoolOptions::new()
    .max_connections(8)
    .acquire_timeout(Duration::from_secs(5))
    .connect(&url)
    .await?;
let mut tx = pool.begin().await?;
sqlx::query("INSERT INTO audit (note) VALUES ($1)").bind("pooled").execute(&mut *tx).await?;
tx.commit().await?;
```

```
use deadpool_postgres::{Manager, ManagerConfig, Pool, RecyclingMethod, Runtime};
use std::time::Duration;
use tokio_postgres::{Config, NoTls};

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let config: Config = std::env::var("DATABASE_URL")?.parse()?;
    let manager = Manager::from_config(
        config,
```

```
        NoTls,
        ManagerConfig {
            recycling_method: RecyclingMethod::Fast,
        },
    );
    let pool = Pool::builder(manager)
        .max_size(8)
        .runtime(Runtime::Tokio1)
        .wait_timeout(Some(Duration::from_secs(5)))
        .build()?;

    let mut conn = pool.get().await?;
    let tx = conn.transaction().await?;
    tx.batch_execute("CREATE TEMP TABLE audit (note text) ON COMMIT DROP")
        .await?;
    let insert = tx
        .prepare_cached("INSERT INTO audit (note) VALUES ($1)")
        .await?;
    assert_eq!(tx.execute(&insert, &[&"pooled"]).await?, 1);
    tx.commit().await?;
    Ok(())
}
```

	sqlx	tokio-postgres + deadpool
Same	A bounded pool with a wait timeout; a transaction borrows one connection	A bounded pool with a wait timeout; a transaction borrows one connection
Different	The pool is part of the driver, and checks each connection before lending it by default (`test_before_acquire`)	The pool is a separate crate over `tokio_postgres::Client`; the check on reuse (`RecyclingMethod`) is your choice, and `prepare_cached` caches statements on request

In stockroom

`db::pool::create` (§13.3) is the whole pool: `Fast` recycling, three 5-second timeouts. The comment on it is the contract that makes `Fast` safe:

```
// stockroom/src/db/pool.rs (excerpt)
/// `RecyclingMethod::Fast` hands a returned connection to the next caller after checking
only
/// that it is not closed. That is right for this service because no handler leaves session
/// state behind: no `SET` (only `SET LOCAL`, which ends with its transaction), no `LISTEN`
/// (the listener has its own connection), and no temporary table that outlives a
/// transaction. A service that cannot promise that wants `RecyclingMethod::Clean`.
```

Handlers get a connection through an extractor, so a handler's signature says that it uses the database, and the connection returns to the pool when the handler's future ends:

```
// stockroom/src/http/mod.rs (excerpt)
pub struct Db(pub Object);

impl FromRequestParts<AppState> for Db {
    type Rejection = AppError;

    async fn from_request_parts(
        _parts: &mut Parts,
        state: &AppState,
    ) -> Result<Self, Self::Rejection> {
```

```
        Ok(Db(state.pool.get().await?))
    }
}
```

stockroom/src/http/mod.rs

A pool that cannot lend a connection in time is an overloaded service, and `AppError` says so with 503 rather than an error page:

```
// stockroom/src/error.rs (excerpt)
impl From<PoolError> for AppError {
    fn from(error: PoolError) -> Self {
        match error {
            PoolError::Backend(e) => e.into(),
            PoolError::Timeout(_) => {
                AppError::Unavailable("no database connection became free in time".into())
            }
            other => AppError::Unavailable(other.to_string()),
        }
    }
}
```

stockroom/src/error.rs

Chapter 15 builds the rest of the HTTP layer on these pieces. `GET /readyz` reports the pool's `status()` (Chapter 16), and the size is a flag, `--pool-size`, because it belongs to the deployment, not the code.

Summary

- One shared `Client` cannot run transactions (`&mut self`), suffers head-of-line blocking, and fails as a whole. A pool gives each borrower its own session.
- A pool is a semaphore (how many), a stack of idle clients (which one), a health check (is it alive) and a guard that returns the client on drop, in the right order.
- deadpool-postgres: `Manager::from_config(config, tls, ManagerConfig { recycling_method })`, then `Pool::builder(...).max_size(n).runtime(Runtime::Tokio1)` with wait, create and recycle timeouts. `Fast` checks `is_closed()` only, `Verified` adds a round trip, `Clean` also resets session state.
- `Object` derefs to `tokio_postgres::Client`; it adds `prepare_cached`, its own `transaction`, and a `GenericClient` with `prepare_cached`. `Object::take` detaches a connection.
- Measured: over a 2 ms delay, one pipelined client ran independent queries 8× faster than a pool of 8; transactions need a pool, and their throughput is bounded by round trips per transaction. Size pools for concurrent transactions, within `max_connections`.
- Never leave session state on a pooled connection: no `LISTEN`, no unread streams, no session `SET`, no session locks or temp tables. PgBouncer's transaction mode enforces the same rules; `query_typed` works behind any pooler.

Exercises

1. **Leak detection.** Write a `Custom` recycling query that makes recycling *fail* if the returned connection has its `application_name` changed, so that such a connection is discarded instead of reused, and

test it with a pool of one. *Hint:* recycling fails when the query errors; `DO $$ BEGIN IF … THEN RAISE EXCEPTION …; END IF; END $$` raises one, and `current_setting('application_name')` reads the setting.

2. **Measure `Verified`.** Run §13.4's checkout in a loop of 1,000 through lagproxy with a 1 ms delay, once with `Fast` and once with `Verified`. Predict the difference first. *Hint:* one round trip per checkout.
3. **Too small.** stockroom's pool has 5 seconds of `wait_timeout`. With `--pool-size 2`, run ten concurrent `GET /reports/sales?delayMs=3000` requests. What do the clients see, and in what order? *Hint:* each request holds its connection for three seconds.
4. **minipool, verified.** minipool's `Options::verify` calls `check_connection()` on every checkout. Repeat §13.3's terminate experiment against minipool with `verify` on and off. *Hint:* minipool is a library; add it to the playground with a path dependency.

Solutions

1. deadpool runs a `Custom` query with `simple_query` when it reuses a connection; an error fails the recycling, and the pool drops that connection and opens another:

```
use deadpool_postgres::{Manager, ManagerConfig, Pool, RecyclingMethod, Runtime};
use tokio_postgres::{Config, NoTls};

const CHECK: &str = "DO $$
BEGIN
    IF current_setting('application_name') <> '' THEN
        RAISE EXCEPTION 'session state left behind';
    END IF;
END $$";

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let config: Config = std::env::var("DATABASE_URL")?.parse()?;
    let manager = Manager::from_config(
        config,
        NoTls,
        ManagerConfig {
            recycling_method: RecyclingMethod::Custom(CHECK.to_owned()),
        },
    );
    let pool = Pool::builder(manager)
        .max_size(1)
        .runtime(Runtime::Tokio1)
        .build()?;

    let leaky: i32 = {
        let conn = pool.get().await?;
        conn.batch_execute("SET application_name = 'left behind'")
            .await?;
        conn.query_one("SELECT pg_backend_pid()", &[]).await?.get(0)
    };
    let conn = pool.get().await?;
    let row = conn
        .query_one(
            "SELECT pg_backend_pid(), current_setting('application_name')",
            &[],
        )
        .await?;
    let (pid, name): (i32, String) = (row.get(0), row.get(1));
    assert_ne!(pid, leaky, "the leaky connection was discarded");
    assert_eq!(name, "");
    Ok(())
}
```

2. Each `Verified` checkout costs one more round trip: twice the delay plus lagproxy's timer overhead, roughly 5 to 8 ms at a 1 ms delay on the machine of Chapter 4. So 1,000 checkouts take several seconds longer than with `Fast`, whose checkout makes no round trip at all.

3. Two requests get a connection at once and finish after about three seconds. Two more were waiting and get the freed connections, finishing after about six seconds. The remaining six have waited five seconds by then, and receive 503 ("no database connection became free in time") at the five-second mark, without ever running their query.

4. With `verify` off, minipool behaves like `Fast`: the killed connection is lent, and the first query fails. With `verify` on, `check_connection()` fails on the dead connection, minipool drops it and opens a new one, and the first query succeeds.

CHAPTER 14

Schema, migrations and the data layer

A service that talks to a database makes two promises about it. The schema it expects must exist, in exactly the shape it expects, on every database it runs against: the developer's, the tests', and production's. And its SQL must be somewhere a reader can find it, check it, and change it without hunting through HTTP handlers.

This chapter covers both. The first half is migrations: how stockroom carries its schema in its binary and applies it safely, even when several instances start at once, and what the ready-made alternative, refinery, does. The second half is the data layer: repository functions that work on any connection or transaction, rows turned into types in one place, and domain types that keep invalid values out of both the program and the database. It ends with a tool that generates that layer from SQL.

14.1 Embedded migrations

A migration is a script that moves the schema from one version to the next. The usual design, and stockroom's, has four parts:

- **Numbered SQL files** (`migrations/0001_init.sql`, `0002_order_events.sql`), never edited once applied: a change is a new file.
- **Compiled into the binary** with `include_str!`, so that a build carries the schema it expects, and a deployment cannot pair the wrong files with the wrong code.
- **A table that records what was applied** (`schema_migrations`), with each migration's version and a checksum of its text.
- **A runner** that applies, in order, the migrations not yet recorded, and refuses to run if an applied migration's text has changed since.

```
// stockroom/src/db/migrate.rs (excerpt)
pub const MIGRATIONS: &[Migration] = &[
    Migration {
        version: 1,
        name: "init",
        sql: include_str!("../../migrations/0001_init.sql"),
    },
    Migration {
        version: 2,
        name: "order_events",
        sql: include_str!("../../migrations/0002_order_events.sql"),
    },
];
```

stockroom/src/db/migrate.rs

The runner fits in a page. This one is stockroom's, reduced to its essentials, and run the way a deployment runs it: two instances start at the same moment, and both try to migrate. (It works in a schema of its own, which it drops at the end.)

```
use std::collections::HashMap;
use tokio_postgres::Client;

type Error = Box<dyn std::error::Error>;

struct Migration {
    version: i32,
    name: &'static str,
    sql: &'static str,
}

const MIGRATIONS: &[Migration] = &[
    Migration {
        version: 1,
        name: "products",
        sql: "CREATE TABLE product (sku text PRIMARY KEY, name text NOT NULL);",
    },
    Migration {
        version: 2,
        name: "prices",
        sql: "ALTER TABLE product ADD COLUMN price_cents bigint NOT NULL DEFAULT 0;",
    },
];

/// FNV-1a, 64 bits, as hex: enough to notice an edited migration.
fn checksum(sql: &str) -> String {
    let hash = sql.bytes().fold(0xcbf2_9ce4_8422_2325_u64, |hash, byte| {
        (hash ^ u64::from(byte)).wrapping_mul(0x0000_0100_0000_01b3)
    });
    format!("{hash:016x}")
}

/// Applies the migrations not yet recorded, each in its own transaction, under an advisory
/// lock so that concurrent runners take turns. Returns the versions it applied.
async fn migrate(client: &mut Client, migrations: &[Migration]) -> Result<Vec<i32>, Error> {
    // One lock per schema, so that this example cannot collide with anything else.
    let lock = "SELECT pg_advisory_lock(hashtext(current_schema()))";
    client.execute(lock, &[]).await?;
    let result = apply(client, migrations).await;
    client
        .execute("SELECT pg_advisory_unlock(hashtext(current_schema()))", &[])
        .await?;
    result
}

async fn apply(client: &mut Client, migrations: &[Migration]) -> Result<Vec<i32>, Error> {
    client
        .batch_execute(
            "CREATE TABLE IF NOT EXISTS schema_migrations (
                 version    integer     PRIMARY KEY,
                 name       text        NOT NULL,
                 checksum   text        NOT NULL,
                 applied_at timestamptz NOT NULL DEFAULT now()
             )",
        )
        .await?;
    let applied: HashMap<i32, String> = client
        .query("SELECT version, checksum FROM schema_migrations", &[])
        .await?
        .iter()
        .map(|row| (row.get(0), row.get(1)))
        .collect();
    for m in migrations {
```

```
        if let Some(recorded) = applied.get(&m.version)
            && *recorded != checksum(m.sql)
        {
            return Err(format!(
                "migration {} ({}) was edited after it was applied",
                m.version, m.name
            )
            .into());
        }
    }

    let mut done = Vec::new();
    for m in migrations
        .iter()
        .filter(|m| !applied.contains_key(&m.version))
    {
        let tx = client.transaction().await?;
        tx.batch_execute(m.sql).await?;
        tx.execute(
            "INSERT INTO schema_migrations (version, name, checksum) VALUES ($1, $2, $3)",
            &[&m.version, &m.name, &checksum(m.sql)],
        )
        .await?;
        tx.commit().await?;
        done.push(m.version);
    }
    Ok(done)
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    // A schema of our own, so that the example leaves nothing behind. Its name is made of
    // digits generated here, never of input, which is what makes formatting it into SQL
safe.
    let schema = format!("ch14_{}", std::process::id());
    let mut a = playground::connect().await?;
    let mut b = playground::connect().await?;
    a.batch_execute(&format!("CREATE SCHEMA {schema}")).await?;
    for client in [&a, &b] {
        client
            .batch_execute(&format!("SET search_path = {schema}"))
            .await?;
    }

    // Two instances start at once: one migrates; the other waits for the lock, then finds
    // nothing left to do.
    let (first, second) = tokio::join!(migrate(&mut a, MIGRATIONS), migrate(&mut b,
MIGRATIONS));
    let mut applied = [first?, second?];
    applied.sort();
    assert_eq!(applied, [vec![], vec![1, 2]]);

    // Running again is a no-op...
    assert!(migrate(&mut a, MIGRATIONS).await?.is_empty());

    // ...and an applied migration whose text changed is refused.
    let edited = [
        Migration {
            sql: MIGRATIONS[0].sql,
            ..MIGRATIONS[0]
        },
        Migration {
            sql: "ALTER TABLE product ADD COLUMN price_cents numeric;",
            ..MIGRATIONS[1]
        },
    ];
    let err = migrate(&mut a, &edited).await.unwrap_err();
    println!("{err}");

    a.batch_execute(&format!("DROP SCHEMA {schema} CASCADE"))
```

```
        .await?;
    Ok(())
}
```

```
migration 2 (prices) was edited after it was applied
```

The checksum is not about security; it catches the honest mistake of fixing a migration in place after it has run somewhere, which leaves databases that ran the old text and databases that ran the new one with different schemas and the same version number.

14.2 Running migrations safely

> **SQL lens:** in PostgreSQL, DDL is transactional. `CREATE TABLE`, `ALTER TABLE`, `CREATE FUNCTION` and nearly all their kin take part in the surrounding transaction and roll back with it. So a migration that fails halfway leaves *nothing* behind: not half a table, not a column without its index. That is why the runner gives each migration its own transaction and records it in the same one. The exceptions are the commands that cannot run in a transaction block at all, chiefly `CREATE INDEX CONCURRENTLY` (and `DROP INDEX CONCURRENTLY`, `VACUUM`, `CREATE DATABASE`).

```
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let mut client = playground::connect().await?;

    // A migration that fails halfway: its first two statements are undone with it.
    let tx = client.transaction().await?;
    let err = tx
        .batch_execute(
            "CREATE TEMP TABLE supplier (id int PRIMARY KEY);
             ALTER TABLE supplier ADD COLUMN name text NOT NULL;
             CREATE INDEX ON supplier (nmae);",
        )
        .await
        .unwrap_err();
    println!("{}", err.as_db_error().expect("a server error").message());
    drop(tx);
    let exists: bool = client
        .query_one("SELECT to_regclass('pg_temp.supplier') IS NOT NULL", &[])
        .await?
        .get(0);
    assert!(!exists, "nothing of the failed migration is left");

    // The exception: CREATE INDEX CONCURRENTLY refuses to run in a transaction block...
    let schema = format!("ch14_{}", std::process::id());
    client
        .batch_execute(&format!(
            "CREATE SCHEMA {schema}; CREATE TABLE {schema}.supplier (id int, name text)"
        ))
        .await?;
    let tx = client.transaction().await?;
    let err = tx
        .batch_execute(&format!(
            "CREATE INDEX CONCURRENTLY ON {schema}.supplier (name)"
        ))
        .await
```

```
        .unwrap_err();
    println!("{}", err.as_db_error().expect("a server error").message());
    assert_eq!(err.code(), Some(&SqlState::ACTIVE_SQL_TRANSACTION)); // 25001
    drop(tx);

    // ...including the implicit one around several statements in one batch_execute.
    let err = client
        .batch_execute(&format!(
            "SELECT 1; CREATE INDEX CONCURRENTLY ON {schema}.supplier (name)"
        ))
        .await
        .unwrap_err();
    assert_eq!(err.code(), Some(&SqlState::ACTIVE_SQL_TRANSACTION));

    // Alone, it works.
    client
        .batch_execute(&format!(
            "CREATE INDEX CONCURRENTLY ON {schema}.supplier (name)"
        ))
        .await?;
    client
        .batch_execute(&format!("DROP SCHEMA {schema} CASCADE"))
        .await?;
    Ok(())
}
```

```
column "nmae" does not exist
CREATE INDEX CONCURRENTLY cannot run inside a transaction block
```

(The example uses a real table for the second half because PostgreSQL builds indexes on temporary tables non-concurrently anyway.) A runner that supports such migrations must run them outside its usual transaction, one statement per `batch_execute`, and accept that a failure in the middle leaves an invalid index behind to drop and rebuild. Many teams keep them out of automatic migrations and run them by hand.

Several instances, one migration. When three replicas of a service start together and each migrates on startup, two of them must wait. A session-level advisory lock does that without a table to lock: `pg_advisory_lock(key)` blocks until no other session holds `key`, and the lock is held across the transactions that follow until `pg_advisory_unlock`, or until the session ends, which matters if the instance crashes mid-migration. The second instance then reads `schema_migrations`, finds everything applied, and moves on. The `tokio::join!` in §14.1 is that scenario in miniature. stockroom's key is a constant everyone agrees on:

```
// stockroom/src/db/migrate.rs (excerpt)
/// The advisory lock all instances agree on: "STOCKROO" in ASCII.
const LOCK_KEY: i64 = 0x5354_4f43_4b52_4f4f;
```

Three more details from stockroom's runner are worth copying:

- **Lift the guard rails for the migration session.** stockroom's sessions carry a 5-second `statement_timeout` and a 2-second `lock_timeout` (Chapter 12); a migration may legitimately take longer and wait for locks, so the runner runs `SET statement_timeout = 0; SET lock_timeout = 0` on its own connection first.

- **Clear the type cache afterwards.** tokio-postgres caches what it learned about custom types (Chapter 5); after a migration changes one, `client.clear_type_cache()` makes the next query look it up afresh.
- **Log the notices.** `CREATE TABLE IF NOT EXISTS` reports "relation … already exists, skipping" as a notice. stockroom's `db::connect` drives its connection with `poll_message` (Chapter 11) and logs every notice, so a migration's server-side messages end up in the service's log.

Migrate either at startup, under the lock (simple, and what stockroom's `serve` does unless started with `--migrate false`), or in a separate deployment step before the new version starts (`stockroom migrate`), which keeps a failed migration from taking down every starting instance. Either way, write migrations that the *previous* version of the code can live with, because during a rolling deployment both run at once: add a column before using it, stop using it before dropping it.

14.3 refinery, the ready-made alternative

refinery (0.10.0, checked 2026-11-06) is the most used migration crate for tokio-postgres. Its migrations are files named `V1__init.sql`, `V2__order_events.sql` (or `U…` for non-contiguous versions), embedded at compile time by a macro, and recorded in a `refinery_schema_history` table with their checksums:

```
// refinery (illustration): refinery = { version = "0.10", features = ["tokio-postgres"] }
mod embedded {
    refinery::embed_migrations!("migrations");
}

let report = embedded::migrations::runner()
    .set_abort_divergent(true) // the default: refuse edited migrations
    .run_async(&mut client)    // a &mut tokio_postgres::Client
    .await?;
for migration in report.applied_migrations() {
    tracing::info!(%migration, "applied");
}
```

It does what §14.1's runner does, with more options: `set_grouped(true)` runs all pending migrations in one transaction, `set_target` stops at a version, and migrations can also be Rust functions that return SQL. Through deadpool, `run_async` wants the `tokio_postgres::Client` underneath the `Object` (`object.deref_mut().deref_mut()`, as its README shows).

What it does not do is lock. Its tokio-postgres driver opens a transaction per migration and nothing else (`refinery_core/src/drivers/tokio_postgres.rs`), so with several instances starting together, take an advisory lock around `run_async` yourself, exactly as §14.1 does, or migrate in a separate deployment step. This book writes its own runner because it is short and teaches both of those lessons; refinery is a sound choice when you would rather not own one.

14.4 The repository layer

stockroom keeps every SQL statement in `repo/`, one module per aggregate (`stores`, `products`, `stock`, `orders`, `reports`, `outbox`), as plain async functions. Handlers never contain SQL; they call repository functions. Three conventions make that work.

Functions take any client. A function that only reads or writes should not care whether it runs on a pooled connection or inside a transaction someone else opened. tokio-postgres's `GenericClient` (Chapter 8) expresses "a `Client` or a `Transaction`"; deadpool-postgres has a `GenericClient` of its own, implemented by `Object` and deadpool's `Transaction`, which adds `prepare_cached`. Both are sealed and `#[async_trait]` traits: you can write functions generic over them, `&impl GenericClient`, but not implement them for your own types, nor use them as `dyn GenericClient`.

Statements are prepared once per connection. `prepare_cached` (Chapter 13) means each function's SQL is parsed and planned once per pooled connection, and then costs no extra round trip.

Rows become types in one place. Each type implements `TryFrom<&Row>`, reading columns by name, so a function's last line converts, and a column added to the `SELECT` but not to the type is noticed in one place.

```
use deadpool_postgres::{GenericClient, Manager, Pool, Runtime};
use tokio_postgres::{Config, Error, NoTls, Row};

#[derive(Debug, PartialEq)]
struct Product {
    sku: String,
    price_cents: i64,
}

impl TryFrom<&Row> for Product {
    type Error = Error;

    fn try_from(row: &Row) -> Result<Self, Self::Error> {
        Ok(Product {
            sku: row.try_get("sku")?,
            price_cents: row.try_get("price_cents")?,
        })
    }
}

// The repository: SQL in one place, for any client.

async fn insert(db: &impl GenericClient, sku: &str, price_cents: i64) -> Result<Product,
Error> {
    let stmt = db
        .prepare_cached(
            "INSERT INTO product (sku, price_cents) VALUES ($1, $2)
             RETURNING sku, price_cents",
        )
        .await?;
    Product::try_from(&db.query_one(&stmt, &[&sku, &price_cents]).await?)
}

async fn by_sku(db: &impl GenericClient, sku: &str) -> Result<Option<Product>, Error> {
    let stmt = db
        .prepare_cached("SELECT sku, price_cents FROM product WHERE sku = $1")
        .await?;
    db.query_opt(&stmt, &[&sku])
        .await?
        .as_ref()
        .map(Product::try_from)
```

```
        .transpose()
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let config: Config = std::env::var("DATABASE_URL")?.parse()?;
    // One connection, so that the temporary table is visible to every borrower.
    let pool = Pool::builder(Manager::new(config, NoTls))
        .max_size(1)
        .runtime(Runtime::Tokio1)
        .build()?;
    let mut conn = pool.get().await?;
    conn.batch_execute("CREATE TEMP TABLE product (sku text PRIMARY KEY, price_cents
bigint)")
        .await?;

    // On the pooled connection...
    insert(&conn, "CHAIR-01", 4999).await?;

    // ...and inside a transaction, with the same functions.
    let tx = conn.transaction().await?;
    insert(&tx, "TABLE-01", 12900).await?;
    assert!(by_sku(&tx, "TABLE-01").await?.is_some());
    drop(tx); // rolled back

    assert_eq!(
        by_sku(&conn, "CHAIR-01").await?,
        Some(Product {
            sku: "CHAIR-01".into(),
            price_cents: 4999
        })
    );
    assert_eq!(by_sku(&conn, "TABLE-01").await?, None);
    Ok(())
}
```

stockroom's functions have the same shape, with `AppError` as the error type and a `tracing` span that names the function but never the parameter values:

```
// stockroom/src/repo/stores.rs (excerpt)
#[tracing::instrument(level = "debug", skip_all)]
pub async fn list(db: &impl GenericClient) -> Result<Vec<Store>, AppError> {
    let stmt = db
        .prepare_cached("SELECT id, code, name, address, created_at FROM stores ORDER BY
code")
        .await?;
    from_rows(&db.query(&stmt, &[]).await?)
}

#[tracing::instrument(level = "debug", skip_all, fields(code))]
pub async fn id_by_code(db: &impl GenericClient, code: &str) -> Result<Uuid, AppError> {
    let stmt = db
        .prepare_cached("SELECT id FROM stores WHERE code = $1")
        .await?;
    match db.query_opt(&stmt, &[&code]).await? {
        Some(row) => Ok(row.try_get(0)?),
        None => Err(AppError::NotFound("store")),
    }
}
```

stockroom/src/repo/stores.rs · stockroom/src/repo/mod.rs

`from_rows`, in `repo/mod.rs`, is the one generic helper: `rows.iter().map(T::try_from)`, collected, for any `T` that implements `TryFrom<&Row>`.

Functions that only make sense inside a transaction say so in their signature: `orders::place` takes `&Transaction<'_>`, because placing an order is several statements that must commit together, and the compiler then refuses to call it on a bare connection. Chapter 8's `in_transaction` retry helper is how handlers call those.

> **Pitfall:** a repository function that takes `&impl GenericClient` and runs two statements that must succeed together is a bug waiting for a caller who passes a pooled connection. Make it take a `&Transaction`, or open one itself.

14.5 Domain types at the boundary

A value that has been checked should not need checking again. stockroom parses input into domain types at the edge, in the HTTP layer, and from then on functions take `Sku`, not `&str`, and `Cents`, not `i64`. The database enforces the same rules once more, as a constraint, for everything that does not go through this code:

Type	Rust	Database	Conversion (Chapter 5)
`Sku`	`Sku::parse` checks the format	the domain `sku`, with the `CHECK` constraint `sku_format`	hand-written `ToSql`/`FromSql` that accept the domain and its base type
`Cents`	an `i64` newtype	`bigint` with `CHECK (price_cents > 0)`	`#[postgres(transparent)]`
`Quantity`	positive `i32`	the domain `positive_quantity`	derived, for a domain
`OrderStatus`	an enum	the enum type `order_status`	derived, with `allow_mismatch`
`Address`	a struct	the composite type `address`	derived

```rust
// stockroom/src/domain/sku.rs (excerpt)
/// 3–12 upper-case letters or digits, optionally followed by a dash and 1–6 more:
`CHAIR-01`.
/// The same rule as the `sku_format` constraint on the domain.
#[derive(Debug, Clone, PartialEq, Eq, PartialOrd, Ord, Hash, Serialize, Deserialize)]
#[serde(try_from = "String", into = "String")]
pub struct Sku(String);
```

stockroom/src/domain/sku.rs

`#[serde(try_from = "String")]` makes JSON deserialization go through `Sku::parse`, so a request body with a malformed SKU is rejected by the extractor with a 422 before any handler runs (Chapter 15). And the private field means the only ways to get a `Sku` are parsing and reading one from the database.

Why repeat the rule in the database? Because not every write goes through this code: a psql session, a script, the next service. The constraint's name, `sku_format`, is also part of the API: `AppError` maps a violation of it to a 422 with a message for the client (Chapter 15), which is why constraint names in stockroom are chosen with care and listed in `error::constraint_message`.

14.6 Generating the data layer: Cornucopia

Hand-written repository functions are a few lines each, and stockroom has a few dozen. A larger code base may prefer to generate them. Cornucopia (1.0.1, checked 2026-11-06) generates Rust from annotated SQL files:

```
-- queries/products.sql

--! product_by_sku
SELECT id, sku, name, price_cents FROM products WHERE sku = :sku;

--! insert_product
INSERT INTO products (sku, name, price_cents) VALUES (:sku, :name, :price_cents);
```

It prepares every query against a real database, which it either connects to (`cornucopia live <url>`), creates for the occasion on an existing server (`cornucopia fresh`), or starts in a container from your schema files (`cornucopia schema`). From the parameter and column types the server reports (the same `Statement::params()` and `columns()` of Chapter 4), it generates a crate with one typed function per query and one struct per row shape, over tokio-postgres (asynchronous by default, `--sync` for the `postgres` crate):

```
// Cornucopia (illustration): the generated crate in use
let product = queries::products::product_by_sku()
    .bind(&client, &"CHAIR-01")
    .one()
    .await?;
```

The trade is the one sqlx's macros make, done ahead of time: every query is checked against a real schema before the program is built, at the cost of a generation step, and of a database at generation time. Version 1.0 merged the Clorinde fork back into Cornucopia; projects that used Clorinde should follow Cornucopia's migration guide.

In stockroom

- `migrations/0001_init.sql` and `0002_order_events.sql`: the schema, with named constraints.
- `db/migrate.rs`: §14.1's runner with the details of §14.2. `stockroom migrate` runs it, and so does `serve` at startup by default; `tests/migrations.rs` checks that a migrated database has nothing left to do, that an edited migration is refused, and that a failed migration leaves no trace.
- `repo/`: one module per aggregate, functions over deadpool's `GenericClient`, statements through `prepare_cached`, rows through `TryFrom<&Row>`.
- `domain/`: the types of §14.5, each with its tests in `tests/types.rs`.

Summary

- Embed numbered migrations with `include_str!`, record them with a checksum, and refuse edited ones. Never edit an applied migration: add a new one.

- DDL is transactional in PostgreSQL: one transaction per migration, recorded in the same transaction, means a failure leaves nothing behind. `CREATE INDEX CONCURRENTLY` is the exception and must run alone, outside any transaction, even an implicit one.
- A session-level `pg_advisory_lock` lets several instances migrate at startup safely. Lift statement and lock timeouts for the migration session, clear the type cache after it, and log its notices.
- refinery (0.10) embeds `V1__name.sql` files and runs them with `run_async`; it does not lock, so add the advisory lock yourself.
- Repository functions take `&impl GenericClient` (deadpool's has `prepare_cached`), keep all SQL in one module per aggregate, and convert rows with `TryFrom<&Row>`. Functions that need a transaction take `&Transaction`.
- Parse input into domain types at the edge, and repeat the rules as named database constraints.
- Cornucopia generates typed query functions from annotated SQL, checked against a real database at generation time.

Exercises

1. **A concurrent index.** Extend §14.1's runner so that a migration can be marked `transactional: false`, and runs such migrations statement by statement outside a transaction. What should the runner record if the second of three statements fails? *Hint:* the `schema_migrations` row is the last thing to write, and only on success.
2. **Down migrations.** stockroom has none. Argue for or against adding them, considering a migration that drops a column. *Hint:* what would the down migration put back?
3. **One more repository function.** Add `by_tag(db, tag, limit)` in §14.4's style that returns the SKUs of the products carrying `tag`, and call it both on a pooled connection and inside a transaction. *Hint:* write the condition as `tags @> ARRAY[$1]`, which a GIN index on `tags` can serve; `$1 = ANY(tags)` means the same but cannot use one.
4. **Where does the check fail?** Insert a product with the SKU `chair` (lower case) three ways: through `Sku::parse`, with a `&str` and `$1::text`, and from psql. Where is each one stopped, and with what error? *Hint:* Chapter 5's finding about parameters that target a domain column.

Solutions

1. The runner cannot make such a migration atomic, so it should make it *restartable*: write each statement so that it can run again (`CREATE INDEX CONCURRENTLY IF NOT EXISTS`, `DROP INDEX CONCURRENTLY IF EXISTS`), run them in order, and insert the `schema_migrations` row only after the last one succeeds. If the second statement fails, nothing is recorded; the next run starts the migration from the top, and the first statement does nothing the second time. A failed `CREATE INDEX CONCURRENTLY` leaves an *invalid* index behind, which `IF NOT EXISTS` would then skip, so the migration should drop invalid indexes first (`pg_index.indisvalid` tells).

2. Against, for most services. A down migration for "drop column" would have to recreate the column, but not its data, so it does not undo the change; it only makes the schema look as if it had. Rolling

back a deployment is safer when migrations are additive and the previous version of the code can run on the new schema (§14.2), so that no schema change needs undoing. Down migrations are useful in development, to iterate on a migration that has not been shared.

3.

```
use deadpool_postgres::{GenericClient, Manager, Pool, Runtime};
use tokio_postgres::{Config, Error, NoTls};

async fn by_tag(db: &impl GenericClient, tag: &str, limit: i64) -> Result<Vec<String>,
Error> {
    let stmt = db
        .prepare_cached("SELECT sku FROM product WHERE tags @> ARRAY[$1] ORDER BY sku LIMIT
$2")
        .await?;
    Ok(db
        .query(&stmt, &[&tag, &limit])
        .await?
        .iter()
        .map(|row| row.get(0))
        .collect())
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let config: Config = std::env::var("DATABASE_URL")?.parse()?;
    let pool = Pool::builder(Manager::new(config, NoTls))
        .max_size(1)
        .runtime(Runtime::Tokio1)
        .build()?;
    let mut conn = pool.get().await?;
    conn.batch_execute(
        "CREATE TEMP TABLE product (sku text PRIMARY KEY, tags text[] NOT NULL);
         INSERT INTO product VALUES
             ('CHAIR-01', '{seating,oak}'), ('STOOL-01', '{seating}'), ('TABLE-01',
'{oak}');",
    )
    .await?;

    assert_eq!(
        by_tag(&conn, "seating", 10).await?,
        ["CHAIR-01", "STOOL-01"]
    );
    let tx = conn.transaction().await?;
    assert_eq!(by_tag(&tx, "oak", 1).await?, ["CHAIR-01"]);
    tx.commit().await?;
    Ok(())
}
```

4. `Sku::parse("chair")` fails in Rust, before any SQL: the request never reaches the database. A `&str` passed for a parameter that targets the `sku` column fails on the *client*, because tokio-postgres describes that parameter as the domain and `&str` does not accept it (Chapter 5); with `$1::text`, the cast moves the check to the server, where the domain's `sku_format` constraint fails with 23514 (`check_violation`). From psql, the same constraint fails with the same error. The constraint is the only check that every path meets.

CHAPTER 15

axum I: a database-backed JSON API

Everything so far comes together in an HTTP service. A request arrives, borrows a connection from the pool, runs a repository function or a transaction, and turns the result, or the error, into a JSON response with the right status code. Most of the work is in the seams: how a handler gets its connection, how a unique violation becomes a 409 and a check violation a 422, how a transaction that hits a deadlock is retried without the client noticing, how a list is paged, and how two clients editing the same product are kept from overwriting each other.

This chapter builds those seams with axum 0.8, one at a time, each as a small runnable service. None of the examples binds a port: they send requests to the router in memory, which is also how Chapter 17 tests stockroom. Chapter 16 adds the long-lived responses: streams, server-sent events and background work.

15.1 The shape of the app

An axum service is a `Router`: routes, each with a handler, plus *state* that every handler can reach, plus layers (middleware) around them all. For a database service the state is, at least, the pool:

```
use axum::extract::State;
use axum::http::{Request, StatusCode};
use axum::routing::get;
use axum::{Json, Router};
use deadpool_postgres::Pool;
use serde_json::{Value, json};

/// What every handler can reach. It is cloned for each request, so every field is a cheap
/// handle: a `Pool` is an `Arc` inside.
#[derive(Clone)]
struct AppState {
    pool: Pool,
}

async fn ready(State(state): State<AppState>) -> Result<Json<Value>, StatusCode> {
    let db = state
        .pool
        .get()
        .await
        .map_err(|_| StatusCode::SERVICE_UNAVAILABLE)?;
    db.check_connection()
        .await
        .map_err(|_| StatusCode::SERVICE_UNAVAILABLE)?;
    let pool = state.pool.status();
    Ok(Json(
        json!({ "status": "ready", "size": pool.size, "available": pool.available }),
    ))
}
```

```
fn router(state: AppState) -> Router {
    Router::new().route("/readyz", get(ready)).with_state(state)
}

#[tokio::main]
async fn main() {
    let app = router(AppState {
        pool: playground::pool(4),
    });
    let request = Request::get("/readyz")
        .body(axum::body::Body::empty())
        .expect("a valid request");
    let (status, _headers, body) = playground::call(&app, request).await;
    println!("{status} {body}");
    assert_eq!(status, StatusCode::OK);
    assert_eq!(body["status"], "ready");
}
```

```
200 OK {"available":0,"size":1,"status":"ready"}
```

(`available` is 0 because the handler itself holds the pool's only open connection while it reports.) Two helpers from the playground make these examples short. `pool(n)` builds a deadpool pool as Chapter 13 did:

```
use deadpool_postgres::{Manager, ManagerConfig, Pool, RecyclingMethod, Runtime};

/// A deadpool pool of at most `max_size` connections to `DATABASE_URL` (Chapter 13). Part
/// III's examples mostly use a pool of one, so that every request sees the same session,
and
/// with it the example's temporary tables.
pub fn pool(max_size: usize) -> Pool {
    let config: tokio_postgres::Config = std::env::var("DATABASE_URL")
        .expect("set DATABASE_URL, e.g. postgres://postgres:secret@localhost:5432/tpbook")
        .parse()
        .expect("DATABASE_URL is a valid connection string");
    let manager = Manager::from_config(
        config,
        NoTls,
        ManagerConfig {
            recycling_method: RecyclingMethod::Fast,
        },
    );
    Pool::builder(manager)
        .max_size(max_size)
        .runtime(Runtime::Tokio1)
        .wait_timeout(Some(std::time::Duration::from_secs(5)))
        .build()
        .expect("a valid pool configuration")
}
```

And `call` sends one request through a router with `tower::ServiceExt::oneshot`, which is all a `Router` needs to answer: no socket, no port, no server.

```
use axum::body::Body;
use axum::http::{HeaderMap, Request, StatusCode};
use http_body_util::BodyExt;
use tower::ServiceExt;

/// Sends one request through `app`, in memory, without binding a port, and returns the
/// response's status, headers and body: parsed as JSON, as a JSON string if it is plain
text,
/// or `Null` when it is empty.
pub async fn call(
    app: &axum::Router,
```

```
    request: Request<Body>,
) -> (StatusCode, HeaderMap, serde_json::Value) {
    let response = app
        .clone()
        .oneshot(request)
        .await
        .expect("a Router's error type is Infallible");
    let (parts, body) = response.into_parts();
    let bytes = body
        .collect()
        .await
        .expect("an in-memory body cannot fail")
        .to_bytes();
    let json = if bytes.is_empty() {
        serde_json::Value::Null
    } else {
        serde_json::from_slice(&bytes).unwrap_or_else(|_| {
            serde_json::Value::String(String::from_utf8_lossy(&bytes).into_owned())
        })
    };
    (parts.status, parts.headers, json)
}
```

In a real service the router is served on a socket with `axum::serve(listener, app)`, and the request goes through layers first: tracing, a timeout, a body limit. stockroom's `http::router` (§ In stockroom) shows the full set, and its `main` sets up `tracing` so that every request, and every repository call, logs its duration.

15.2 A `Db` extractor

Every handler that touches the database starts the same way: get a connection from the pool, and answer 503 if none is free. An *extractor* says it once. Implement `FromRequestParts`, and any handler can take the connection as an argument:

```
use axum::body::Body;
use axum::extract::{FromRequestParts, Path};
use axum::http::request::Parts;
use axum::http::{Request, StatusCode};
use axum::routing::get;
use axum::{Json, Router};
use deadpool_postgres::{Object, Pool};
use serde_json::{Value, json};

#[derive(Clone)]
struct AppState {
    pool: Pool,
}

/// A pooled connection, checked out for as long as the handler runs.
struct Db(Object);

impl FromRequestParts<AppState> for Db {
    type Rejection = (StatusCode, String);

    async fn from_request_parts(
        _parts: &mut Parts,
        state: &AppState,
    ) -> Result<Self, Self::Rejection> {
        let conn = state
            .pool
            .get()
            .await
            .map_err(|e| (StatusCode::SERVICE_UNAVAILABLE, e.to_string()))?;
```

```
        Ok(Db(conn))
    }
}

async fn product(Db(db): Db, Path(sku): Path<String>) -> Result<Json<Value>, (StatusCode,
String)> {
    let row = db
        .query_opt("SELECT sku, name FROM product WHERE sku = $1", &[&sku])
        .await
        .map_err(|e| (StatusCode::INTERNAL_SERVER_ERROR, e.to_string()))?
        .ok_or_else(|| (StatusCode::NOT_FOUND, format!("no product {sku}")))?;
    let (sku, name): (String, String) = (row.get(0), row.get(1));
    Ok(Json(json!({ "sku": sku, "name": name })))
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let pool = playground::pool(1);
    pool.get()
        .await?
        .batch_execute(
            "CREATE TEMP TABLE product (sku text PRIMARY KEY, name text NOT NULL);
             INSERT INTO product VALUES ('CHAIR-01', 'Oak chair');",
        )
        .await?;
    let app = Router::new()
        .route("/products/{sku}", get(product))
        .with_state(AppState { pool });

    let (status, _, body) = playground::call(
        &app,
        Request::get("/products/CHAIR-01").body(Body::empty())?,
    )
    .await;
    assert_eq!(
        (status, body["name"].as_str()),
        (StatusCode::OK, Some("Oak chair"))
    );

    let (status, _, body) =
        playground::call(&app, Request::get("/products/
NOPE-01").body(Body::empty())?).await;
    println!("{status} {body}");
    assert_eq!(status, StatusCode::NOT_FOUND);
    Ok(())
}
```

```
404 Not Found "no product NOPE-01"
```

The connection goes back to the pool when the handler's future ends, whatever the outcome. Two details deserve a sentence each:

- **Extractor order.** axum runs extractors in argument order, and only the last one may read the body (`Json`, `Body`). A handler that takes `Db` first holds a connection while its body arrives, which is fine for small JSON bodies and wasteful for uploads, as Chapter 16 notes.
- **One connection per request.** A handler that needs two connections at once, to run two transactions say, is almost always a design mistake, and with a pool of one it would deadlock against itself.

15.3 `AppError`: from SQLSTATE to HTTP status

Chapter 6 classified database errors by SQLSTATE. A service turns that classification into HTTP statuses, once, in one `From<tokio_postgres::Error>` implementation, so that handlers can use `?` everywhere and still answer correctly:

SQLSTATE	Meaning	Status
23505 `unique_violation`	the thing already exists	409 Conflict
23514 `check_violation`, 23502 `not_null_violation`, 23503 `foreign_key_violation`	the request breaks a rule of the data	422 Unprocessable Entity
40001 `serialization_failure`, 40P01 `deadlock_detected`, 55P03 `lock_not_available`	concurrency, not the request; retry	503, with `Retry-After`
57014 `query_canceled`	the query hit a timeout	503
57P01 `admin_shutdown`, a closed connection, no connection from the pool	the database is unavailable	503
anything else	a bug, or something for an operator	500, details only in the log

Two more rules make the mapping useful to a client. The *constraint name* (`DbError::constraint`, Chapter 6) says which rule was broken, and so which request field to blame. And the server's own message never reaches the client: it can contain table names and the values of other rows (Chapter 6), so it goes to the log, and the client gets a stable code and a message written for it.

```
use axum::body::Body;
use axum::extract::{FromRequestParts, Path};
use axum::http::request::Parts;
use axum::http::{HeaderValue, Request, StatusCode, header};
use axum::response::{IntoResponse, Response};
use axum::routing::{get, post};
use axum::{Json, Router};
use deadpool_postgres::{Object, Pool, PoolError};
use serde::Deserialize;
use serde_json::{Value, json};
use tokio_postgres::error::SqlState;

#[derive(Debug)]
enum AppError {
    NotFound(&'static str),
    Invalid {
        field: Option<&'static str>,
        message: &'static str,
    },
    Conflict {
        field: Option<&'static str>,
        message: &'static str,
    },
    Busy,
    Unavailable,
    Internal,
}

impl From<tokio_postgres::Error> for AppError {
    fn from(error: tokio_postgres::Error) -> Self {
        let Some(db) = error.as_db_error() else {
            return if error.is_closed() {
```

```
                AppError::Unavailable
            } else {
                AppError::Internal
            };
        };
        // The constraint's name says which rule, and so which field, the request broke.
        let field = match db.constraint() {
            Some("product_pkey") => Some("sku"),
            Some("product_price_positive") => Some("priceCents"),
            _ => None,
        };
        match *db.code() {
            SqlState::UNIQUE_VIOLATION => AppError::Conflict {
                field,
                message: "this already exists",
            },
            SqlState::CHECK_VIOLATION
            | SqlState::NOT_NULL_VIOLATION
            | SqlState::FOREIGN_KEY_VIOLATION => AppError::Invalid {
                field,
                message: "the request breaks a rule of the data",
            },
            SqlState::T_R_SERIALIZATION_FAILURE | SqlState::T_R_DEADLOCK_DETECTED =>
AppError::Busy,
            _ => {
                eprintln!("database error: {}", db.message()); // the log, not the client
                AppError::Internal
            }
        }
    }
}

impl From<PoolError> for AppError {
    fn from(_: PoolError) -> Self {
        AppError::Unavailable
    }
}

impl IntoResponse for AppError {
    fn into_response(self) -> Response {
        let (status, body) = match self {
            AppError::NotFound(what) => (
                StatusCode::NOT_FOUND,
                json!({ "error": "not_found", "message": format!("{what} not found") }),
            ),
            AppError::Invalid { field, message } => (
                StatusCode::UNPROCESSABLE_ENTITY,
                json!({ "error": "invalid", "field": field, "message": message }),
            ),
            AppError::Conflict { field, message } => (
                StatusCode::CONFLICT,
                json!({ "error": "conflict", "field": field, "message": message }),
            ),
            AppError::Busy | AppError::Unavailable => (
                StatusCode::SERVICE_UNAVAILABLE,
                json!({ "error": "unavailable", "message": "try again" }),
            ),
            AppError::Internal => (
                StatusCode::INTERNAL_SERVER_ERROR,
                json!({ "error": "internal", "message": "something went wrong on our
side" }),
            ),
        };
        let mut response = (status, Json(body)).into_response();
        if status == StatusCode::SERVICE_UNAVAILABLE {
            response
                .headers_mut()
                .insert(header::RETRY_AFTER, HeaderValue::from_static("1"));
        }
        response
```

```
    }
}

#[derive(Clone)]
struct AppState {
    pool: Pool,
}

struct Db(Object);

impl FromRequestParts<AppState> for Db {
    type Rejection = AppError;

    async fn from_request_parts(_: &mut Parts, state: &AppState) -> Result<Self, AppError> {
        Ok(Db(state.pool.get().await?))
    }
}

#[derive(Deserialize)]
#[serde(rename_all = "camelCase")]
struct NewProduct {
    sku: String,
    name: String,
    price_cents: i64,
}

async fn create(Db(db): Db, Json(new): Json<NewProduct>) -> Result<StatusCode, AppError> {
    let stmt = db
        .prepare_cached("INSERT INTO product (sku, name, price_cents) VALUES ($1, $2, $3)")
        .await?;
    db.execute(&stmt, &[&new.sku, &new.name, &new.price_cents])
        .await?;
    Ok(StatusCode::CREATED)
}

async fn get_one(Db(db): Db, Path(sku): Path<String>) -> Result<Json<Value>, AppError> {
    let row = db
        .query_opt(
            "SELECT name, price_cents FROM product WHERE sku = $1",
            &[&sku],
        )
        .await?
        .ok_or(AppError::NotFound("product"))?;
    let (name, price): (String, i64) = (row.get(0), row.get(1));
    Ok(Json(
        json!({ "sku": sku, "name": name, "priceCents": price }),
    ))
}

fn post_json(uri: &str, body: Value) -> Request<Body> {
    Request::post(uri)
        .header(header::CONTENT_TYPE, "application/json")
        .body(Body::from(body.to_string()))
        .expect("a valid request")
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let pool = playground::pool(1);
    pool.get()
        .await?
        .batch_execute(
            "CREATE TEMP TABLE product (
                 sku         text   PRIMARY KEY,
                 name        text   NOT NULL,
                 price_cents bigint NOT NULL CONSTRAINT product_price_positive CHECK
(price_cents > 0)
             )",
        )
        .await?;
```

```
    let app = Router::new()
        .route("/products", post(create))
        .route("/products/{sku}", get(get_one))
        .with_state(AppState { pool });

    let chair = json!({ "sku": "CHAIR-01", "name": "Oak chair", "priceCents": 4999 });
    let (status, _, _) = playground::call(&app, post_json("/products",
chair.clone())).await;
    assert_eq!(status, StatusCode::CREATED);

    let (status, _, body) = playground::call(&app, post_json("/products", chair)).await;
    println!("{status} {body}");
    assert_eq!(
        (status, body["field"].as_str()),
        (StatusCode::CONFLICT, Some("sku"))
    );

    let free = json!({ "sku": "LAMP-01", "name": "Lamp", "priceCents": -1 });
    let (status, _, body) = playground::call(&app, post_json("/products", free)).await;
    println!("{status} {body}");
    assert_eq!(
        (status, body["field"].as_str()),
        (StatusCode::UNPROCESSABLE_ENTITY, Some("priceCents"))
    );

    let missing = Request::get("/products/NOPE-01").body(Body::empty())?;
    let (status, _, body) = playground::call(&app, missing).await;
    println!("{status} {body}");
    assert_eq!(status, StatusCode::NOT_FOUND);
    Ok(())
}
```

```
409 Conflict {"error":"conflict","field":"sku","message":"this already exists"}
422 Unprocessable Entity {"error":"invalid","field":"priceCents","message":"the request
breaks a rule of the data"}
404 Not Found {"error":"not_found","message":"product not found"}
```

Note what the handlers do *not* do. `create` does not check whether the SKU exists before inserting it: that check would race with a concurrent insert, and the unique constraint is the only check that cannot. It does not validate the price either: the `CHECK` constraint does, and its name tells `AppError` which field to name. The database already enforces these rules, and the error mapping turns its verdicts into answers.

> **Pitfall:** mapping *every* database error to 400 or 422 blames the client for your bugs. A syntax error in your SQL (42601), a missing column (42703) or a wrong parameter type is a 500. Classify the codes you mean, and let everything else be internal.

15.4 Writes in transactions

A handler that writes several rows opens a transaction on its connection, and Chapter 8's rules apply unchanged: `?` anywhere drops the `Transaction`, which rolls it back, so an error halfway through leaves nothing behind. `Db(mut db)` because `transaction()` takes `&mut self`:

```
use axum::body::Body;
use axum::extract::State;
use axum::http::{Request, StatusCode, header};
```

```
use axum::routing::post;
use axum::{Json, Router};
use deadpool_postgres::Pool;
use serde::Deserialize;
use serde_json::json;
use tokio_postgres::error::SqlState;

#[derive(Deserialize)]
struct Transfer {
    from: String,
    to: String,
    amount: i64,
}

/// A database error as a response: a broken rule is the client's 422, the rest our 500.
fn db_error(error: tokio_postgres::Error) -> (StatusCode, String) {
    match error.code() {
        Some(&SqlState::CHECK_VIOLATION) => (
            StatusCode::UNPROCESSABLE_ENTITY,
            "insufficient funds".to_owned(),
        ),
        _ => (
            StatusCode::INTERNAL_SERVER_ERROR,
            "internal error".to_owned(),
        ),
    }
}

async fn transfer(
    State(pool): State<Pool>,
    Json(t): Json<Transfer>,
) -> Result<StatusCode, (StatusCode, String)> {
    let mut db = pool
        .get()
        .await
        .map_err(|_| (StatusCode::SERVICE_UNAVAILABLE, "no connection".to_owned()))?;
    let tx = db.transaction().await.map_err(db_error)?;
    // Credit first, on purpose: when the debit fails, the credit must be undone.
    let credited = tx
        .execute(
            "UPDATE account SET balance = balance + $2 WHERE id = $1",
            &[&t.to, &t.amount],
        )
        .await
        .map_err(db_error)?;
    let debited = tx
        .execute(
            "UPDATE account SET balance = balance - $2 WHERE id = $1",
            &[&t.from, &t.amount],
        )
        .await
        .map_err(db_error)?; // a CHECK violation returns here; `tx` is dropped and rolls
back
    if credited != 1 || debited != 1 {
        return Err((StatusCode::NOT_FOUND, "no such account".to_owned())); // rolls back too
    }
    tx.commit().await.map_err(db_error)?;
    Ok(StatusCode::NO_CONTENT)
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let pool = playground::pool(1);
    pool.get()
        .await?
        .batch_execute(
            "CREATE TEMP TABLE account (
                 id      text    PRIMARY KEY,
                 balance bigint NOT NULL CHECK (balance >= 0)
             );
```

```
                INSERT INTO account VALUES ('alice', 100), ('bob', 0);",
            )
            .await?;
        let app = Router::new()
            .route("/transfers", post(transfer))
            .with_state(pool.clone());
        let send = |amount: i64| {
            Request::post("/transfers")
                .header(header::CONTENT_TYPE, "application/json")
                .body(Body::from(
                    json!({ "from": "alice", "to": "bob", "amount": amount }).to_string(),
                ))
                .expect("a valid request")
        };

        let (status, _, _) = playground::call(&app, send(60)).await;
        assert_eq!(status, StatusCode::NO_CONTENT);
        let (status, _, body) = playground::call(&app, send(60)).await; // only 40 left
        println!("{status} {body}");
        assert_eq!(status, StatusCode::UNPROCESSABLE_ENTITY);

        // The failed transfer's credit to bob was rolled back.
        let balances: Vec<i64> = pool
            .get()
            .await?
            .query("SELECT balance FROM account ORDER BY id", &[])
            .await?
            .iter()
            .map(|row| row.get(0))
            .collect();
        assert_eq!(balances, [40, 60]);
        Ok(())
    }
```

```
422 Unprocessable Entity "insufficient funds"
```

This handler takes `State<Pool>` and gets the connection itself, which also works: the state can be the pool alone when nothing else is shared.

Transactions that can collide with others, at `REPEATABLE READ` or `SERIALIZABLE`, or that can deadlock, also need a retry. Chapter 8's `in_transaction` helper does it, and in a handler it is called with an `async move` closure that owns the request:

```
// stockroom/src/http/orders.rs (excerpt)
pub async fn place(
    Db(mut db): Db,
    AppJson(order): AppJson<NewOrder>,
) -> Result<impl IntoResponse, AppError> {
    // `async move`: the closure owns the request, and each attempt borrows it from there.
    let order = db::in_transaction(&mut db, IsolationLevel::ReadCommitted, async move |tx| {
        orders::place(tx, &order).await
    })
    .await?;
    Ok((StatusCode::CREATED, etag(order.version), Json(order)))
}
```

stockroom/src/http/orders.rs · stockroom/src/db/retry.rs

> **Rust refresher:** axum requires a handler's future to be `Send`, since the server may move it between threads. An async closure that captures a *reference* (to `order`, to any local, even a `&'static str`) makes the retry helper's future fail that check with "implementation of `Send` is not general enough", a current compiler limitation (Chapter 8). `async move`, with the closure owning `order`, compiles; each attempt then lends `&order` to `orders::place` from inside the closure.

`in_transaction` returns `AppError::Busy` when its retries run out, which `AppError` turns into 503 with `Retry-After: 1`: the client is told to try again, not that it did something wrong.

15.5 Keyset pagination with an opaque cursor

Chapter 9 explained why lists are paged by key, not by `OFFSET`. Over HTTP the key travels as an opaque cursor: the client gets `nextCursor` with each page and sends it back as `?cursor=` for the next, without knowing what is in it. The handler asks for one row more than the page, to know whether there is a next page without another query:

```
use axum::body::Body;
use axum::extract::{Query, State};
use axum::http::{Request, StatusCode};
use axum::routing::get;
use axum::{Json, Router};
use deadpool_postgres::Pool;
use serde::{Deserialize, Serialize};

#[derive(Deserialize)]
struct ListQuery {
    cursor: Option<String>,
    limit: Option<i64>,
}

#[derive(Serialize)]
#[serde(rename_all = "camelCase")]
struct Page {
    items: Vec<String>,
    #[serde(skip_serializing_if = "Option::is_none")]
    next_cursor: Option<String>,
}

/// The last SKU of a page, hex-encoded: opaque to clients, trivial for us.
fn encode_cursor(sku: &str) -> String {
    sku.bytes().map(|b| format!("{b:02x}")).collect()
}

fn decode_cursor(cursor: &str) -> Option<String> {
    let bytes: Option<Vec<u8>> = (0..cursor.len())
        .step_by(2)
        .map(|i| u8::from_str_radix(cursor.get(i..i + 2)?, 16).ok())
        .collect();
    String::from_utf8(bytes?).ok()
}

async fn list(
    State(pool): State<Pool>,
    Query(query): Query<ListQuery>,
) -> Result<Json<Page>, StatusCode> {
    let limit = query.limit.unwrap_or(50).clamp(1, 500);
    let after = match query.cursor.as_deref() {
        Some(cursor) =>
Some(decode_cursor(cursor).ok_or(StatusCode::UNPROCESSABLE_ENTITY)?),
```

```
        None => None,
    };
    let db = pool
        .get()
        .await
        .map_err(|_| StatusCode::SERVICE_UNAVAILABLE)?;
    let stmt = db
        .prepare_cached(
            "SELECT sku FROM product WHERE sku > coalesce($1, '') ORDER BY sku LIMIT $2",
        )
        .await
        .map_err(|_| StatusCode::INTERNAL_SERVER_ERROR)?;
    // One more than asked for: if it comes back, there is a next page.
    let mut items: Vec<String> = db
        .query(&stmt, &[&after, &(limit + 1)])
        .await
        .map_err(|_| StatusCode::INTERNAL_SERVER_ERROR)?
        .iter()
        .map(|row| row.get(0))
        .collect();
    let next_cursor = if items.len() as i64 > limit {
        items.truncate(limit as usize);
        items.last().map(|last| encode_cursor(last))
    } else {
        None
    };
    Ok(Json(Page { items, next_cursor }))
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let pool = playground::pool(1);
    pool.get()
        .await?
        .batch_execute(
            "CREATE TEMP TABLE product (sku text PRIMARY KEY);
             INSERT INTO product SELECT 'SKU-' || i FROM generate_series(1, 7) i;",
        )
        .await?;
    let app = Router::new().route("/products", get(list)).with_state(pool);

    let mut uri = "/products?limit=3".to_owned();
    let mut pages = Vec::new();
    loop {
        let (status, _, body) =
            playground::call(&app, Request::get(&uri).body(Body::empty())?).await;
        assert_eq!(status, StatusCode::OK);
        pages.push(body["items"].as_array().map_or(0, Vec::len));
        match body["nextCursor"].as_str() {
            Some(cursor) => uri = format!("/products?limit=3&cursor={cursor}"),
            None => break,
        }
    }
    assert_eq!(pages, [3, 3, 1]); // and no empty request to discover the end
    Ok(())
}
```

`sku` is unique, so it can be the whole key. For a sort column with duplicates, the cursor holds the column and a unique tie-breaker, as Chapter 9's Exercise 3 did. Hex is enough to make a cursor opaque; it is not a secret, and clients that decode it learn only a SKU they already saw. A cursor that encodes something private should be signed or encrypted.

15.6 ETag and `If-Match`: optimistic concurrency over HTTP

Chapter 8 kept two writers from overwriting each other with a `version` column: an update succeeds only if the version is the one the writer read. HTTP has a matching mechanism. The server sends the version with the resource as an *entity tag*, `ETag: "3"`, and a client that wants to update sends it back, `If-Match: "3"`. If the resource has changed meanwhile, the server answers 412 Precondition Failed instead of applying the change:

```
use axum::body::Body;
use axum::extract::{Path, State};
use axum::http::{HeaderMap, HeaderValue, Request, StatusCode, header};
use axum::response::{IntoResponse, Response};
use axum::routing::get;
use axum::{Json, Router};
use deadpool_postgres::Pool;
use serde::Deserialize;
use serde_json::{Value, json};

fn etag(version: i32) -> [(header::HeaderName, HeaderValue); 1] {
    let value = HeaderValue::from_str(&format!("\"{version}\"")).expect("digits and
quotes");
    [(header::ETAG, value)]
}

/// The version an `If-Match` header names; `None` without one.
fn if_match(headers: &HeaderMap) -> Result<Option<i32>, StatusCode> {
    let Some(value) = headers.get(header::IF_MATCH) else {
        return Ok(None);
    };
    let text = value.to_str().map_err(|_| StatusCode::BAD_REQUEST)?;
    text.trim()
        .trim_matches('"')
        .parse()
        .map(Some)
        .map_err(|_| StatusCode::BAD_REQUEST)
}

#[derive(Deserialize)]
#[serde(rename_all = "camelCase")]
struct Patch {
    price_cents: i64,
}

type ApiError = (StatusCode, Json<Value>);

fn api_error(status: StatusCode, code: &str) -> ApiError {
    (status, Json(json!({ "error": code })))
}

async fn get_one(State(pool): State<Pool>, Path(sku): Path<String>) -> Result<Response,
ApiError> {
    let unavailable = |_| api_error(StatusCode::SERVICE_UNAVAILABLE, "unavailable");
    let internal = |_| api_error(StatusCode::INTERNAL_SERVER_ERROR, "internal");
    let db = pool.get().await.map_err(unavailable)?;
    let row = db
        .query_opt(
            "SELECT price_cents, version FROM product WHERE sku = $1",
            &[&sku],
        )
        .await
        .map_err(internal)?
        .ok_or_else(|| api_error(StatusCode::NOT_FOUND, "not_found"))?;
    let (price, version): (i64, i32) = (row.get(0), row.get(1));
    Ok((etag(version), Json(json!({ "priceCents": price }))).into_response())
}

async fn update(
```

```
    State(pool): State<Pool>,
    Path(sku): Path<String>,
    headers: HeaderMap,
    Json(patch): Json<Patch>,
) -> Result<Response, ApiError> {
    let expected = if_match(&headers).map_err(|status| api_error(status, "bad_if_match"))?;
    let db = pool
        .get()
        .await
        .map_err(|_| api_error(StatusCode::SERVICE_UNAVAILABLE, "unavailable"))?;
    let internal = |_| api_error(StatusCode::INTERNAL_SERVER_ERROR, "internal");
    // Applied only if the version is still the expected one (or no version was given).
    let updated = db
        .query_opt(
            "UPDATE product SET price_cents = $3, version = version + 1
             WHERE sku = $1 AND version = coalesce($2, version)
             RETURNING version",
            &[&sku, &expected, &patch.price_cents],
        )
        .await
        .map_err(internal)?;
    if let Some(row) = updated {
        return Ok((etag(row.get(0)), StatusCode::OK).into_response());
    }
    // Nothing updated: no such product, or someone else changed it first.
    let current = db
        .query_opt("SELECT version FROM product WHERE sku = $1", &[&sku])
        .await
        .map_err(internal)?;
    Err(match current {
        None => api_error(StatusCode::NOT_FOUND, "not_found"),
        Some(row) => (
            StatusCode::PRECONDITION_FAILED,
            Json(json!({ "error": "stale_version", "currentVersion": row.get::<_,
i32>(0) })),
        ),
    })
}

fn patch(price: i64, if_match: Option<&str>) -> Request<Body> {
    let mut request =
        Request::patch("/products/CHAIR-01").header(header::CONTENT_TYPE, "application/
json");
    if let Some(tag) = if_match {
        request = request.header(header::IF_MATCH, tag);
    }
    request
        .body(Body::from(json!({ "priceCents": price }).to_string()))
        .expect("a valid request")
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let pool = playground::pool(1);
    pool.get()
        .await?
        .batch_execute(
            "CREATE TEMP TABLE product (
                 sku text PRIMARY KEY, price_cents bigint NOT NULL, version int NOT NULL
DEFAULT 1
             );
             INSERT INTO product VALUES ('CHAIR-01', 4999);",
        )
        .await?;
    let app = Router::new()
        .route("/products/{sku}", get(get_one).patch(update))
        .with_state(pool);

    // Two clients read the product, and both see version 1.
    let (_, headers, _) = playground::call(
```

```
        &app,
        Request::get("/products/CHAIR-01").body(Body::empty())?,
    )
    .await;
    assert_eq!(headers[header::ETAG], "\"1\"");

    // The first one's change applies, and the version moves on.
    let (status, headers, _) = playground::call(&app, patch(4500, Some("\"1\""))).await;
    assert_eq!(
        (status, &headers[header::ETAG]),
        (StatusCode::OK, &HeaderValue::from_static("\"2\""))
    );

    // The second one's change was based on version 1: refused, with the current version.
    let (status, _, body) = playground::call(&app, patch(5500, Some("\"1\""))).await;
    println!("{status} {body}");
    assert_eq!(status, StatusCode::PRECONDITION_FAILED);
    assert_eq!(body["currentVersion"], 2);

    // Without If-Match, the last writer wins: allowed here, though an API may require it.
    let (status, _, _) = playground::call(&app, patch(5500, None)).await;
    assert_eq!(status, StatusCode::OK);
    Ok(())
}
```

```
412 Precondition Failed {"currentVersion":2,"error":"stale_version"}
```

The whole check is the `WHERE version = coalesce($2, version)` of one `UPDATE`: no lock is held between the client's read and its write, which may be minutes apart, and nothing is read before writing. A client that gets 412 re-reads the resource, shows the user what changed, and tries again with the new tag.

15.7 Validation at the edge

Three layers check a request, and each catches what the others cannot:

1. **Deserialization** turns the body into a typed struct, and rejects what does not fit: a missing field, a string where a number belongs, a SKU that `Sku::parse` refuses (stockroom's `#[serde(try_from = "String")]`, Chapter 14).
2. **The handler** checks rules that need several fields, or that the database cannot express cheaply: "a batch has 1–100 products", "the period ends before it starts".
3. **The database** enforces the rest, as constraints (§15.3).

axum's `Json` extractor rejects a bad body by itself, but with a plain-text body and its own status codes. To keep one error format for the whole API, wrap it. Implementing `FromRequest` by hand shows what the wrapper does:

```
use axum::body::Body;
use axum::extract::{FromRequest, Request};
use axum::http::{StatusCode, header};
use axum::response::{IntoResponse, Response};
use axum::routing::post;
use axum::{Json, Router};
use serde::Deserialize;
use serde::de::DeserializeOwned;
use serde_json::json;

/// `Json`, but its rejections use the API's error format.
struct AppJson<T>(T);
```

```
impl<S, T> FromRequest<S> for AppJson<T>
where
    S: Send + Sync,
    T: DeserializeOwned,
{
    type Rejection = Response;

    async fn from_request(request: Request, state: &S) -> Result<Self, Response> {
        match Json::<T>::from_request(request, state).await {
            Ok(Json(value)) => Ok(AppJson(value)),
            Err(rejection) => Err((
                rejection.status(),
                Json(json!({ "error": "invalid", "message": rejection.body_text() })),
            )
                .into_response()),
        }
    }
}

/// A SKU, checked when it is made: deserializing one runs `parse`.
#[derive(Debug, Deserialize)]
#[serde(try_from = "String")]
struct Sku(String);

impl TryFrom<String> for Sku {
    type Error = String;
    fn try_from(text: String) -> Result<Self, String> {
        let ok = (3..=12).contains(&text.len())
            && text
                .bytes()
                .all(|b| b.is_ascii_uppercase() || b.is_ascii_digit() || b == b'-');
        if ok {
            Ok(Sku(text))
        } else {
            Err(format!("{text:?} is not a SKU"))
        }
    }
}

#[derive(Debug, Deserialize)]
#[serde(rename_all = "camelCase", deny_unknown_fields)]
struct NewProduct {
    sku: Sku,
    name: String,
}

async fn create(AppJson(new): AppJson<NewProduct>) -> impl IntoResponse {
    // Rules that need the whole request belong here, after the types are known to be right.
    if new.name.trim().is_empty() {
        return (
            StatusCode::UNPROCESSABLE_ENTITY,
            Json(json!({ "error": "invalid", "field": "name", "message": "a name is
required" })),
        );
    }
    (StatusCode::CREATED, Json(json!({ "sku": new.sku.0 })))
}

#[tokio::main]
async fn main() {
    let app = Router::new().route("/products", post(create));
    let post = |body: &str, content_type: &str| {
        axum::http::Request::post("/products")
            .header(header::CONTENT_TYPE, content_type)
            .body(Body::from(body.to_owned()))
            .expect("a valid request")
    };
    let json = "application/json";
```

```
    for (body, content_type) in [
        (r#"{"sku": "chair", "name": "Oak chair"}"#, json),
        (
            r#"{"sku": "CHAIR-01", "name": "Oak chair", "colour": "red"}"#,
            json,
        ),
        (r#"{"sku": "CHAIR-01""#, json),
        (r#"{"sku": "CHAIR-01", "name": "Oak chair"}"#, "text/plain"),
        (r#"{"sku": "CHAIR-01", "name": " "}"#, json),
        (r#"{"sku": "CHAIR-01", "name": "Oak chair"}"#, json),
    ] {
        let (status, _, body) = playground::call(&app, post(body, content_type)).await;
        println!(
            "{} {}",
            status.as_u16(),
            body["message"].as_str().unwrap_or("-")
        );
    }
}
```

```
422 Failed to deserialize the JSON body into the target type: sku: "chair" is not a SKU at
line 1 column 15
422 Failed to deserialize the JSON body into the target type: colour: unknown field
`colour`, expected `sku` or `name` at line 1 column 49
400 Failed to parse the request body as JSON: EOF while parsing an object at line 1 column
18
415 Expected request with `Content-Type: application/json`
422 a name is required
201 -
```

axum's statuses are worth keeping: 400 for a body that is not JSON, 415 for the wrong content type, 422 for JSON of the wrong shape. stockroom maps all three to 422 with the same JSON format, through a one-line derive (`#[from_request(via(axum::Json), rejection(AppError))]`, with axum's `macros` feature), and `QueryRejection` the same way for query strings.

15.8 The pipelined dashboard

`GET /reports/dashboard` runs five independent aggregates. On one pooled connection they are pipelined with `try_join!` (Chapter 7), so the handler costs about one round trip, not five:

```
// stockroom/src/http/reports.rs (excerpt)
pub async fn dashboard(Db(db): Db) -> Result<Json<Dashboard>, AppError> {
    // `&db` is a `&Object`; deref coercion turns it into the `&tokio_postgres::Client` the
    // report needs for `query_one_scalar`.
    Ok(Json(reports::dashboard(&db).await?))
}
```

stockroom/src/http/reports.rs · stockroom/src/repo/reports.rs

The comment points at a small trap. `query_one_scalar` exists only on `tokio_postgres::Client` (added in 0.7.16, Chapter 2), not on `Transaction` or either `GenericClient`, so `reports::dashboard` takes `&Client`, and the handler passes `&db`: an `&Object` that derefs, twice, to `&Client`. A function that must also run inside a transaction would use `query_one` and `get(0)` instead.

Side by side — *an axum handler*

The task: `GET /products/{sku}` returns the product as JSON, or 404.

```
// sqlx (illustration)
#[derive(sqlx::FromRow, Serialize)]
struct Product {
    sku: String,
    name: String,
}

async fn product(
    State(pool): State<PgPool>,
    Path(sku): Path<String>,
) -> Result<Json<Product>, StatusCode> {
    sqlx::query_as::<_, Product>("SELECT sku, name FROM product WHERE sku = $1")
        .bind(&sku)
        .fetch_optional(&pool)
        .await
        .map_err(|_| StatusCode::INTERNAL_SERVER_ERROR)?
        .map(Json)
        .ok_or(StatusCode::NOT_FOUND)
}
```

```
use axum::body::Body;
use axum::extract::{Path, State};
use axum::http::{Request, StatusCode};
use axum::routing::get;
use axum::{Json, Router};
use deadpool_postgres::Pool;
use serde::Serialize;

#[derive(Serialize)]
struct Product {
    sku: String,
    name: String,
}

async fn product(
    State(pool): State<Pool>,
    Path(sku): Path<String>,
) -> Result<Json<Product>, StatusCode> {
    let db = pool
        .get()
        .await
        .map_err(|_| StatusCode::SERVICE_UNAVAILABLE)?;
    let row = db
        .query_opt("SELECT sku, name FROM product WHERE sku = $1", &[&sku])
        .await
        .map_err(|_| StatusCode::INTERNAL_SERVER_ERROR)?
        .ok_or(StatusCode::NOT_FOUND)?;
    Ok(Json(Product {
        sku: row.get("sku"),
        name: row.get("name"),
    }))
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let pool = playground::pool(1);
    pool.get()
        .await?
        .batch_execute(
            "CREATE TEMP TABLE product (sku text PRIMARY KEY, name text NOT NULL);
             INSERT INTO product VALUES ('CHAIR-01', 'Oak chair');",
        )
        .await?;
    let app = Router::new()
        .route("/products/{sku}", get(product))
        .with_state(pool);
    let (status, _, body) = playground::call(
        &app,
        Request::get("/products/CHAIR-01").body(Body::empty())?,
```

```
    )
    .await;
    assert_eq!(
        (status, body["name"].as_str()),
        (StatusCode::OK, Some("Oak chair"))
    );
    let (status, _, _) =
        playground::call(&app, Request::get("/products/
NOPE-01").body(Body::empty())?).await;
    assert_eq!(status, StatusCode::NOT_FOUND);
    Ok(())
}
```

	sqlx	tokio-postgres
Same	The pool is the state; the handler runs one parameterized query and maps "no row" to 404	The pool is the state; the handler runs one parameterized query and maps "no row" to 404
Different	The query runs on the pool directly; `FromRow` derives the mapping	A connection is checked out first (`pool.get()`); the mapping is written by hand, or with `TryFrom<&Row>`

In stockroom

The router lists every endpoint, then wraps them all in a timeout and request tracing:

```
// stockroom/src/http/mod.rs (excerpt)
pub fn router(state: AppState, options: RouterOptions) -> Router {
    Router::new()
        .route("/healthz", get(health::live))
        .route("/readyz", get(health::ready))
        .route("/admin/sessions", get(health::sessions))
        .route("/stores", get(stores::list).post(stores::create))
        // …
        .route("/orders", post(orders::place))
        .route("/orders/events", get(events::orders))
        .route("/orders/{id}", get(orders::get))
        .route("/orders/{id}/transition", post(orders::transition))
        .route("/reports/dashboard", get(reports::dashboard))
        .route("/reports/sales", get(reports::sales))
        .layer(TimeoutLayer::with_status_code(
            StatusCode::SERVICE_UNAVAILABLE,
            options.request_timeout,
        ))
        .layer(TraceLayer::new_for_http())
        .with_state(state)
}
```

stockroom/src/http/mod.rs

`TimeoutLayer::with_status_code` (tower-http 0.7) answers 503 when a handler takes longer than the request timeout, by dropping the handler's future, which Chapter 12 showed is not the same as stopping its query. The slow report guards against that with `CancelOnDrop`.

`AppError` (`src/error.rs`) is §15.3 in full: eleven variants, a stable `error` code for each, the fields a client needs (`currentVersion` for 412, the SKU and quantities for insufficient stock), `Retry-After` on `Busy`, and server errors logged with every `DbError` field but sent to the client as "something went wrong on our side". Its mapping from the database, by SQLSTATE and then by constraint name:

```
// stockroom/src/error.rs (excerpt)
match *db.code() {
    SqlState::UNIQUE_VIOLATION => {
        let (field, message) =
            constraint_message(db).unwrap_or((None, "this already exists"));
        AppError::Conflict {
            field,
            message: message.into(),
        }
    }
    SqlState::CHECK_VIOLATION
    | SqlState::NOT_NULL_VIOLATION
    | SqlState::FOREIGN_KEY_VIOLATION => {
        let (field, message) = constraint_message(db)
            .unwrap_or((None, "the request breaks a rule of the data"));
        AppError::Invalid {
            field,
            message: message.into(),
        }
    }
    SqlState::T_R_SERIALIZATION_FAILURE
    | SqlState::T_R_DEADLOCK_DETECTED
    | SqlState::LOCK_NOT_AVAILABLE => AppError::Busy(error),
    SqlState::QUERY_CANCELED => AppError::Timeout(error),
    SqlState::ADMIN_SHUTDOWN
    | SqlState::CANNOT_CONNECT_NOW
    | SqlState::TOO_MANY_CONNECTIONS => AppError::Unavailable(db.message().to_owned()),
    _ => AppError::Db(error),
}
```

stockroom/src/error.rs

The rest of `http/` follows this chapter's patterns: `products::list` pages with a hex cursor (§15.5), `products::get` and `update` send and check `ETag`/`If-Match` (§15.6), as does `orders::transition`; `products::create_batch` uses a savepoint per item (Chapter 8) to keep the good items of a batch when some fail; and `stockroom/stockroom.http` holds a request for every endpoint, ready to send from an editor.

Summary

- The `Router`'s state holds the pool (and other cheap handles); layers add tracing, timeouts and body limits. Tests and examples drive the router in memory with `oneshot`.
- A `Db` extractor (`FromRequestParts`) checks a connection out for the handler; it returns when the handler's future ends. Put body extractors last.
- Map database errors once, in `From<tokio_postgres::Error>`: 23505 → 409, 23514/23502/23503 → 422 (with the field named by the constraint), 40001/40P01/55P03 → 503 with `Retry-After`, 57014 → 503, the rest → 500 with details only in the log.
- Let constraints do the checking; do not check-then-insert. Transactions roll back on `?`. Retried transactions in handlers use an `async move` closure that owns the request.
- Page with an opaque keyset cursor and one extra row. Turn the version column into `ETag`, and `If-Match` into `WHERE version = …`; answer 412 with the current version.
- Validate in three layers: types (deserialization), the handler, the database. Wrap `Json` to keep one error format.

Exercises

1. **Map lock timeouts.** stockroom's sessions have `lock_timeout = 2000`. Which status does a request that waited too long for a row lock get, and is that the right one? Would you retry it inside `in_transaction`? *Hint:* 55P03 is classified with serialization failures.
2. **A 409 with details.** Change §15.3's `create` so that a duplicate SKU answers 409 with the existing product's name in the body, without a check-then-insert race. *Hint:* `INSERT … ON CONFLICT (sku) DO NOTHING RETURNING …`, then a `SELECT` when nothing came back.
3. **Require If-Match.** Change §15.6's `update` to answer 428 Precondition Required when the request has no `If-Match`. Which clients does that protect, and which does it annoy? *Hint:* RFC 6585 defines 428 for exactly this.
4. **A cursor that lies.** In §15.5, what happens when a client sends `?cursor=zz`, and when it sends a valid hex cursor for a SKU that was deleted since? Is either a problem? *Hint:* `sku > $1` does not need `$1` to exist.

Solutions

1. 503 with `Retry-After: 1`, as `AppError::Busy`, and `in_transaction` retries it like a serialization failure. That is reasonable for short waits behind another request's transaction; the retry usually succeeds once that transaction ends. If the wait was behind something long (a migration, a stuck session), the retries run out quickly and the client is told to come back, which is better than holding a pool connection for the whole duration.

2. Insert with `ON CONFLICT DO NOTHING`; a row back means created, no row means it existed, and the follow-up `SELECT` reads the existing one. There is still a tiny window (the existing product could be deleted between the two statements), which the second query handles by returning 404 or retrying:

```
use axum::body::Body;
use axum::extract::State;
use axum::http::{Request, StatusCode, header};
use axum::routing::post;
use axum::{Json, Router};
use deadpool_postgres::Pool;
use serde::Deserialize;
use serde_json::{Value, json};

#[derive(Deserialize)]
struct NewProduct {
    sku: String,
    name: String,
}

async fn create(
    State(pool): State<Pool>,
    Json(new): Json<NewProduct>,
) -> Result<(StatusCode, Json<Value>), StatusCode> {
    let db = pool
        .get()
        .await
        .map_err(|_| StatusCode::SERVICE_UNAVAILABLE)?;
    let inserted = db
        .query_opt(
            "INSERT INTO product (sku, name) VALUES ($1, $2)
             ON CONFLICT (sku) DO NOTHING RETURNING sku",
            &[&new.sku, &new.name],
```

```
        )
        .await
        .map_err(|_| StatusCode::INTERNAL_SERVER_ERROR)?;
    if inserted.is_some() {
        return Ok((StatusCode::CREATED, Json(json!({ "sku": new.sku }))));
    }
    let existing = db
        .query_opt("SELECT name FROM product WHERE sku = $1", &[&new.sku])
        .await
        .map_err(|_| StatusCode::INTERNAL_SERVER_ERROR)?
        .ok_or(StatusCode::CONFLICT)?; // deleted in between: still a conflict, without
details
    let name: String = existing.get(0);
    Ok((
        StatusCode::CONFLICT,
        Json(json!({ "error": "conflict", "field": "sku", "existingName": name })),
    ))
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let pool = playground::pool(1);
    pool.get()
        .await?
        .batch_execute("CREATE TEMP TABLE product (sku text PRIMARY KEY, name text NOT
NULL)")
        .await?;
    let app = Router::new()
        .route("/products", post(create))
        .with_state(pool);
    let send = |name: &str| {
        Request::post("/products")
            .header(header::CONTENT_TYPE, "application/json")
            .body(Body::from(
                json!({ "sku": "CHAIR-01", "name": name }).to_string(),
            ))
            .expect("a valid request")
    };
    let (status, _, _) = playground::call(&app, send("Oak chair")).await;
    assert_eq!(status, StatusCode::CREATED);
    let (status, _, body) = playground::call(&app, send("Pine chair")).await;
    assert_eq!(status, StatusCode::CONFLICT);
    assert_eq!(body["existingName"], "Oak chair");
    Ok(())
}
```

3. Return `StatusCode::PRECONDITION_REQUIRED` when `if_match` returns `None`. It protects every client from silently overwriting someone else's change, including clients written by people who never heard of `ETag`; it annoys scripts and quick `curl` commands, which must now `GET` first. Many APIs require it for `PATCH` and `PUT` on contended resources and not elsewhere.

4. `zz` is not hex, so `decode_cursor` returns `None` and the handler answers 422: a client error, correctly. A cursor for a deleted SKU is fine: `sku > $1` compares values and does not need the row to exist, so the next page starts right after where the deleted row would have been. That robustness is one of keyset pagination's advantages over cursors that point at rows.

CHAPTER 16

axum II: streaming, real-time and background work

Chapter 15's handlers all had the same shape: borrow a connection, run a query, return a response, give the connection back. This chapter is about the responses and the work that do not fit that shape. A body that streams a whole catalog must keep its connection for as long as it streams. An upload must flow into `COPY` as it arrives. A live feed of order events must stay open for minutes, fed by the single `LISTEN` connection of Chapter 11. A slow report that the client abandoned must be cancelled on the server. And some work belongs to nobody's request at all: the outbox worker that delivers jobs in the background.

Each section is one of stockroom's endpoints or tasks, reduced to a runnable example. The last sections cover what ties them together at the end: shutting down without cutting anything off, and telling an orchestrator whether the service is alive and ready.

16.1 NDJSON from a `RowStream`: who owns the connection?

Chapter 9 streamed rows with `query_raw`, and found that a `RowStream` does not borrow the client. In a handler that is both what makes streaming possible and a trap. The handler can return a body built from the stream; the body outlives the handler; and the pooled `Object` the handler borrowed is dropped when the handler returns, going back to the pool *while the stream still runs on it*. The next request to borrow that connection queues its query behind the rest of the rows.

The fix is to move the `Object` into the body's stream, so that the connection stays checked out until the last row has been sent (or the client goes away and the body is dropped). Here are both versions, side by side in one program, with a pool of two connections; while the first response's body is still unread, a second request needs a connection:

```
use axum::Router;
use axum::body::{Body, Bytes};
use axum::extract::State;
use axum::http::{Request, StatusCode};
use axum::response::{IntoResponse, Response};
use axum::routing::get;
use deadpool_postgres::Pool;
use futures_util::TryStreamExt;
use http_body_util::BodyExt;
use std::pin::pin;
use std::time::Duration;
```

```rust
use tower::ServiceExt;

const MANY: &str = "SELECT i FROM generate_series(1, 200000) i";

fn unavailable<E>(_: E) -> StatusCode {
    StatusCode::SERVICE_UNAVAILABLE
}

/// Wrong: `db` goes back to the pool when this function returns, while `rows` still uses
it.
async fn leaky(State(pool): State<Pool>) -> Result<Response, StatusCode> {
    let db = pool.get().await.map_err(unavailable)?;
    let rows = db
        .query_raw(MANY, playground::slice_iter(&[]))
        .await
        .map_err(unavailable)?;
    let lines = rows.map_ok(|row| Bytes::from(format!("{}\n", row.get::<_, i32>(0))));
    Ok(Body::from_stream(lines).into_response())
}

/// Right: the body's stream owns `db`, so the connection stays checked out until it ends.
async fn owned(State(pool): State<Pool>) -> Result<Response, StatusCode> {
    let db = pool.get().await.map_err(unavailable)?;
    let rows = db
        .query_raw(MANY, playground::slice_iter(&[]))
        .await
        .map_err(unavailable)?;
    let lines = async_stream::try_stream! {
        let _checked_out = db;
        let mut rows = pin!(rows);
        while let Some(row) = rows.try_next().await? {
            yield Bytes::from(format!("{}\n", row.get::<_, i32>(0)));
        }
    };
    Ok(Body::from_stream::<_>(lines.map_err(|e: tokio_postgres::Error| e)).into_response())
}

async fn one(State(pool): State<Pool>) -> Result<String, StatusCode> {
    let db = pool.get().await.map_err(unavailable)?;
    let row = db.query_one("SELECT 1", &[]).await.map_err(unavailable)?;
    Ok(row.get::<_, i32>(0).to_string())
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let pool = playground::pool(2);
    for (name, stream_route) in [("leaky", get(leaky)), ("owned", get(owned))] {
        let app = Router::new()
            .route("/stream", stream_route)
            .route("/one", get(one))
            .with_state(pool.clone());

        // The streaming response arrives; its body is not read yet.
        let streaming = app
            .clone()
            .oneshot(Request::get("/stream").body(Body::empty())?)
            .await?;
        // Meanwhile, another request.
        let other = tokio::time::timeout(
            Duration::from_millis(500),
            playground::call(&app, Request::get("/one").body(Body::empty())?),
        )
        .await;
        println!(
            "{name}: the other request {}",
            if other.is_ok() {
                "was answered"
            } else {
                "timed out"
            }
```

```
        );
        assert_eq!(other.is_ok(), name == "owned");

        // Either way the stream itself is complete.
        let body = streaming.into_body().collect().await?.to_bytes();
        assert_eq!(body.iter().filter(|&&b| b == b'\n').count(), 200_000);
    }
    Ok(())
}
```

```
leaky: the other request timed out
owned: the other request was answered
```

In the leaky version, the pool's only idle connection was the one that had just "returned", so the second request got it, and its `SELECT 1` waited behind 200,000 unread rows. In the owned version, the stream kept its connection, so the second request got the other one. `let _checked_out = db;` is the whole fix: the underscore-prefixed name keeps the value alive to the end of the stream, where a bare `_` would drop it at once.

Backpressure works end to end through this chain. A client that reads slowly stops the body; the body stops polling the stream; the stream's channel fills; the connection stops reading the socket; and the server waits in `ClientWrite` (Chapter 9). No layer buffers more than a little, so memory stays flat however large the result and however slow the client. The price is that a slow client holds a pooled connection for as long as it takes: set the pool size, and consider a cap on concurrent streams, with that in mind.

> **Pitfall:** once the first bytes of a streamed body are sent, the status (200) is final. An error after that can only cut the body short, which the client sees as a truncated response: an NDJSON client should treat a missing final newline, or a count that does not match, as a failure. Decide before streaming whatever can still fail early, and stream only after.

16.2 A CSV export with `copy_out`

When the response is CSV, PostgreSQL can produce it: `COPY … TO STDOUT (FORMAT csv, HEADER)` formats every row, quoting included, and `copy_out` (Chapter 10) hands the chunks over as they come. The handler only moves bytes, and the same ownership rule applies:

```
// stockroom/src/http/products.rs (excerpt)
pub async fn export_csv(Db(db): Db) -> Result<Response, AppError> {
    let copy = db.copy_out(products::EXPORT_CSV).await?;
    let body = Body::from_stream(async_stream::stream! {
        let _checked_out = db;
        let mut copy = pin!(copy);
        while let Some(chunk) = copy.next().await {
            yield chunk;
        }
    });
    Ok((
        [
            (header::CONTENT_TYPE, "text/csv"),
            (
```

```
                header::CONTENT_DISPOSITION,
                "attachment; filename=\"products.csv\"",
            ),
        ],
        body,
    )
        .into_response())
}
```

stockroom/src/http/products.rs

The next section's example exports with exactly this shape and reads the result back in.

16.3 A CSV import: from the request body into `COPY`

The other direction is the same idea reversed. axum's `Body` is a stream of `Bytes` (`body.into_data_stream()`), and a `CopyInSink` accepts `Bytes` (Chapter 10), so an upload can go from the socket into `COPY` chunk by chunk, never held whole in memory. Chapter 10's staging table makes it safe: the rows land in a temporary table, are checked and merged in one transaction, and the upload goes in completely or not at all. A body limit keeps a client from uploading a terabyte:

```
use axum::Json;
use axum::Router;
use axum::body::Body;
use axum::extract::State;
use axum::http::{Request, StatusCode, header};
use axum::response::{IntoResponse, Response};
use axum::routing::{get, post};
use deadpool_postgres::Pool;
use futures_util::{SinkExt, StreamExt};
use serde_json::{Value, json};
use std::pin::pin;
use tower_http::limit::RequestBodyLimitLayer;

type ApiError = (StatusCode, Json<Value>);

fn error(status: StatusCode, message: impl ToString) -> ApiError {
    (status, Json(json!({ "error": message.to_string() })))
}

async fn export(State(pool): State<Pool>) -> Result<Response, ApiError> {
    let db = pool
        .get()
        .await
        .map_err(|e| error(StatusCode::SERVICE_UNAVAILABLE, e))?;
    let copy = db
        .copy_out("COPY (SELECT sku, price_cents FROM product ORDER BY sku) TO STDOUT
(FORMAT csv, HEADER)")
        .await
        .map_err(|e| error(StatusCode::INTERNAL_SERVER_ERROR, e))?;
    let body = Body::from_stream(async_stream::stream! {
        let _checked_out = db;
        let mut copy = pin!(copy);
        while let Some(chunk) = copy.next().await {
            yield chunk;
        }
    });
    Ok(([(header::CONTENT_TYPE, "text/csv")], body).into_response())
}

async fn import(State(pool): State<Pool>, body: Body) -> Result<Json<Value>, ApiError> {
    let internal = |e: tokio_postgres::Error| error(StatusCode::INTERNAL_SERVER_ERROR, e);
    let mut db = pool
```

```
        .get()
        .await
        .map_err(|e| error(StatusCode::SERVICE_UNAVAILABLE, e))?;
    let tx = db.transaction().await.map_err(internal)?;
    tx.batch_execute(
        "CREATE TEMP TABLE price_import (sku text, price_cents bigint) ON COMMIT DROP",
    )
    .await
    .map_err(internal)?;
    let sink = tx
        .copy_in("COPY price_import FROM STDIN (FORMAT csv, HEADER MATCH)")
        .await
        .map_err(internal)?;
    let mut sink = pin!(sink);
    let mut chunks = body.into_data_stream();
    while let Some(chunk) = chunks.next().await {
        // A body error here is the client's: an upload cut short, or one over the limit.
        let chunk = chunk.map_err(|e| error(StatusCode::BAD_REQUEST, e))?;
        sink.send(chunk)
            .await
            .map_err(|e| error(StatusCode::UNPROCESSABLE_ENTITY, e))?;
    }
    let rows = sink.as_mut().finish().await.map_err(|e| {
        // A malformed line is reported with its line number (Chapter 10).
        let db = e.as_db_error();
        let at = db.and_then(|d| d.where_()).unwrap_or_default();
        let message = db
            .map(|d| d.message())
            .unwrap_or("could not read the upload");
        error(
            StatusCode::UNPROCESSABLE_ENTITY,
            format!("{message} ({at})"),
        )
    })?;
    let changed = tx
        .execute(
            "MERGE INTO product AS p USING price_import AS i ON p.sku = i.sku
             WHEN MATCHED THEN UPDATE SET price_cents = i.price_cents
             WHEN NOT MATCHED THEN INSERT VALUES (i.sku, i.price_cents)",
            &[],
        )
        .await
        .map_err(internal)?;
    tx.commit().await.map_err(internal)?;
    Ok(Json(json!({ "rows": rows, "changed": changed })))
}

fn upload(csv: &str) -> Request<Body> {
    Request::post("/import")
        .header(header::CONTENT_TYPE, "text/csv")
        .header(header::CONTENT_LENGTH, csv.len())
        .body(Body::from(csv.to_owned()))
        .expect("a valid request")
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let pool = playground::pool(1);
    pool.get()
        .await?
        .batch_execute("CREATE TEMP TABLE product (sku text PRIMARY KEY, price_cents
bigint)")
        .await?;
    let app = Router::new()
        .route("/export.csv", get(export))
        .route(
            "/import",
            post(import).layer(RequestBodyLimitLayer::new(1024)),
        )
        .with_state(pool);
```

```
    let (status, _, body) = playground::call(
        &app,
        upload("sku,price_cents\nA-1,4999\nA-2,12900\nA-3,2500\n"),
    )
    .await;
    assert_eq!(status, StatusCode::OK);
    assert_eq!(body["rows"], 3);

    let (_, _, csv) =
        playground::call(&app, Request::get("/export.csv").body(Body::empty())?).await;
    print!("{}", csv.as_str().unwrap_or_default());

    // A bad line: nothing is imported, and the client learns where.
    let (status, _, body) =
        playground::call(&app, upload("sku,price_cents\nA-1,1\nA-2,lots\n")).await;
    println!("{status} {}", body["error"]);
    assert_eq!(status, StatusCode::UNPROCESSABLE_ENTITY);

    // Too big: refused before the handler runs.
    let big = format!("sku,price_cents\n{}", "A-1,1\n".repeat(200));
    let (status, _, _) = playground::call(&app, upload(&big)).await;
    println!("{status}");
    assert_eq!(status, StatusCode::PAYLOAD_TOO_LARGE);
    Ok(())
}
```

```
sku,price_cents
A-1,4999
A-2,12900
A-3,2500
422 Unprocessable Entity "invalid input syntax for type bigint: \"lots\" (COPY price_import,
line 3, column price_cents: \"lots\")"
413 Payload Too Large
```

Three details carry over to stockroom's `POST /products/import`:

- **The limit is a layer on the route**, `RequestBodyLimitLayer` from tower-http. A request with a `Content-Length` over the limit gets 413 before the handler runs; one without (a chunked upload) has its body cut off at the limit, and the handler sees a body error, which aborts the `COPY` and rolls back the transaction.
- **Errors in the data are the client's**, class 22, reported with the line `DbError::where_()` names. They quote the client's own CSV back, which is the one case where a `DbError`'s text may reach a client (Chapter 6).
- **The handler holds a connection while the upload arrives**, since the `COPY` runs as the bytes come in. A slow uploader holds a pooled connection; the body limit and a request timeout bound how long.

16.4 Server-sent events from `LISTEN`

`GET /orders/events` keeps a response open and sends an event each time an order changes: server-sent events, a `text/event-stream` body of `event:` and `data:` lines that browsers read with `EventSource`. The events come from Chapter 11's design: one dedicated `LISTEN` connection for the whole process, feeding a `tokio::sync::broadcast` channel, from which each open SSE response subscribes. So a thousand open event streams cost one database connection, not a thousand.

axum's `Sse` turns a `Stream` of `Event`s into such a response, and `KeepAlive` sends a comment line every 15 seconds by default, so that proxies do not close an idle stream:

```
use axum::Router;
use axum::body::Body;
use axum::extract::State;
use axum::http::Request;
use axum::response::sse::{Event, KeepAlive, Sse};
use axum::routing::get;
use futures_util::{Stream, StreamExt};
use http_body_util::BodyExt;
use std::convert::Infallible;
use std::time::Duration;
use tokio::sync::broadcast::{self, error::RecvError};
use tokio_postgres::AsyncMessage;
use tokio_util::sync::CancellationToken;
use tower::ServiceExt;

#[derive(Clone)]
struct AppState {
    events: broadcast::Sender<String>,
    shutdown: CancellationToken,
}

async fn order_events(
    State(state): State<AppState>,
) -> Sse<impl Stream<Item = Result<Event, Infallible>>> {
    let mut events = state.events.subscribe();
    let stream = async_stream::stream! {
        loop {
            match events.recv().await {
                Ok(payload) => yield Ok(Event::default().event("order").data(payload)),
                // Missed some: tell the client to reload, rather than pretend (Chapter 11).
                Err(RecvError::Lagged(_)) => yield
Ok(Event::default().event("resync").data("reload")),
                Err(RecvError::Closed) => break,
            }
        }
    };
    // End with the service, or a graceful shutdown would wait for this stream forever.
    let stream = stream.take_until(state.shutdown.clone().cancelled_owned());
    Sse::new(stream).keep_alive(KeepAlive::default())
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    // The one listener connection, forwarding notifications into the broadcast channel.
    let (listener, mut messages) = playground::connect_with_messages().await?;
    listener.batch_execute("LISTEN order_events").await?;
    let (events, _) = broadcast::channel(64);
    let forward = events.clone();
    tokio::spawn(async move {
        while let Some(message) = messages.recv().await {
            if let AsyncMessage::Notification(n) = message {
                let _ = forward.send(n.payload().to_owned()); // Err: no subscriber right
now
            }
        }
    });

    let shutdown = CancellationToken::new();
    let app = Router::new()
        .route("/orders/events", get(order_events))
        .with_state(AppState {
            events,
            shutdown: shutdown.clone(),
        });

    // A client connects...
    let response = app
```

```
        .oneshot(Request::get("/orders/events").body(Body::empty())?)
        .await?;
    println!("{:?}", response.headers()["content-type"]);
    let mut body = response.into_body();

    // ...an order changes, in another session...
    let writer = playground::connect().await?;
    writer
        .execute(
            "SELECT pg_notify('order_events', $1)",
            &[&r#"{"orderId":42,"status":"paid"}"#],
        )
        .await?;

    // ...and the event arrives on the open response.
    let frame = body.frame().await.expect("an event")?;
    print!(
        "{}",
        String::from_utf8_lossy(frame.data_ref().expect("data"))
    );

    // At shutdown the stream ends, and with it the response.
    shutdown.cancel();
    let end = tokio::time::timeout(Duration::from_secs(1), body.frame()).await?;
    assert!(end.is_none(), "the body has ended");
    Ok(())
}
```

```
"text/event-stream"
event: order
data: {"orderId":42,"status":"paid"}
```

Notice what the SSE handler does *not* hold: a database connection. It holds a broadcast receiver, so an idle event stream costs a few hundred bytes. If the handler needs the database first (stockroom looks up the store a client filters on), it borrows a connection for that lookup and lets it go before returning the stream.

The `take_until` line matters more than it looks. axum's graceful shutdown (§16.7) waits for open responses to finish, and an event stream never finishes by itself: without it, a SIGTERM would leave the process waiting for every connected browser to leave. stockroom's test for this measured a shutdown in under 50 ms with an SSE client attached.

16.5 Timeouts and client disconnects

A request can be abandoned two ways: the router's timeout fires, or the client disconnects. In both cases axum *drops the handler's future*, and Chapter 12 showed what that leaves behind: a query still running on the server, and a pooled connection that goes back to the pool busy.

`tower_http::timeout::TimeoutLayer` gives a slow handler a deadline and answers 503 when it passes. A guard that owns the pooled connection, and cancels its query when dropped before the handler finishes, makes the abandonment complete:

```
use axum::Router;
use axum::body::Body;
use axum::extract::State;
use axum::http::{Request, StatusCode};
use axum::routing::get;
```

```rust
use deadpool_postgres::{Object, Pool};
use std::ops::Deref;
use std::time::Duration;
use tokio_postgres::NoTls;
use tower_http::timeout::TimeoutLayer;

/// Owns a pooled connection while a slow query runs. Dropped before `finish` (the request
was
/// abandoned), it cancels the query and discards the connection.
struct CancelOnDrop(Option<Object>);

impl CancelOnDrop {
    fn finish(mut self) -> Object {
        self.0.take().expect("present until finished or dropped")
    }
}

impl Deref for CancelOnDrop {
    type Target = Object;
    fn deref(&self) -> &Object {
        self.0.as_ref().expect("present until finished or dropped")
    }
}

impl Drop for CancelOnDrop {
    fn drop(&mut self) {
        let Some(conn) = self.0.take() else { return };
        let token = conn.cancel_token();
        // `drop` cannot await: send the cancel request from a task.
        tokio::spawn(async move {
            let _ = token.cancel_query(NoTls).await;
            // Never back to the pool: a late cancel request could hit its next user.
            drop(Object::take(conn));
        });
    }
}

async fn unguarded(State(pool): State<Pool>) -> Result<String, StatusCode> {
    let db = pool
        .get()
        .await
        .map_err(|_| StatusCode::SERVICE_UNAVAILABLE)?;
    let row = db
        .query_one("SELECT 'done' FROM pg_sleep(3) /* unguarded report */", &[])
        .await
        .map_err(|_| StatusCode::INTERNAL_SERVER_ERROR)?;
    Ok(row.get(0))
}

async fn guarded(State(pool): State<Pool>) -> Result<String, StatusCode> {
    let db = CancelOnDrop(Some(
        pool.get()
            .await
            .map_err(|_| StatusCode::SERVICE_UNAVAILABLE)?,
    ));
    let row = db
        .query_one("SELECT 'done' FROM pg_sleep(3) /* guarded report */", &[])
        .await
        .map_err(|_| StatusCode::INTERNAL_SERVER_ERROR)?;
    drop(db.finish()); // finished: disarm the guard, and the connection goes back to the
pool
    Ok(row.get(0))
}

/// How many sessions are running the query marked `marker`, after `wait`.
async fn running(observer: &tokio_postgres::Client, marker: &str, wait: Duration) -> i64 {
    tokio::time::sleep(wait).await;
    observer
        .query_one(
            "SELECT count(*) FROM pg_stat_activity
```

```
             WHERE state = 'active' AND query LIKE '%' || $1 || '%' AND pid <>
pg_backend_pid()",
            &[&marker],
        )
        .await
        .expect("a query on pg_stat_activity")
        .get(0)
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let observer = playground::connect().await?;
    let app = Router::new()
        .route("/unguarded", get(unguarded))
        .route("/guarded", get(guarded))
        .layer(TimeoutLayer::with_status_code(
            StatusCode::SERVICE_UNAVAILABLE,
            Duration::from_millis(200),
        ))
        .with_state(playground::pool(4));

    for (uri, marker) in [
        ("/unguarded", "unguarded report"),
        ("/guarded", "guarded report"),
    ] {
        let (status, _, _) = playground::call(&app,
Request::get(uri).body(Body::empty())?).await;
        let left = running(&observer, marker, Duration::from_millis(300)).await;
        println!("{uri}: {status}, queries still running on the server: {left}");
        assert_eq!(status, StatusCode::SERVICE_UNAVAILABLE);
        assert_eq!(left, if uri == "/guarded" { 0 } else { 1 });
    }
    Ok(())
}
```

```
/unguarded: 503 Service Unavailable, queries still running on the server: 1
/guarded: 503 Service Unavailable, queries still running on the server: 0
```

Both clients got their 503 after 200 ms. Only one of the servers stopped working. stockroom's `CancelOnDrop` (Chapter 12) is this guard with tracing and a configurable TLS connector, and `GET /reports/sales?delayMs=5000` uses it; a test in `tests/reports.rs` (`an_abandoned_report_is_cancelled_on_the_server`) checks that no `pg_sleep` is left running after a 300 ms router timeout.

A server-side `statement_timeout` (Chapter 12) is the other half: the guard handles requests that were abandoned, the timeout handles queries that run too long whether or not anyone is waiting. stockroom sets a 5-second statement timeout on every session, and its test `statement_timeout_is_a_503` checks that a query ended by it becomes a 503 with the error code `timeout`.

16.6 The outbox worker

Some work must happen *because* a transaction committed, but not *inside* it: sending the "your order is ready" email, calling a partner's webhook. Doing it in the request is wrong both ways. Sent before the commit, the email goes out for an order that may still roll back; sent after, it is lost if the process dies between the commit and the send.

The *transactional outbox* splits the work in two. The transaction that changes the order also inserts a job row into an `outbox` table (stockroom's trigger does it, Chapter 11), so the job exists if and only

if the change committed. A background worker then claims due jobs, delivers them, and marks them done. Several workers can share the table safely because of `FOR UPDATE SKIP LOCKED`: each worker locks the rows it claims, and the others *skip* locked rows instead of waiting for them (Chapter 8). A worker that crashes mid-batch releases its locks with its connection, and its jobs become claimable again:

```
type Error = Box<dyn std::error::Error>;

const CLAIM: &str = "SELECT id FROM outbox
     WHERE done_at IS NULL ORDER BY id LIMIT $1
     FOR UPDATE SKIP LOCKED";

async fn claim(tx: &tokio_postgres::Transaction<'_>, limit: i64) -> Result<Vec<i64>, Error>
{
    Ok(tx
        .query(CLAIM, &[&limit])
        .await?
        .iter()
        .map(|row| row.get(0))
        .collect())
}

async fn mark_done(tx: &tokio_postgres::Transaction<'_>, ids: &[i64]) -> Result<(), Error> {
    tx.execute(
        "UPDATE outbox SET done_at = now() WHERE id = ANY($1)",
        &[&ids],
    )
    .await?;
    Ok(())
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    // Workers are separate sessions, so the table must be a real one: in a schema of our
own.
    let schema = format!("ch16_{}", std::process::id());
    let mut a = playground::connect().await?;
    let mut b = playground::connect().await?;
    let mut c = playground::connect().await?;
    a.batch_execute(&format!(
        "CREATE SCHEMA {schema};
         CREATE TABLE {schema}.outbox (id bigint PRIMARY KEY, done_at timestamptz);
         INSERT INTO {schema}.outbox SELECT generate_series(1, 5);"
    ))
    .await?;
    for worker in [&a, &b, &c] {
        worker
            .batch_execute(&format!("SET search_path = {schema}"))
            .await?;
    }
    let mut delivered: Vec<i64> = Vec::new();

    // Worker A claims three jobs and is still delivering them...
    let tx_a = a.transaction().await?;
    let batch_a = claim(&tx_a, 3).await?;
    // ...while worker B claims too: it skips A's rows instead of waiting for them.
    let tx_b = b.transaction().await?;
    let batch_b = claim(&tx_b, 3).await?;
    assert_eq!(
        (batch_a.as_slice(), batch_b.as_slice()),
        (&[1, 2, 3][..], &[4, 5][..])
    );

    delivered.extend(&batch_a);
    mark_done(&tx_a, &batch_a).await?;
    tx_a.commit().await?;

    // B delivers its jobs, then dies before committing: its locks go, the jobs stay due.
    delivered.extend(&batch_b);
```

```
    drop(tx_b);
    drop(b);

    // Worker C finds them, and delivers them again.
    let tx_c = c.transaction().await?;
    let batch_c = claim(&tx_c, 3).await?;
    delivered.extend(&batch_c);
    mark_done(&tx_c, &batch_c).await?;
    tx_c.commit().await?;

    println!("deliveries: {delivered:?}");
    assert_eq!(delivered, [1, 2, 3, 4, 5, 4, 5]); // at least once, not exactly once
    let pending: i64 = c
        .query_one("SELECT count(*) FROM outbox WHERE done_at IS NULL", &[])
        .await?
        .get(0);
    assert_eq!(pending, 0);

    a.batch_execute(&format!("DROP SCHEMA {schema} CASCADE"))
        .await?;
    Ok(())
}
```

```
deliveries: [1, 2, 3, 4, 5, 4, 5]
```

That last line is the outbox's guarantee and its cost. Every job is delivered *at least once*: none is lost, whatever crashes when. Some may be delivered twice, so the receiving side must be idempotent, for instance by deduplicating on the job id that travels with each delivery.

stockroom's worker adds two things. It does not poll in a tight loop: it sleeps until the listener's `outbox` notification wakes it (Chapter 11), with a 30-second poll as a safety net for a missed wake-up, and it drains full batches back to back. And a failed delivery is not retried at once: the job's `available_at` moves forward with exponential backoff (1 s, 2 s, 4 s, up to an hour), and `last_error` records why.

```
// stockroom/src/worker/outbox.rs (excerpt)
pub async fn run(pool: Pool, hub: Hub, deliver: impl Deliver, shutdown: CancellationToken) {
    loop {
        match drain(&pool, &deliver).await {
            Ok(n) if n as i64 == BATCH => continue, // a full batch: there may be more
            Ok(_) => {}
            Err(error) => tracing::warn!(%error, "outbox drain failed"),
        }
        tokio::select! {
            () = hub.outbox_woken() => {}
            () = tokio::time::sleep(POLL_INTERVAL) => {}
            () = shutdown.cancelled() => return,
        }
    }
}
```

stockroom/src/worker/outbox.rs · stockroom/src/repo/outbox.rs

One trade-off is deliberate: the worker holds its claimed rows locked, and a pooled connection, while it delivers. That is what makes a job belong to one worker at a time, and it means deliveries need short timeouts of their own. A design that must make slow calls instead marks jobs as "in progress until" a lease time, commits, delivers, and comes back to mark them done.

16.7 Graceful shutdown

When an orchestrator stops the service (SIGTERM, Ctrl-C), in-flight requests should finish, background tasks should stop between units of work, and the pool should close its connections cleanly. One `tokio_util::sync::CancellationToken`, cloned into everything long-lived, carries the signal:

```
// stockroom/src/app.rs (excerpt)
let tcp = TcpListener::bind(args.bind).await?;
tracing::info!("listening on http://{}", tcp.local_addr()?);
tokio::spawn(cancel_on_signal(shutdown.clone()));

// Stop accepting connections when `shutdown` fires, then wait for in-flight requests.
axum::serve(tcp, app)
    .with_graceful_shutdown(shutdown.clone().cancelled_owned())
    .await?;

shutdown.cancel();
let _ = tokio::join!(listener, worker);
pool.close();
tracing::info!("shut down");
```

stockroom/src/app.rs

The order is the point. `with_graceful_shutdown` stops accepting connections when the token fires and returns once in-flight responses are done; event streams end with the token (`take_until`, §16.4), so "done" comes promptly. The listener and the worker see the same token in their `select!` loops and return; the worker finishes the batch it is delivering first, because the token is only checked between batches. Then `pool.close()` closes the idle connections. The process exits with nothing half-done that matters: an outbox batch interrupted by a hard kill is simply delivered again (§16.6).

16.8 Health and readiness

Orchestrators ask two different questions, and conflating them causes outages:

- **Liveness**, `GET /healthz`: is the process working at all? If not, restart it. stockroom's answer is a constant `"ok"`: it deliberately does not touch the database, because a database outage is not fixed by restarting every instance of every service that uses it, and a liveness check that fails during one makes things worse.
- **Readiness**, `GET /readyz`: should this instance receive traffic right now? Here the database matters: stockroom borrows a connection and calls `check_connection()` (one round trip, no statement, Chapter 12), and reports the pool's state, so a load balancer stops sending requests to an instance that cannot serve them.

```
// stockroom/src/http/health.rs (excerpt)
pub async fn ready(State(state): State<AppState>) -> Result<Json<Value>, AppError> {
    let db = state.pool.get().await?;
    db.check_connection().await?; // one round trip, no statement
    let pool = state.pool.status();
    Ok(Json(json!({
        "status": "ready",
        "pool": { "maxSize": pool.max_size, "size": pool.size, "available": pool.available,
"waiting": pool.waiting },
        "listening": state.hub.server_info().is_some(),
    })))
}
```

stockroom/src/http/health.rs

When the pool cannot lend a connection within its wait timeout, `AppError` turns that into 503, which is exactly what a readiness probe should see. The `waiting` count is worth graphing: a pool that often has waiters is too small, or its connections are held too long (Chapter 13).

In stockroom

Endpoint or task	Section	File
`GET /products/stream` (NDJSON)	16.1	`http/products.rs`
`GET /products/export.csv`	16.2	`http/products.rs`
`POST /products/import` (64 MiB limit)	16.3	`http/products.rs`, `repo/products.rs`
`GET /orders/events` (SSE, `?store=` filter)	16.4	`http/events.rs`, `notify/`
`GET /reports/sales?delayMs=` with `CancelOnDrop`	16.5	`http/reports.rs`, `db/cancel.rs`
the outbox worker	16.6	`worker/outbox.rs`, `repo/outbox.rs`
graceful shutdown	16.7	`app.rs`
`GET /healthz`, `GET /readyz`, `GET /admin/sessions`	16.8	`http/health.rs`

`stockroom/stockroom.http` has a request for each; for the event stream, run `curl -N localhost:8080/orders/events` in one terminal and place an order from another.

Summary

- A streamed body outlives its handler, and a `RowStream` or `CopyOutStream` does not borrow the connection: move the pooled `Object` into the body's stream (`let _checked_out = db;`), or the next borrower queues behind the unread rows. Backpressure then reaches the server.
- Once a streamed body starts, its status is final; errors can only truncate it.
- CSV out: `copy_out` straight into the body. CSV in: `body.into_data_stream()` into `copy_in`, through a staging table and a merge, in one transaction, under `RequestBodyLimitLayer`.
- SSE: one `LISTEN` connection feeds a `broadcast`; each response subscribes, turns `Lagged` into a resync event, keeps alive, and ends with the shutdown token.
- An abandoned request leaves its query running unless a guard cancels it on drop and discards the connection; pair it with a server-side `statement_timeout`.
- The outbox: jobs written in the changing transaction, claimed with `FOR UPDATE SKIP LOCKED`, delivered at least once; woken by `NOTIFY`, with polling as a fallback.
- Shut down in order: stop accepting, let responses finish (streams end on the token), stop background tasks between units of work, close the pool. Liveness ignores the database; readiness checks it.

Exercises

1. **Count the rows.** Make §16.1's NDJSON stream end with a trailer line, `{"count": 200000}`, so that a client can tell a complete response from a truncated one. *Hint:* `async_stream::try_stream!` can keep a counter and `yield` once more after the loop.
2. **Filtered events.** Add a `?store=` query parameter to §16.4's handler that only forwards events whose JSON has that `storeId`. Where should the filtering happen: in SQL, in the listener task, or in each subscriber? *Hint:* there is one `LISTEN` for all subscribers.
3. **A lease instead of a lock.** Sketch the SQL for an outbox where a worker claims jobs by setting `locked_until = now() + interval '1 minute'`, commits, delivers, and then marks them done. What happens to a job whose worker died, and what new failure mode appears? *Hint:* `WHERE done_at IS NULL AND (locked_until IS NULL OR locked_until < now())`.
4. **Shutdown with a slow request.** In stockroom, a request is running a 4-second report when SIGTERM arrives. What does the client get, and how long does the process take to exit? *Hint:* graceful shutdown waits for in-flight requests; the request timeout still applies.

Solutions

1.

```rust
use axum::Router;
use axum::body::{Body, Bytes};
use axum::extract::State;
use axum::http::{Request, StatusCode};
use axum::response::{IntoResponse, Response};
use axum::routing::get;
use deadpool_postgres::Pool;
use futures_util::TryStreamExt;
use std::pin::pin;

async fn stream(State(pool): State<Pool>) -> Result<Response, StatusCode> {
    let db = pool
        .get()
        .await
        .map_err(|_| StatusCode::SERVICE_UNAVAILABLE)?;
    let rows = db
        .query_raw(
            "SELECT i FROM generate_series(1, 1000) i",
            playground::slice_iter(&[]),
        )
        .await
        .map_err(|_| StatusCode::INTERNAL_SERVER_ERROR)?;
    let lines = async_stream::try_stream! {
        let _checked_out = db;
        let mut rows = pin!(rows);
        let mut count = 0u64;
        while let Some(row) = rows.try_next().await? {
            count += 1;
            yield Bytes::from(format!("{{\"i\": {}}}\n", row.get::<_, i32>(0)));
        }
        yield Bytes::from(format!("{{\"count\": {count}}}\n"));
    };
    Ok(Body::from_stream::<_>(lines.map_err(|e: tokio_postgres::Error| e)).into_response())
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let app = Router::new()
        .route("/stream", get(stream))
```

```
        .with_state(playground::pool(1));
    let (_, _, body) = playground::call(&app, Request::get("/
stream").body(Body::empty())?).await;
    let text = body.as_str().unwrap_or_default().to_owned();
    let last = text.lines().last().unwrap_or_default();
    assert_eq!(last, r#"{"count": 1000}"#);
    assert_eq!(text.lines().count(), 1001);
    Ok(())
}
```

A client reads lines until the trailer and compares the count with the lines it saw; a response without a trailer was cut short.

2. In each subscriber, as stockroom does. The single listener receives every event for every store, and the broadcast hands each one to every subscriber, which keeps the one it cares about. Filtering in SQL would need one channel per store, and so one `LISTEN` per store on the shared connection, managed as subscribers come and go; it pays off only when stores are many and events frequent.

3. Claim with `UPDATE outbox SET locked_until = now() + interval '1 minute' WHERE id IN (SELECT id FROM outbox WHERE done_at IS NULL AND (locked_until IS NULL OR locked_until < now()) ORDER BY id LIMIT $1 FOR UPDATE SKIP LOCKED) RETURNING id, topic, payload`, commit, deliver, then `UPDATE outbox SET done_at = now() WHERE id = ANY($1)`. A dead worker's jobs become claimable again when their lease expires, without holding a connection meanwhile. The new failure mode is a *slow* worker: one whose delivery outlasts the lease, so a second worker claims and delivers the same job while the first is still at it. Deliveries must still be idempotent, and the lease longer than any delivery's timeout.

4. The client still gets its answer: graceful shutdown stops accepting new connections but lets the in-flight request finish. If the report finishes within the request timeout (10 s by default in stockroom), the client gets 200 and the process exits right after, about 4 seconds after the signal. If the report would run longer, the router's timeout answers 503 first, and `CancelOnDrop` cancels the query, so the process still exits within the request timeout.

CHAPTER 17

Testing against a real database

The code in this book is mostly SQL and the glue around it, and most of its bugs live in the glue's assumptions about the SQL: that a constraint has the name the error mapping expects, that a transaction really rolls back, that fifty concurrent orders cannot sell twenty-one chairs. A mock of the database can confirm none of that, because it answers whatever the test told it to. So stockroom's tests run against PostgreSQL itself, and this chapter is about making that fast, isolated and reliable.

It covers what to test at which layer, two ways to isolate tests from each other (a database per test, and a transaction per test, with the second one's limits), testing the HTTP layer in memory, testing concurrency on purpose rather than by luck, and provoking each error code the error mapping handles. It ends with running PostgreSQL in CI.

17.1 What to test where

Layer	What can go wrong	How stockroom tests it	Database?
Pure functions	parsing, checksums, cursors, retry delays	`#[test]` unit tests next to the code	no
Types	`ToSql`/`FromSql` against the real column types, domains, enums	`tests/types.rs`: round trips through the database	yes
Repository and migrations	SQL that does not parse, wrong columns, constraint names, transactional DDL	through the HTTP tests, plus `tests/migrations.rs`	yes
HTTP	status codes, error bodies, headers, streaming	`tests/*.rs` with `oneshot` (§17.4)	yes
Concurrency	overselling, deadlocks, lost updates	`tests/concurrency.rs` with many tasks at once (§17.5)	yes

Most of stockroom's tests are at the HTTP layer, and that is deliberate. A test that sends `POST /orders` and checks the status, the body and the stock levels afterwards exercises the extractor, the validation, the transaction, the SQL, the constraints and the error mapping in one go, through the same interface a client uses, and it survives refactoring of everything behind that interface. Repository functions get dedicated tests only where the HTTP layer cannot reach them easily.

Run them with:

```
cargo test -p stockroom                    # 10 unit tests, 34 against PostgreSQL
cargo test -p stockroom --test concurrency # just the concurrency tests
```

```
running 6 tests
test result: ok. 6 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.89s
```

(One of the eight test binaries, measured with PG 18.6 on localhost. Each finishes in under a second, because tests within a binary run in parallel, each on a database of its own.)

17.2 A database per test, cloned from a template

Tests that share a database see each other's rows, and then a test passes or fails depending on which ran first. The cleanest isolation is a fresh database per test, and PostgreSQL can make one in milliseconds: `CREATE DATABASE new TEMPLATE old` copies an existing database file by file, schema and data, without running a single migration. So the harness migrates one *template* database once, then clones it for every test:

```
// stockroom/tests/common/mod.rs (excerpt)
pub async fn new() -> TestDb {
    let template = template();
    let name = format!("stockroom_test_{}", Uuid::now_v7().simple());
    let (admin, task) = connect(&config_for("postgres")).await;
    // A session that has just left the template can linger for a moment; cloning
    // fails with 55006 (object in use) until it is gone.
    let mut attempts = 0;
    loop {
        match admin
            .batch_execute(&format!(
                "CREATE DATABASE \"{name}\" TEMPLATE \"{template}\""
            ))
            .await
        {
            Ok(()) => break,
            Err(e) if e.code() == Some(&SqlState::OBJECT_IN_USE) && attempts < 50 => {
                attempts += 1;
                tokio::time::sleep(Duration::from_millis(50)).await;
            }
            Err(e) => panic!("cannot create {name}: {e}"),
        }
    }
    // …
```

stockroom/tests/common/mod.rs

The details are where the time went when building it:

- **The template's name includes a checksum of all migrations** (`stockroom_template_<checksum>`). Change a migration and the next test run builds a new template, and drops the old ones; leave them alone and every run reuses it.
- **Only one process builds it.** Cargo runs test binaries one after another, but a developer may run two `cargo test`s at once; an advisory lock (Chapter 14) makes the second wait.
- **The template must have no sessions when it is cloned**, or `CREATE DATABASE … TEMPLATE` fails with 55006 (`object_in_use`). The harness waits for its own migration connection's task to end, and retries the clone briefly for sessions that linger.

- **It is built outside any test's runtime.** Each `#[tokio::test]` has a runtime of its own, and a connection spawned on one test's runtime dies with it. The template is built once, in a `OnceLock`, on a thread with a runtime of its own.
- **Names are generated, never input**, which is why `format!` into `CREATE DATABASE` is acceptable here: identifiers cannot be parameters, and a UUID is a safe identifier.

Cleaning up without async `Drop`

A test database must be dropped when the test ends, whether it passed or panicked, and the natural place is `Drop`. But `Drop::drop` cannot `.await`, and it may run on a Tokio worker thread, where blocking on a future with `Handle::block_on` panics. The way out is a thread of its own, with a runtime of its own, which the drop waits for. The same technique, applied to a schema instead of a database:

```
use std::sync::atomic::{AtomicU32, Ordering};
use tokio_postgres::Client;

/// A schema for one test, dropped when the guard is, even if the test panics.
struct TestSchema {
    name: String,
}

impl TestSchema {
    async fn new(client: &Client) -> Result<TestSchema, tokio_postgres::Error> {
        // Generated from the process id and a counter: no input reaches the SQL.
        static NEXT: AtomicU32 = AtomicU32::new(0);
        let name = format!(
            "test_{}_{}",
            std::process::id(),
            NEXT.fetch_add(1, Ordering::Relaxed)
        );
        client
            .batch_execute(&format!("CREATE SCHEMA {name}"))
            .await?;
        Ok(TestSchema { name })
    }
}

impl Drop for TestSchema {
    /// `Drop` cannot await, and it may run inside a runtime, where blocking on a future
    /// panics. So the cleanup runs on a thread with a runtime of its own, and drop waits.
    fn drop(&mut self) {
        let sql = format!("DROP SCHEMA IF EXISTS {} CASCADE", self.name);
        let _ = std::thread::spawn(move || {
            tokio::runtime::Builder::new_current_thread()
                .enable_all()
                .build()
                .expect("a runtime")
                .block_on(async move {
                    let client = playground::connect().await?;
                    client.batch_execute(&sql).await
                })
        })
        .join();
    }
}

async fn exists(client: &Client, name: &str) -> Result<bool, tokio_postgres::Error> {
    let row = client
        .query_one(
            "SELECT EXISTS (SELECT FROM pg_namespace WHERE nspname = $1)",
            &[&name],
        )
        .await?;
    Ok(row.get(0))
```

```
}

#[tokio::main]
async fn main() -> Result<(), tokio_postgres::Error> {
    let client = playground::connect().await?;

    // A test that passes.
    let name = {
        let schema = TestSchema::new(&client).await?;
        client
            .batch_execute(&format!("CREATE TABLE {}.item (id int)", schema.name))
            .await?;
        schema.name.clone()
    }; // dropped here
    assert!(!exists(&client, &name).await?);

    // A test that panics: the guard is dropped while the task unwinds.
    let schema = TestSchema::new(&client).await?;
    let name = schema.name.clone();
    let outcome = tokio::spawn(async move {
        let _schema = schema;
        assert_eq!(1 + 1, 3, "a failing assertion");
    })
    .await;
    assert!(outcome.is_err());
    assert!(!exists(&client, &name).await?);
    Ok(())
}
```

(Running it prints the failing assertion's panic message, from the task that was meant to fail.) stockroom's `TestDb` does exactly this with `DROP DATABASE … WITH (FORCE)`, where `FORCE` (PG 13+) ends the sessions the test's pool may still hold.

17.3 A transaction per test, and its limits

The other common isolation technique opens a transaction at the start of each test, runs the test inside it, and rolls it back at the end: nothing to create, nothing to clean up, and very fast. It works for code that takes a `&impl GenericClient` (Chapter 14), which can be handed the test's transaction. It has blind spots, though, and they are exactly the behaviours this book cares about:

```
use std::time::Duration;
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let (mut test, mut messages) = playground::connect_with_messages().await?;
    let mut other = playground::connect().await?;
    // A real table, visible to both sessions, in a schema of our own.
    let schema = format!("ch17_{}", std::process::id());
    test.batch_execute(&format!(
        "CREATE SCHEMA {schema};
         CREATE TABLE {schema}.item (id int CONSTRAINT item_id_key UNIQUE DEFERRABLE
INITIALLY DEFERRED);"
    ))
    .await?;
    for client in [&test, &other] {
        client
            .batch_execute(&format!("SET search_path = {schema}"))
            .await?;
    }
    test.batch_execute("LISTEN items").await?;

    {
```

```
        // The test, in a transaction that will be rolled back.
        let tx = test.transaction().await?;
        // A duplicate, but the constraint is deferred: no error until COMMIT.
        tx.execute("INSERT INTO item VALUES (1), (1)", &[]).await?;
        tx.execute("SELECT pg_notify('items', 'added')", &[])
            .await?;
        let mine: i64 = tx.query_one("SELECT count(*) FROM item", &[]).await?.get(0);
        // Any other connection, such as a second pooled one, sees nothing.
        let theirs: i64 = other
            .query_one("SELECT count(*) FROM item", &[])
            .await?
            .get(0);
        assert_eq!((mine, theirs), (2, 0));
    } // rolled back: the test "passed"

    // The notification never left.
    let notified = tokio::time::timeout(Duration::from_millis(200), messages.recv()).await;
    assert!(notified.is_err(), "a rolled-back NOTIFY is never delivered");

    // The same work, committed, fails: at COMMIT, which the rolled-back test never reached.
    let tx = other.transaction().await?;
    tx.execute("INSERT INTO item VALUES (1), (1)", &[]).await?;
    let err = tx.commit().await.unwrap_err();
    println!(
        "at commit: {}",
        err.as_db_error().expect("a server error").message()
    );
    assert_eq!(err.code(), Some(&SqlState::UNIQUE_VIOLATION));

    test.batch_execute(&format!("DROP SCHEMA {schema} CASCADE"))
        .await?;
    Ok(())
}
```

```
at commit: duplicate key value violates unique constraint "item_id_key"
```

So a transaction per test cannot test:

- **anything that happens at commit**: deferred constraints, `NOTIFY`, the outbox trigger's effects as other sessions see them;
- **anything with more than one connection**: the pool, concurrency, a listener, a second request that should see the first one's write;
- **code that manages its own transactions**, such as `in_transaction`'s retries, since a nested `BEGIN` is a warning and not a transaction, and `COMMIT` would end the test's transaction.

That is most of Part II. stockroom therefore uses a database per test throughout; a transaction per test remains a good fit for a large suite of plain repository queries.

17.4 HTTP tests with `oneshot`

The router is a `tower::Service`, so a test can call it like a function: build a `Request`, call `oneshot`, read the `Response`. No port, no HTTP client, no server task: Chapters 15 and 16 have been doing it all along with `playground::call`. stockroom's harness wraps it in a `TestApp` with helpers for its fixtures, created through the API itself so that fixtures cannot drift from what clients can do:

```
// stockroom/tests/orders.rs (excerpt)
#[tokio::test]
async fn an_order_for_more_than_is_available_reserves_nothing() {
```

```
    let db = TestDb::new().await;
    let app = shop(&db).await;
    // …
```

```
// stockroom/tests/common/mod.rs (excerpt)
pub async fn order(&self, store: &str, lines: &[(&str, i32)]) -> Reply {
    let lines: Vec<Value> = lines
        .iter()
        .map(|(sku, q)| json!({ "sku": sku, "quantity": q }))
        .collect();
    self.post(
        "/orders",
        json!({ "store": store, "customer": { "name": "Ada", "email": "ada@example.com" },
"lines": lines }),
    )
    .await
}
```

stockroom/tests/common/mod.rs · stockroom/tests/orders.rs

The router under test is the real one, `stockroom::http::router`, on a pool connected to the test's database. Tests that need the listener or the outbox worker start them explicitly (`tests/events.rs`), so that the others do not pay for background tasks they do not use.

17.5 Concurrency tests

The bugs that matter most in a service like stockroom appear only when requests overlap, and they will not show up in a test that sends one request at a time. A concurrency test creates the overlap on purpose: many tasks at once, on a multi-threaded runtime, against a pool with more than one connection, followed by an assertion about the *total* outcome:

```
// stockroom/tests/concurrency.rs (excerpt)
/// 20 chairs, 50 customers each wanting one, all at the same moment: exactly 20 orders.
#[tokio::test(flavor = "multi_thread", worker_threads = 8)]
async fn fifty_customers_twenty_chairs() {
    let db = TestDb::new().await;
    let app = db.app(16);
    app.store("mitte").await;
    app.product("CHAIR-01", 4999).await;
    app.stock("mitte", "CHAIR-01", 20).await;

    let attempts = (0..50).map(|_| {
        let app = app.clone();
        tokio::spawn(async move { app.order("mitte", &[("CHAIR-01", 1)]).await.status })
    });
    let statuses: Vec<StatusCode> = join_all(attempts)
        .await
        .into_iter()
        .map(Result::unwrap)
        .collect();
    // …
    assert_eq!((created, refused), (20, 30), "{statuses:?}");
    assert_eq!(app.level("mitte", "CHAIR-01").await, (20, 20, 0));
}
```

stockroom/tests/concurrency.rs

Two practices make such tests trustworthy. **Repeat them**: a race that loses one run in fifty passes most runs, so while building stockroom the oversell test ran 50 times in a loop, and passed all 50. And **force the interleaving when you can**: instead of hoping two transactions overlap, drive two connections

step by step, so that the conflict happens on every run. §17.7's example does that for a serialization failure and a deadlock.

`tests/concurrency.rs` also checks that orders listing the same products in opposite orders never deadlock (they lock rows in a fixed order, Chapter 8), and that 40 concurrent transfers between two stores lose no stock.

17.6 PostgreSQL in CI: a service, or testcontainers

The tests need a PostgreSQL server, and two setups are common.

A service container. The CI system starts PostgreSQL next to the job, and the tests connect to it through an environment variable, exactly as on a developer's machine. In GitHub Actions:

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:18
        env:
          POSTGRES_PASSWORD: postgres
        ports: ["5432:5432"]
        options: >-
          --health-cmd pg_isready --health-interval 2s --health-timeout 3s --health-retries
30
    env:
      ADMIN_DATABASE_URL: postgres://postgres:postgres@localhost:5432/postgres
    steps:
      - uses: actions/checkout@v4
      - run: cargo test --workspace
```

This is what stockroom's harness expects: `ADMIN_DATABASE_URL` names a server and an account that may create databases.

testcontainers. The test itself starts a throwaway container through the Docker API, and connects to the port it was given. testcontainers-modules (0.15.0, on testcontainers 0.27; checked 2026-11-06) has a ready-made PostgreSQL image, with `postgres` as the user, password and database by default:

```rust
// testcontainers (illustration): testcontainers-modules = { version = "0.15", features =
["postgres"] }
use testcontainers_modules::postgres::Postgres;
use testcontainers_modules::testcontainers::runners::AsyncRunner;

let container = Postgres::default().start().await?;
let port = container.get_host_port_ipv4(5432).await?;
let url = format!("postgres://postgres:postgres@127.0.0.1:{port}/postgres");
let (client, connection) = tokio_postgres::connect(&url, NoTls).await?;
// … the container stops when `container` is dropped
```

It needs nothing from the CI configuration but Docker, and each test binary can have a server of its own. The price is startup time, seconds per container rather than milliseconds per cloned database; combine the two (one container per test binary, a template database cloned per test) to get both isolation and speed.

17.7 Provoking every SQLSTATE the error mapping handles

Chapter 15's `AppError` turns SQLSTATEs into HTTP statuses. Each arm of that `match` is code, and code that never runs in a test is code you hope works. Each error is easy to provoke on purpose, and deterministic if the test drives the interleaving itself:

```
use std::time::Duration;
use tokio_postgres::Client;

type Error = Box<dyn std::error::Error>;

/// The SQLSTATE of the error `result` holds; panics if it succeeded.
fn code<T>(result: Result<T, tokio_postgres::Error>) -> String {
    match result {
        Ok(_) => panic!("expected an error"),
        Err(e) => e
            .code()
            .map(|c| c.code().to_owned())
            .unwrap_or_else(|| e.to_string()),
    }
}

async fn session(schema: &str) -> Result<Client, Error> {
    let client = playground::connect().await?;
    client
        .batch_execute(&format!("SET search_path = {schema}"))
        .await?;
    Ok(client)
}

#[tokio::main]
async fn main() -> Result<(), Error> {
    let schema = format!("ch17_codes_{}", std::process::id());
    let setup = playground::connect().await?;
    setup
        .batch_execute(&format!(
            "CREATE SCHEMA {schema};
             SET search_path = {schema};
             CREATE TABLE store (id int PRIMARY KEY);
             CREATE TABLE stock (
                 id    int PRIMARY KEY,
                 store int NOT NULL REFERENCES store,
                 units int NOT NULL CHECK (units >= 0)
             );
             INSERT INTO store VALUES (1);
             INSERT INTO stock VALUES (1, 1, 10), (2, 1, 10);"
        ))
        .await?;
    let (mut a, mut b) = (session(&schema).await?, session(&schema).await?);
    let mut seen = Vec::new();

    // Integrity: 23505, 23514, 23502, 23503.
    seen.push((
        "23505",
        code(a.execute("INSERT INTO stock VALUES (1, 1, 1)", &[]).await),
    ));
    seen.push((
        "23514",
        code(
            a.execute("UPDATE stock SET units = -1 WHERE id = 1", &[])
                .await,
        ),
    ));
    seen.push((
        "23502",
        code(
            a.execute("INSERT INTO stock VALUES (3, NULL, 1)", &[])
                .await,
```

```
        ),
    ));
    seen.push((
        "23503",
        code(a.execute("INSERT INTO stock VALUES (3, 9, 1)", &[]).await),
    ));

    // 55P03: a lock that is not available, without waiting.
    let hold = a.transaction().await?;
    hold.execute("SELECT * FROM stock WHERE id = 1 FOR UPDATE", &[])
        .await?;
    seen.push((
        "55P03",
        code(
            b.execute("SELECT * FROM stock WHERE id = 1 FOR UPDATE NOWAIT", &[])
                .await,
        ),
    ));
    hold.rollback().await?;

    // 57014: a statement that runs out of time.
    b.batch_execute("SET statement_timeout = '50ms'").await?;
    seen.push(("57014", code(b.execute("SELECT pg_sleep(1)", &[]).await)));
    b.batch_execute("SET statement_timeout = 0").await?;

    // 40001: write skew under SERIALIZABLE, interleaved step by step.
    {
        let serializable = tokio_postgres::IsolationLevel::Serializable;
        let ta = a
            .build_transaction()
            .isolation_level(serializable)
            .start()
            .await?;
        let tb = b
            .build_transaction()
            .isolation_level(serializable)
            .start()
            .await?;
        ta.query("SELECT sum(units) FROM stock", &[]).await?;
        tb.query("SELECT sum(units) FROM stock", &[]).await?;
        ta.execute("UPDATE stock SET units = units - 1 WHERE id = 1", &[])
            .await?;
        // The conflict is reported by the second UPDATE or by the second COMMIT.
        let second = match tb
            .execute("UPDATE stock SET units = units - 1 WHERE id = 2", &[])
            .await
        {
            Ok(_) => {
                ta.commit().await?;
                tb.commit().await
            }
            Err(e) => Err(e),
        };
        seen.push(("40001", code(second)));
    }

    // 40P01: two transactions that each wait for the other's row.
    {
        let ta = a.transaction().await?;
        let tb = b.transaction().await?;
        ta.execute("UPDATE stock SET units = units WHERE id = 1", &[])
            .await?;
        tb.execute("UPDATE stock SET units = units WHERE id = 2", &[])
            .await?;
        let (first, second) = tokio::join!(
            ta.execute("UPDATE stock SET units = units WHERE id = 2", &[]),
            async {
                tokio::time::sleep(Duration::from_millis(100)).await;
                tb.execute("UPDATE stock SET units = units WHERE id = 1", &[])
                    .await
```

```
            }
        );
        // PostgreSQL picks one victim, whose transaction aborts and releases its locks; the
        // other then gets the lock it was waiting for.
        seen.push(("40P01", code(first.and(second))));
    }

    for (expected, got) in &seen {
        println!("{expected}: {got}");
        assert_eq!(expected, got);
    }
    setup
        .batch_execute(&format!("DROP SCHEMA {schema} CASCADE"))
        .await?;
    Ok(())
}
```

```
23505: 23505
23514: 23514
23502: 23502
23503: 23503
55P03: 55P03
57014: 57014
40001: 40001
40P01: 40P01
```

The deadlock takes about a second: PostgreSQL looks for deadlocks only after a lock wait reaches `deadlock_timeout` (1 s by default). stockroom's tests provoke the same codes through the API, and check the statuses: 409 for the duplicate SKU, 422 with the field for the check violations, 412 for a stale `If-Match`, and 503 with `timeout` for the statement timeout (`tests/reports.rs`).

In stockroom

File	Tests	What it covers
`src/**` (unit)	10	SKU parsing and its serde, cents arithmetic, order-status transitions, migration checksums and order, cursors, retry backoff
`tests/types.rs`	5	domains, enums (and a new label), composites, arrays
`tests/migrations.rs`	5	idempotence, edited and unknown migrations, rollback, two instances at once
`tests/products.rs`	7	errors a client can cause, `If-Match`, pagination, savepoints, CSV in and out, NDJSON
`tests/orders.rs`	5	pricing and reservations, transitions, cancellation, 404
`tests/concurrency.rs`	3	overselling, deadlock-free ordering, transfers
`tests/events.rs`	6	`NOTIFY` on commit only, reconnect and resync, SSE, outbox, health
`tests/reports.rs`	3	the dashboard, cancellation of an abandoned report, statement timeout

All of them run against a database cloned from the migrated template, and leave nothing behind.

Summary

- Test against PostgreSQL itself: mocks cannot check constraint names, transactional behaviour or concurrency. Test mostly through the HTTP layer, in memory with `oneshot`.
- A database per test, cloned with `CREATE DATABASE … TEMPLATE` from a template migrated once (named after a checksum of the migrations), gives full isolation in milliseconds. Retry the clone on 55006 while sessions linger.
- `Drop` cannot await: clean up from a thread with a runtime of its own, and wait for it.
- A transaction per test is fast but blind to commit-time behaviour, `NOTIFY`, several connections and code that manages its own transactions.
- Concurrency tests run many tasks at once on a multi-threaded runtime and assert the total; repeat them, and force interleavings step by step where you can.
- In CI, use a service container with `ADMIN_DATABASE_URL`, or testcontainers for a server per test binary.
- Provoke every SQLSTATE the error mapping handles, and check the resulting statuses.

Exercises

1. **Template versus migrations.** Measure creating a test database by cloning stockroom's template, and by creating an empty one and running the migrations. Which grows with the size of the schema, and which with the size of the data in the template? *Hint:* time `CREATE DATABASE … TEMPLATE` and `CREATE DATABASE` plus `migrate::run`.
2. **Break the guard.** In stockroom's `orders::place`, replace the guarded reservation (`… AND available >= $3`) with a read-then-write: `SELECT available`, check it in Rust, then an unguarded `UPDATE`. Run `fifty_customers_twenty_chairs` a few times. What fails, and what keeps the data correct anyway? *Hint:* look at the `CHECK` constraints on `stock` in `0001_init.sql`.
3. **57P01.** Add a case to §17.7's example for `admin_shutdown`: a query that is running when its backend is terminated. What does the client see, and what does stockroom's `AppError` make of it? *Hint:* run `pg_sleep` in a spawned task, then `pg_terminate_backend` from another session.
4. **Fixtures through SQL.** stockroom creates fixtures through the API. When would inserting them with SQL be the better choice? *Hint:* think of a report over a year of data.

Solutions

1. Cloning copies files, so its time grows with the size of the template's data and is nearly independent of how many migrations built it; running migrations grows with the number and cost of the migrations, and is independent of data (an empty database has none). For stockroom both are fast, and the template wins by skipping the DDL; the gap widens as migrations accumulate.

2. At `READ COMMITTED`, two transactions can both read `available = 1` and both decide to reserve, and the unguarded `UPDATE`s then raise `reserved` above `on_hand`. The constraint `stock_reserved_within_on_hand` (`reserved <= on_hand`) refuses the second one with 23514, so no chair is sold twice; but `AppError` maps that check violation to 422, and the test, which accepts only 201 and 409, fails whenever two orders interleave that way, which fifty tasks on sixteen connections make likely. The guarded `UPDATE` makes

the check and the write one atomic statement, re-evaluated after any lock wait (Chapter 8), so the race cannot happen; the constraint is the safety net that turns the bug into a wrong status instead of oversold stock.

3. The running query fails with a `DbError` of severity `FATAL` and code 57P01 ("terminating connection due to administrator command"), and the connection is closed afterwards. stockroom maps `ADMIN_SHUTDOWN` to `AppError::Unavailable`, a 503, and the pool discards the closed connection on its next checkout:

```
use std::sync::Arc;
use std::time::Duration;
use tokio_postgres::error::SqlState;

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let victim = Arc::new(playground::connect().await?);
    let admin = playground::connect().await?;
    let pid: i32 = victim
        .query_one("SELECT pg_backend_pid()", &[])
        .await?
        .get(0);

    let running = {
        let victim = Arc::clone(&victim);
        tokio::spawn(async move { victim.execute("SELECT pg_sleep(5)", &[]).await })
    };
    tokio::time::sleep(Duration::from_millis(100)).await;
    admin
        .execute("SELECT pg_terminate_backend($1)", &[&pid])
        .await?;

    let err = running.await?.unwrap_err();
    let db = err.as_db_error().expect("a server error");
    println!("{} {}: {}", db.severity(), db.code().code(), db.message());
    assert_eq!(db.code(), &SqlState::ADMIN_SHUTDOWN);
    Ok(())
}
```

4. When the API would make the fixture slow or impossible to build: a report test needs a year of orders with dates in the past, which no endpoint creates (`POST /orders` stamps the current time). Insert those rows with SQL, or binary `COPY` as `stockroom seed` does, and keep creating through the API the fixtures that the API can create.

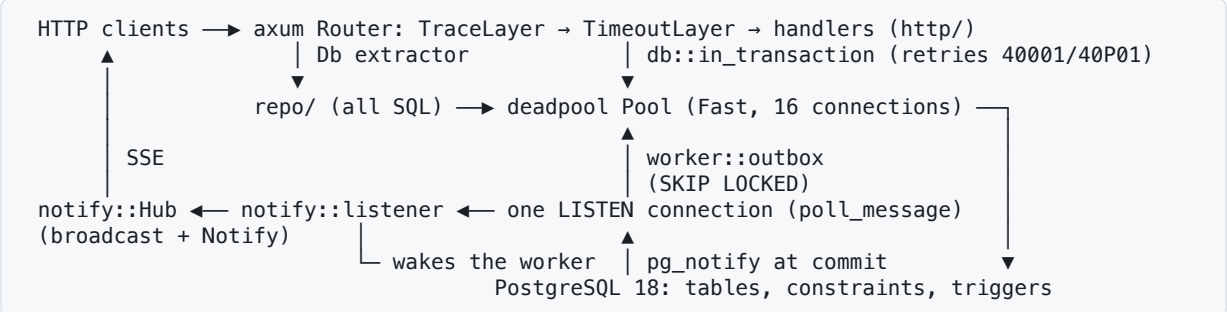
CHAPTER 18

Capstone: stockroom end to end

stockroom has appeared in pieces in every chapter: a migration here, a repository function there, a handler, a guard, a worker. This chapter puts it back together. It shows the whole shape of the service, follows one order from the HTTP request to the email it triggers, maps every tokio-postgres feature the book covered to the file that uses it, and runs the service for real, with the output of an actual session. It ends with what a production deployment would add, and with exercises that extend the project.

stockroom is a click-and-collect backend for a small retail chain: stores hold stock of products, customers order for pickup at a store, and staff move orders from *pending* through *paid* and *ready* to *collected*. About 3,900 lines of Rust and two SQL migrations.

18.1 The shape



Directory	What it holds	Chapters
`migrations/`	the schema: domains, enum, composite, constraints with names, triggers	5, 11, 14
`src/db/`	connecting, the pool, migrations, the retry helper, `CancelOnDrop`, TLS	1, 8, 12, 13, 14
`src/domain/`	`Sku`, `Cents`, `Quantity`, `OrderStatus`, `Address`, `Customer`	3, 5, 14
`src/repo/`	every SQL statement, one module per aggregate	2–10, 14
`src/http/`	the router, extractors and handlers	15, 16
`src/error.rs`	`AppError`: SQLSTATE and constraint names → HTTP	6, 15
`src/notify/`	the `LISTEN` connection and the in-process hub	11, 16
`src/worker/`	the outbox worker	16
`src/cli/`	`seed` (binary `COPY`) and `export` (portal, binary `COPY` out)	9, 10

Directory	What it holds	Chapters
`tests/`	34 tests against PostgreSQL, a database each	17

Three connection roles, three rules. **Pooled connections** serve requests and the worker, and never keep session state (Chapter 13). **One dedicated connection** listens, driven with `poll_message` and reconnected with backoff (Chapter 11). **Short-lived connections** run migrations, `seed` and `export`, with their notices logged (Chapters 10, 14).

18.2 A request's life

Follow `POST /orders` for two units of `SKU00001` at `store-01`, as it happened in the session of §18.4.

1. The router. `TraceLayer` opens a span; `TimeoutLayer` starts the 10-second clock. The `Db` extractor borrows a connection from the pool (or answers 503 after 5 seconds of waiting); `AppJson<NewOrder>` deserializes the body, and `Sku` and `Quantity` validate themselves on the way (Chapters 14, 15).

2. The transaction. The handler calls `db::in_transaction` at `READ COMMITTED` with an `async move` closure that owns the order (Chapters 8, 15). Inside, `repo::orders::place`:

- merges repeated SKUs and checks the line count;
- looks up the store and the products in **one round trip**: two queries pipelined with `try_join!` (Chapter 7), the products with `sku = ANY($1)`, where `$1` is `text[]` because a domain compares as its base type (Chapter 5);
- sorts the lines by product id, so that every order takes its stock locks in the same order and two orders can wait for each other but never deadlock (Chapter 8);
- reserves every line with a **guarded** `UPDATE`, … `SET reserved = reserved + $3 WHERE … AND available >= $3`, all of them pipelined in order with `try_join_all`; a line that updates 0 rows means not enough stock, answered with 409 and the available quantity (Chapters 7, 8);
- inserts the order, with the customer as `Json<Customer>` into `jsonb` (Chapter 3), and all its lines in one statement with `unnest` over three arrays (Chapter 10).

```rust
// stockroom/src/repo/orders.rs (excerpt)
// The same lock order in every transaction: orders sharing products queue, never deadlock.
lines.sort_by_key(|line| line.product_id);
// …
// Every reservation at once: pipelined, sent in this order, run one by one by the server.
let reserve = tx
    .prepare_cached(
        "UPDATE stock SET reserved = reserved + $3
         WHERE store_id = $1 AND product_id = $2 AND available >= $3",
    )
    .await?;
```

stockroom/src/repo/orders.rs

3. The trigger. The `INSERT INTO orders` fires `announce_order_event()`, in the same transaction: it queues a `pg_notify('order_events', …)` with a small JSON event, inserts a job row into `outbox`, and queues a payload-less `pg_notify('outbox', '')` (Chapter 11). None of it is visible to anyone yet.

4. The commit. `in_transaction` commits. If the commit or anything before it had failed with a serialization failure or a deadlock, it would have rolled back and tried again, up to eight attempts in all, with jittered backoff (Chapter 8). At the commit, PostgreSQL delivers the two notifications, and the order, its reservations and the outbox job become visible together. The handler answers 201 with `ETag: "1"`.

5. The listener. The dedicated connection's driver task receives both notifications through `poll_message` (Chapter 11). `order_events` becomes `HubEvent::Order` on the broadcast channel; `outbox` wakes the worker through a `Notify`.

6. The event stream. Every open `GET /orders/events` response (filtered by store) receives the event from its broadcast subscription and writes it as an SSE `event: order` (Chapter 16). None of them holds a database connection.

7. The outbox worker. Woken, it borrows a pooled connection, claims due jobs with `FOR UPDATE SKIP LOCKED`, delivers them (stockroom's delivery logs a line, standing in for an email), marks them done, and commits. A crash in the middle would leave the jobs due, for the next worker to deliver again: at least once (Chapter 16). In the session below, the job was delivered 16 ms after the order was created.

Then a member of staff marks the order paid, with `If-Match: "1"`; the update checks the version in its `WHERE` clause (Chapters 8, 15), the status trigger announces the change, and the whole chain runs again.

18.3 The feature map

Every public item of tokio-postgres 0.7.18 that the book teaches, with the chapter that teaches it and where stockroom or an app uses it. (PLAN.md §5.3 has the same matrix, which the book was planned from.)

API	Chapter	Used in
`connect`, `NoTls`, spawning the `Connection`	1	`db/connect.rs`, `tests/common`
`Config` (URL and key/value forms, setters)	1, 12	`config.rs`
`Connection::parameter`	1	`notify/listener.rs` → `/admin/sessions`
`is_closed`, `check_connection`	1, 12, 13	`http/health.rs` (`/readyz`), pglisten
`batch_execute`	2	`db/migrate.rs`, staging tables
`execute`, `query`, `query_one`, `query_opt`	2	`repo/`
`query_scalar`, `query_one_scalar`, `query_opt_scalar`	2	`repo/reports.rs` (dashboard)
`simple_query`, `SimpleQueryMessage`, `SimpleQueryRow`	2	pgq, `repo/reports.rs` (`sessions`)
`Row` (`get`, `try_get`), `Column`	3	`TryFrom<&Row>` in `repo/`
`prepare`, `Statement` (`prepare_typed` in Chapter 4 only)	4	pgq `--describe`, `cli/export.rs`
`query_typed` and family	4	minipool's benchmark
`ToSql`, `FromSql`, derives, `Json<T>`, feature flags	3, 5	`domain/`

API	Chapter	Used in
`clear_type_cache`	5	`db/migrate.rs`
`Error`, `DbError`, `SqlState`	6	`error.rs`
pipelining (`join!`, `try_join!`, `try_join_all`)	7	`repo/orders.rs`, `repo/reports.rs`
`transaction`, `build_transaction`, `IsolationLevel`, savepoints	8	`db/retry.rs`, `http/products.rs` (batch), `cli/export.rs`
`GenericClient`	8, 14	`repo/` (deadpool's)
`query_raw`, `RowStream`, `rows_affected`	9	`http/products.rs` (`/products/stream`)
`bind`, `query_portal`, `Portal`	9	`cli/export.rs`
`simple_query_raw`	9	pgq `--stream`
`copy_in`, `CopyInSink`, `copy_out`, `CopyOutStream`	10	`repo/products.rs` (import), `http/products.rs` (export)
`binary_copy`	10	`cli/seed.rs`, `cli/export.rs`, pgload
`poll_message`, `AsyncMessage`, `Notification`	11	`notify/listener.rs`, `db/connect.rs`, pglisten
`cancel_token`, `CancelToken`	12	`db/cancel.rs`
timeouts, keepalives, `options`	12	`config.rs`
`MakeTlsConnect`, tokio-postgres-rustls	12	`db/tls.rs` (`--features tls`)
`connect_raw`	12	lagproxy users: pgload, minipool
deadpool-postgres (`Pool`, `prepare_cached`, `Object::take`)	13	`db/pool.rs`, `db/cancel.rs`

18.4 Running it

With the database of Chapter 0 and `STOCKROOM_DATABASE_URL` set in `.env`:

```
cargo run -p stockroom -- migrate
cargo run -p stockroom -- seed --reset
cargo run -p stockroom -- serve
```

`migrate` on an already migrated database logs the server's notice and does nothing; `seed` fills the tables with binary `COPY`, deterministically (the same `--seed` gives the same data):

```
INFO stockroom::db::connect: relation "schema_migrations" already exists, skipping
severity="NOTICE" code="42P07"
the schema is up to date

seeded 3 stores, 1000 products, 2995 stock rows, 20000 movements
```

`serve` listens on `127.0.0.1:8080`, migrating first unless told not to, and starts the listener and the worker:

```
INFO stockroom::app: listening on http://127.0.0.1:8080
INFO stockroom::notify::listener: listening for order events
```

The session below was run with curl; `stockroom/stockroom.http` has the same requests, and more, for an editor's REST client. Readiness first: the pool has opened one connection of its 16, and the listener is connected:

```
curl -s localhost:8080/readyz
```

```
{"listening":true,"pool":{"available":0,"maxSize":16,"size":1,"waiting":0},"status":"ready"}
```

In another terminal, a client subscribes to the store's events:

```
curl -N 'localhost:8080/orders/events?store=store-01'
```

An order for two units of `SKU00001`, of which `store-01` has 50 available:

```
curl -s -i -X POST localhost:8080/orders -H 'content-type: application/json' \
  -d '{"store":"store-01","customer":{"name":"Ada","email":"ada@example.com"},"lines":
[{"sku":"SKU00001","quantity":2}]}'
```

```
HTTP/1.1 201 Created
content-type: application/json
etag: "1"
content-length: 316
date: Fri, 06 Nov 2026 20:01:04 GMT

{"id":"01a1b273-50f5-71ea-8e39-
c645892c55f3","store":"store-01","status":"pending","customer":
{"name":"Ada","email":"ada@example.com"},"totalCents":158200,"version":1,"createdAt":"2026-11-06T20:01:04.242
[{"sku":"SKU00001","quantity":2,"unitPriceCents":79100}]}
```

(The id is a UUID version 7, from `uuidv7()` in the schema: its first digits are a timestamp, so ids sort by creation time.) Marking it paid, with the version from the `ETag`, and then trying again with the same, now stale, version:

```
curl -s -i -X POST localhost:8080/orders/$ID/transition -H 'content-type: application/json'
\
  -H 'If-Match: "1"' -d '{"to":"paid"}'
curl -s -i -X POST localhost:8080/orders/$ID/transition -H 'content-type: application/json'
\
  -H 'If-Match: "1"' -d '{"to":"ready"}'
```

```
HTTP/1.1 200 OK
etag: "2"
{"previous":"pending","order":{"id":"01a1b273-50f5-71ea-8e39-
c645892c55f3","store":"store-01","status":"paid", … ,"version":2, … }}
HTTP/1.1 412 Precondition Failed
{"currentVersion":2,"error":"stale_version","message":"the resource has changed: its current
version is 2"}
```

The stock shows the reservation, and the event stream received both changes:

```
[{'sku': 'SKU00001', 'onHand': 50, 'reserved': 2, 'available': 48}]
```

```
event: order
id: 01a1b273-50f5-71ea-8e39-c645892c55f3:1
data: {"orderId":"01a1b273-50f5-71ea-8e39-c645892c55f3","storeId":"01a1b273-085c-7487-
b7e8-500106fafe63","status":"pending","previous":null,"version":1}

event: order
id: 01a1b273-50f5-71ea-8e39-c645892c55f3:2
data: {"orderId":"01a1b273-50f5-71ea-8e39-c645892c55f3","storeId":"01a1b273-085c-7487-
b7e8-500106fafe63","status":"paid","previous":"pending","version":2}
```

The server's log shows the outbox worker delivering each job (16 ms after the order's `createdAt`, for the first):

```
INFO stockroom::worker::outbox: delivered topic="order.pending"
payload={"orderId":"01a1b273-50f5-71ea-8e39-
c645892c55f3","previous":null,"status":"pending","storeId":"01a1b273-085c-7487-
b7e8-500106fafe63","version":1}
INFO stockroom::worker::outbox: delivered topic="order.paid"
payload={"orderId":"01a1b273-50f5-71ea-8e39-
c645892c55f3","previous":"pending","status":"paid","storeId":"01a1b273-085c-7487-
b7e8-500106fafe63","version":2}
```

And the pipelined dashboard (abridged):

```
{"products": 1000, "stores": 3, "openOrders": 1, "ordersToday": 1, "revenueTodayCents":
158200,
 "averageOrderCents": "158200.00", "lowStock": [{"store": "store-02", "sku": "SKU00447",
"available": 3}, …]}
```

Finally, SIGTERM, with the event stream still connected. The stream ended with the shutdown token, the server finished in-flight requests, the worker and listener returned, and the pool closed:

```
INFO stockroom::app: shutting down
INFO stockroom::app: shut down
```

The process exited 1.5 ms after the signal (measured on this machine, localhost).

Over TLS, against the `docker compose` server of Chapter 12:

```
cargo run -p stockroom --features tls -- --db-ca-file docker/certs/ca.crt migrate
```

18.5 What production would add

stockroom is complete as a teaching service, and deliberately stops short of a few things a production deployment would need:

- **Metrics.** The pool's `status()` (size, available, waiting), query durations from the `tracing` spans every repository function already opens, the outbox's pending count (`repo::outbox::pending`), and `pg_notification_queue_usage()`. Export them with an OpenTelemetry or Prometheus layer; alert on pool waiters and outbox age.
- **Read replicas.** Reports and the catalog can read from a replica: a second pool whose `Config` lists the replicas with `target_session_attrs=read-only` and `load_balance_hosts=random` (Chapter 12), and repository functions that say which pool they need. Mind replication lag: a client that just wrote must read from the primary.
- **Partitioning.** `stock_movements` grows forever. Partition it by month (`PARTITION BY RANGE (at)`), so that old months can be detached and archived, and queries over recent movements touch only recent partitions.
- **PgBouncer.** Many service instances, each with a pool of 16, exhaust `max_connections`. PgBouncer in transaction mode multiplexes them onto fewer server connections; stockroom already keeps no session state between transactions, and `prepare_cached` works with PgBouncer 1.21+ (Chapter

13). The `LISTEN` connection must bypass it: in transaction mode a session does not survive between transactions.

- **Backups and migrations as a deployment step**, with `--migrate false` on the instances, and migrations written to be compatible with the previous release (Chapter 14).
- **Delivery for real.** `LogDelivery` stands in for an email service; a real `Deliver` implementation needs timeouts shorter than the pool's patience, idempotency keys (the job id), and a dead-letter state after too many attempts.

Summary

- stockroom combines three kinds of connections: pooled (requests and the worker, no session state), one dedicated listener, and short-lived ones for the command-line tools.
- An order is one retried transaction: pipelined lookups, sorted locks, guarded pipelined reservations, `unnest` inserts. A trigger in the same transaction queues the notifications and the outbox job, which the commit releases together.
- After the commit, the listener fans the event out to SSE clients through a broadcast channel, and wakes the outbox worker, which delivers at least once with `SKIP LOCKED`.
- Every tokio-postgres feature the book teaches is used by stockroom or one of the apps; §18.3 maps each to its file.
- Production adds metrics, replicas, partitioning, an external pooler, a migration step and real delivery, all on the foundations already in place.

Exercises

These extend stockroom itself. Each needs a migration, repository code, a handler or a task, and tests in `tests/`.

1. **Pickup deadlines.** Orders that stay `ready` for more than three days should be cancelled and their stock released. Add a background task that does it every minute, safely with several instances running. *Hint:* `UPDATE … WHERE status = 'ready' AND updated_at < now() - interval '3 days'` with `FOR UPDATE SKIP LOCKED` in a subquery, through the existing transition logic so that the trigger announces each cancellation.
2. **A low-stock alert.** When a reservation brings a product's available stock below 5, publish a `low_stock` event to SSE clients, at most once per product per hour. *Hint:* a trigger on `stock`, and a table that remembers when each alert was last sent.
3. **Idempotent orders.** Mobile clients retry `POST /orders` when the network drops, creating duplicate orders. Accept an `Idempotency-Key` header and return the original order for a repeated key. *Hint:* a unique column for the key, `INSERT … ON CONFLICT DO NOTHING RETURNING`, and the same transaction as the order.

4. **A read replica for reports.** Add a second, optional pool for `/reports/*`, configured with `--replica-url`, falling back to the primary pool when it is not set. How does `/readyz` change? *Hint:* `AppState` gets an `Option<Pool>`; the readiness check should report both pools.

Solutions

1. Claim the expired orders in batches inside a transaction: `SELECT id FROM orders WHERE status = 'ready' AND updated_at < now() - interval '3 days' ORDER BY id LIMIT 100 FOR UPDATE SKIP LOCKED`, then run `repo::orders::transition(tx, id, Cancelled, None)` for each, which releases the reservation and fires the status trigger. `SKIP LOCKED` lets several instances run the task at once without waiting for each other or cancelling an order twice, and a staff member's concurrent transition of the same order is serialized by the row lock. Test it by creating a `ready` order, moving its `updated_at` back with SQL (§17's fixtures through SQL), running one pass, and checking the stock levels and the event on the hub.

2. An `AFTER UPDATE OF reserved, on_hand ON stock` trigger compares the old and new `available` values; when the new one crosses below 5, it inserts or updates a row in a `stock_alerts (store_id, product_id, sent_at)` table with `ON CONFLICT … DO UPDATE … WHERE stock_alerts.sent_at < now() - interval '1 hour'`, and calls `pg_notify('stock_events', …)` only when that statement affected a row. The listener gains a channel and a `HubEvent::LowStock`; the SSE handler a second event type. The hour-long throttle lives in the database, so it holds across instances and restarts.

3. Add `idempotency_key text UNIQUE` to `orders` (nullable, so that keyless orders are unaffected). In `place`, before reserving anything, try `SELECT id FROM orders WHERE idempotency_key = $1`: if found, return that order with 200. Otherwise proceed, inserting the key with the order; a concurrent request with the same key then fails on the unique constraint at the insert, and its transaction rolls back, reservations included, so the handler can catch that specific constraint (`orders_idempotency_key_key`) and return the winner's order. The key must be scoped to the customer in a real system, and expire.

4. `AppState { pool, reports: Option<Pool>, … }`, and the report handlers use `state.reports.as_ref().unwrap_or(&state.pool)`. The replica pool's `Config` comes from `--replica-url` with `target_session_attrs=read-only`. `/readyz` should check the primary pool as now and report the replica pool's status separately, without failing readiness when only the replica is down, since reports can fall back to the primary; whether they should, under load, is an operational decision worth a flag.

Appendix — cheat sheets, phrasebook, glossary and ecosystem

Reference material, for looking things up rather than reading through: every method of `Client` and `Transaction` with its return type, the type-mapping table, a SQLSTATE table with suggested HTTP statuses, a phrasebook for readers coming from sqlx or diesel, a glossary, the crates built on tokio-postgres, and where to read further. Everything here is for tokio-postgres 0.7.18, postgres-types 0.2.14 and deadpool-postgres 0.14.2, taken from their source and documentation on 2026-11-06.

A API cheat sheet

`Client`

All methods take `&self` except the two that start a transaction. "Text or statement" means any `T: ToStatement`: a `&str` (prepared on the fly, two round trips, Chapter 4) or a `&Statement`. `params` is `&[&(dyn ToSql + Sync)]`, and typed params are `&[(&(dyn ToSql + Sync), Type)]`.

Method	Takes	Returns	Chapter
`batch_execute`	SQL text, any number of statements	`Result<(), Error>`	2
`simple_query`	SQL text	`Result<Vec<SimpleQueryMessage>, Error>`	2
`simple_query_raw`	SQL text	`Result<SimpleQueryStream, Error>`	9
`execute`	text or statement, params	`Result<u64, Error>` (rows affected)	2
`query`	text or statement, params	`Result<Vec<Row>, Error>`	2
`query_one`	text or statement, params	`Result<Row, Error>` (error unless exactly one row)	2
`query_opt`	text or statement, params	`Result<Option<Row>, Error>` (error if more than one)	2
`query_scalar` (0.7.16+)	text or statement, params	`Result<Vec<R>, Error>`, `R: FromSqlOwned`	2
`query_one_scalar` (0.7.16+)	text or statement, params	`Result<R, Error>`	2
`query_opt_scalar` (0.7.16+)	text or statement, params	`Result<Option<R>, Error>`	2
`query_raw`	text or statement, iterator of `BorrowToSql`	`Result<RowStream, Error>`	9

Method	Takes	Returns	Chapter
`execute_raw`	text or statement, iterator	`Result<u64, Error>`	9
`prepare`	SQL text	`Result<Statement, Error>`	4
`prepare_typed`	SQL text, `&[Type]`	`Result<Statement, Error>`	4
`query_typed` (0.7.11+)	SQL text, typed params	`Result<Vec<Row>, Error>` (one round trip)	4
`query_typed_one`, `query_typed_opt` (0.7.17+)	SQL text, typed params	`Result<Row, Error>`, `Result<Option<Row>, Error>`	4
`query_typed_raw`	SQL text, iterator of `(value, Type)`	`Result<RowStream, Error>`	4, 9
`execute_typed` (0.7.17+)	SQL text, typed params	`Result<u64, Error>`	4
`copy_in`	a `COPY … FROM STDIN` statement	`Result<CopyInSink<U>, Error>`, `U: Buf`	10
`copy_out`	a `COPY … TO STDOUT` statement	`Result<CopyOutStream, Error>`	10
`transaction`	`&mut self`	`Result<Transaction<'_>, Error>`	8
`build_transaction`	`&mut self`	`TransactionBuilder<'_>`	8
`cancel_token`		`CancelToken`	12
`check_connection` (0.7.15+)		`Result<(), Error>` (one round trip, no statement)	12
`is_closed`		`bool`	1, 12
`clear_type_cache`		`()`	5
`cancel_query`, `cancel_query_raw`		deprecated since 0.6.0: use `cancel_token()`	12

`Transaction<'_>`

The same query methods as `Client` (`batch_execute`, `simple_query`, `execute`, `query`, `query_one`, `query_opt`, `query_raw`, `execute_raw`, `prepare`, `prepare_typed`, the `query_typed` family, `execute_typed`, `copy_in`, `copy_out`, `cancel_token`), **without** the `query_*_scalar` methods and `simple_query_raw`, plus:

Method	Returns	Chapter
`commit(self)`	`Result<(), Error>`	8
`rollback(self)`	`Result<(), Error>`; dropping rolls back too, but loses the rollback's error	8
`transaction(&mut self)`	`Result<Transaction<'_>, Error>`: a savepoint with a generated name	8
`savepoint(&mut self, name)`	`Result<Transaction<'_>, Error>`: a named savepoint	8
`bind(&statement, params)`, `bind_raw`	`Result<Portal, Error>`	9
`query_portal(&portal, max_rows)`	`Result<Vec<Row>, Error>`	9
`query_portal_raw(&portal, max_rows)`	`Result<RowStream, Error>`	9
`client()`	`&Client`	8

`TransactionBuilder`: `isolation_level(IsolationLevel)`, `read_only(bool)`, `deferrable(bool)`, `start() -> Result<Transaction<'_>, Error>`. `IsolationLevel`: `ReadUncommitted`, `ReadCommitted`, `RepeatableRead`, `Serializable`.

`GenericClient`

Sealed, `#[async_trait]`, implemented for `Client` and `Transaction<'_>`: `execute`, `execute_raw`, `execute_typed`, `query`, `query_one`, `query_opt`, `query_raw`, `query_typed`, `query_typed_one`, `query_typed_opt`, `query_typed_raw`, `prepare`, `prepare_typed`, `transaction`, `batch_execute`, `simple_query`, `client`. deadpool-postgres has its own `GenericClient`, also sealed and `#[async_trait]`, implemented for `Object` and deadpool's `Transaction`, which adds `prepare_cached` and `prepare_typed_cached` (Chapters 13, 14).

The rest of the crate

Item	What it is	Chapter
`connect(url, tls)`	`Result<(Client, Connection<Socket, T::Stream>), Error>`	1
`Config`	parsed from a URL or key/value string; builder setters; `connect`, `connect_raw`	1, 12
`Connection`	a `Future<Output = Result<(), Error>>`; `poll_message`, `parameter(name)`	1, 11
`AsyncMessage`	`Notice(DbError)`, `Notification(Notification)`; non-exhaustive	11
`Row`	`get`, `try_get` (by index or name), `columns`, `len`, `is_empty`, `raw_size_bytes`	3
`Statement`	`params() -> &[Type]`, `columns() -> &[Column]`	4
`RowStream`	`Stream<Item = Result<Row, Error>>`, `!Unpin`; `rows_affected()` after the end	9
`SimpleQueryMessage`	`Row`, `CommandComplete(u64)`, `RowDescription` (0.7.11+); non-exhaustive	2
`CopyInSink<T>`	`Sink<T>`; `finish()` returns the row count	10
`CopyOutStream`	`Stream<Item = Result<Bytes, Error>>`, one chunk per row	10
`binary_copy::{BinaryCopyInWriter, BinaryCopyOutStream, BinaryCopyOutRow}`	binary `COPY` with declared types	10
`CancelToken`	`cancel_query(tls)`, `cancel_query_raw(stream, tls)`	12
`error::{Error, DbError, SqlState, Severity, ErrorPosition}`	Chapter 6's error model	6
`tls::{MakeTlsConnect, TlsConnect, TlsStream, NoTls}`	the TLS connector traits	12
`types`	re-export of postgres-types: `ToSql`, `FromSql`, `Type`, `Kind`, `Json`, …	3, 5

B Types

Built in

From the documentation of `ToSql` and `FromSql` in postgres-types 0.2.14:

Rust type	PostgreSQL type(s)
`bool`	`BOOL`
`i8`	`"char"`
`i16`	`SMALLINT`, `SMALLSERIAL`
`i32`	`INT`, `SERIAL`
`u32`	`OID`
`i64`	`BIGINT`, `BIGSERIAL`
`f32`	`REAL`
`f64`	`DOUBLE PRECISION`
`&str`, `String`	`VARCHAR`, `CHAR(n)`, `TEXT`, `CITEXT`, `NAME`, `UNKNOWN`, `LTREE`, `LQUERY`, `LTXTQUERY`
`&[u8]`, `Vec<u8>` (and `[u8; N]` for `ToSql`)	`BYTEA`
`HashMap<String, Option<String>>`	`HSTORE`
`SystemTime`	`TIMESTAMP`, `TIMESTAMP WITH TIME ZONE`
`IpAddr`	`INET`
`Option<T>`	any of the above, nullable
`Vec<T>`, `Box<[T]>`, and `&[T]` for `ToSql`; `[T; N]` with the `array-impls` feature	one-dimensional arrays
`Json<T>` (`T: Serialize/Deserialize`)	`JSON`, `JSONB` (needs `with-serde_json-1`)

There is no built-in mapping for `numeric` (use `rust_decimal`'s `db-tokio-postgres` feature, Chapter 3), `interval`, or `money`. Read them as text (`::text`) or with a crate that provides one, or implement `FromSql` yourself (Chapter 5).

Behind features

Enable these on tokio-postgres (it forwards them to postgres-types). The book uses `with-jiff-0_2`, `with-serde_json-1`, `with-uuid-1` and `array-impls`.

Feature	Rust type(s)	PostgreSQL type(s)
`with-chrono-0_4`	`chrono::NaiveDateTime`, `DateTime<Utc | Local | FixedOffset>`, `NaiveDate`, `NaiveTime`	`TIMESTAMP`, `TIMESTAMPTZ`, `DATE`, `TIME`
`with-time-0_2`, `with-time-0_3`	`time::PrimitiveDateTime`, `OffsetDateTime`, `Date`, `Time`	`TIMESTAMP`, `TIMESTAMPTZ`, `DATE`, `TIME`
`with-jiff-0_1`, `with-jiff-0_2`	`jiff::civil::DateTime`, `jiff::Timestamp`, `jiff::civil::Date`, `jiff::civil::Time`	`TIMESTAMP`, `TIMESTAMPTZ`, `DATE`, `TIME`
`with-uuid-0_8`, `with-uuid-1`	`uuid::Uuid`	`UUID`

Feature	Rust type(s)	PostgreSQL type(s)
`with-serde_json-1`	`serde_json::Value`, `Json<T>`	`JSON`, `JSONB`
`with-cidr-0_2`, `with-cidr-0_3`	`cidr::IpCidr`, `cidr::IpInet`	`CIDR`, `INET`
`with-eui48-0_4`, `with-eui48-1`	`eui48::MacAddress`	`MACADDR`
`with-geo-types-0_6`, `with-geo-types-0_7`	`geo_types::Point<f64>`, `Rect<f64>`, `LineString<f64>`	`POINT`, `BOX`, `PATH`
`with-bit-vec-0_6` … `with-bit-vec-0_9`	`bit_vec::BitVec`	`BIT`, `VARBIT`
`with-smol_str-01`	`smol_str::SmolStr`	the text types
`array-impls`	Rust arrays `[T; N]` (and `[u8; N]` for `ToSql`)	arrays (`BYTEA` for `[u8; N]`)

Derived (postgres-types `derive` feature, Chapter 5)

Attribute	For
`#[derive(ToSql, FromSql)]` on an enum	a PostgreSQL `ENUM`; `#[postgres(name = "…")]` on the type and variants; `rename_all = "…"`; `allow_mismatch` to tolerate labels on either side
on a struct with named fields	a composite type, field by field
on a newtype with `#[postgres(transparent)]`	the inner type, exactly
on a newtype without `transparent`	a domain over the inner type, matched by name

C SQLSTATE cheat sheet

The codes a service meets most, the `SqlState` constant to match in Rust (Chapter 6), and a suggested HTTP status (Chapter 15). "Retry" means the whole transaction may succeed if run again.

SQLSTATE	`SqlState::…`	Meaning	Suggested status
23505	`UNIQUE_VIOLATION`	duplicate key	409
23503	`FOREIGN_KEY_VIOLATION`	missing or still referenced row	422 (or 409 on delete)
23502	`NOT_NULL_VIOLATION`	a required value is missing	422
23514	`CHECK_VIOLATION`	a `CHECK` constraint (domains included) failed	422
23P01	`EXCLUSION_VIOLATION`	an exclusion constraint (overlapping ranges) failed	409
22001	`STRING_DATA_RIGHT_TRUNCATION`	value too long for `varchar(n)`	422
22003	`NUMERIC_VALUE_OUT_OF_RANGE`	a number overflowed its type	422

SQLSTATE	`SqlState::…`	Meaning	Suggested status
22P02	`INVALID_TEXT_REPRESENTATION`	text that does not convert (`'abc'::int`)	422
22007, 22008	`INVALID_DATETIME_FORMAT`, `DATETIME_FIELD_OVERFLOW`	a bad date or time	422
22P04	`BAD_COPY_FILE_FORMAT`	malformed `COPY` data	422, with `where_()`
21000	`CARDINALITY_VIOLATION`	one row changed twice by `ON CONFLICT/MERGE`	422
40001	`T_R_SERIALIZATION_FAILURE`	serialization failure	retry, then 503 + `Retry-After`
40P01	`T_R_DEADLOCK_DETECTED`	deadlock	retry, then 503 + `Retry-After`
55P03	`LOCK_NOT_AVAILABLE`	`NOWAIT` or `lock_timeout`	retry, then 503
57014	`QUERY_CANCELED`	`statement_timeout` or a cancel request	503 (or 504)
25P02	`IN_FAILED_SQL_TRANSACTION`	a statement after an error in the transaction	500 (a bug)
25001	`ACTIVE_SQL_TRANSACTION`	e.g. `CREATE INDEX CONCURRENTLY` inside a transaction	500
25P01	`NO_ACTIVE_SQL_TRANSACTION`	e.g. `DECLARE` outside a transaction	500
25P03	`IDLE_IN_TRANSACTION_SESSION_TIMEOUT`	session ended (FATAL)	503
25P04	(no constant in 0.7.18)	`transaction_timeout` (PG 17+), session ended	503
34000	`INVALID_CURSOR_NAME`	a portal or cursor that no longer exists	500
0A000	`FEATURE_NOT_SUPPORTED`	includes "cached plan must not change result type"	500
42601, 42703, 42P01, 42883, 42804	`SYNTAX_ERROR`, `UNDEFINED_COLUMN`, `UNDEFINED_TABLE`, `UNDEFINED_FUNCTION`, `DATATYPE_MISMATCH`	the SQL is wrong	500 (a bug)
42501	`INSUFFICIENT_PRIVILEGE`	missing grant	500 (a deployment bug)
53300	`TOO_MANY_CONNECTIONS`	`max_connections` reached	503
57P01, 57P02, 57P03	`ADMIN_SHUTDOWN`, `CRASH_SHUTDOWN`, `CANNOT_CONNECT_NOW`	the server is going away or starting	503
08000, 08006	`CONNECTION_EXCEPTION`, `CONNECTION_FAILURE`	connection problems	503

SQLSTATE	`SqlState::…`	Meaning	Suggested status
08P01	`PROTOCOL_VIOLATION`	e.g. a binary `COPY` with the wrong type size	422 for `COPY` input, else 500
28000, 28P01	`INVALID_AUTHORIZATION_SPECIFICATION`, `INVALID_PASSWORD`	rejected by `pg_hba.conf`, or a wrong password	at startup: fail fast
55006	`OBJECT_IN_USE`	e.g. cloning a template that has sessions	retry briefly
P0001	`RAISE_EXCEPTION`	`RAISE EXCEPTION` in PL/pgSQL	depends on the function

A connection that closed, or an error from the client side (a conversion), has no SQLSTATE at all: `Error::code()` returns `None`, and `Error::is_closed()` tells the first case apart.

D Phrasebook: from sqlx and diesel

How common tasks read in each library. The sqlx column matches the illustrations in the chapters; the diesel column shows diesel's query builder, with diesel-async for asynchronous code (§F covers how diesel-async sits on tokio-postgres).

Task	sqlx	tokio-postgres	Chapter
Connect	`PgPool::connect(url)`	`tokio_postgres::connect(url, tls)` + spawn the `Connection`	1
A pool	`PgPoolOptions::new().max_connections(8)`	deadpool-postgres `Pool::builder(manager).max_size(8)`	13
Run DDL	`sqlx::raw_sql(sql).execute(&pool)`	`client.batch_execute(sql)`	2
Insert, count rows	`sqlx::query(sql).bind(a).execute(&pool)` → `rows_affected()`	`client.execute(sql, &[&a])` → `u64`	2
One row into a struct	`query_as::<_, T>(sql).bind(a).fetch_one(&pool)` with `#[derive(FromRow)]`	`client.query_one(sql, &[&a])` + `TryFrom<&Row>`	2, 3
Zero or one row	`fetch_optional`	`query_opt`	2
A single value	`query_scalar(sql).fetch_one`	`query_one_scalar` (0.7.16+), or `query_one(…)?.get(0)`	2
Stream rows	`.fetch(&pool)`	`query_raw` → `RowStream`	9
Compile-time checked SQL	`query!`, `query_as!` macros	no macros; Cornucopia generates code ahead of time (§F)	4, 14
Prepared statements	cached implicitly per connection	explicit `prepare`, or deadpool's `prepare_cached`	4, 13
Concurrent queries	a pool: one connection per query in flight	pipelining on one connection (`join!`)	7
Transaction	`pool.begin()`, `tx.commit()`	`client.transaction()`, `tx.commit()`; rollback on drop in both	8

Task	sqlx	tokio-postgres	Chapter
Isolation level	`SET TRANSACTION ISOLATION LEVEL …` as SQL	`build_transaction().isolation_level(…)`	8
`COPY FROM STDIN`	`conn.copy_in_raw(sql)`, `send`, `finish`	`copy_in` → `CopyInSink`, `send`, `finish`; binary rows with `BinaryCopyInWriter`	10
`LISTEN`	`PgListener::connect_with(&pool)`, `listen`, `recv` (reconnects itself)	drive the `Connection` with `poll_message` (reconnecting is yours)	11
Cancel a query	(see sqlx's documentation)	`cancel_token().cancel_query(tls)`	12
Error code	`err.as_database_error()?.code()`	`err.code() == Some(&SqlState::…)`	6

Task	diesel (with diesel-async for async)	tokio-postgres
Schema	`table!` macros generated by `diesel print-schema`	none: SQL text
A typed row	`#[derive(Queryable, Selectable)]`	`TryFrom<&Row>`, or `#[derive(FromSql)]` for composite values
A query	the query builder: `products.filter(sku.eq(x)).first(&mut conn)`	SQL text with `$1` parameters
Raw SQL	`diesel::sql_query(sql).bind::<Text, _>(x).load(&mut conn)`	`client.query(sql, &[&x])`
Migrations	`diesel_migrations::embed_migrations!`	your own (Chapter 14) or refinery

The biggest conceptual differences, beyond syntax: tokio-postgres makes every round trip visible (preparing, pipelining, portals), exposes the protocol features the others hide (`COPY` as a sink, notifications as messages, cancel requests), and leaves the mapping between rows and types to you.

E Glossary

Backend. The server process that serves one connection (one session). Its PID is `pg_backend_pid()`, and a `Notification`'s `process_id()` names the sender's.

Binary format. The encoding of values that tokio-postgres uses for parameters and results, and that binary `COPY` uses; as opposed to the text format of the simple protocol and text `COPY`.

Cancel request. A message sent on a new connection, naming a backend by PID and secret key, asking it to stop its current statement (Chapter 12).

Connection (tokio-postgres). The half of a connection that does the IO: a future to spawn, or to drive with `poll_message`. The other half is the `Client`.

Cursor. A portal created with SQL `DECLARE`, read with `FETCH` (Chapter 9).

Domain. A named type over a base type with constraints (`CREATE DOMAIN sku AS text CHECK …`). Parameters that target a domain column are described as the domain (Chapter 5).

Extended protocol. Parse, Bind, Describe, Execute and Sync messages: typed parameters, prepared statements and portals. What `query`, `execute` and friends use.

Head-of-line blocking. A request on a connection waiting behind a slow one ahead of it, because a session runs one statement at a time (Chapter 7).

MVCC. Multi-version concurrency control: each transaction sees a snapshot of committed row versions, so readers do not block writers (Chapter 8).

Notice. A non-error message from the server (`RAISE NOTICE`, "already exists, skipping"), delivered as `AsyncMessage::Notice` (Chapter 11).

Notification. A message delivered to sessions that `LISTEN` on a channel, when a transaction that ran `NOTIFY` commits (Chapter 11).

Outbox. A table of jobs written in the same transaction as the change they report, delivered afterwards by a worker, at least once (Chapter 16).

Pipelining. Sending several requests on one connection before their answers arrive; tokio-postgres does it whenever several futures are in flight on one client (Chapter 7).

Pool. A set of open connections lent out to tasks, one at a time each (Chapter 13).

Portal. The server-side state of an executing query, from which rows can be fetched in batches; named portals live until the end of their transaction (Chapter 9).

Prepared statement. A parsed and described statement kept on the server for its session, executed many times with different parameters (Chapter 4).

Round trip. One request and its answer over the network: the unit of latency that most of this book counts.

Savepoint. A point inside a transaction to which it can roll back without ending (Chapter 8).

Session. The server-side state of one connection: settings, prepared statements, temporary tables, `LISTEN`s, advisory locks.

Simple protocol. One `Query` message with SQL text, possibly several statements, no parameters, results as text (Chapter 2).

SQLSTATE. The five-character error code of a server error, grouped in classes by its first two characters (Chapter 6).

SSI. Serializable snapshot isolation: PostgreSQL's `SERIALIZABLE`, which detects dangerous patterns of reads and writes and fails one transaction with 40001 (Chapter 8).

Statement timeout. The server-side limit on how long one statement may run (`statement_timeout`), ending it with 57014 (Chapter 12).

Sync. The extended-protocol message that ends a request and makes the server answer `ReadyForQuery`; every tokio-postgres request ends with one.

F The crates built on tokio-postgres

On 2026-11-06, crates.io listed 883 crates that depend on tokio-postgres directly. This section covers the ones a reader of this book is most likely to meet, grouped by what they do, with what each wraps or exposes, how it relates to the chapters, and when to choose it. Versions, dates and download counts are crates.io's on that day; API facts come from each crate's documentation and published source.

Crate	Version (released)	Downloads	What it is
`postgres`	0.19.14 (2026-06-12)	21.4 M	the synchronous wrapper
`postgres-types`	0.2.14 (2026-06-12)	89.8 M	`ToSql`/`FromSql`, the derives
`postgres-protocol`	0.6.12 (2026-06-12)	90.7 M	the wire protocol, SCRAM, escaping
`deadpool-postgres`	0.14.2 (2026-08-26)	28.2 M	a pool (Chapter 13)
`bb8-postgres`	0.9.0 (2024-12-09)	7.2 M	a pool
`mobc-postgres`	0.9.0 (2025-06-25)	0.1 M	a pool
`tokio-postgres-rustls`	0.14.0 (2026-05-21)	19.5 M	TLS with rustls (Chapter 12)
`tokio-postgres-rustls-improved`	0.17.1 (2026-10-13)	0.08 M	a fork of the above
`postgres-native-tls`, `postgres-openssl`	0.5.3 (2026-03-30)	10.5 M, 5.4 M	TLS with native-tls, OpenSSL
`diesel-async`	0.9.2 (2026-06-19)	17.0 M	Diesel's async connections
`toasty`, `toasty-driver-postgresql`	0.11.0 (2026-09-25)	0.09 M	the Tokio project's ORM
`refinery`	0.10.0 (2026-10-02)	12.8 M	migrations (Chapter 14)
`cornucopia`	1.0.1 (2026-08-14)	0.2 M	code generation from SQL (Chapter 14)
`postgres-from-row`	0.5.2 (2023-05-26)	0.4 M	`#[derive(FromRow)]`
`tokio-pg-mapper`	0.2.0 (2021-01-04)	0.3 M	`#[derive(PostgresMapper)]`
`rust_decimal`	1.43.0 (2026-09-02)	171.6 M	`numeric` as `Decimal`
`pgvector`	0.4.2 (2026-05-22)	21.8 M	the `vector` extension's types

The family: `postgres`, `postgres-types`, `postgres-protocol`

All three live in the rust-postgres repository with tokio-postgres.

- **`postgres`** is "a lightweight wrapper over tokio-postgres": its `Client` holds a `tokio_postgres::Client` and a private current-thread Tokio runtime that drives the `Connection` with `poll_message`, and every method blocks on that runtime and takes `&mut self`. It re-exports tokio-postgres's `types`, `error`, `Statement`, `Portal`, `IsolationLevel` and `NoTls`, so the type and error chapters of this book apply unchanged. Choose it for synchronous programs (scripts, CLIs, threads); never call it from inside a Tokio runtime, where its `block_on` panics with "Cannot start a runtime from within a runtime". It has no `query_scalar` family in 0.19.14; `check_connection` arrived in 0.19.12 and the typed one-row methods in 0.19.13.

- **`postgres-types`** is where `ToSql`, `FromSql`, `Type` and `Json` live; `tokio_postgres::types` is a re-export of it. Depend on it directly for two reasons: the `derive` feature (Chapter 5), which only it has, and implementing the traits for your own types. Third-party crates do the same: `rust_decimal` and `pgvector` depend on `postgres-types`, not on tokio-postgres.
- **`postgres-protocol`** is the message layer underneath: frontend and backend messages, SCRAM authentication, the binary formats. It is "not typically used directly", with two useful exceptions: `password::scram_sha_256(password)` hashes a password on the client before an `ALTER ROLE … PASSWORD`, so the plain text is never sent to the server, and `escape::escape_identifier`, an alternative to pglisten's `quote_ident` for the identifiers that cannot be parameters.

Pools

- **deadpool-postgres** is the pool of Chapter 13 and stockroom.
- **bb8-postgres**: `PostgresConnectionManager<Tls>::new(tokio_postgres::Config, tls)`, for the generic `bb8` pool; `pool.get()` returns a `PooledConnection` that derefs to `tokio_postgres::Client`. It spawns each `Connection` as a plain future, so the connection's error is dropped, and notices only logged (Chapter 11). bb8 tests every connection on checkout by default (`test_on_check_out`, an empty query: one round trip, like deadpool's `Verified`) and closes idle and old connections in the background. It has no statement cache. Its last release is from December 2024.
- **mobc-postgres**: `PgConnectionManager<Tls>::new(config, tls)` for the `mobc` pool, also lending `tokio_postgres::Clients`, with a health check (an empty query) on every checkout by default. Its feature list stops at `with-uuid-0_8` and `with-time-0_2`: enable newer ones on tokio-postgres directly. The crate's listed repository is archived; the code now lives in mobc's repository.

All three hand out plain `tokio_postgres::Clients`, so everything in Chapters 2–12 works on their connections. The differences are deadpool's `prepare_cached` and recycling methods, and how much checking happens on checkout (§13.3).

TLS connectors

All implement `MakeTlsConnect<Socket>` (Chapter 12).

- **tokio-postgres-rustls** 0.14: `MakeRustlsConnect::new(rustls::ClientConfig)`, plus `with_webpki_roots()` and `with_native_certs()` behind features of the same names; you enable rustls's crypto provider (`ring` or `aws-lc-rs`) yourself. It sets no ALPN, so direct TLS needs `alpn_protocols` on the `ClientConfig` (§12.6). Channel binding follows the certificate's signature algorithm (SHA-256, SHA-384 or SHA-512), with none for Ed25519, for which RFC 5929 defines no digest. **Use 0.14 or later:** in 0.13.0 a certificate-parsing mistake made channel binding silently absent for *every* certificate, so `channel_binding=require` failed and `prefer` quietly did without; 0.14.0 fixed it (PR #32).
- **tokio-postgres-rustls-improved** is a fork created while that fix was waiting upstream. With the fix released, what it still adds is channel binding for ECDSA-with-SHA-512 certificates (with the `aws-lc-rs` provider), `aws-lc-rs` as the default provider, and a connector whose configuration can be reloaded at run time.

- **postgres-native-tls** (`MakeTlsConnector::new(native_tls::TlsConnector)`) uses the platform's TLS (Security.framework, SChannel or OpenSSL) and its trust store; **postgres-openssl** (`MakeTlsConnector::new(SslConnector)`, with `set_callback` for per-connection settings) uses OpenSSL. Both have `set_postgresql_alpn()`, "required when using `sslnegotiation=direct`", since 0.5.1.

Several other rustls connectors exist (`postgres_rustls`, `rustls-tokio-postgres`, `tokio-postgres-generic-rustls`); they do the same job with different defaults.

Above the driver: ORMs, code generation, migrations

- **diesel-async** (feature `postgres`) gives Diesel 2.3 an asynchronous `AsyncPgConnection`, which wraps a `tokio_postgres::Client`. `AsyncPgConnection::establish(url)` connects without TLS; for TLS, connect with tokio-postgres yourself and call `AsyncPgConnection::try_from_client_and_connection(client, connection)`, which drives the `Connection` with `poll_message` and exposes its notifications. Its documentation says pipelining "happens automatically when futures are polled concurrently", as Chapter 7 showed for the client underneath, and `cancel_token()` returns tokio-postgres's `CancelToken`. Choose it when you want Diesel's compile-time-checked query builder in async code; its pools come from bb8, deadpool (the generic crate) or mobc, through `diesel_async::pooled_connection`.

```
// diesel-async (illustration), adapted from its pooled-with-rustls example
let tls = tokio_postgres_rustls::MakeRustlsConnect::new(client_config);
let (client, connection) = tokio_postgres::connect(&url, tls).await?;
let mut conn = AsyncPgConnection::try_from_client_and_connection(client,
connection).await?;
```

- **toasty** (`features = ["postgresql"]`) is an asynchronous ORM from the Tokio project for SQLite, PostgreSQL, MySQL and DynamoDB. Its PostgreSQL driver holds a `tokio_postgres::Client` (`impl From<tokio_postgres::Client>` for its connection), prepares every statement with `prepare_typed` and caches up to 100, and uses tokio-postgres-rustls for TLS. It is young: its 2024 announcement called it a preview, and it is at 0.11 with breaking changes in each release.
- **cornucopia** 1.0 (§14.6) generates a crate of typed query functions from annotated SQL. The generated code defines its own `GenericClient` trait, implemented for `tokio_postgres::Client` and `Transaction` and, with its default `deadpool` feature, for deadpool's `Client` and `Transaction`, where it uses `prepare_cached`; a `sync` mode targets `postgres` instead. It absorbed the Clorinde fork in 2026, and the separate `cornucopia_async` runtime crate is no longer needed.
- **refinery** (§14.3): with the `tokio-postgres` feature, `run_async` takes a `&mut tokio_postgres::Client` and runs each migration in a transaction of its own; it takes no lock. Note that its `tokio-postgres-rustls` feature still asks for tokio-postgres-rustls 0.13, the release with broken channel binding: connect yourself with 0.14 and pass the client in.

Row mapping

Three crates derive the `TryFrom<&Row>` that Chapter 3 writes by hand. Check their dates first:

- **postgres-from-row** 0.5.2 (May 2023): `#[derive(FromRow)]` gives `from_row(&Row)`, which panics on a mismatch, and `try_from_row(&Row) -> Result<Self, tokio_postgres::Error>`, with `#[from_row(flatten)]`, `rename`, `from` and `try_from` attributes. Columns are matched by name.
- **tokio-pg-mapper** 0.2.0 (January 2021): `#[derive(PostgresMapper)]` with a `#[pg_mapper(table = "…")]` attribute, which is required (the derive panics without it, though the README's example leaves it out), and `from_row` / `from_row_ref`.
- **serde_postgres** 0.2.0 (2020) depends on tokio-postgres **0.5**, so its `from_row` cannot take a 0.7 `Row` at all.

For new code, a `TryFrom<&Row>` per type (Chapter 14) is a few lines and has no such risk; `postgres-from-row` is the reasonable choice if a derive is wanted.

Types that plug in through postgres-types

- **rust_decimal** with `db-tokio-postgres` (identical to `db-postgres`): `Decimal` for `numeric`. `NaN` and the infinities, which `numeric` can hold, fail to convert, as do values beyond its 96-bit mantissa (about 28 digits); digits beyond scale 28 are rounded away. For exact arbitrary-precision values, read `numeric` as text, or use a big-decimal crate.
- **pgvector** with `postgres`: `Vector`, `HalfVector` (feature `halfvec`), `SparseVector` and `Bit` for the extension's types, matched by type *name* because extension OIDs differ between databases, the same reason Chapter 5's derives match custom types by name.
- **fastnum** (feature `tokio-postgres`) is another decimal type with the same integration.

Next to the driver, not on it

- **pgwire-replication** 0.5.0 is a logical-replication client (`pgoutput`) written directly on the wire protocol; it "intentionally avoids … `tokio-postgres`" for the replication path, and uses tokio-postgres only in its tests. tokio-postgres 0.7.18 has no replication protocol of its own (rust-postgres issue #116 is open), so change-data capture means a crate like this one, or polling and `LISTEN` (Chapter 11).
- **tokio-cron-scheduler** (feature `postgres_storage`) keeps its job schedule in PostgreSQL through tokio-postgres, and Cloudflare's **worker** crate implements tokio-postgres's `TlsConnect` for its sockets, so that Workers can use tokio-postgres directly: two examples of how far the driver travels beyond servers like stockroom.

G Further reading

- **tokio-postgres** documentation, 0.7.18: docs.rs/tokio-postgres/0.7.18, and its source and issues at github.com/rust-postgres/rust-postgres.
- **postgres-types**: docs.rs/postgres-types/0.2.14, for `ToSql`, `FromSql` and the derives.
- **deadpool-postgres**: docs.rs/deadpool-postgres/0.14.2.
- **The PostgreSQL protocol**: Frontend/Backend Protocol, especially Message Flow, which explains pipelining, `COPY`, cancel requests and TLS negotiation at the level this book works at.
- **Concurrency**: Transaction Isolation and Explicit Locking.

- **Errors**: PostgreSQL Error Codes, the complete SQLSTATE list.
- **`COPY`**, **`NOTIFY`**, **`DECLARE`**: the SQL reference pages, COPY, NOTIFY, DECLARE.
- **Server settings** used in the book: Client Connection Defaults (`statement_timeout`, `lock_timeout`, `idle_in_transaction_session_timeout`, `transaction_timeout`).
- **axum**: docs.rs/axum/0.8.9, and tower-http's timeout and limit layers.
- **Release notes** of the PostgreSQL versions whose features the book uses: 15 (`MERGE`, `HEADER MATCH`), 16 (`pg_input_is_valid`), 17 (`ON_ERROR`, `MERGE … RETURNING`, `transaction_timeout`, direct TLS), 18 (`uuidv7()`, virtual generated columns, `RETURNING old/new`, `REJECT_LIMIT`).